CANTEEN EXPRESS: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff, and Faculty at Palawan State University Main Canteen

A Capstone Project Presented to the

Faculty of the College of Sciences

Palawan State University

In Partial Fulfillment

of Requirement for the Degree

Bachelor of Science in Information Technology

By:

Rivera, Vince Alshie V.

Fajiculay, Jonathan G.

Madia, Norelyn R.

Tumalac, Juliet Lou T.

November 2026

EXECUTIVE SUMMARY

The Palawan State University (PalSU) Main Canteen offers food services to students, faculty and staff. During meal periods, customers visit the canteen to purchase food, while canteen staff attend to orders, payments and food preparation. Customers may also wish to view available food items and place orders before going to the canteen.

To provide another way of ordering food, Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff, and Faculty at Palawan State University Main Canteen was developed. Student/Walk-In customers can browse the menu and place orders for counter pickup using their smartphones through the mobile kiosk interface. Faculty and Staff can create accounts, place advance orders, choose counter pickup or eligible on-campus delivery, monitor order statuses, communicate with Delivery Personnel and earn loyalty points. Canteen Staff can access the unified POS and Kitchen Board, while the Administrator manages user accounts, menu items, and reports. Delivery Personnel can receive requests and update delivery statuses.

The system brings together the ordering activities of customers and canteen personnel in one platform. It also includes features for order monitoring and eligible on-campus delivery. After development, Canteen Express underwent User Acceptance Testing (UAT) with selected users. Their feedback was gathered to assess its functionality and usability using selected characteristics of the ISO/IEC 25010 software quality model and the System Usability Scale (SUS).

Table of Contents

Title Page

Executive Summary…….....i

Table of Content……...……...ii

List of Figures ….....v

List of Tables ….....viii

Chapter 1: INTRODUCTION.....1

Project Context ….....3

Purpose and Description ….....4

Objectives

General Objective..….....11

Specific Objectives.….....11

Significance of the Study ….....13

Scope and Limitations ….....14

Scope……..14

Limitations…….17

Operational Definition of Terms ….....18

Chapter 2: REVIEW OF RELATED LITERATURE/SYSTEMS.....24

Local….....24

Foreign….....28

Matrix of Related Literature……..…….....32

Chapter 3: TECHNICAL BACKGROUND….....45

Physical Architecture…….....45

Organizational Chart.….....48

System Flowchart.….....50

Hardware and Software Specifications.….....59

List of Hardware Specifications for the Development of the System.….....59

List of Software Specifications for the Development of the System…….....62

List of Hardware Specifications for the Implementation of the System…….....64

List of Software Specifications for the Implementation of the System…….....67

Chapter 4: METHODOLOGY….....71

Software Development Life Cycle.….....71

Conceptual Framework.….....77

Requirement Analysis.….....79

Use-Case Diagram ….....81

User Stories ….....83

System/Software Requirements Specification ….....89

Functional Requirements ….....91

Non-Functional Requirements.....97

Requirements Documentation ….....99

Ethical Consideration.….....100

Work Plan (Gantt Chart).….....104

Cost Benefit Analysis.….....104

Design of Software, Systems, Product and/or Processes ….....106

Data Flow Diagram ….....107

Entity Relationship Diagram ….....109

Network Topology and Architecture.....113

Chapter 5: RESULTS AND DISCUSSION….....115

Development and Testing ….....115

Implementation Plan ….....120

Implementation Results ….....122

Results and Discussion ….....123

Chapter 6: CONCLUSION AND RECOMMENDATIONS ….....139

Conclusion ….....139

Recommendations ….....142

REFERENCES ….....….....….....….....146

APPENDICES ….....….....148

List of Figures

Figure 3.1 Physical Architecture……....45

Figure 3.2 Organization Chart……...48

Figure 3.3 Canteen Staff/Admin Flowchart…….……..50

Figure 3.4 Delivery Personnel Flowchart……...……...52

Figure 3.5 Customer (Student/Walk-In) Flowchart...……54

Figure 3.6 Customer (Faculty/Staff) Flowchart...……..56

Figure 4.1 Agile Methodology……...73

Figure 4.2 Conceptual Framework……78

Figure 4.3 Use Case Diagram……82

Figure 4.4 Gantt Chart…….104

Figure 4.5 Data Flow Diagram……108

Figure 4.6 Entity Relationship Diagram…….….111

Figure 4.7 Network Topology and Architecture……..114

Figure 5.1 Prototype Development……...116

Figure 5.2 System Development……..117

Figure 5.3 Usability and Navigation……124

Figure 5.4 Ordering and Checkout Functions……..125

Figure 5.5 Queue Slip, Barcode Generation and Menu Availability……...……126

Figure 5.6 System Responsiveness and Visual Clarity……...….128

Figure 5.7 Overall Functionality & Recommendation Among Students & Walk-In Users…....129

Figure 5.8 Account Access and Role-Based Capabilities……..…..131

Figure 5.9 Loyalty Points and Order History……..…….131

Figure 5.10 Campus Delivery Requests and Real-Time Tracking…….….132

Figure 5.11 In-App Communications for Delivery Updates……133

Figure 5.12 Counter POS and Barcode Processing…….134

Figure 5.13 Mobile PWA Experience and Menu Browsing……135

Figure 5.14 Overall User Satisfaction and Reliability Among Faculty and Staff……...136

Figure 6.1 Onboarding Screen of Screen of Student/Walk-In Users……...173

Figure 6.2 Student/Walk-In Menu Interface, Cart and Queue Slip Screens……174

Figure 6.3 Cancel Queue Slip and Order History Screens……...175

Figure 6.4 Faculty/Staff Sign In, Sign Up and Terms Modal……..176

Figure 6.5 Customer Menu Browsing, Cart and Order Checkout Flow……..177

Figure 6.6 Live Tracking, Delivery and Post Order Features……..179

Figure 6.7 Customer Notification System and Account Navigation Menu…….180

Figure 6.8 Delivery Personnel Portal and Rider Hub Workflow…….181

Figure 6.9 Rider Dispatch, Live GPS Tracking and Delivery Fulfillment Workflow…….182

Figure 6.10 Canteen Staff Portal Login and Management Dashboard Overview……...183

Figure 6.11 Menu and Stock Management Interface……...184

Figure 6.12 Registered Users Management View……185

Figure 6.13 Delivery Personnel and Campus Dispatch View with Account Creation Modal….186

Figure 6.14 Feedback and Ratings Dashboard……187

Figure 6.15 Sales Reports, Analytics and Rider Rankings Dashboard……188

Figure 6.16 Activity Log Page……188

Figure 6.17 Change Password……..189

Figure 6.18 Kitchen Order Display and Counter POS Interface…….189

Figure 6.19 E-Receipt Student/Walk-In Customer through Canteen Staff POS…….190

Figure 6.20 Sample Input…….....……196

Figure 6.21 Sample Output…….....…….…197

Figure 6.22 Sample Reports…….....…….…198

Figure 6.23 Process Flow…….....…….…211

Figure 6.24 Data Flow…….....…….…212

Figure 6.25 Information Flow…….....…….…213

Figure 6.26 Mobile Kiosk Interface & E-Receipt Generation Workflow……...……..…214

List of Tables

Table 2.1 Matrix of Similarities and Differences of the Research (Local)……38

Table 2.2 Matrix of Similarities and Differences of the Research (Foreign)…….44

Table 3.1 List of Hardware Specifications for the Development of the System……59

Table 3.2 List of Software Specifications for the Development of the System…….....62

Table 3.3 List of Hardware Specifications for the Implementation of the System……64

Table 3.4 List of Software Specifications for the Implementation of the System…….67

Table 4.1 User Stories……88

CHAPTER 1

INTRODUCTION

In today's academic environment, students, faculty and staff often have limited time to purchase meals between classes and work responsibilities. During peak meal periods, an increase in customer volume at university canteens may contribute to longer queues and waiting times. These conditions highlight the importance of organizing food orders during busy periods while maintaining the canteen's regular operations.

Digital ordering systems offer another way for customers to browse food items and submit orders. Bari et al. (2025) discussed an online food ordering system for college canteens and its potential to support waiting-time management and order organization. Digital platforms have also been used in campus dining services in other countries. For example, the University of Notre Dame and The Ohio State University have used Grubhub Campus Dining to support food ordering at participating dining locations (Notre Dame Campus Dining, 2024; Grubhub, 2021; Ohio State University Dining Services.). These examples illustrate how digital ordering can be incorporated into campus dining services, although the available functions depend on the system and institution.

In the Philippines, university canteens use different approaches to serve their customers. Pangasinan State University (n.d.), for example, describes a cafeteria that uses a self-service buffet arrangement. At Palawan State University (PalSU), the Main Canteen serves students, faculty and staff through its existing food service and counter-ordering process. During peak meal periods, several customers may arrive within a limited time, increasing activity around the ordering and serving areas. This situation may be challenging for students with short breaks

between classes and for faculty and staff who need to manage their work schedules. An additional ordering option may help organize order submission while complementing the canteen's existing services.

To address this need, the researchers developed Canteen Express, a web/mobile-based Progressive Web Application (PWA) for the PalSU Main Canteen. The system provides a shared menu-browsing and digital ordering process for Student/Walk-In and Faculty/Staff customers. Student/Walk-In customers can access the application through their personal smartphones and submit orders for counter pickup without creating an account using the Mobile Kiosk. Faculty/Staff customers use their institutional email addresses and a six-digit one-time password (OTP) to access additional functions, including advance ordering and eligible on-campus delivery. Both customer types can browse available menu items and submit orders, but delivery is available only to eligible Faculty/Staff customers.

The system also provides functions for processing cash payments, managing food preparation, updating order statuses and handling eligible deliveries. Pickup orders are paid for at the canteen counter, while eligible delivery orders are paid for in cash upon receipt. Order-status updates help customers follow the progress of their orders, while GPS-based location information is available for eligible deliveries. Canteen Staff, Delivery Personnel and the Administrator are provided with role-specific interfaces to support their respective responsibilities. Canteen Express is intended to complement the existing ordering process by providing an additional way to submit and manage orders, particularly during peak meal periods.

Project Context

The Palawan State University (PalSU) Main Canteen serves meals to students, faculty and staff as part of the university's daily campus activities. The canteen's ordering and serving activities are important parts of this service. During peak meal periods, an increase in the number of customers may create more activity around the ordering and serving areas. Customers with limited time between classes or scheduled work responsibilities may need to consider the time required to place and receive their orders.

Customers also need access to available menu items and relevant food information when selecting their meals. Since food availability may change throughout the day, updated menu information can help customers make choices based on the items listed in the system. An additional way to browse the menu and submit orders may support customers while allowing the canteen to continue its existing counter-based operations.

To better understand the existing ordering process and identify possible areas for improvement, the researchers coordinated with Mrs. Janeth G. Dalojo, Food Supervisor II of the PalSU Main Canteen. The project was developed based on the identified needs and the intended scope of the study. The resulting system, Canteen Express, is a web/mobile-based PWA that provides a common menu-browsing and ordering process for Student/Walk-In and Faculty/Staff customers. Both customer types can browse available food items and submit orders, while their access permissions differ according to their customer mode. Student/Walk-In customers can place counter-pickup orders without an account, whereas authenticated Faculty/Staff customers can choose counter pickup or eligible on-campus delivery.

The system also provides interfaces for Canteen Staff to verify orders, process cash payments and manage food preparation; for Delivery Personnel to handle accepted delivery requests; and for the Administrator to maintain system information and review recorded data. The study focuses on the development and evaluation of these functions in relation to the needs of the PalSU Main Canteen. It does not assume that the system will automatically eliminate queues or improve service performance. Instead, its suitability is examined through the system's implemented functions and the evaluation procedures described in the study.

Purpose and Description

Canteen Express is designed to offer students, faculty and staff of Palawan State University an additional channel for ordering food from the PalSU Main Canteen via a web/mobile-based Progressive Web Application (PWA). The system enables digital menu browsing, order submission, cash-based order fulfillment and order management tailored to each role. Its main goal is to help organize orders while complementing the canteen's existing services.

The application offers a common basic ordering process for both Student/Walk-In and Faculty/Staff customers. Both types of customers can browse the available menu items and view the nutritional information stored in the system. What mainly sets them apart is the access method and the extra functions available to authenticated Faculty/Staff customers.

Student/Walk-In customers can use Canteen Express on their personal smartphones through a compatible web browser. The project refers to this setup as a Mobile Kiosk, meaning the customer's own device acts as the interface for browsing the menu and submitting a

counter-pickup order. It does not refer to a separate physical kiosk. Student/Walk-In customers are not required to create an account to use this ordering mode.

Faculty/Staff customers must register or log in with their PalSU institutional email address and complete the required six-digit OTP verification. Once verified, they can use the same basic menu-browsing and ordering functions that are available to Student/Walk-In customers. They can also place orders in advance and choose counter pickup or eligible on-campus delivery. Whether delivery is available depends on the system's delivery-area restrictions and the applicable conditions of the delivery service.

For Student/Walk-In counter orders, the system produces a digital order slip that includes the queue number, order details, total amount and barcode. The customer shows the slip at the canteen counter, where Canteen Staff use the Counter Point-of-Sale (POS) interface to retrieve and verify the order and to process the cash payment. Once payment is confirmed the order moves on to kitchen preparation. Canteen Staff use the Kitchen Board to update the order status as it progresses through the preparation and fulfillment stages. Unpaid orders may be cancelled automatically based on the time limit configured in the system.

Faculty/Staff customers who choose counter pickup also pay in cash at the canteen counter. For orders that qualify for delivery, payment is collected in cash when the order arrives. Faculty and Staff customers are able to check order-status updates and, for deliveries that apply, use GPS-based location data to track the assigned Delivery Personnel. Communication regarding eligible deliveries is supported through an in-app chat feature. Eligible Faculty/Staff customers may also provide ratings and feedback once a transaction is complete and they can take part in the loyalty rewards feature, which grants one point for every ₱100 spent on eligible purchases.

Points that have accumulated can be redeemed for available discount vouchers in accordance with the system's rules.

Canteen Staff rely on the Counter POS and Kitchen Board to process orders and coordinate food preparation. Even when using the same device as Canteen Staff, the Administrator logs in separately to access administrative functions. Delivery Personnel have a separate interface where they can view and accept eligible requests, update delivery statuses, access location and route information, communicate with customers, upload proof-of-delivery photos and review earnings tied to completed deliveries.

The following are the features of the system for each type of user:

For Customers:

The system allows customers to browse available canteen menu items and view their corresponding nutritional information to support their food selection.

For Students/Walk-In:

- Account-Free Access: Student/Walk-In customers can access the ordering interface through a compatible browser on their personal smartphones without creating an account.
- Digital Counter Ordering: Customers can select menu items, add them to their cart and submit orders for counter pickup.
- Digital Order Slip: After submitting an order, the system generates a digital slip containing the queue number, order list, total amount and barcode.
- Barcode Verification: Customers present the digital order slip at the counter so Canteen Staff can retrieve and verify the order through the Counter POS.

- Cash Payment and Pickup: Customers pay in cash at the counter. After the order is prepared and marked ready, they collect it at the canteen.

For Faculty/Staff:

- Account Registration and Verification: Faculty/Staff customers register or log in using their institutional email addresses and complete the required six-digit OTP verification.
- Shared Menu and Ordering Functions: Authenticated customers can browse the menu, review available food information, select items and submit orders.
- Advance Ordering: Customers can place orders in advance through their authenticated accounts.
- Pickup and Eligible Delivery: Customers can choose counter pickup or request delivery to an eligible location within the designated service area.
- Order-Status Monitoring: Customers can view the order-status updates provided as their orders progress through fulfillment.
- GPS-Based Delivery Monitoring: For eligible delivery orders, customers can view location information for their assigned Delivery Personnel, subject to GPS and network availability.
- In-App Chat: Customers can communicate with assigned Delivery Personnel about eligible delivery orders.
- Ratings and Feedback: Customers can submit feedback about their order or delivery experience after the transaction is completed.

- Loyalty Rewards: Eligible purchases earn one point for every ₱100 spent. Accumulated points can be redeemed for available discount vouchers according to the system's rules.

For Canteen Staff:

Canteen Staff handle daily order processing, counter transactions and food preparation at the PalSU Main Canteen. They use the Counter POS and Kitchen Board to retrieve order information, process payments and coordinate fulfillment.

- Counter POS: Staff use the Counter POS to scan order barcodes or retrieve orders using available order information, verify the order details and process cash payments at the counter.
- Payment Confirmation: Staff confirm counter payments through the system so that unpaid orders can proceed to kitchen preparation.
- Kitchen Board: Staff use the Kitchen Board to view incoming orders and monitor their preparation status.
- Order-Status Management: Staff update order statuses through the applicable stages, including Pending, Preparing, Ready and Completed.
- Order Coordination: The system provides order details and status information to support food preparation and the release of completed orders.

For Admin:

The Administrator is responsible for maintaining system information and accessing administrative functions. The Administrator uses a separate login with permissions distinct from

those of Canteen Staff. Although both roles may use the same device, their system access and responsibilities remain separate.

- User Account Management: The Administrator manages customer and Delivery Personnel accounts through the available administrative functions.
- Menu Management: The Administrator adds, edits or removes menu items and maintains their prices, availability and nutritional information.
- Promotion and Voucher Management: The Administrator manages supported promotions and discount vouchers.
- Reports and Analytics: The Administrator reviews reports and analytics based on recorded transactions and system data.
- Feedback Management: The Administrator reviews customer feedback and handles concerns through the available functions.
- System Configuration: The Administrator manages supported settings and other administrative functions.

The Administrator's responsibilities differ from those of Canteen Staff. Staff focus on counter transactions and food preparation, while the Administrator maintains system information and oversees the administrative functions available in Canteen Express.

For Delivery Personnel:

Delivery Personnel handle accepted delivery requests within the system's designated service area. They use the delivery interface to view available requests, accept eligible orders, update delivery progress and access location information.

The main features for Delivery Personnel include:

- Delivery Request Management: Delivery Personnel view and accept available requests that qualify for on-campus delivery.
- GPS Location and Route Guidance: The interface provides location and route information to assist Delivery Personnel in reaching the specified delivery destination.
- Delivery-Status Updates: Delivery Personnel update the delivery progress through the available status controls.
- In-App Chat: Delivery Personnel communicate with customers regarding their assigned deliveries.
- Proof of Delivery: Delivery Personnel upload a photo documenting the completed handoff when required by the system.
    - Earnings Monitoring: Delivery Personnel can review earnings associated with completed deliveries through the available interface.
- Delivery-Area Restriction: The system restricts delivery requests to the designated service area within a 0.8-kilometer radius of the specified campus-center coordinates. Requests outside the permitted area are rejected.

PalSU students may apply to become part-time Delivery Personnel. However submitting an application does not automatically authorize an applicant to fulfill delivery requests. Recruitment, screening, selection, scheduling and supervision remain the responsibility of PalSU Main Canteen management and are not automated by the system.

Overall, Canteen Express brings customer ordering, counter transactions, food preparation, eligible delivery fulfillment and administrative functions together in one web/mobile-based platform. The functions available to each user depend on the assigned role and customer mode. The system's usability and quality are examined through the testing and evaluation procedures of the study and its findings are based on the data collected rather than assumed operational improvements.

Objectives

General Objective

The general objective of the study was to design, develop and evaluate Canteen Express, a web/mobile-based Progressive Web Application (PWA) for digital food ordering and order management at the Palawan State University (PalSU) Main Canteen. The system provides shared menu-browsing and basic ordering functions for Student/Walk-In and Faculty/Staff customers, with authenticated Faculty/Staff customers receiving additional access to advance ordering and eligible on-campus delivery. It also provides role-specific functions for Canteen Staff, Delivery Personnel and the Administrator.

Specific Objectives

1. Develop a responsive web/mobile-based ordering application that enables Student/Walk-In and Faculty/Staff customers to browse available menu items, view recorded nutritional information, select food items and submit orders, while providing account-free counter ordering for Student/Walk-In customers and institutional email and OTP verification for Faculty/Staff customers.

2. Provide additional ordering options for authenticated Faculty/Staff customers by enabling advance ordering, counter pickup and eligible on-campus delivery, with cash payment at the counter for pickup orders and cash payment upon receipt for delivery orders.

3. Develop an order-processing and kitchen-management interface that allows Canteen Staff to retrieve and verify orders, process cash payments through the Counter POS and update order statuses through the Kitchen Board during preparation and fulfillment.

4. Provide delivery-management functions that allow authorized Delivery Personnel to view and accept eligible requests, access GPS location and route information, update delivery statuses, communicate with customers, submit proof-of-delivery photos and review earnings associated with completed deliveries.

5. Develop administrative functions that allow the Administrator to manage user accounts, menu items, prices, nutritional information, menu stock, promotions, discount vouchers, system settings, reports and analytics based on recorded system data.

6. Provide a loyalty rewards feature for eligible Faculty/Staff customers that awards one point for every ₱100 spent on eligible purchases and allows accumulated points to be redeemed for available discount vouchers according to the system's rules.

7. Evaluate the developed system through User Acceptance Testing (UAT), using selected software quality characteristics from ISO/IEC 25010 and the System Usability Scale (SUS), based on the study's evaluation procedures and the data collected from intended users.

Significance of the Study

This study presents the development and evaluation of Canteen Express, a web/mobile-based Progressive Web Application for the PalSU Main Canteen. The system's functions and evaluation findings may provide information about the use of digital ordering and order management in a university canteen setting.

The Student/Walk-In customers system provides an additional way to browse available menu items and submit counter-pickup orders through personal smartphones. It retains cash payment at the canteen counter and does not require Student/Walk-In customers to create an account. The study may help explain how account-free digital ordering can be incorporated into an existing campus food service process.

Authenticated Faculty/Staff customers can use the same basic menu-browsing and ordering functions as Student/Walk-In customers. They also have access to advance ordering, order-status monitoring and eligible on-campus delivery. The study documents how these additional functions are implemented and evaluated for users with different ordering needs.

For Canteen Staff the system provides interfaces for verifying digital orders, confirming counter payments and updating order statuses during food preparation. The study documents how these functions support the recording and coordination of orders while retaining the staff's responsibility for payment verification, preparation and order release.

For PalSU Main Canteen management the delivery interface provides functions for accepting eligible delivery requests, updating delivery statuses, accessing GPS and route information, communicating with customers, submitting proof of delivery and reviewing

earnings. The study documents how these tools are used within the system's limited on-campus delivery scope.

For delivery personnel the delivery interface provides functions for accepting eligible delivery requests, updating delivery statuses, accessing GPS and route information, communicating with customers, submitting proof of delivery and reviewing earnings. The study documents how these tools are used within the system's limited on-campus delivery scope.

Finally for future researchers this study may serve as a reference for future work involving web/mobile-based ordering systems, Progressive Web Applications, and order management in educational institutions. Its documentation of user roles, system functions, limitations and evaluation procedures may help future researchers compare approaches or adapt similar systems to other institutional settings.

Scope and Limitations

Scope

Canteen Express was created for the Palawan State University (PalSU) Main Campus. It allows the community of PalSU to have another option of ordering their meals from the PalSU Main Canteen via a web/mobile-based Progressive Web Application (PWA). It has various options such as online ordering, pre-ordering and eligible delivery. The application contains four primary user roles, namely, Customer, Canteen Staff, Delivery Personnel and Administrator. The Customer role can be segmented into two types: Student/Walk-In and Faculty/Staff. This allows the appropriate roles to have access to the correct features using role-based access control (RBAC).

The Student/Walk-In role allows customers to browse available menu items, view nutritional information and submit orders for counter pickup using their personal smartphones through a compatible web browser. After checkout, the system generates a digital order slip containing the queue number, order list, total amount and barcode. Customers present the order slip at the canteen counter, where Canteen Staff verify the order and process cash payment before the order proceeds to preparation.

The Faculty/Staff role contains a variety of features. It allows customers to pre-order and have eligible delivery. To access this, the customer must register/login using their institutional email and enter the correct 6 digit One Time Password (OTP). Once verified, they can view available menu items, nutritional information, place pre-orders. The faculty/staff can either choose to collect their order from the counter or have it delivered to them if eligible. If they are collecting from the counter, they pay cash. If they are having it delivered, they pay cash when the order arrives. The Customer can track their order status. For eligible deliveries, they can track their assigned Delivery Personnel live using GPS. They can also message their assigned rider. They can leave feedback and reviews once they have completed their transaction. For eligible Faculty/Staff customers, they receive 1 Loyalty Point per every ₱100 they spend. They can then use these points to redeem any available discount vouchers.

The Counter Point Of Sale (POS) and Kitchen Board allows the Canteen Staff to manage orders. The POS allows the staff to scan the digital order receipts. They can view what the order contains, verify the order and take payment in cash. The Kitchen Board allows the staff to view the incoming orders and update the status of the order. This includes: Pending , Preparing , Ready and Completed .

The Administrator has their own login and is used to manage the overall system. They can manage customer and Delivery Personnel accounts, menu items, pricing, nutritional information, stock, promotions, discount vouchers, reports, analytics and other settings. They can also view customer feedback and resolve any issues raised.

The Delivery Personnel can view eligible orders using their own interface. They can select an order to deliver. They can update order status, message customers using in-app messaging and use the GPS to view their location and route. Once they have delivered the order, they can upload a proof of delivery photo. They can also view how much they have earned. There is a geofence set up, so that they only deliver within the service area. This has been set as a radius of 0.8km from the centre of campus. If they try to deliver outside of the allowed area it will not allow them to. Whilst PalSU students can apply to be part-time Delivery Personnel, the recruitment, screening, selection and administration of these individuals is the responsibility of the PalSU Main Canteen Management.

Canteen Express has been created as a Responsive Web Application using a PWA. This means it can be accessed via a browser on an Android or iOS device. It is not a native Android/iOS application. There is a database set up to store and manage customer account information, menu items, orders and transaction information. You will need to be connected to the internet to use the online features. Authentication has been put in place to keep customers safe and role-based access control (RBAC) to ensure they can only access certain features. We took into account the Data Privacy Act of 2012 (Republic Act No. 10173) and other legal requirements. Testing was carried out and the results were reported.

Limitations

Canteen Express is only available at the Palawan State University (PalSU), Main Canteen. Student/Walk-In customers may order using their own personal mobile phones, while Faculty/Staff customers must register/login using their institutional account in order to utilise the extra features provided such as pre-ordering and on-campus deliveries if applicable. All orders are paid using cash as there is no online payment system.

Delivery Personnel are not automated in terms of recruitment, selection, scheduling or payroll. These are handled by the management of the PalSU Main Canteen. This means that the amount of Delivery Personnel currently available will dictate how quickly orders can be delivered. Deliveries are also only available in the set delivery area. This is set to be within a 0.8km radius of the given campus centre coordinates. Orders cannot be delivered out of this area.

The menu items, availability, price and nutritional information will depend on what has been inputted into the system by an authorised person. If the information is old or incorrect then this may cause issues. Order status updates will depend on Canteen Staff and Delivery Personnel inputting the correct stage. This means that the system will not automatically track all stages of the order. GPS tracking is available for applicable deliveries but will depend on the accuracy of the GPS from the customer and Delivery Personnel devices.

Canteen Express needs an active internet connection to communicate with the backend. There is no option to order whilst offline. This means that if there is no internet connection then you may not be able to place your order, see updated information about the order or utilise delivery features.

Overall the map and delivery features rely on the GPS from the device, internet connection and third party map and delivery services. The map data is provided by OpenStreetMap and delivery routes are provided by the set delivery services. If there is inaccurate GPS data, incomplete map data, or the services are not working this may cause issues with the location and suggested routes. This means that there is no guarantee that there will be no issues with live tracking.

Operational Definition of Terms

Administrator (Admin) - An authorized system role with access to administrative functions, including account management, menu and menu stock maintenance, promotion and voucher management, reports, analytics, feedback management and supported system settings. The Administrator uses a separate login with permissions distinct from those of Canteen Staff.

Barcode Verification - The process through which Canteen Staff scan or use the barcode on a customer's digital order slip to retrieve and verify the corresponding order through the Counter POS.

Canteen Express - The web/mobile-based Progressive Web Application developed for the PalSU Main Canteen. It supports shared menu-browsing and ordering functions for Student/Walk-In and Faculty/Staff customers, counter pickup, eligible Faculty/Staff delivery and role-specific functions for Canteen Staff, Delivery Personnel and the Administrator.

Canteen Staff - Authorized canteen personnel who handle order verification, counter transactions, cash-payment processing, food preparation coordination and order-status updates through the Counter POS and Kitchen Board interfaces.

Counter Pickup - An order-fulfillment option in which the customer collects the prepared order at the PalSU Main Canteen. Counter-pickup orders are paid for in cash at the counter.

Counter POS (Point of Sale) - The interface used by Canteen Staff to retrieve and verify orders, scan digital order slips where supported, process cash payments and confirm payment before an order proceeds to preparation.

Customer - A person who uses Canteen Express through either the Student/Walk-In or Faculty/Staff customer mode. Both modes support menu browsing and order submission, while authentication and available fulfillment options differ.

Delivery Fee - The fee charged for eligible on-campus delivery orders according to the system's implemented calculation rules. The fee description in the thesis must match the actual fee calculation used by the application.

Delivery Personnel - Individuals authorized by PalSU Main Canteen management to fulfill eligible on-campus delivery requests. Through the delivery interface, they can accept requests, update delivery statuses, access GPS and route information, communicate with customers, submit proof-of-delivery photos and review earnings associated with completed deliveries.

Digital Order Slip - The electronic record generated after a customer submits an order. It contains the order details and payment information, including the queue number, item list, total amount and barcode used for order retrieval and verification.

Discount Voucher - A digital discount reward available for redemption through the loyalty rewards feature according to the system's implemented rules.

Faculty/Staff Customer - A PalSU faculty member or staff member who authenticates through the required institutional email and six-digit OTP process. This customer mode supports menu browsing and order submission, as well as advance ordering, counter pickup, eligible on-campus delivery, order-status monitoring and other available features.

Feedback and Ratings - A feature that allows eligible customers to submit ratings or comments about their order or delivery experience after a completed transaction. The available feedback can be reviewed through the appropriate system functions.

GPS-Based Delivery Monitoring - A feature that displays location information associated with an eligible delivery using GPS data from the relevant devices. Its accuracy and availability depend on device permissions, location signals and network connectivity.

In-App Chat - A communication feature that allows customers and their assigned Delivery Personnel to exchange messages concerning eligible delivery orders.

Kitchen Board - A digital interface used by Canteen Staff to view incoming orders, monitor their progress and update the available order statuses during food preparation and fulfillment.

Loyalty Rewards System - A feature available to eligible Faculty/Staff customers that awards one point for every ₱100 spent on eligible purchases. Accumulated points may be redeemed for available discount vouchers according to the system's rules.

Menu Management - Administrative functions used to maintain menu items, prices, availability, nutritional information in the system.

Mobile Kiosk - The BYOD (Bring Your Own Device) arrangement through which Student/Walk-In customers use their own compatible smartphones to access the Canteen Express ordering interface. It refers to a mobile ordering interface, not a separate physical kiosk.

Nutritional Information - Food-related information entered and maintained by authorized personnel and displayed with applicable menu items to help customers review the details available in the system.

One-Time Password (OTP) - A six-digit verification code used by Faculty/Staff customers as part of the institutional account authentication process.

On-Campus Delivery - An order-fulfillment option in which authorized Delivery Personnel deliver an eligible Faculty/Staff order to a specified location within the system's designated campus service area. Delivery requests outside the configured boundary are not accepted.

Order Fulfillment - The process of handling an order from submission through payment, preparation and completion. Depending on the customer's mode and selected option, fulfillment may involve counter pickup or eligible on-campus delivery.

Order-Status Updates - Information displayed by the system as Canteen Staff and Delivery Personnel update the progress of an order. These updates depend on the status information entered through the available interfaces.

Peak Hours - Periods during the school day when the number of customers visiting the PalSU Main Canteen is relatively high and ordering and serving activities may become busier.

Palawan State University (PalSU) Main Canteen - The food service facility at the PalSU Main Campus in Tiniguiban Heights, Puerto Princesa City, Palawan. It is the designated client and implementation site of Canteen Express for this study.

Pre-Ordering - The function that allows authenticated Faculty/Staff customers to submit food orders in advance and choose counter pickup or eligible on-campus delivery.

Progressive Web Application (PWA) - A web application designed to be accessed through compatible browsers on supported devices and to provide selected app-like features. Canteen Express is a PWA rather than a native Android or iOS application.

Proof of Delivery (PoD) - A feature that allows Delivery Personnel to upload a photo documenting the handoff of a completed delivery.

Rider Hub - The delivery interface used by authorized Delivery Personnel to view and accept eligible delivery requests, update delivery statuses, access GPS and route information, communicate with customers, submit proof-of-delivery photos and review available earnings information.

Role-Based Access Control (RBAC) - An access-control approach that assigns permissions according to user roles. Canteen Express has four primary roles: Customer, Canteen Staff, Delivery Personnel and Administrator. The Customer role has two modes: Student/Walk-In and Faculty/Staff.

Student/Walk-In Customer - A customer, including an enrolled student or eligible walk-in patron, who accesses Canteen Express through a personal smartphone or other supported browser-enabled device to browse the menu, submit a counter-pickup order, receive a digital order slip and pay in cash at the canteen counter. This mode does not require an account and does not include access to Faculty/Staff-only delivery and loyalty functions.

User Acceptance Testing (UAT) - The testing and evaluation process used to gather feedback from selected intended users regarding the developed system, according to the study's test procedures and evaluation instruments.

Verification Process - The procedure through which Faculty/Staff customers confirm their institutional account using the required six-digit OTP before accessing the functions associated with their customer mode.

CHAPTER 2

REVIEW OF RELATED LITERATURE/ SYSTEMS

This chapter presents a review of related literature and systems that the researchers considered when strengthening the importance of the present study. This chapter also looks at past studies and systems to help improve the project and make sure it is based on useful and proven ideas. Lastly, it offers a synthesis to better understand the research and gain a deeper understanding of the thesis project.

Local

Anino Jr. Comillas and Malabo (2021), in their study "Food Ordering System with Non-Primary Ingredients Customization", developed a mobile-based food ordering system for Shield Diner, a restaurant located in Pueblo de Oro, Cagayan de Oro City. The study investigated how a mobile application could support restaurant ordering by allowing customers to browse the menu, place orders and customize non-primary ingredients according to their preferences. It also aimed to help restaurant staff monitor incoming orders and address issues associated with manual order-taking, particularly during busy periods. The researchers used the Waterfall Model in developing the system and evaluated its usability through a survey involving ten Shield Diner employees using the USE questionnaire, which assessed usefulness, ease of use, ease of learning and user satisfaction. The study reported an overall mean score of 3.74, interpreted as "Agree", indicating generally positive evaluations from the respondents. The researchers concluded that the system supported the restaurant's ordering process and provided functions for menu browsing, ingredient customization and order monitoring (Anino Jr. et al.,2021)

Synthesis

The study by Anino Jr., Comillas and Malabo (2019) investigated the use of a mobile-based food ordering system that allows customers to browse menus, place orders and customize non-primary ingredients according to their preferences. The researchers evaluated the system's usability through a survey of restaurant employees, with results indicating generally positive evaluations in terms of usefulness, ease of use, ease of learning and user satisfaction. This study is relevant to Canteen Express because both systems use digital technology to support food ordering and provide a basis for considering menu browsing and digital order management in the development of a food ordering system for the PalSU Main Canteen. However, the study focuses on a restaurant setting, while Canteen Express is designed for a university canteen and includes functions for advance ordering status updates and on-campus delivery. These functions address the specific ordering and delivery requirements of the PalSU Main Canteen. The available findings from Anino et al. (2021) do not establish whether their system supports these campus-specific functions. Therefore, the relevance of the study lies primarily in its application of mobile technology to food ordering and its evaluation of system usability.

Buenaventura, Ignacio and Laspoña (2021), in their study "Mobile Ordering Application for a Generic Fast Food Restaurant," developed a mobile ordering application that allows customers to browse the menu, place orders in advance and receive an order summary converted into a barcode that cashiers can scan to process orders quickly. The main goal of their study was to reduce waiting time and address the long queues and service delays commonly experienced in fast-food restaurants during peak hours. The system was developed using Agile Methodology, which allowed the researchers to build and improve the application in a flexible and iterative process. The study evaluated the system using a survey conducted among five restaurant owners

and fifty customers, measuring the application based on design, system performance, system integrity and security. The application received an overall mean score of 4.45, interpreted as “Very Good”, indicating positive evaluations from the respondents. The study therefore examined both the implementation of mobile-based ordering and users’ assessments of the application’s acceptability (Buenaventura et al.,2021).

Synthesis

The study by Buenaventura et al. (2021) examined the use of a mobile ordering application that allows customers to browse menus, place orders in advance and present a barcode containing order information for cashier processing. The study is relevant to Canteen Express because both systems use mobile technology to support advanced food ordering and address concerns related to queues and order processing in food service establishments. The findings on application acceptability also provide a reference for evaluating the usability and functionality of Canteen Express. While Buenaventura et al. focused on mobile ordering and barcode-based order processing in a fast-food restaurant, Canteen Express is designed for the PalSU Main Canteen and has additional functions for on-campus delivery and order status updates. These functions are intended to support the ordering and delivery needs of the university community. The study’s reported evaluation does not establish whether its application included these campus-specific functions, so the relevance of the research lies primarily in its approach to mobile ordering, advance order placement and application evaluation.

Lambino and Murcia (2025), in their study "A Path Analysis of Factors that Influence Acceptance of Online Food Ordering Systems Among Davao City Customers," investigated the key factors that affect how customers in Davao City accept and use online food ordering systems. Using the Technology Acceptance Model (TAM) as a theoretical framework, the

researchers examined personal IT innovativeness, social influence, user convenience and system quality in relation to perceived ease of use, perceived usefulness and users' attitudes toward online food ordering applications. The study involved 367 respondents who had used online food ordering platforms such as GrabFood and Foodpanda. The researchers applied path analysis to examine the relationships between the factors and user acceptance. The findings identified system quality and perceived usefulness in understanding consumer acceptance of online food ordering applications.

Synthesis

The study by Lambino and Murcia (2025) is relevant to the study because it examines factors associated with customer acceptance of online food ordering systems in the Philippine context. The study identified system quality and perceived usefulness as important determinants of users' attitudes toward online food ordering applications. These findings are relevant to Canteen Express because they highlight the importance of developing a system with useful functions and reliable system performance. While Lambino and Murcia focused on analyzing factors that influence user acceptance, Canteen Express applied the concept of online food ordering within the specific setting of the PalSU Main Canteen. It provides functions for browsing menu items, placing orders in advance, viewing order status and selecting pickup or on-campus delivery. The findings of Lambino and Murcia may also inform the evaluation of Canteen Express by providing a basis for examining users' perceptions of the system's usefulness and quality. However, the study does not establish whether these factors will produce the same outcomes in a university canteen setting, making user evaluation important for determining how the system is received by its intended users.

Foreign

Mollayev and Atabayeva (2024), in their study "Oguz Canteen App: Revolutionizing Campus Dining," presented a proposed mobile application intended for the canteen of Oguz Han Engineering and Technology University of Turkmenistan. The paper discussed how mobile technology could be applied to campus food services to address concerns such as long queues, manual order processing and payment handling. The proposed application included features for advance meal ordering digital payments, real-time order tracking, menu browsing and user feedback. It also outlined functions for canteen management, including an order dashboard, menu stocks management, sales analytics and access to user feedback. In addition, the paper described proposed technical components and security measures, such as data encryption and two-factor authentication. The study discussed the potential of these features to support campus dining operations and the ordering experience of the students, faculty and staff. However, the article primarily presents the proposed application and its intended functions rather than reporting empirical user testing or measured improvements in waiting time and operational efficiency (Mollayev & Atabeyava, 2024)

Synthesis

The study by Mollayev and Atabayeva (2024) is relevant to Canteen Express because both projects focus on applying mobile technology to university canteen services. The Oguz Canteen App proposed features such as advance meal ordering, menu browsing, order tracking and canteen management functions to address issues associated with campus dining. These features are relevant to Canteen Express, which also provides digital ordering functions for the PalSU Main Canteen. However, the two projects differ in their intended context and described functionality. Canteen Express is designed to support not only advance ordering but also

on-campus delivery, order status updates and separate functions for customers, canteen staff, delivery personnel and the administrator. The Oguz Canteen App paper discusses proposed canteen management features, while Canteen Express incorporates operational functions for processing orders and handling delivery activities. The article does not report empirical testing or measured outcomes that would establish the effectiveness of its proposed features. Therefore, it provides a reference for considering mobile ordering and canteen management in a campus setting, while the implementation and user evaluation of Canteen Express address the specific requirements of the PalSU Main Canteen.

Thakare and Dhruvkumar (2025), in their study "FoodGet: College Campus Food Delivery System", presented the development and evaluation of a web-based food ordering application designed for college campuses. The system was developed to address issues associated with campus food ordering, including long queues, limited food availability during peak hours and difficulties in managing orders manually. Its features include menu browsing, online order placement, real-time menu updates, order tracking, payment gateway integration and vendor management. The researchers used React.js for the frontend, PHP with Laravel for the backend and MySQL for database management. The development process included requirements gathering, system design, implementation, testing and evaluation. According to the article, the researchers conducted a two-week pilot feedback through surveys and interviews. The authors reported improvements in order processing time, order accuracy and user satisfaction. They also reported that the average order processing time decreased from 15 minutes under manual ordering to 5 minutes using the system. These findings were presented by the authors as evidence of the potential use of digital ordering systems in campus services (Dinesh & Thackeray, 2025).

Synthesis

The study “FoodGet: College Campus Food Delivery System” by Dinesh and Thakare (2025) is in relevance to the study because both studies apply digital technology to food ordering in a campus setting. FoodGet addresses issues associated with campus food services, including long queues and manual order processing, through features such as menu browsing, online order placement, order tracking and vendor management. These features are relevant to Canteen Express, which also provides digital ordering functions for the PalSU Main Canteen. However, the two systems differ in their described functionality and intended operational context. Canteen Express includes on-campus delivery, delivery personnel functions, GPS route guidance, in-chat communication and loyalty rewards. It also provides separate access and functions for Student/Walk-in Customers, Faculty/Staff Customers, Canteen Staff, Delivery Personnel and the Administrator. Although FoodGet includes order tracking and vendor management, the article does not establish whether it supports all of these specific functions. The study by Dinesh and Thakare (2025) provides a reference for the application of digital ordering technology in camps food services, while Canteen Express addresses the ordering, order-processing and on-campus delivery requirements identified for the PalSU Main Canteen. The functional and usability of Canteen Express was assessed through its own testing and user evaluation.

Manaf, Awang, Ahmad, Arifin and Wan Ismail (2025), in their study "Development of an Integrated Mobile Application for Food Ordering and Delivery," developed QuickBites, a mobile application designed for students, lectures and staff at Universiti Teknologi MARA (UiTM) Machang campus in Malaysia. The study addressed challenges associated with campus food services, including long queues and the inconvenience of physically visiting food stalls during limited breaks between classes. The application was developed using Android Studio and

Firebase for real-time data synchronization, following the Rapid Application Development (RAD) approach. Quickbites provides features for menu browsing, order placements, real-time order tracking and coordination with campus food vendors. The application also supports different roles for managing customer orders and delivery activities. The study aimed to evaluate the application's functionality and usability. The authors described QuickBites as a campus food ordering and delivery platform and identified cash on delivery as its payment method in the current version (Manaf et al., 2025).

Synthesis

The study by Manaf et al. (2025) is relevant to Canteen Express because both systems apply mobile technology to food ordering and delivery within a university campus. QuickBites was developed to address challenges such as long queues and the inconvenience of visiting food stalls during limited breaks between classes. It includes functions for menu browsing, order placement, order tracking and coordination of food preparation and delivery. These functions are relevant to Canteen Express, which also supports digital ordering and on-campus delivery at the PalSU Main Canteen. However, the systems differ in their described features and intended context. Canteen Express includes loyalty rewards, nutritional information, dietary preference filtering, vendor-managed promotions and functions for managing delivery activities. The QuickBites study describes a multi-role ordering and delivery application, but the available information does not establish whether it includes these same features. The study by Manaf et al. (2025) therefore provides a reference for applying mobile technology to campus food ordering and delivery, while Canteen Express addresses the specific ordering, administrative and delivery requirements identified for the PalSU Main Canteen. The usability and effectiveness of Canteen Express was assessed through its own testing and user evaluation.

Similarities and Differences of the Researches in the Review of Related Literature or Systems

FEATURES OF THE STUDY/SYSTEM	FOOD ORDERING SYSTEM WITH NON-PRIMARY INGREDIENTS CUSTOMIZATION - LOCAL LITERATURE 1 (Anino Jr. et al., 2021)	MOBILE ORDERING APPLICATION FOR A GENERIC FAST FOOD RESTAURANT - LOCAL LITERATURE 2 (Buenaventura et al., 2021)	A PATH ANALYSIS OF FACTORS THAT INFLUENCE ACCEPTANCE OF ONLINE FOOD ORDERING SYSTEMS AMONG DAVAO CITY CUSTOMERS - LOCAL LITERATURE 3 (Lambino & Murcia, 2025)	CANTEEN EXPRESS: A WEB/MOBILE-BASED PRE-ORDER AND DELIVERY SYSTEM FOR STUDENTS, STAFF AND FACULTY AT PALAWAN STATE UNIVERSITY MAIN CANTEEN
Integration of Advanced Solutions		✔		✔

Study/System Approach	Mobile-based food ordering system with ingredient customization and interconnected subsystems for administration, kitchen operations and customer ordering through a centralized system.	Mobile ordering application with barcode-based order processing and an administrative panel for menu management.	Survey-based study using path analysis to examine factors influencing Filipino customers' acceptance of online food ordering systems.	Web/Mobile pre-order and delivery system with four user roles, cash payment, real-time order tracking, loyalty rewards, nutritional information and canteen staff and admin dashboards.

Technologies or Languages Used	Android-based mobile application developed using the Waterfall Model. The specific programming languages and technologies should be listed only if confirmed in the article.	Not specified; focused on Agile methodology and system evaluation.	Quantitative research methods using surveys and path analysis to examine factors associated with acceptance of online food ordering systems.	Python 3.12 and Django 5.1.5 for backend development. Supabase PostgreSQL and SQLite for data storage. Tailwind CSS, Alpine.js and JavaScript libraries for the frontend. Leaflet.js, OpenStreetMap, OSRM and Valhalla for mapping and routing. For hosting, Railway is used, along with Node.js and Git for development and version control.

Focus on Food Ordering and Campus Context	Focused on a restaurant-based food ordering system, not campus-specific.	Focused on fast-food restaurant ordering but not campus-specific	Examines factors influencing customers' acceptance of online food ordering systems in Davao City. No developed system.	Built specifically for Palawan State University Main Canteen. Supports advance food ordering, order processing, pickup and on-campus delivery.

Workflow	Customer uses mobile device to browse menu, places and customizes order, kitchen receives and processes order, staff monitors order status, food is prepared and served to customer.	Customer uses mobile application to browse menu, places order, receives barcode, cashier scans barcode to process order	No system workflow. The study collects survey responses and applies path analysis to examine relationships among factors associated with online food ordering acceptance.	Student/Walk-In customers place orders through the mobile kiosk and present the generated order slip at the counter and make the payment through cash. Faculty/Staff customers log-in to browse the menu, place orders, select pickup or on-campus delivery and view order status. Canteen Staff process orders and update preparation statuses, while Delivery Personnel can view and accept available

				delivery orders, update delivery status and complete deliveries. The administrator oversees the system.
Addressing Security Concerns	Basic system evaluation through usability testing (USE questionnaire); no detailed implementation of security features discussed.	The study evaluates application acceptability, this includes security-related criteria. Specific security mechanisms should be listed only if described in the article.	No system developed, security is not applicable.	The system implemented these security features: Institutional RBAC (@psu.palawan.edu.ph + OTP + Admin PIN), 0.8 GPS geofencing, HTTPS/HSTS, CSRF protection, Django ORM parameter safety and activity logging.

Unique Features of the Study/System	Ability to customize non-primary ingredients in food orders, along with a centralized system that connects customer, kitchen and admin interfaces for efficient order management.	Barcode-based order processing system that allows cashiers to quickly scan and fulfill pre-orders at the counter.	Examines factors associated with customer acceptance of online food ordering systems, identifying system quality and perceived usefulness as important determinants of users' attitudes.	Full campus dining solution with on-campus delivery with GPS guidance, loyalty rewards, nutritional information and delivery personnel incentive system.

Table 2.1: Matrix of Similarities and Differences of the Research (Local)

FEATURES OF THE STUDY/SYSTEM	OGUZ CANTEEN APP: REVOLUTIONIZING CAMPUS DINING - FOREIGN LITERATURE 1 (Mollayev & Atabayeva, 2024)	FOODGET: COLLEGE CAMPUS FOOD DELIVERY SYSTEM - FOREIGN LITERATURE 2 (Dinesh & Thakare, 2025)	DEVELOPMENT OF AN INTEGRATED MOBILE APPLICATION FOR FOOD ORDERING AND DELIVERY - FOREIGN LITERATURE 3 (Manaf et al., 2025)	CANTEEN EXPRESS: A WEB/MOBILE-BASED PRE-ORDER AND DELIVERY SYSTEM FOR STUDENTS, STAFF AND FACULTY AT PALAWAN STATE UNIVERSITY

Integration of Advanced Solutions	✔	✔	✔	✔
Study/System Approach	Proposed mobile application for university canteen services, focusing on advance meal ordering and addressing queues on campus.	Full-stack web-based campus food ordering system with real-time menu updates, secure payment integration, order tracking and vendor management features.	Campus-based mobile food ordering and delivery application with multiple user roles, order tracking and Firebase integration.	Pre-order and delivery system with four user roles, real-time order tracking, loyalty rewards, nutritional information and canteen staff/admin dashboard.

Technologies or Languages Used	Specific programming languages and technologies are not fully specified. The study focuses on the proposed digitalization of university canteen services.	React.js (frontend), PHP with Laravel (backend), MySQL (database), JavaScript frameworks, Stripe/PayPalPay for payment integration.	Android Studio, Firebase, Rapid Application Development (RAD).	Python and Django for backend development. Supabase PostgreSQL for production and SQLite for local development. Tailwind CSS, Alpine.js, Javascript - Frontend. Leaflet.js, OpenStreetMap, OSRM and Valhalla - Mapping and Routing. Railway - Hosting. Git - Version Control.

Focus on Food Ordering and Campus Context	Proposed mobile application for university canteen services, focusing on advance meal ordering, digital payments, order tracking and customer feedback.	Full-stack web-based food ordering and delivery system designed for college campuses, serving students and faculty and supporting campus food vendors through menu and order management.	Mobile food ordering and delivery application developed for students, lectures and staff at UiTM Machang campus, with real-time order tracking, campus food vendor integration and cash-on-delivery payment.	Web/mobile-based pre-order and delivery system developed specifically for the PalSU Main Canteen, supporting mobile kiosk ordering for student/walk-in customers and account-based ordering, pickup and on-campus delivery for Faculty/Staff customers.

Workflow	Users browse food options, pre-order meals, pay through digital payments, track orders and submit feedback. The paper of this study does not provide enough detail to confirm a complete operational workflow for canteen staff.	User logs in, browses menu, places order, make secure payment, vendor processes order, system updates order status in real time, user tracks order until completion	Customers browse menus and place food orders, while the system supports order tracking and delivery coordination with campus food vendors. Payment is made through cash on delivery in the current version.	Students/Walk-in customers use the mobile kiosk to browse items and place orders, then receive an order slip to present at the counter for scanning and cash payment. Faculty/Staff customers log in, browse the menu, place orders and select pickup or on-campus delivery. Canteen staff process orders, update preparation statuses and handle counter transactions. Delivery personnel can view and accept available

				delivery orders, update delivery status and complete deliveries.
Addressing Security Concerns	The paper discusses a proposed campus canteen application, but specific security measures are not clearly detailed in the available description.	The paper describes user authentication, role-based access and payment security measures, including Stripe/PayPal integration, SSL encryption and two-factor authentication (2FA) for vendors.	Firebase authentication used for user login; basic role-based access implemented	The system implemented these security features: Institutional RBAC (@psu.palawan.edu.ph + OTP + Admin Password), 0.8 GPS geofencing, HTTPS/HSTS, CSRF protection, Django ORM parameter safety and activity logging.

Unique Features of the Study/System	Proposes advance meal ordering, digital payments, order tracking, customer feedback and canteen management features for a university setting.	Describes a campus food ordering system with menu management, order tracking, vendor management and online payment integration.	Provides mobile food ordering, order tracking and campus vendor integration, with cash on delivery in its current version.	Combines mobile kiosk ordering, account-based pre-ordering, pickup, on campus delivery, GPS route guidance, loyalty rewards (voucher, discounts), nutritional information and delivery personnel incentives.

Table 2.2: Matrix of Similarities and Differences of the Research (Foreign)

CHAPTER 3

TECHNICAL BACKGROUND

This chapter presents the technical background of Canteen Express. It describes the physical architecture, organizational chart, system flowcharts, hardware and software specifications and the technical requirements used in the development and implementation of the system.

Physical Architecture

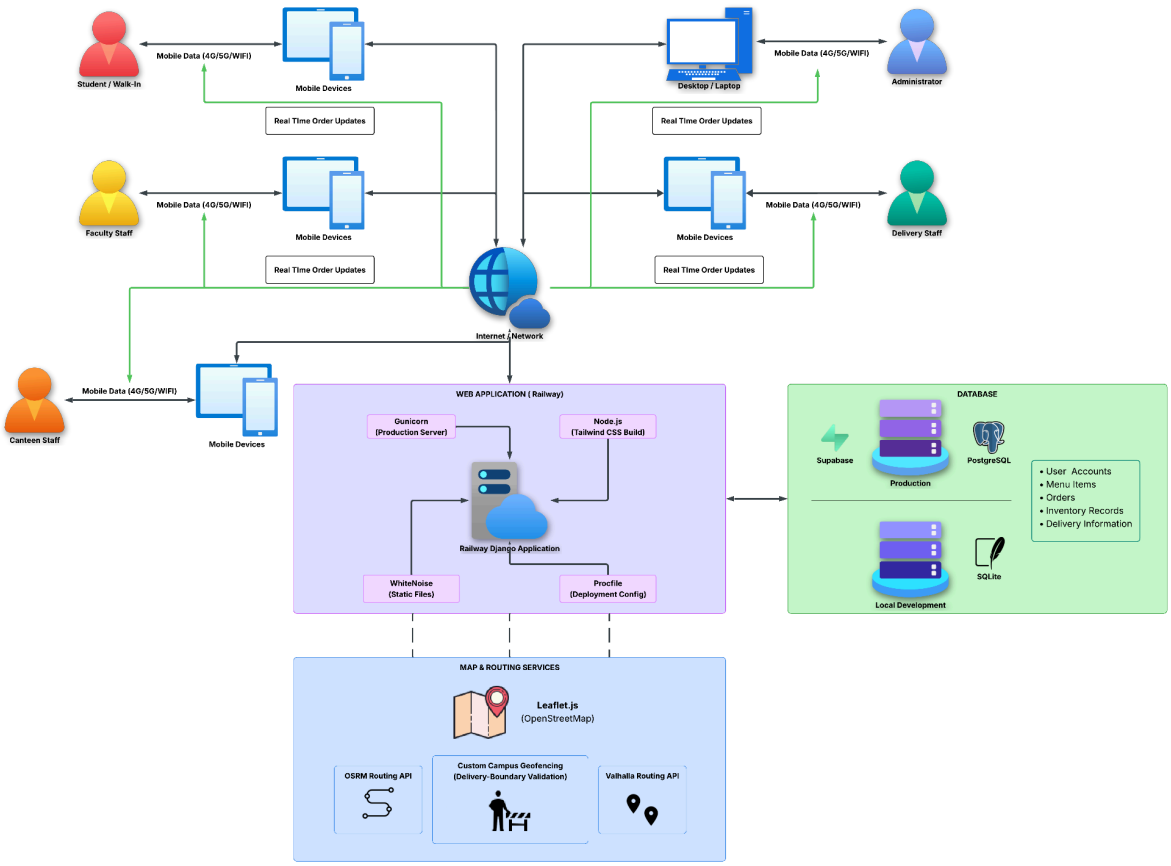


Figure 3.1: The Physical Architecture of Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff and Faculty at Palawan State University Main Canteen.

The web application is developed using Django 5.1.5 and Python 3.12 and is deployed on Railway. Gunicorn serves the Django application in the production environment, while WhiteNoise handles static file serving. The frontend uses HTML, CSS, Tailwind CSS, JavaScript and Alpine.js to provide the browser-based interfaces and interactions for the system's different roles and access modes. Node.js supports the frontend asset-processing and build workflow.

Supabase PostgreSQL serves as the production database, storing and retrieving system information, including user accounts, menu items, orders, transaction and payment records, menu information, delivery activities, loyalty records and other data associated with the implemented features. SQLite is used as the local development database when configured. This separates the local development environment from the production environment used by the deployed application.

For delivery-related functions, Leaflet.js provides the interactive map interface and OpenStreetMap supplies the map data. The system integrates the OSRM Routing API for route information, device GPS for location data and custom geofence validation to determine whether a delivery destination falls within the designated service area. The delivery boundary is configured as a 0.8-kilometer radius from the specified campus-center coordinates (9.77778, 118.73333). These components support delivery navigation and location monitoring, while the accuracy and availability of location information remain dependent on device GPS, permissions, network connectivity and external mapping services.

The application also supports order-status updates across the ordering, payment, food preparation and delivery workflows. Customer requests are processed by the hosted application and the relevant records are stored in the production database. These functions allow authorized

users to submit orders, confirm payments, manage preparation statuses, process eligible deliveries and access the information associated with their assigned roles. The delivery workflow integrates location information, route data and delivery-area validation to support eligible on-campus deliveries.

Organizational Chart

The organizational chart presents the project structure of Canteen Express and illustrates the relationships, responsibilities and reporting arrangement among the project stakeholders and development team.

Figure 3.2: Organization Chart of Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff and Faculty at Palawan State University Main Canteen

The organizational chart identifies the Client, Adviser and Lead/Web Developer involved in the project. The Client represents the intended recipient or stakeholder of the system and provides requirements related to the canteen's operational needs. The Adviser provides guidance throughout the proposal, development and documentation stages, including reviewing project progress and recommending improvements. The Lead/Web Developer is responsible for developing and configuring the application, implementing its backend logic and user interfaces and integrating the system components according to the established requirements.

The organizational chart describes the project team and its responsibilities. It is distinct from the system's user-role structure, which determines the functions available to Student/Walk-In customers, Faculty/Staff customers, Canteen Staff, Delivery Personnel and the Administrator.

System Flowchart

The system flowcharts describe the workflows performed by the different user roles in Canteen Express. These workflows cover authentication, order placement, cash payment, food preparation, rider assignment, delivery and administrative functions. Each flowchart represents the activities and status changes associated with the corresponding user interface.

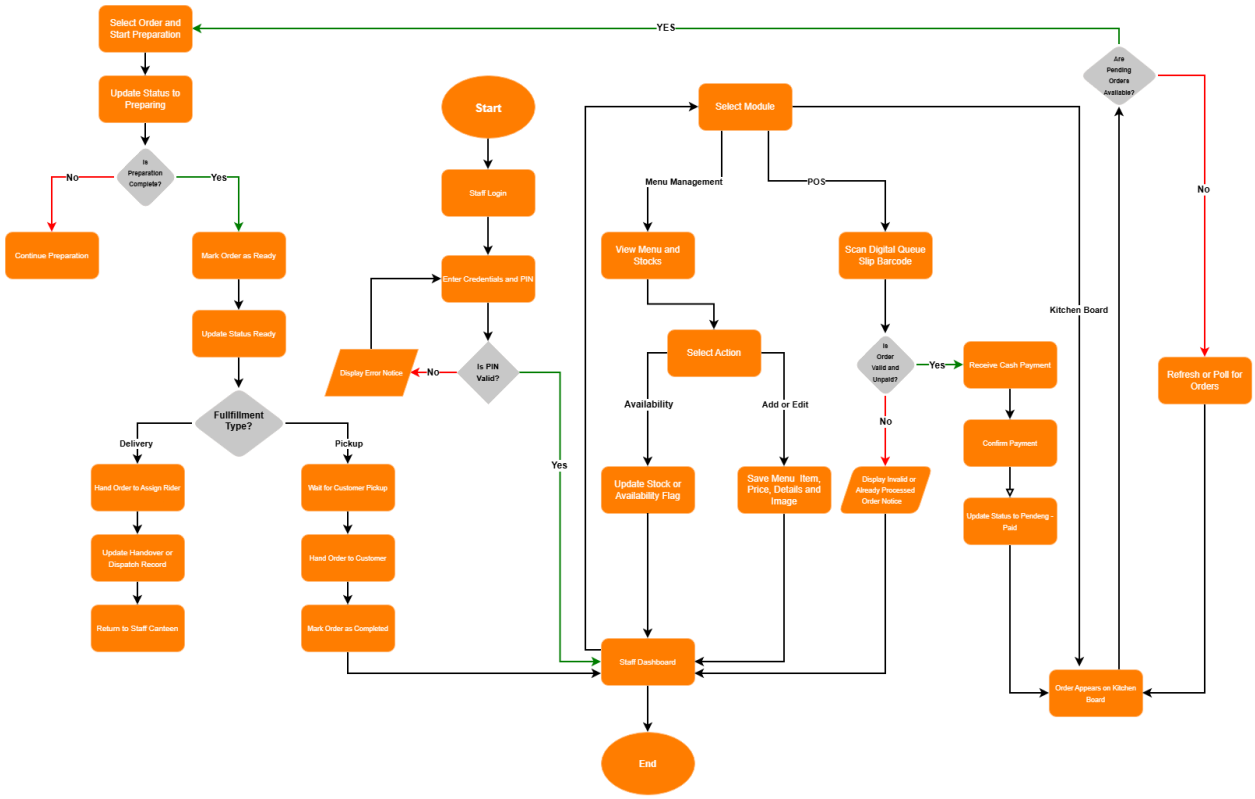


Figure 3.3: Canteen Staff/Admin Flowchart

The flowchart illustrates the workflow performed by Canteen Staff through the Counter POS and Kitchen Board. Staff access the operational functions using their assigned credentials. When a Student/Walk-In customer presents a digital queue slip, staff use the Counter POS or the barcode-processing function to retrieve and verify the associated order. After checking the

transaction, staff collect cash or verify the applicable counter-payment arrangement and confirm payment through the system.

Upon confirmation, the kiosk order changes from Unpaid to Pending and the system makes it available for kitchen processing. The Kitchen Board displays the order so staff can monitor and update its preparation status through Pending, Preparing, Ready and Completed. Once preparation is finished, the order is released for counter pickup and the transaction is completed in accordance with the implemented workflow.

Canteen Staff can also manage applicable operational tasks, including menu availability and stock updates where authorized. When an order is cancelled through the supported staff workflow, the system voids the associated payment records, restores applicable menu stock and returns redeemed loyalty points where relevant. Administrative functions remain restricted to the Administrator, even when both roles use the same physical device.

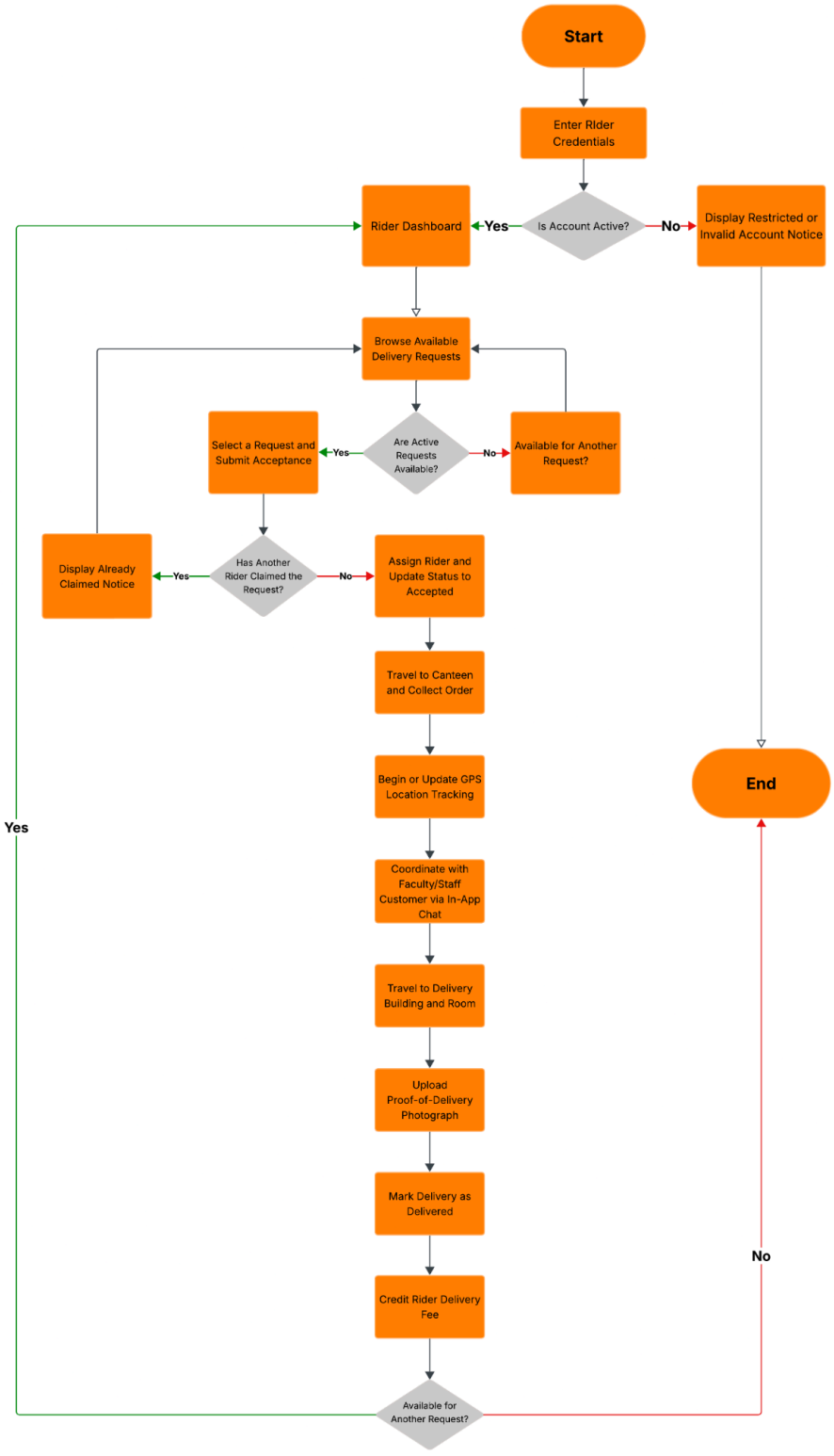


Figure 3.4: Delivery Personnel Flowchart

The flowchart illustrates the workflow followed by authorized Delivery Personnel in the Canteen Express system. The process begins when the rider enters their credentials on the delivery login page. The system validates the rider's account status before granting access to the Rider Dashboard. If the account is inactive or invalid, the system displays a restricted or invalid account notice and terminates the process.

Once access is granted, the rider can browse available delivery requests. If an active request is available, the rider can select a request and submit an acceptance request. The system checks whether another rider has already claimed the selected request. If the request has already been claimed, the system displays an appropriate notice and allows the rider to return to browsing available requests. Otherwise, the system assigns the request to the rider and updates its status to Accepted. If no active request is available, the rider can return to browsing for another request.

After accepting a delivery request, the rider travels to the canteen to collect the order. During the delivery process, the rider's device provides location information for GPS tracking, while the configured routing component assists with navigation. The rider can coordinate with the Faculty/Staff customer through the system's DeliveryMessage feature. The rider then proceeds to the customer's designated delivery building and room. For cash-on-delivery transactions, the rider collects the applicable payment from the customer upon handoff.

After delivering the order, the rider uploads the required proof-of-delivery photograph and marks the delivery as Delivered. The system records the completion timestamp, updates the relevant delivery and order information, credits the applicable rider delivery fee and credits

eligible loyalty points to the customer's account. The customer can also provide feedback or a rating through the implemented system features.

After completing the delivery, the rider indicates whether they are available to accept another request. If available, the rider returns to the Rider Dashboard to browse other delivery requests. Otherwise, the process ends.

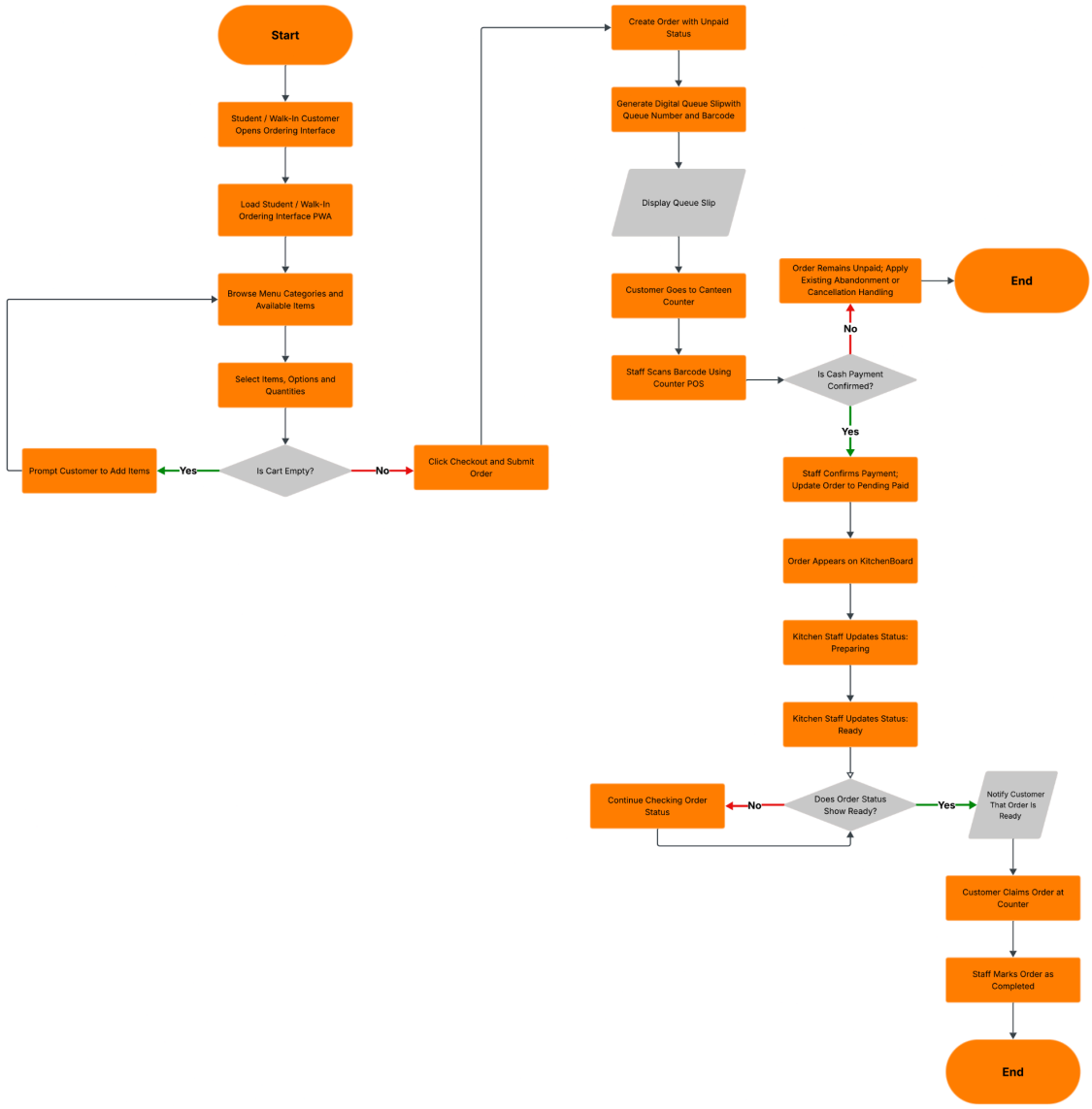


Figure 3.5: Student/Walk-In/s Customer Flowchart

The flowchart illustrates the ordering and counter-pickup process for Student/Walk-In customers using the Canteen Express system. The process begins when the customer opens the Student/Walk-In ordering interface through the Progressive Web Application (PWA). The customer browses the available menu categories and food items, selects the desired items, options and quantities and proceeds to checkout once the cart contains at least one item. If the cart is empty, the system prompts the customer to add items before continuing.

After the customer submits the order, the system creates an order with an Unpaid status and generates a digital queue slip containing the queue number and barcode. The customer presents the queue slip at the canteen counter, where staff scan the barcode through the Counter POS to retrieve the associated order and verify the transaction.

The order remains unpaid until staff confirm receipt of the cash payment. If payment is not confirmed, the order remains unpaid and follows the applicable abandonment or cancellation handling process. Once payment is confirmed, the staff update the order status to Pending (Paid), making the order available on the Kitchen Board for preparation.

Kitchen staff update the order status to Preparing and then Ready when preparation is complete. The system checks the order status and continues monitoring it until it reaches Ready. Once the order is ready, the customer is notified and proceeds to the counter to claim the order. After the customer receives the order, staff mark it as Completed, concluding the ordering process.

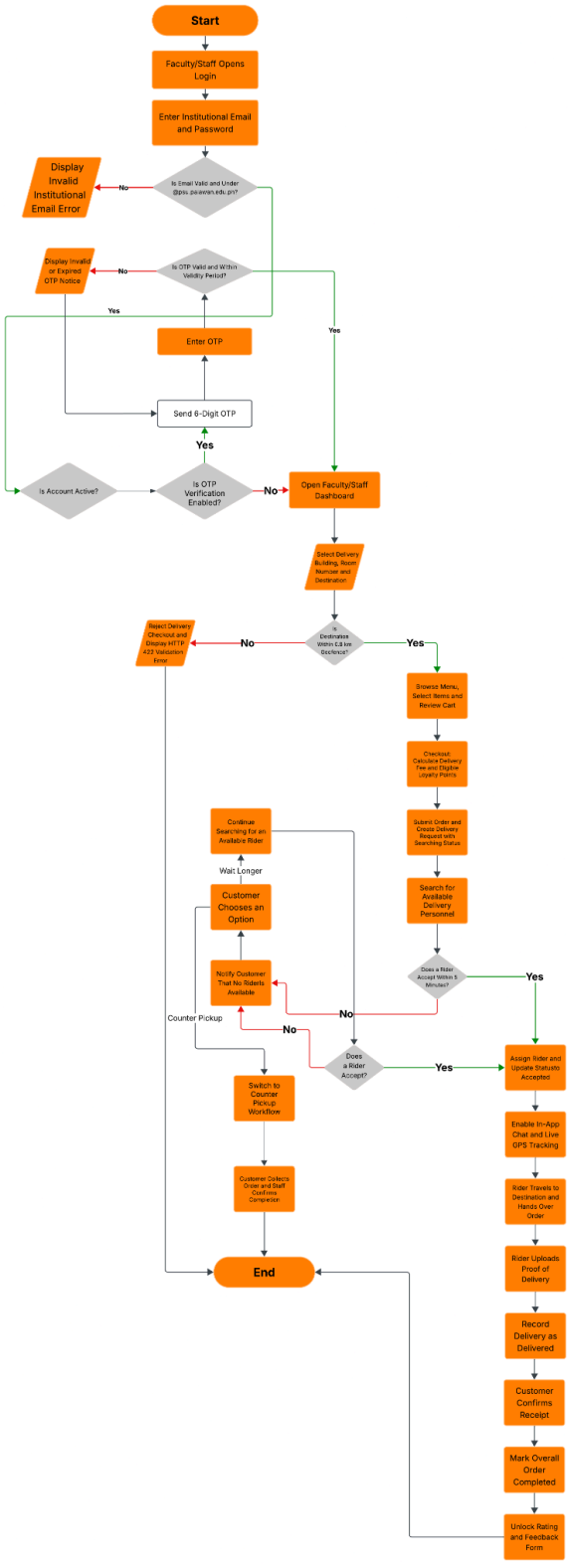


Figure 3.6: Faculty/Staff Customer Flowchart

The flowchart illustrates the account-based ordering and delivery workflow for Faculty/Staff customers using the Canteen Express system. The process begins when the customer opens the Faculty/Staff login page and enters their institutional email address and password. The system checks whether the email is valid and uses the required @psu.palawan.edu.ph domain. Invalid institutional email credentials result in an error notice. The system also checks the account status and applies the implemented six-digit One-Time Password (OTP) verification process, including validation of the OTP and its validity period, before granting access to the Faculty/Staff Dashboard.

After successful authentication, the customer selects a delivery building, room number and destination. The system validates whether the delivery destination is within the configured 0.8-kilometer geofence centered at PSU Tiniguiban Heights, approximately 9.77778 latitude and 118.73333 longitude. If the destination is outside the permitted delivery area or fails the required validation, the system rejects the delivery checkout and returns the applicable validation error. Otherwise, the customer can browse available menu items, select products and quantities and review the cart before proceeding to checkout.

During checkout, the system calculates the order subtotal and applicable delivery fee based on the implemented pricing rules. Eligible customers can also redeem available loyalty points according to the system's checkout rules. After the customer submits the order, the system creates the order and a delivery request with a Searching status while it searches for available Delivery Personnel.

The system provides a five-minute acceptance window for eligible riders. If a rider accepts the request within the allotted period, the system assigns the rider and updates the request

status to Accepted. The system then enables the supported in-app chat and live GPS tracking features so the rider and customer can coordinate during delivery. The rider travels to the destination and hands over the order to the customer. The rider uploads proof of delivery and the delivery is recorded as Delivered. The customer then confirms receipt, after which the overall order is marked as Completed and the rating and feedback form becomes available.

If no rider accepts the request within the acceptance window, the system notifies the customer that no rider is available. The customer can choose an available option, such as switching to the counter-pickup workflow or continuing to wait for another rider, depending on the options presented by the application. If counter pickup is selected, the order follows the applicable counter-pickup process, where the customer collects the order and staff confirm its completion.

Upon successful completion of the applicable order workflow, the system records the relevant completion information and processes eligible loyalty-point credits according to the implemented rules. This workflow provides authenticated Faculty/Staff customers with access to pre-ordering and eligible on-campus delivery while enforcing account and delivery-location restrictions.

Hardware and Software Specification

List of Hardware Specifications for the Development of the System

The following table below shows the hardware specification requirements for the development of the application system.

Table 3.1: List of Hardware Specifications for the Development of the System

Name	Specification	Uses
Developer Workstation (Laptop/Desktop Computer)	Processor: Intel Core i5 / AMD Ryzen 5 or later **RAM**: 8 GB or later. **Storage**: 256 GB SSD or higher **Graphics**: Integrated graphics processor **Operating System**: Windows 11 64-bit **Connectivity**: Wi-Fi or Ethernet	Used by the developers to write code, debug, manage application files and databases.

Mobile Testing Devices (Android Smartphone)	**Operating System:** Android 10 or later **RAM:** 3GB minimum; 4 GB recommended. **Storage:** 16 GB internal storage or higher **Processor:** Quad-core Processor or higher **Connectivity:** Wi-Fi or mobile data preferably 4G or 5G. **Additional Requirements:** GPS/location services, camera access where required and a compatible, updated web browser.	Used by developers to test mobile responsiveness, PWA functionality, menu browsing, order placement, digital queue slips and eligible GPS-based delivery features. Camera access is also used to test proof-of-delivery photo uploads.

Mobile Testing Device **(iOS Smartphone)**	**Operating System:** iOS 12 or later **Storage:** 32 GB internal storage or higher. **Processor:** Apple A12 Bionic or higher. **Connectivity:** Wi-Fi or mobile data preferably 4G or 5G. **Additional Requirements:** GPS/location service, camera access where required and a compatible, updated web browser.	Used by developers to test application accessibility, responsive layouts, ordering functions, order-status viewing eligible delivery-tracking features and proof-of-delivery photo uploads on compatible Apple devices.

The table presents the hardware resources relevant to the development and testing of Canteen Express. The developer workstation is used for programming, running the local development environment, managing the source code and preparing documentation. Smartphones and other devices compatible to support testing of the system's browser-based interfaces across different screen sizes. A compatible counter device and barcode scanner or supported camera are used to test the order retrieval and payment-confirmation workflow.

Internet connectivity is required to access the deployed application and test the functions that depend on network or location services.

The specifications describe the development and testing requirements and should not be interpreted as mandatory minimum requirements for every end user.

List of Software Specifications for the Development of the System

The following information below shows the software specifications required for the development of the system.

Table 3.2: List of Software Specifications for the Development of the System

Name	Specifications	Uses
Backend	Django 5.1.5; Python 3.12	Handles backend processing, authentication, URL routing and database interaction
Frontend / UI	HTML5, CSS3, Tailwind CSS v3, Alpine.js, JavaScript (ES6+)	Used for responsive user interfaces and client-side interaction
Charts	Chart.js	Used to handle data visualization and administrative analytics.

Barcode	JsBarcode	Used to generate digital queue-slip barcodes.
Icons and Fonts	FontAwesome; Google Fonts	Used in interface icons and typography
Routing	OSRM Routing API; Valhalla Routing API	Used for route computation for eligible on-campus delivery guidance
Geofencing	Custom campus geofencing	Used for validation of eligible delivery destinations within the defined campus boundary.
Production Server	Gunicorn	Used to serve the Django application in production.
Static Files	WhiteNoise	Used to serve the static assets in the deployed Django application.
Deployment	Railway (PaaS); Procfile	Used to host and start the deployed application.
Build Tool	Node.js	Supports the Tailwind CSS build process

Version Control	Git / Github	Used for source code version control and change management.
Development Environment	Visual Studio Code (VS Code)	It is used as a development environment for writing, editing, debugging and managing the source of Canteen Express.

The table above presents the software requirements that collectively supported the testing and development of the system. The core of the backend used Django 5.1.5 and Python 3.12 for backend processing, HTML5, CSS3, Tailwind CSS v3, JavaScript ES6+ and Alpine.js were used for the frontend and user interfaces. Chart.js was used for data visualization while JsBarcode was used for barcode generation. FontAwesome and Google Fonts supported interface presentation. Supabase PostgreSQL served as the production database while SQLite was used for local development. Leaflet.js and OpenStreetMap provided map display while OSRM and Valhalla routing APIs supported route computation. Geofencing validates delivery destinations; Node.js supports the Tailwind CSS build process and the deployed Django application was served through Gunicorn and WhiteNoise on Railway using the projects Procfile.

List of Hardware Specifications for the Implementation of the System

The following table below shows the hardware specification requirements for the implementation of the application system.

Table 3.3: List of Hardware Specifications for the Implementation of the System

Name	Specification	Uses
Laptop / Desktop Computer	**Processor:** Intel Core i5 / AMD Ryzen 5 or higher **RAM:** 8 GB or higher **Storage:** 256 GB SSD or higher **Graphics:** Integrated Graphics Processor or equivalent Operating System: Windows 11, 64-bit	Used by Canteen Staff to access Counter POS and Kitchen Board. The Administrator uses a separate login on a compatible device to access administrative functions.
Smartphone - Android Device	**Operating System:** Android 10 or later **RAM:** 3 GB minimum ; 4GB recommended. **Storage:** 16 GB internal storage or higher with sufficient available space. **Processor:** Quad-Core Processor or equivalent **Connectivity:** Wi-Fi or mobile data preferably 4G or 5G **Additional Requirements:** GPS / location services for eligible delivery	Use by Student/Walk-In customers to check food availability, place orders and generate digital queue slips. Faculty/Staff use smartphones to access accounts, OTP verification, pre-order, view order status and track eligible deliveries.

	functions; camera access for delivery personnel who upload proof-of-delivery-photo; compatible and updated web browser.	
Smartphone - iOS Device	**Operating System:** iOS 12 or higher **Storage:** 32 GB internal storage or higher with sufficient available space. Processor: A12 Bionic or higher Connectivity: Wi-Fi or mobile data preferably 4G or 5G Additional Requirements: GPS/location services for eligible delivery functions; camera access for proof-of-delivery photo uploads; Safari or another compatible, updated web browser.	Provide an alternative device for access to Canteen Express. Student/Walk-In customers can check menu availability, place orders and display digital queue slips. Faculty/Staff can access their accounts, orders and monitor order status and eligible deliveries. Delivery Personnel can access delivery requests, GPS-assisted route guidance, customer communication, order-status updates and proof-of-delivery functions provided the required device permissions are enabled.

Network Equipment	Wi-Fi router or mobile internet	Provides connectivity for devices that access the deployed web application.

The table presents the hardware requirements for implementing Canteen Express. Canteen Staff use a compatible laptop or desktop computer to access the Counter POS and Kitchen Board, while the Administrator uses a separate login to access authorized administrative functions. Student/Walk-In customers use compatible smartphones to browse the menu, place orders and display digital queue slips. Faculty/Staff customers use smartphones to authenticate, place pre-orders, view order statuses and access eligible delivery features. Delivery Personnel use compatible smartphones to view delivery requests, communicate with customers, use supported GPS features and upload proof-of-delivery photographs. Internet connectivity through Wi-Fi or mobile data is required to access the deployed application.

List of Software Specifications for the Implementation of the System

The following table below shows the software specification requirements for the implementation of the application system.

Table 3.4: List of Software Specifications for the Implementation of the System

Software	Specification	Uses
Application Backend	Django 5.1.5; Python 3.12	To run the Canteen Express application and handle its core system processes.

Production	Gunicorn	To serve the Django application in the production environment.
Static File Handling	WhiteNoise	Serves static assets in the deployed Django application.
Deployment Platform	Railway (PaaS); Procfile	Host and start the deployed application.
Production Database	Supabase PostgreSQL	Store persistent production data.
Frontend	HTML5, CSS3, JavaScript (ES6+), Tailwind CSS v3, Alpine.js	To provide web/mobile-based user interfaces.
Mapping	Leaflet.js; OpenStreetMap	Provides interactive map displays.
Routing	OSRM Routing API; Valhalla Routing API	Supports route computation for eligible on-campus delivery guidance.
Geofencing	Custom campus geofencing	Validates eligible delivery destinations within the defined campus boundary.

Analytics	Chart.js	Displays system analytics and charts.
Barcode	JsBarcode	Generates queue-slip barcodes.
Version Control	Git / Github	Source code version control and change management.
Development Environment	Visual Studio Code (VS Code)	Provides an environment for writing, editing, debugging and managing the source code of Canteen Express.

The table above presents the software technologies used to deploy and operate the Canteen Express system. Django 5.1.5 and Python 3.12 support backend processing and system requests, while Railway hosts the deployed application. Gunicorn serves the Django application in the production environment and WhiteNoise serves its static assets.

Supabase PostgreSQL stores persistent production data, including user accounts, menu information, orders, menu records and delivery information. The system uses HTML5, CSS3, Tailwind CSS v3, JavaScript (ES6+) and Alpine.js to provide its web/mobile-based interfaces. Leaflet.js and OpenStreetMap support interactive maps, while OSRM and Valhalla routing APIs support route computation where implemented. Custom campus geofencing validates eligible delivery destinations against the defined service boundary. Chart.js provides analytics and charts and JsBarcode generates digital queue-slip barcodes.

These technologies work together to support the deployed Canteen Express application. User devices connect through the Internet to the application hosted on Railway, which processes requests and communicates with Supabase PostgreSQL for persistent data storage. This arrangement separates client devices, network connectivity, application hosting and database services within the system architecture.

CHAPTER 4

METHODOLOGY

This chapter presents the methodology and procedures used to gather requirements, develop, test and evaluate Canteen Express. It discusses the Software Development Life Cycle (SDLC) as the overall framework for system development and Agile as the development approach. This chapter also presents the requirements analysis, use case diagram, user stories, Software Requirements Specification (SRS), conceptual framework, data flow diagram, entity relationship diagram, network topology, work plan and cost-benefit analysis while the Agile approach supported incremental development and refinement of system functional.

The researchers used the SDLC to organize the development process from identifying system requirements to designing, implementing, testing and evaluating Canteen Express. Agile was selected as the development methodology because it supports incremental development, regular testing and adjustments based on feedback. This approach allowed the researchers to develop the system in stages and refine its functions throughout the development process.

The development process involved four main iterations. The first iteration focused on requirements gathering and initial system design. The researchers identified the needs of the PalSU Main Canteen and its intended users, including customer access, ordering procedures, payment processes and delivery requirements. These requirements guided the initial system design.

The second iteration focused on developing the core ordering functions. The researchers developed menu browsing and ordering features for Student/Walk-In customers and for

Faculty/Staff customers through account-based access. These functions were tested by the researchers and client to identify issues in the ordering process.

The third iteration focused on order processing and delivery functions. The researchers developed and tested the Kitchen Board, Counter POS, order status updates and delivery functions, including the ability of Delivery Personnel to view and accept available delivery orders and update delivery statuses. Issues identified during testing were addressed through system revisions.

The fourth iteration focused on system integration, testing and refinement. The researcher tested the integrated system and gathered user feedback to assess its functionality and usability. The results helped identify areas requiring improvement and appropriate revisions were made as part of the final refinement process.

Through these iterations the researchers developed and refined Canteen Express according to the identified requirements and testing results. The use of Agile supported incremental development, while SDLC provided the overall structure for organizing the system development process.

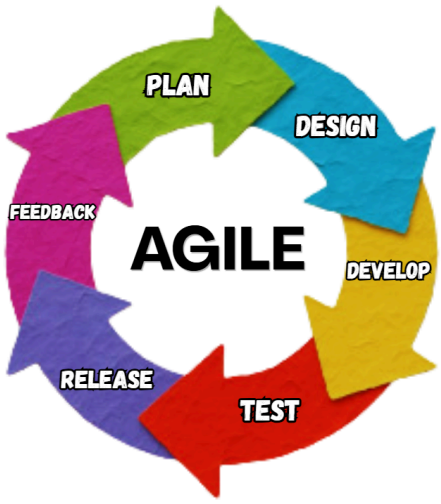


Figure 4.1: Agile Methodology

Planning

During the planning phase, the researchers communicated with the key stakeholders at the PalSU Main Canteen on February 25, 2026, including Mrs. Janeth G. Dalojo, Food Supervisor II, as well as canteen staff and regular student, staff and faculty patrons, in order to gather system requirements. Surveys, interviews and direct observation of canteen operations were conducted to understand existing workflows and identify the areas for development of a centralized order management system.

The data gathered during this phase guided the development of the system's functional and non-functional requirements. Core tasks such as pre-ordering meals, real-time order tracking and GPS-assisted delivery was the primary focus of the functional requirements. Non-functional requirements defined the system's reliability, performance, scalability and security standards, including compliance with the Data Privacy Act of 2012 (Republic Act No. 10173).

Design

Prototypes and mock-ups were designed for the Canteen Express application using different tools like Figma and Wireframe. Different user roles were considered during the design process such as Student/Walk-In Customer, Faculty/Staff Portal, Counter POS, Kitchen Board, Rider Hub and Administrator. Each interface was appropriately designed based on the functions of their respective users.

Student/Walk-In Customer was designed to allow users to view the menu items and place an order. Faculty/Staff Portal was designed to let users place orders with their accounts, view order status and choose between pickup or on campus delivery. The Counter POS was designed to scan order slips and to process cash payments. The Kitchen Board was designed for the Canteen Staff to keep track of order preparation. Rider Hub was designed to give Delivery Personnel access to available delivery orders. Lastly, the Administrator was designed to perform various administrative tasks. This includes managing user accounts, menu and reports and analytics.

Prototypes were shown to Mrs. Janeth G. Dalojo to get her feedback. System architecture, database, permissions and how order processing, payments and GPS navigation would be integrated for the application was also considered during this phase.

Development

The researchers began to develop Canteen Express following the design phase. This was created using Django 5.1.5 and Python as the backend. Supabase PostgreSQL was used as the production database and SQLite for local development. The frontend was created using HTML, Tailwind CSS, Alpine.js and FontAwesome for icons. The application uses PWA features to allow installation or be used on compatible mobile/tablet devices.

The system has several roles with access based on roles. These include Student/Walk-In customers, Faculty/Staff customers, Delivery Personnel, Canteen Staff and Administrator. Student/Walk-In customers have access to a mobile kiosk style interface to browse and order from the menu. They also have access to barcode queue slips. Cash payments can be processed at the Counter POS. Orders can be managed using the Kitchen Board. Faculty/Staff Customers have access to an account-based ordering portal. Delivery Personnel are able to take available orders, provide their GPS location, follow routes created using the maps and chat with customers within the application. The maps are provided by Leaflet.js and OpenStreetMap. Routes are provided using OSRM and Valhalla. Additional features include loyalty points, managing the menu, giving reviews, sales analysis, viewing reports and logs managed by the Administrator.

Git was used throughout development to manage changes to the code. Features were created to match the identified requirements and any changes made to improve the system.

Testing

After the development phase, the researchers moved onto the testing phase. This was done to make sure the system worked on actual devices. A sample of PalSUans gave their feedback. Testing of the system was also conducted regularly with the client starting from August 2026. This was done around 3-4 times a month. The canteen staff and admin were involved in this part of the testing as they will be using the system when it goes live. The research team got feedback from them to help identify issues in the system. They were able to test out the functions of the system and make changes to it according to the needs of the PalSU Main Canteen.

For Student/Walk-In Customers, a sample of at least 10 people from each college (in total there are 9 colleges) in the Palawan State University Main Campus were randomly chosen. For Faculty/Staff, at least 10 people who are currently working as either a Faculty Member or Staff in any department within the Palawan State University Main Canteen were randomly chosen. Data was collected to identify any bugs, errors and what needs to be improved within the system. Functional testing was carried out to ensure all features such as pre-ordering, tracking and GPS delivery were working properly.

Usability testing was also carried out to ensure all roles could easily navigate through the application. Close work was done with the canteen staff and Food Supervisor to make sure the system met the requirements and any last minute changes could be made.

Deployment

Once testing and development was complete, the system could be deployed. However, instead of deploying to native app stores, Canteen Express was deployed as a responsive web application, using Railway as a hosting platform and Gunicorn as a WSGI application server. The production backend was linked to a secure PostgreSQL database hosted by Supabase with SSL encryption enabled. Before deploying to the public, the system underwent some testing and was deployed to a select group of PalSU students, staff, faculty and canteen staff to get final feedback, test database migrations and ensure the server remained stable. Following beta testing, the final system was made and improvements were put into place based on the results and recommendations given.

Review

The system was deployed. Any problems and errors were resolved quickly. Features were improved using user feedback. The system was regularly reviewed to ensure it is running

smoothly, securely and meeting the needs of the PalSU Main Canteen. This informed how we can improve the system in the future.

By using the Agile method we have ensured that the system is being constantly improved to meet user needs. Using this method allows us to work as a team, engage users actively and deliver a high quality system gradually. This is essential due to the multiple user roles involved within the project.

Conceptual Framework

The conceptual framework of the system shows the relationship between the key variables of the study. It presents a visual model that summarizes the core concepts of the system and is accompanied by a narrative explanation of how those concepts interact. The framework is built around the Input-Process-Output (IPO) model, a functional representation that identifies the inputs, the processing tasks required to transform those inputs and the resulting outputs. This model allowed readers to understand the overall direction of the system at a glance, without having to read through the entire study.

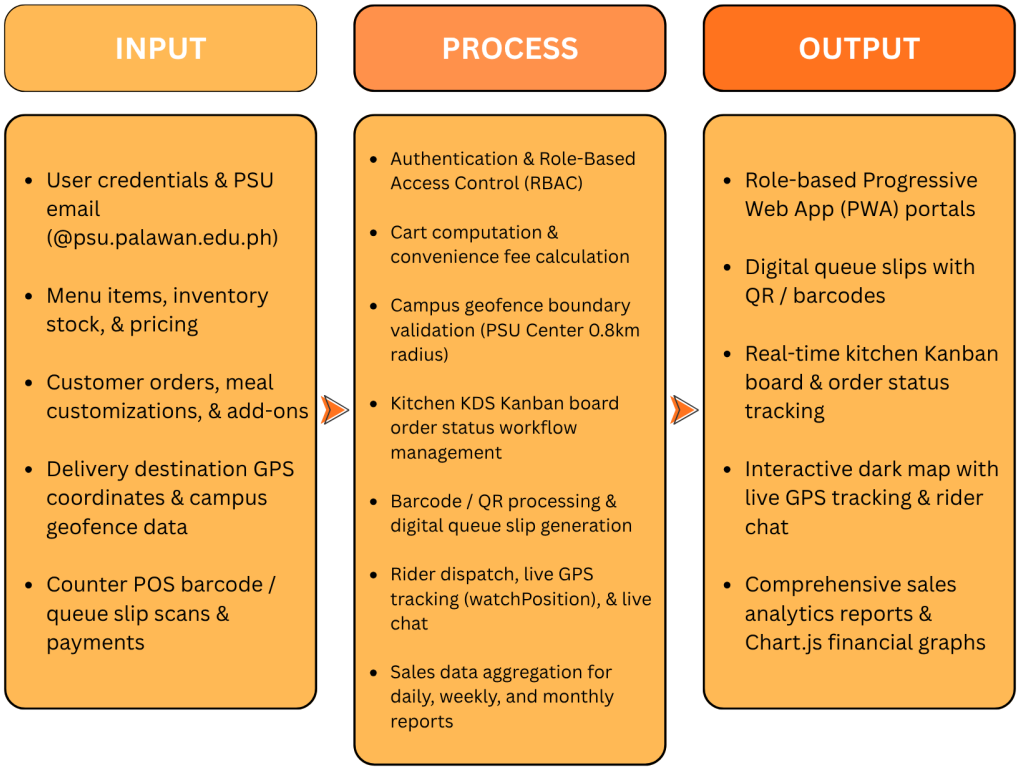


Figure 4.2: Conceptual Framework of Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff and Faculty at PalSU Main Canteen

The Input stage denotes the movement of data and materials that enter the system from external sources. Within the system, inputs comprise user credentials verified through institutional PSU email (@psu.palawan.edu.ph), currently available canteen menu items, menu stock levels and pricing information. Further inputs include customer orders containing meal customizations and add-ons, GPS coordinates of delivery destinations together with geofence data and Counter POS barcode or queue slip scans accompanied by payment details. Collecting these data accurately and promptly is vital for matching users with the food items available and for supporting the smooth operation of the canteen.

The Process stage comprises the central computational logic, business rules and backend workflows that are applied to the incoming data. This encompasses user authentication and Role-Based Access Control (RBAC), real-time shopping cart computation with delivery fee calculations and validation of campus geofence boundaries that enforce the PSU Center 0.8 km radius. It also manages order status transitions on the kitchen KDS Kanban board, barcode and QR processing for digital queue slips, rider dispatching, live GPS tracking (watchPosition), in-app rider chat management and the aggregation of sales data for daily, weekly and monthly analytics reports.

The Output stage returns the completed system results, interfaces and actionable data to users, staff and administrators. The principal outputs comprise role-based Progressive Web App (PWA) portals, digital queue slips with embedded QR codes and barcodes and real-time kitchen Kanban boards for tracking live order status. In addition, the system outputs interactive dark-themed maps with live GPS tracking and rider chat interfaces, together with thorough sales analytics reports and Chart.js financial graphs that support administrative decision-making.

Requirements Analysis

This process demands both technical and interpersonal competencies and entails collaboration within a team. In order to establish the system requirements and constraints, the researchers carried out a requirements analysis. Several essential activities must be accomplished during requirements analysis, including the identification of customer requirements, evaluation of system feasibility, performance of economic and technical analysis, allocation of functions to system components and definition of a timeline with associated constraints.

At present, students, staff and faculty members must endure long queues at the Palawan State University Main Campus Canteen. During peak periods, the staff must handle a substantial volume of walk-in orders. This is an area amenable to improvement and consequently the researchers developed a system intended to help address these issues.

Following the gathering of the requirements, they were documented in an SRS document. These requirements, use cases and user stories were disseminated to stakeholders for review and approval. The business requirements were established during the requirements analysis. This encompassed the canteen's goals and vision as well as the problem that requires resolution. The software requirements were likewise established. This involved specifying in detail the precise processes and features necessary to satisfy the business requirements. The business requirements consist of establishing a faster and better-organized canteen service for PalSU. The software requirements delineate who may access the system, how users can register and log in, how orders can be placed and tracked and how the delivery system operates.

To identify these needs concretely, the researchers gathered data using the following approaches:

Interview

Furthermore, semi-structured interviews were conducted by the researchers with Mrs. Janeth G. Dalojo (Food Supervisor II) and several staff members of the canteen. This enabled us to collect detailed information concerning the process, issues and expected outcomes of the current system. The interview yielded detailed information that a survey would not have allowed us to obtain. Most notably, we were able to gain an understanding of the ordering and payment process within the canteen.

Survey

Survey questionnaires were administered to students, employees and faculty members who regularly utilize the PalSU Main Canteen. A total of up to 10 students from each college (Palawan State University comprises 9 colleges), 20 randomly chosen department employees and Faculty Members across all colleges and 20 Senior High School students from Palawan State University completed the survey. Information was collected regarding users' habits, the problems they encountered daily at the canteen and whether they would be willing to use a pre-ordering system. This yielded valuable insights into which features were most needed, such as menu browsing, pre-ordering, live order tracking, nutritional information and rewards.

The data obtained from the interviews and surveys were compiled and analyzed in order to determine precisely the needs of the students, employees and faculty members. This made it possible to finalize the system's features and its day-to-day processes.

Use Case Diagram

A use case diagram is a graphical depiction of a user's possible interactions with a system. It shows the various use cases the system supports and the different types of users called actors that interact with it. In a use case diagram the use cases are represented by ellipses or circles, while the actors are typically shown as stick figures. The diagram gives stakeholders and developers a clear, visual overview of what the system does and who uses it and is often accompanied by other types of diagrams for deeper detail.

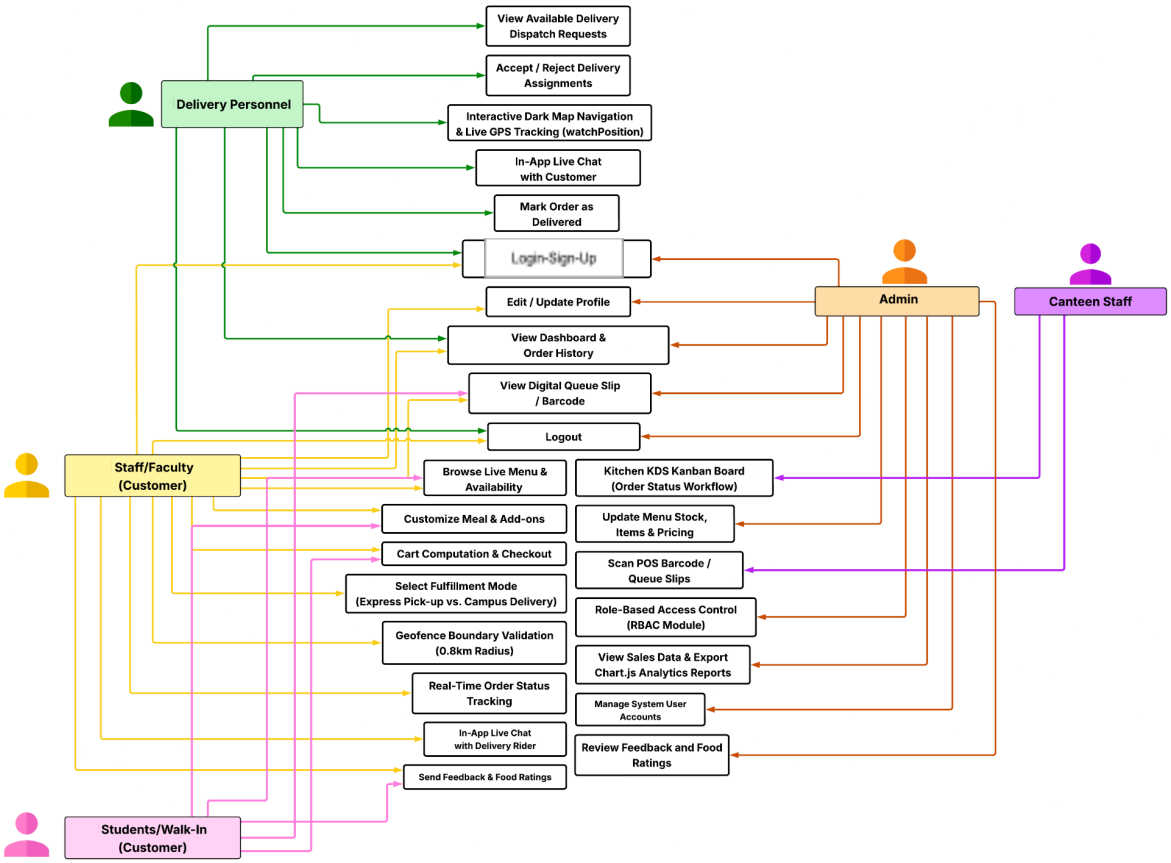


Figure 4.3: Use Case Diagram of Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff and Faculty at Palawan State University Main Canteen

The figure above showcases the Use Case Diagram of the system. The system consists of four main user roles: the Admin, Canteen Staff, Delivery Personnel and the Student/Walk-In Staff/Faculty User.

The Administrator is responsible for overseeing the entire system and managing its administrative functions. These include managing system user accounts, monitoring the dashboard and order history, updating menu stocks, menu items and pricing and viewing sales data through analytics reports. The Administrator also has access to the role-based access control

(RBAC) module and can review feedback and ratings submitted through the system. These functions support the management and monitoring of Canteen Express.

The Canteen Staff focus strictly on daily floor operations and order fulfillment. Staff members use the Counter POS interface to process payments and scan barcodes and manage incoming orders in real time through the Kitchen Board by updating order preparation statuses (pending, preparing, ready, completed) and coordinating with customers and delivery personnel.

The Delivery Personnel was able to receive delivery assignments, view GPS route guidance to the delivery destination, update the delivery status of each order and view their delivery history.

The Student/Staff/Faculty user scope is divided by deployment interface: registered Faculty and Staff utilize the full Web/Mobile application suite to perform Account Registration & Management, Menu Browsing with Dietary Filters, Pre-Order Placement, Real-Time Order Tracking, Loyalty Reward Redemption and Order Rating & Review Submission.

Walk-In/Students interact exclusively with the on-site ordering kiosk to browse the digital menu, apply dietary filters and place immediate counter orders directly without needing account registration.

User Stories

A user story is a small, self-contained unit of development work designed to accomplish a specific goal within a product. It is a brief, plain-language explanation of a feature or functionality written from the user's point of view. In Agile software development, a user story is considered the smallest unit of product development work that can lead to a complete element of

user functionality. User stories follow the standard format: As a description of the user, I want functionality so that benefits. The table below presents the user stories for the system across all four user roles.

User Role	Story ID	User Story
Student/Walk-In	US-01	As a student/walk-in, I want to browse the canteen menu through the smartphone and pre-order meals so that I can minimize my time in the queue and collect my order efficiently using a digital queue slip/Barcode.
Student/Walk-In	US-02	As a student/walk-in, I want to view menu categories, item descriptions and pricing on the smartphone to select affordable meals that fit my preferences.
Student/Walk-In	US-03	As a student/walk-in, I want to rate and review orders upon completion and provide feedback on food quality and counter service.
Faculty/Staff	US-04	As a faculty/staff member, I want to browse the canteen menu via the faculty/staff interface and pre-order meals with options for on-campus delivery or faculty-priority pickup.

Faculty/Staff	US-05	As a faculty/staff member, I want to track my order status in real time including live rider GPS tracking for PSU campus delivery or preparation updates so I know when my food arrives at my office or is ready for pickup.
Faculty/Staff	US-06	As a faculty/staff member, I want to view detailed menu descriptions and nutritional information so that I can find food that suits my health and wellness.
Faculty/Staff	US-07	As a faculty/staff member, I want to earn loyalty points and rewards for recurring meal pre-orders and subscription orders to encourage continued use of the platform.
Faculty/Staff	US-08	As a faculty/staff member, I want to submit ratings and feedback via the faculty feedback and ratings modal in my dashboard to help improve canteen service and delivery efficiency.
Canteen Staff	US-9	As a canteen staff, I want to view incoming orders in real time on the kitchen board and update their preparation statuses so that I can coordinate food fulfillment efficiently and reduce customer wait times.

Canteen Staff	US-10	As a canteen staff, I want to scan customer barcode queue slips at the counter POS to confirm payments instantly and issue receipts so that walk-in transactions are recorded and processed without delays.
Delivery Personnel	US-11	As a delivery personnel, I want to receive delivery assignments with GPS route guidance so that I can deliver orders to the correct location efficiently.
Delivery Personnel	US-12	As a delivery personnel, I want to update the delivery status of an order so that the customer is aware of the order's current location.
Delivery Personnel	US-13	As a delivery personnel, I want to view my delivery history so that I can keep track of completed assignments.
Administrator	US-14	As an admin, I want to monitor all orders placed on the platform so that I can ensure the system is functioning correctly.
Administrator	US-15	As an admin, I want to generate system-wide reports so that I can track usage revenue and performance metrics.

Administrator	US-16	As an admin, I want to manage user accounts so that I can deactivate or remove accounts that violate platform policies.
Student/Walk-In, Faculty/Staff, Canteen Staff (Unavailable-Item Scenario)	US-17	As a customer/canteen staff, I want the system to automatically block checkout or alert me when an ordered menu item is out of stock so that unfulfillable orders are prevented.
Canteen Staff (Invalid/Duplicate Barcode Scenario)	US-18	As a canteen staff, I want the counter POS scanner to reject invalid, expired, or already-paid/duplicate barcodes with an error message so that fraudulent or double redemption is prevented.
Student/Walk-In, Faculty/Staff, Delivery Personnel	US-19	As a customer/delivery personnel, I want the system to reject delivery requests or GPS location updates outside the PSU campus geofence radius so that out-of-bounds orders are disallowed.

(Delivery Failure/Geofence Scenario)		
Student/Walk-In, Faculty/Staff, Canteen Staff (Cancellation Scenario)	US-20	As a customer/canteen staff, I want to cancel unpaid or pending orders so that stocks are automatically restored and loyalty points/fees are refunded appropriately.
All Users (Success Paths)	US-21	As a system user, I want standard workflows (checkout > payment confirmation > kitchen preparation > pickup/delivery > completion) to complete successfully with proper receipt generation and point crediting.
All Users (Failure Paths	US-22	As a system user, I want the system to gracefully handle server errors, expired sessions and network interruptions with clear error codes (e.g., 400, 401, 403, 404, 422, 423) rather than crashing or dumping raw tracebacks.

Table 4.1: User Stories Table of Canteen Express

The table above presents the user stories for the system. It outlines the specific wants and needs of each of the four user roles like Student/Staff/Faculty User, Canteen Staff, Delivery Personnel and Admin. Including these user stories in the development process ensures that the system is built around real user goals, improving the overall experience for all stakeholders. These stories will serve as the primary basis for defining the system's features and development priorities during each Agile iteration.

System/Software Requirements Specification

A Software Requirements Specification (SRS) is a document that delineates the nature of a project, software, or application. It functions as a comprehensive manual for the system, prepared prior to the commencement of development. The SRS addresses the purpose and scope of the system, functional and non-functional requirements, software and hardware requirements, environmental conditions, security requirements and software quality attributes.

With regard to the system, the SRS differentiates between business requirements and software requirements. Business requirements pertain to the canteen's objectives and the core problem that must be addressed, specifically the need for a faster and more organized food ordering and delivery service at PalSU. Software requirements decompose the specific steps required to achieve those business objectives. Whereas the business requirement articulates the "why," the software requirements delineate the "what," detailing who has access to the system, how each user interacts with it, what processes must occur and what outputs the system must produce.

Functional requirements define the fundamental behavior of the system: what the system does or must not do in response to inputs. They encompass calculations, data inputs and business

processes and represent the features that enable the system to function as intended. Non-functional requirements define how effectively the system performs its functions, encompassing usability, security, reliability, performance, availability and scalability.

Screen Layout

- **Students/Walk-Ins, Staff/Faculty**
  - System Size: 1080 × 1920 (Android Phone) / 1125 × 2532 (iOS Phone)
  - Orientation: Portrait (Mobile Phones) or Landscape/Portrait (Tablets)
  - Application Viewport: 360 × 640 or 375 × 812 (Phones) / 768 × 1024
- **Delivery Personnel**
  - System Size: 1080 × 1920 (Android Phone) or 1125 × 2532 (iOS Phone)
  - Orientation: Portrait (Mobile Phones)
  - Application Viewport: 360 × 640 (Android) or 375 × 812 (iOS)
- **Canteen Staff / Administrator**
  - System Size: 1920 × 1080 (Full HD Desktop) or 2560 × 1440 (Widescreen Admin Display)
  - Orientation: Landscape/Portrait
  - Application Viewport: 1600px+ fluid desktop workspace (Desktop) / 1024 × 768 or 768 × 1024 (POS Display)

User Classes

Four types of users interacted with the system: Student/Staff/Faculty Users (User Class 1), Canteen Staff (User Class 2), Delivery Personnel (User Class 3) and Administrators (User Class 4). Each user class will access the same system but will have a distinct set of roles, permissions and functionalities tailored to their specific responsibilities within the canteen service workflow.

Functional Requirements

User Class 1 – Student/Walk-Ins, Faculty/Staff Users

Functional Requirement – Faculty/Staff Account Access and Authentication

ID: FR1.1

Title: Faculty and Staff Account Registration and Authentication

Description: Faculty and Staff users can create an account using their institutional email address and log in using their registered credentials. A password recovery option is available through email verification.

Functional Requirement – Student/Walk-In Order Access

ID: FR1.2

Title: Student/Walk-In Order Access

Description: Student/Walk-In users can place immediate counter or pre-orders through the kiosk interface on their devices without requiring account registration.

Functional Requirement – Menu Browsing and Nutritional Information

ID: FR1.3

Title: Menu Browsing and Dietary Filtering

Description: Users can browse the available canteen menu, view item descriptions and prices and be informed about the nutritional information to find items that suit their preferences.

Functional Requirement – Pre-Order Placement and Counter Order Placement

ID: FR1.4

Title: Pre-Order Placement and Counter Order Placement

Description: Faculty/Staff users can select menu items, specify quantities, choose between pickup or delivery and submit a pre-order through the system before arriving at the canteen. Student/Walk-In users can select menu items and place immediate counter orders through their smartphones.

Functional Requirement – Real-Time Notifications

ID: FR1.5

Title: Real-Time Push & In-App Notifications

Description: Users receive instant in-app alerts and notifications, regarding order status transitions (e.g., received, preparing, ready, out for delivery, completed) and delivery dispatch updates.

Functional Requirement – Real-Time Order Tracking

ID: FR1.6

Title: Real-Time Order Tracking

Description: Faculty/Staff users can monitor the status of their orders from order received, preparation and ready for pickup to out for delivery through the systems order tracking feature.

Functional Requirement – Loyalty Rewards

ID: FR1.7

Title: Loyalty Rewards

Description: Faculty/Staff automatically earn loyalty points for every purchase order. Accumulated points can be redeemed for discounts on future orders according to the canteen reward feature.

Functional Requirement – Ratings and Reviews

ID: FR1.8

Title: Ratings and Reviews

Description: After an order is completed, users can rate the food and canteen service after completing an order and leave a written review to provide feedback to the canteen staff.

Functional Requirement – Campus Geofence Validation

ID: FR1.9

Title: Campus Geofence Boundary Verification

Description: During delivery checkout, the system automatically validates the Faculty/Staff's customer delivery destination coordinates against the designated campus geographic boundary (PSU Tiniguiban Heights radius) and rejects out-of-campus orders with a validation error.

Functional Requirement – Rider Customer Live Chat

ID: FR1.10

Title: In-App Delivery Chat

Description: Faculty/Staff and assigned delivery personnel can communicate directly via a real-time in-app messaging channel while an order is out for delivery to coordinate handoffs.

Functional Requirement – Digital Queue Slip and Barcode Generation

ID: FR1.11

Title: Digital Queue Slip and Barcode Generation

Description: The system automatically generates a digital queue slip complete with a barcode or QR code upon order placement, enabling counter staff to instantly scan and confirm payment.

User Class 2 – Canteen Staff

Functional Requirement – Canteen Staff Login

ID: FR2.1

Title: Canteen Staff Login

Description: Canteen staff members can then log in using the assigned credentials to access the dashboard.

Functional Requirement – Canteen Staff Counter POS & Kitchen Board Access

ID: FR2.2

Title: Canteen Staff Dashboard & POS Access

Description: Canteen staff members can log in using their assigned credentials to access the Counter POS interface for scanning barcode slips and the Kitchen Board for updating order preparation statuses in real time.

Functional Requirement – Canteen Operational Schedule and Status Control

ID: FR2.3

Title: Operational Hours and Emergency Status Management

Description: Canteen staff can manage operating hours, toggle automated open/closed status and issue emergency closure notices with specific closure reasons.

User Class 3 – Delivery Personnel

Functional Requirement – Delivery Assignment View

ID: FR3.1

Title: Delivery Assignment View

Description: Delivery personnel can view their current assigned delivery orders, including the customer's name, delivery address and relevant order details.

Functional Requirement – GPS Route Guidance

ID: FR3.2

Title: GPS Route Guidance

Description: The system will provide GPS route guidance to assist delivery personnel in navigating to the correct customer's delivery destination efficiently.

Functional Requirement – Delivery Status Update

ID: FR3.3

Title: Delivery Status Update

Description: Delivery personnel can update the status of delivery such as pick up, in transit or delivered. These updates are reflected in the system so that customers and canteen staff can monitor the delivery status.

Functional Requirement – Delivery History

ID: FR3.4

Title: Delivery History

Description: Delivery personnel can view a record of their previous completed deliveries for reference and performance monitoring.

User Class 4 – Admin

Functional Requirement – User Account Management

ID: FR4.1

Title: User Account Management

Description: The Admin can view, edit, deactivate, or remove user accounts across the different user classes to maintain platform integrity and enforce system policies.

Functional Requirement – System-Wide Order Monitoring

ID: FR4.2

Title: System-Wide Order Monitoring

Description: The Admin can view and monitor places through the system across different users and canteen staff to oversee overall ordering activity.

Functional Requirement – Report Generation

ID: FR4.3

Title: Report Generation

Description: The Admin can generate system-wide reports on transactions, user activity, sales and canteen staff performance to support system monitoring and decision making.

Functional Requirement – Feedback and Complaints Management

ID: FR4.4

Title: Feedback and Complaints Management

Description: The Admin can view user ratings and reviews, manage complaints to respond to reported concerns and take appropriate administrative action regarding users who violate system policies.

Functional Requirement – Activity Logging and Audit Trail

ID: FR4.4

Title: Activity Logging and Audit Trail

Description: The system automatically records and logs administrative and staff actions across modules to maintain security, traceability and accountability.

Non-Functional Requirements

Usability

The system offers a clean, responsive and intuitive interface that is designed to reduce the learning curve for all four user classes. Shared layout templates, including base.html and role-specific dashboards, are used to ensure that buttons, icons, menus and navigation elements are labeled and styled consistently.

For Faculty and Staff, the interface facilitates account-based pre-ordering for pickup or delivery, live GPS order tracking, loyalty points management and the submission of post-order feedback. For Student/Walk-In users, a Kiosk interface allows orders to be placed directly without account registration, automatically producing a digital queue slip with a scannable barcode for payment processing at the counter POS terminal.

System interaction workflows and interface components are validated by automated unit and integration tests across the application modules, namely customer_portal, canteen_menu, kitchen_display, deliveries and accounts.

Security

The system safeguards user accounts and system data against unauthorized access. Authentication and password management are implemented by means of Django's built-in authentication and security features. Passwords are stored securely through password hashing, whereas role-based access control limits each user class to the features and data pertinent to their assigned responsibilities. Data transmitted between users and the system are protected by HTTPS/SSL encryption in the production environment. Administrative functions are confined to authorized administrators.

Reliability

The system is intended to deliver consistent and dependable operation throughout standard canteen activities. Database records, comprising user accounts, menu information, orders, transactions and delivery records, are stored persistently in order to mitigate the risk of data loss. Functional and integration testing was conducted on the system to confirm that orders, status updates, transactions and other essential operations are processed correctly. Testing further encompassed concurrent user and order activities during peak periods in the canteen.

Performance

The system is intended to deliver prompt responses to user actions; menu browsing, order placement, order status updates and delivery tracking are handled with minimal delay under normal operating conditions. Database queries are appropriately structured so as to minimize unnecessary processing and enhance response time. Images and other media uploaded to the system are likewise optimized in order to avoid unnecessary loading delays.

Availability

The system is deployed via a cloud-based hosting environment so that authorized users can access it during the canteen's intended operating hours. Faculty/Staff Customers as well as Student/Walk-In users are able to access Canteen Express remotely through their compatible devices. Faculty/Staff Customers must log in to use the platform's ordering and delivery features, whereas Student/Walk-In Customers can access the customer ordering interface to browse the available menu items and place orders. Scheduled maintenance is carried out outside peak operating periods whenever feasible and users will be notified of planned system interruptions.

Scalability

The system architecture was designed to accommodate a growing number of users, orders, menu items and canteen staff without the need for a complete redesign. The Django application's modular structure permits the addition of further features and user functions as required. The cloud-based deployment environment and the PostgreSQL database likewise enable the system to support greater data and usage demands as it expands.

Requirements Documentation

Following an initial interview with Mrs. Janeth G. Dalojo, Food Supervisor II of the PalSU Main Canteen, several requirements were identified and integrated into the design of Canteen Express. Among the identified needs was a pre-ordering feature enabling Faculty and Staff users to place orders in advance, thereby helping to reduce the time spent waiting at the canteen counter. The system likewise accommodates immediate counter orders placed through the devices of Student/Walk-In users. Mrs. Dalojo further identified the need for a comprehensive order and transaction history. The system will maintain records of orders,

including pertinent order details, dates, times, payment records and order status. These records are accessible to authorized users, specifically the admin, for monitoring and record-keeping purposes. The system additionally enables the administrator to manage the menu by adding or editing menu items, updating prices, uploading item photos and changing item availability in accordance with current stock.

This will designate items as unavailable once they can no longer be purchased. Furthermore, the system provides the Admin with sales summaries, transaction history and system-wide reporting. Such reports contain information including completed orders, transactions, sales activity and other pertinent system records that can support canteen monitoring and planning. The requirements identified during the interview have been incorporated into the functional and non-functional requirements of Canteen Express. These requirements guided the development, testing and evaluation of the system to ensure that the implemented features address the identified needs of the intended users and of canteen operations.

Ethical Consideration

As mandated by the University Research Manual, all research projects are required to undergo review by the University Research Ethics Review Committee (URERC). However, at the time of this study, the URERC was under a moratorium while awaiting the results of its Level 2 Accreditation. In view of this circumstance, the researchers sought guidance through formal consultation regarding the ethical considerations of the study. Prior to conducting any user interviews, surveys, or system testing, the researchers complied with all comments, recommendations and suggestions provided during the consultation to ensure all research

procedures were ethically sound, responsible and compliant with institutional guidance. This step was essential to safeguard the rights, welfare and privacy of all participants during data collection and testing.

All participants were fully informed about the nature and purpose of the study, mentioning it to the header of the survey questionnaire as for the surveys, while in face-to-face interviews the researchers discussed the purpose, risks and benefits of the system. All participants were informed that the participation is strictly voluntary and they were given permission to withdraw anytime without any penalty.

Personal information was collected with the explicit consent of the users and all gathered data were secured through strong authentication mechanisms, such as secure password hashing (PBKDF2/Argon2) and role-based session tokens. To distinguish between anonymity and confidentiality: survey responses were kept strictly anonymous with no personal identifiers collected, whereas interview recordings and system transaction logs maintained confidentiality through secure storage and restricted access. Furthermore, data prepared for statistical evaluation were anonymized, coded and shared securely with the designated project statistician under strict data privacy protocols, ensuring individual identities remain protected while enabling necessary quantitative analysis.

In terms of risk assessment and participant protection, the system entails minimal physical, psychological, social, or financial risks. Physical risks related to campus deliveries are mitigated by strict campus geofence enforcement (limiting deliveries to a 0.8 km radius around PSU Tiniguiban Heights) and real-time GPS tracking (navigator.geolocation.watchPosition) to ensure safe routing. Financial and transactional risks are addressed through transparent pricing

calculations (fixed delivery fee of ₱15 per ₱300 purchase block), secure order processing and queue slip verification at the counter POS to prevent erroneous billing or double charges. User data privacy and confidentiality are rigorously protected across all accounts (Faculty, Staff and Delivery Personnels).

Digital data, including database records and system logs, are securely stored in encrypted cloud storage (Supabase PostgreSQL with SSL/TLS encryption) and password-protected local drives, while any physical records or interview notes are kept in secure, locked storage. Access is strictly limited to authorized members of the research team and the designated project statistician. All collected participant and system data will be retained temporarily during the evaluation and deployment phase, after which they will be permanently deleted and destroyed by March 2027.

The researchers may have their own opinions and positive views about Canteen Express because they are the developers and researchers of the system. To avoid having biases, the researchers created a survey questionnaire or interview with clear and neutral questions. All the results positive or negative are kept to maintain fair insights about the system. The results and conclusions will be based only on the data collected from the participants.

The development and deployment of the system adhere strictly to the institutional research guidelines, data privacy standards and ethical protocols of Palawan State University (PalSU). Because the system involves user authentication, order processing and delivery logistics for customers (faculty and staff) within the university campus (PSU Tiniguiban Heights), the project follows minimal-risk human participants guidelines. Canteen Express is classified as a low-risk educational software development and administrative system; reviewed and cleared

under institutional guidelines. In terms of data privacy and compliance, user data collection (such as the PalSU email addresses "@psu.palawan.edu.ph" for faculty/staff authentication and delivery contact details) complies with the Data Privacy Act of 2012 (Republic Act No. 10173) of the Philippines, ensuring secure password hashing (Django PBKDF2), session management and restricted database access. All source code, database architecture and documentation for Canteen-Express are original works developed by the project team and have undergone institutional anti-plagiarism screening to ensure similarity indices remain well below allowable academic thresholds. External libraries and frameworks (such as Django and Leaflet.js) are properly attributed in accordance with open-source licenses, while map tiles, UI icons and menu media utilize OpenStreetMap standards, open-source icon sets and authorized or royalty free campus assets solely for academic prototyping purposes.

In addition, the researchers complied with the Data Privacy Act of 2012 (Republic Act No. 10173) to uphold the privacy rights of all students, staff, faculty, canteen staff and other stakeholders involved in the study. All raw data were properly disposed of after the research was completed and accepted, in accordance with applicable data privacy standards.

Work Plan (Gantt Chart)

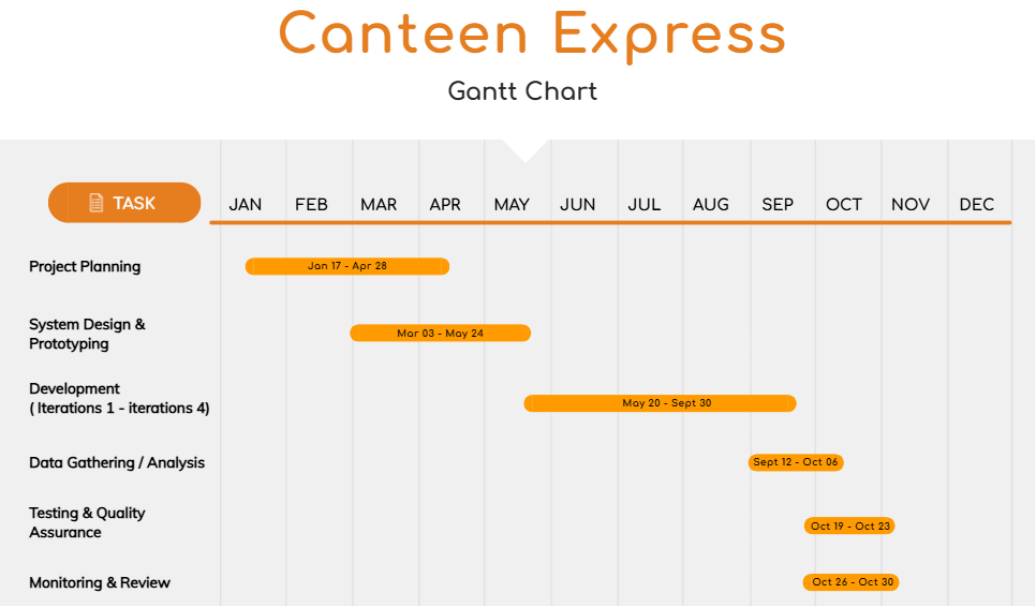


Figure 4.4: Gantt Chart – Canteen Express Development Work Plan

The figure above presents the planned schedule of activities for the development of Canteen Express from January to October 2026. The project schedule includes planning and requirements gathering, system design and prototyping, development, testing and quality assurance and monitoring and review. Some activities overlap to accommodate the coordination of requirements gathering, design revisions, development tasks and testing activities. The schedule serves as a reference for organizing project activities and monitoring progress.

Cost-Benefit Analysis

The following section presents a preliminary cost-benefit analysis for the development and deployment of the system. Given that the project is academic in nature and primarily uses

free and open-source tools, the estimated costs are relatively low compared to the operational and quality-of-life benefits the system is expected to deliver to the PalSU community.

Estimated Project Costs

Cost Item	Estimated Amount
ONE-TIME COSTS	
Documentation & Printing	₱1,500 – ₱4,000
Miscellaneous / Contingency	₱1,000
Testing Devices (Researcher Owned)	Free (₱0)
SUBTOTAL (ONE-TIME)	₱2,500 – ₱5,000
RECURRING COSTS (1 YEAR)	
Cloud Hosting and Database (1 year - Scaled Production Instance)	₱40,000 – ₱70,000
Custom Domain & SSL (1 year)	₱1,000 - ₱3,000

SUBTOTAL (RECURRING)	₱41,000 – ₱73,000
DEVELOPMENT TOOLS & SOFTWARE LICENCES	
Development Tools & Software Licenses	Free / Open Source (Django, Python, HTML5, CSS3, Javascript, Tailwind CSS, Supabase PostgreSQL, Leaflet.js, OpenStreetMap, Chart.js, Git, Github and Visual Studio Code)
Estimated Total	₱43,500 – ₱78,000

Overall the development of the system represents a low-cost investment with high potential returns in terms of operational efficiency, user satisfaction and institutional service quality. The tangible savings from reduced cash handling and the intangible gains from improved student and faculty experience make the project a worthwhile initiative for the PalSU Main Canteen.

Design of Software, Systems, Product and/or Processes

The design of software, systems, product and processes involves three levels of results. The Architectural Design is the highest-level abstraction, identifying the software as a system

with many interacting components. The High-Level Design breaks this down into subsystems and modules, showing how they interact with each other. The Detailed Design goes deeper into each module, defining its logical structure and how it communicates with other parts of the system. Together, these levels guide the researchers in building the system in a structured, logical and maintainable way.

Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation that shows how data moves through a system from inputs, through processes, to outputs. Each process in a DFD must have at least one input and one output. Each data store must have at least one data flow in and one data flow out and all data stored in the system must pass through a process. DFDs are organized into levels that progressively add detail.

DFD Level 0, also called the Context Diagram, is a high-level overview of the entire proposed system. It shows the system as a single process and identifies the external entities that interact with it, namely the Student/Faculty Users, Canteen Vendor/Staff, Delivery Personnel and the Admin. It is designed to be easily understood by a wide audience, including stakeholders and non-technical readers and shows only the major data flows in and out of the system.

DFD Level 1 provides a more detailed breakdown of the Context Diagram. It highlights the main functions carried out within the system, such as user authentication, order processing, payment handling, delivery management and notification dispatch. Each of these functions is shown as a separate process with its own inputs and outputs, giving developers and stakeholders a clearer picture of how the system operates internally.

DFD Level 2 goes one step deeper into the parts of Level 1, providing the most detailed view of specific processes where additional clarity is needed for example, breaking down the order processing module into sub-processes such as order validation, stock checking, status updating and notification triggering.

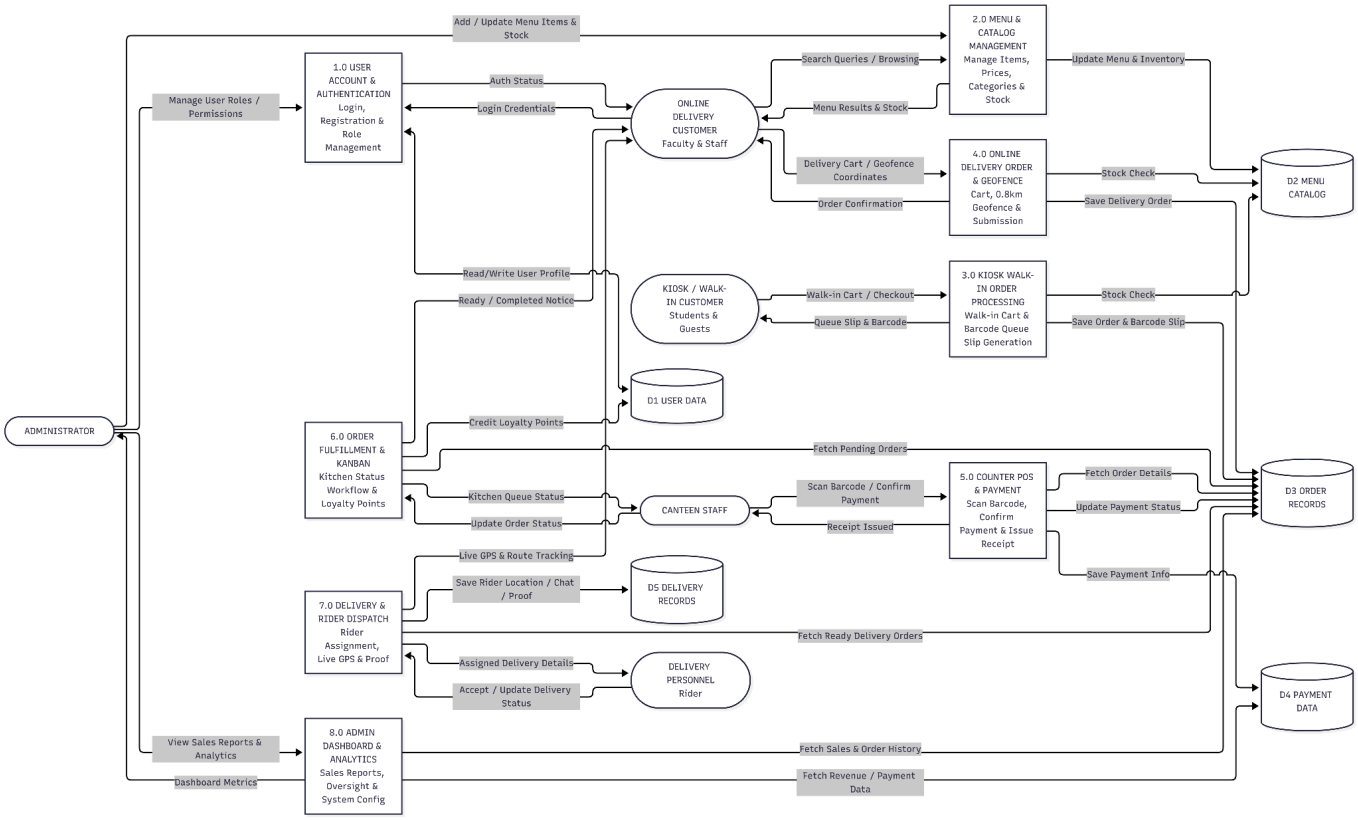


Figure 4.5: Level 1; Data Flow Diagram of Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff and Faculty at Palawan State University Main Canteen

The figure above shows the flow of data and actions within the system. The system accommodates four main types of users: Student/Staff/Faculty Customer, Canteen Staff, Delivery Personnel and the Admin.

The Customers role encompasses both walk-in kiosk users (students and campus visitors) and online delivery users (staff/faculty members). Kiosk customers place walk-in orders and generate barcode queue slips through the customer portal, while faculty and staff log in with institutional credentials, browse real-time menu categories, submit delivery orders with 0.8 km campus geofence validation and track live GPS deliveries and order status updates.

The Canteen Staff role handles counter operations and order fulfillment. Staff members process walk-in payments and scan customer barcode slips via the Counter POS Interface to issue receipts and update payment statuses, while also operating the kitchen display kanban workflow to transition orders from pending to preparing, ready and completed, which automatically credits customer loyalty points.

The Delivery Personnel manages on-campus deliveries. Delivery Personnels log into the rider portal to view dispatched orders fetched from order records, accept deliveries, update real-time delivery statuses, broadcast live GPS location coordinates, communicate with customers via the in-chat feature and upload proof of delivery photos into delivery data records.

The Admin role provides high-level system oversight, user account management and business analytics. Administrators configure role permissions, manage system settings and inspect comprehensive sales reports, revenue metrics and order histories through the admin dashboard, interfacing with user data, menu catalogs and financial transaction records.

Entity Relationship Diagram

An Entity Relationship Diagram (ERD) shows the relationships of entity sets stored in a database. An entity is an object or component of data about which the system needs to store information and an entity set is a collection of similar entities. Entities can have attributes that

define their properties. By identifying the entities, their attributes and the relationships between them, an ERD illustrates the logical structure of the database.

In an ERD, entities are represented by rectangles, relationships between entities are shown using diamond shapes and attributes are represented by ovals or ellipses connected to their respective entities. A weak entity, one that cannot be uniquely identified by its own attributes alone and depends on a relationship with another entity, is represented by a double rectangle. Entities can also be self-linked; for example, an order item may reference another order item in a grouped or combo meal scenario.

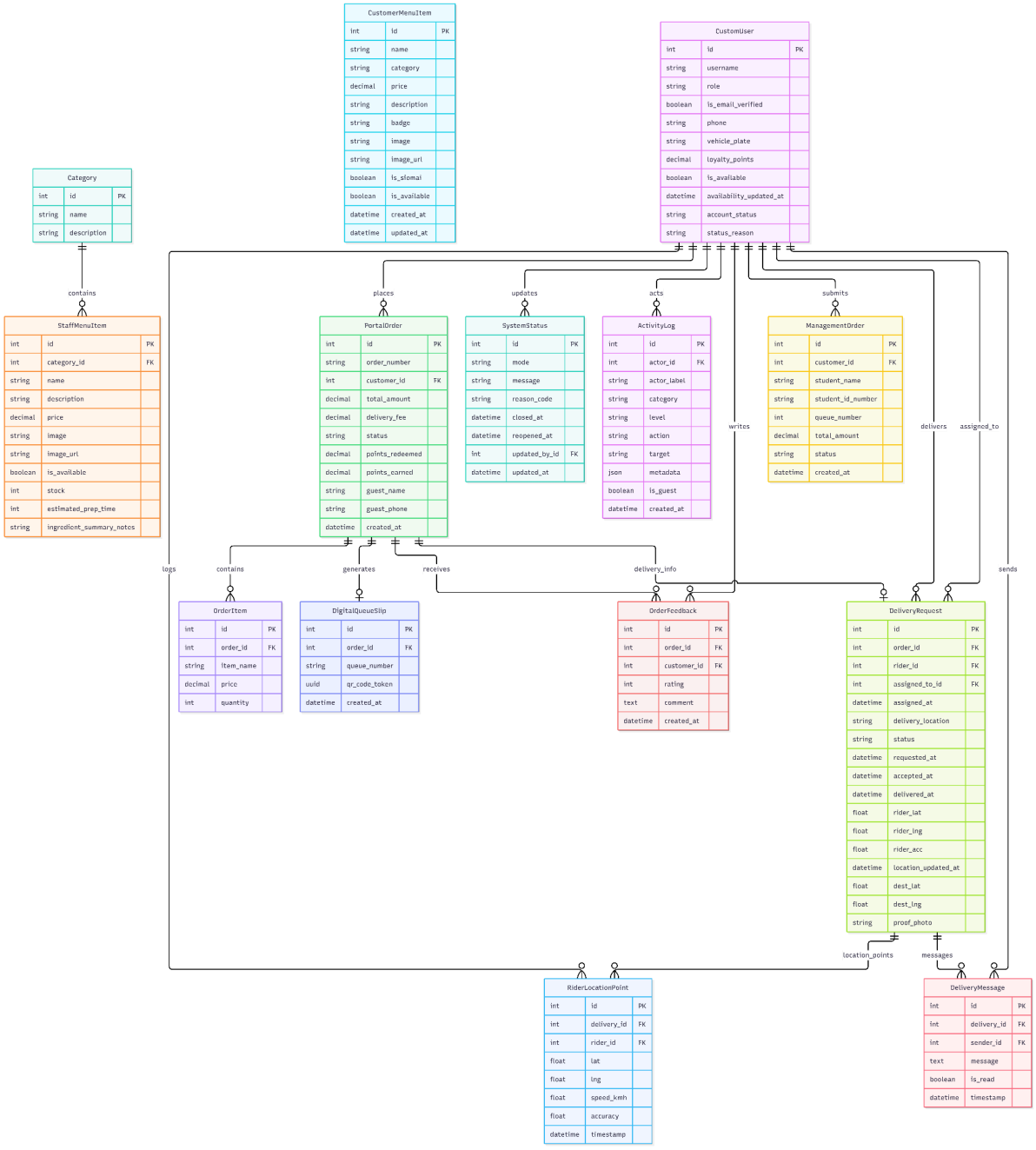


Figure 4.6: Entity Relationship Diagram of Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for PalSU Main Canteen

This Entity-Relationship (ER) diagram illustrates the complete database schema and entity relationships for the system. At the center of user and administrative management is the

CustomUser table, which stores authentication details, user roles (Student/Walk-In, Faculty and Staff, Delivery Personnel), account status, status reasons, contact information, vehicle details for delivery personnels and loyalty points. Surrounding user management are the menu and catalog entities, the Category table group items, the StaffMenuItem table tracks menu stock counts, preparation times and ingredient notes for canteen staff and the CustomerMenuItem table manages portal items categorized by food types with dietary and specialty flags like is_siomai.

For order and queue processing, the schema distinguishes between online and counter workflows. The PortalOrder table handles customer portal orders with fields for total amounts, delivery fees, points redeemed and earned, guest contact details and statuses ranging from unpaid to completed, while the ManagementOrder table supports counter POS orders by recording student names, student ID numbers, queue numbers and processing statuses. Each portal order is linked to individual OrderItem records specifying quantities and prices and generates a unique DigitalQueueSlip containing a queue number and a secure UUID QR code token.

Logistics, tracking and real-time communication are handled through dedicated delivery entities. The DeliveryRequest table coordinates delivery assignments by linking portal orders to designated riders and dispatchers, tracking pickup and destination coordinates (dest_lat, dest_lng), rider accuracy, proof photos and statuses such as searching, accepted, delivered, rejected, or timeout. To support live tracking and interaction, the RiderLocationPoint table records the coordinate tracking history, speeds and accuracy of the delivery personnel, while the DeliveryMessage table facilitates live chat communication between customers and riders.

In addition, the schema incorporates system oversight and auditing entities. The SystemStatus table acts as a durable source of truth for the canteen’s open/close operational

mode, messages and closure reasons updated by staff. The ActivityLog table provides an append-only audit trail recording system actions, actor snapshots, categories, severities, targets and structured metadata. Finally, the OrderFeedback table captures customer ratings and textual reviews tied to completed portal orders and customer profiles.

Network Topology and Architecture

Network topology refers to the manner in which devices and communication components are interconnected within a network. In the case of Canteen Express, the local campus network employs a star topology, whereby client devices communicate via a central network device, such as a router or wireless access point. Such an arrangement simplifies network management and enables connection problems affecting an individual client device to be isolated from other connected devices. Nevertheless, users may also access the system via mobile data, which connects to the deployed application through the Internet rather than through the PalSU Campus Network.

In this study the researchers present a Network Topology Diagram that depicts the relationship among the client devices, local network, internet connection, deployed application and database services of Canteen Express.

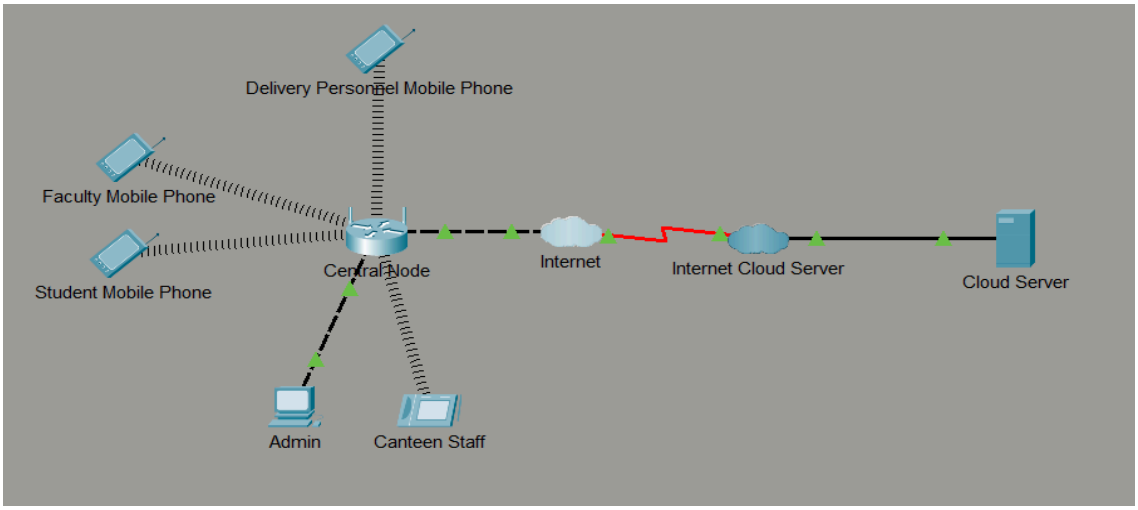


Figure 4.7: Network Topology and Architecture of Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for PalSU Main Canteen

The system is accessed by Student/Walk-in Customers, Faculty/Staff Customers, Canteen Staff, Delivery Personnel and the Administrator, each employing devices suited to their designated functions. Student/Walk-In customers and Faculty/Staff users access the web/mobile application via their own smartphones or other compatible browser-enabled devices, whereas Canteen Staff and the Administrator utilize compatible devices for counter operations and administration. Delivery Personnel utilize a compatible mobile device for their assigned delivery functions. Devices connected via the PalSU campus network utilize the available Wi-Fi router or wireless access point, while devices relying on mobile data connect to the application through the Internet. Both approaches provide access to the deployed system.

CHAPTER 5

RESULT AND DISCUSSION

This chapter presents the results of the development, testing and implementation of Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff and Faculty at Palawan State University Main Canteen. It discusses how the system was developed, the tests conducted, the implementation process and the results gathered from the respondents.

Development and Testing

Before developing the system the researchers gathered information about the current process used at the Palawan State University Main Canteen. An interview was conducted with Mrs. Janeth G. Dalojo, the Food Supervisor II of the PalSU Main Canteen. The interview provided information about the existing workflow, operational concerns and requirements for pre-ordering for Faculty and Staff users, kiosk ordering for Students and Walk-In users, order and transaction records, menu management, delivery management and reporting functions.

The information gathered from the interview was used as a basis for defining the functional and non-functional requirements of Canteen Express. These requirements guided the design and development of the system and were also considered during the testing and evaluation stages.

Prototype Development

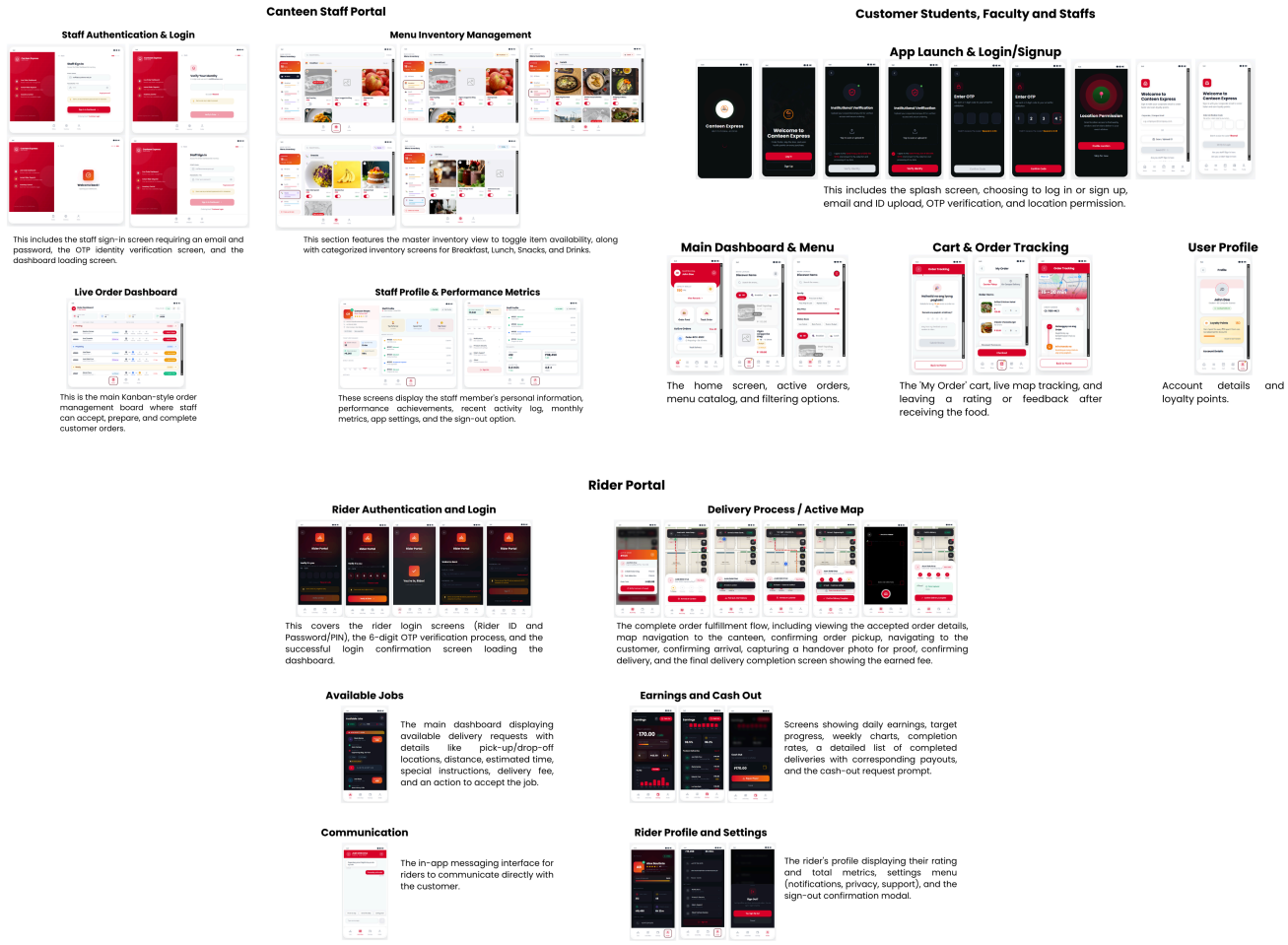


Figure 5.1: Prototype Development

Before developing the actual system the researchers prepared prototypes and wireframes to establish the planned interface, navigation and arrangement of the system feature. The design considered the different user roles supported by Canteen Express including: Student/Walk-in users, Faculty and Staff users, Delivery Personnel, Canteen Staff and the Administrator.

The prototype included the major interfaces required for menu browsing, ordering, order monitoring, delivery management and administrative functions. The prototypes were reviewed

during the design stage and were presented to the stakeholder for feedback before the actual development of the system. This process allowed the researchers to identify necessary adjustments to the interface and system workflow before implementation.

System Development

Figure 5.2: System Development

After completing the prototype the researchers proceeded with the development of the actual system. Canteen Express was developed as a web/mobile-based platform using Django 5.1.5 and Python as the primary backend technologies. HTML5, CSS3, Javascript and Tailwind CSS were used to create the responsive interface for the different user roles.

The system merged Leaflet.js and OpenStreetMap for map-related functions and delivery route guidance while Chart.js was used to present administrative analytics. Supabase PostgreSQL served as the primary database with SQLite used as a local development database. Visual Studio was used as the development environment, implementation specifications also included Railway as the deployment platform and Gunicorn as the production WSGI server.

The development system included role-based interfaces and functions for ordering, canteen operations, delivery management and administrative monitoring. These functions were based on the requirements identified during the requirements analysis and design stages.

The researchers followed an iterative development approach consistent with the Agile methodology described in Chapter 4. The system was developed in phases so that specific functions could be implemented, reviewed, tested and improved as development progressed. The development was divided into four major phases.

The first phase focused on Account and User Management where the researchers established the basic account and access functions of the system. This included user registration, login, account verification and role-based access. The access structure was designed to provide different interfaces and functions according to the user assigned roles. After completing the account and access functions the researchers proceeded to the second phase which focused on Customer Ordering. The researchers developed the main ordering functions as well as menu browsing, nutritional information, shopping cart, checkout, mobile kiosk interface ordering and pre-ordering. Faculty and Staff users were provided with the Faculty Portal where they could place orders in advance while Student/Walk-In users could place their orders through their smartphone's mobile kiosk interface. The third phase focused on Canteen and Delivery Operations. During this phase the researchers developed the functions needed by Canteen Staff and Delivery Personnel to manage and process orders. These included the Counter POS, Kitchen Kanban, menu stocks monitoring, order status management, Rider Hub, delivery assignment, in-app chat, GPS route guidance, proof of delivery and campus delivery geofence. These functions allowed the system to support the processing and monitoring of orders from the time they were placed until they were picked up or delivered.

The final phase focused on Additional Features and Final Improvements. At this stage the researcher completed additional features such as loyalty points, vouchers and discounts, ratings and feedback, report, analytics and dispute handling. The researchers also reviewed the different parts of the system and refined the interfaces and functions based on the feedback and testing conducted during development. Through these phases the researchers gradually developed and improved Canteen Express while ensuring that its major functions were implemented, tested and reviewed before the completion of the system.

System Testing

Testing was performed throughout the development process to verify whether the implemented functions operated according to their intended requirements. Individual functions were first checked before related functions were tested together as part of the complete system.

The testing covered important system functions such as account login, checkout, order processing, loyalty point computation, delivery fee computation, barcode or queue slip processing, order status updates and campus delivery geofence validation. Functional testing was also used to verify major features related to ordering, tracking and delivery. Usability testing was included to determine whether the different user roles could navigate and use the system properly.

Security Used

Security measures were merged into Canteen Express to protect user accounts, system functions and stored information. Role-Based Access Control was used to restrict access to the

system functions according to the user assigned role. Account verification and authentication were also included as part of the account management process.

The system was designed with Django security mechanisms, session management, restricted database access and password protection. The deployed system also used HTTPS/SSL and Supabase PostgreSQL with SSL/TLS to help protect data transmitted between the application and database. Sensitive configuration values were managed through environment variables where applicable. The system security and privacy considerations were also aligned with the Data Privacy Act of 2012 (Republic Act No. 10173) as discussed in the earlier chapters.

For delivery transactions the system also merged campus geofence validation and proof of delivery. The geofence limits delivery operations to the defined campus delivery area while proof of delivery provides a record of the completed handover.

Implementation Plan

After completing the development and testing stages the researchers prepared Canteen Express for implementation. Testing was conducted throughout the development process of the system to verify whether the implemented functions operated according to their intended requirements. Individual functions were first checked before related functions were tested together as part of the complete system. The testing covered important system functions such as account login, checkout, order processing, loyalty point computation, delivery fee computation, barcode or queue slip processing, order status updates and campus delivery geofence validation. Functional Testing was also conducted to verify the major features related to ordering, tracking

and delivery while the usability testing was included to determine whether the different user roles could properly navigate and use the system.

After the testing and necessary revisions the researchers presented the developed system to the research adviser on the 28th day of October 2026. The demonstration covered all the features of the system, these include the dashboards of all the four roles: Customers (Students/Walk-In, Faculty/Staff), Delivery Personnel, Canteen Staff and Admin. During the presentation the adviser provided comments and suggestions concerning the sign-up process. The adviser suggested a “Continue with Google” feature that lets the Faculty and Staff customers continue signing up with their institutional email. The researchers decided to put this feedback to Chapter 6 which contains the recommendations.

After including the adviser recommendations the researcher presented the updated system to Mrs. Janeth G. Dalojo Food Supervisor II of the PalSU Main Canteen on the 28th day of October 2026. The demonstration focused on the functions related to the canteen operations, including the dashboards of all the four roles. Feedback from the stakeholders concerning the separation of the POS page and Kitchen Board was documented and considered in the succeeding revisions to be combined in one page. This activity allowed the researchers to determine whether the developed functions corresponded with the operational requirements identified during the requirements analysis.

Once the required revisions and testing were completed the researchers prepared Canteen Express for deployment using the selected cloud hosting environment. Railway was identified as the deployment platform used while Supabase PostgreSQL served as the primary database environment. The researchers also conducted an orientation for the intended user of the system.

The orientation covered the basic procedures and functions relevant to each user role including customer ordering, order monitoring, canteen operations, delivery functions and administrative functions. Through these implementation activities the researchers prepared the system for actual and user evaluation.

Implementation Result

Following the implementation activities the researchers conducted final checking of the systems major functions. The purpose of this stage was to determine whether the implemented features operated properly and whether the system was ready for User Acceptance Testing (UAT).

During the implementation period Canteen Express recorded an estimated amount of 167 completed test or implementation transactions. These included 147 mobile kiosk orders and 167 pre-orders and 20 delivery transactions if applicable.

The researchers also documented issues encountered during the implementation, specifically instances where One-Time Passwords (OTPs) failed to dispatch via email, periods of server downtime on Railway and overall delayed system response times. These issues were systematically reviewed and resolved through iterative system revisions prior to conducting the final user evaluation.

Result and Discussion

The evaluation result of Canteen Express was conducted through User Acceptance Testing (UAT) among the selected users identified in the study. The evaluation focused on the three criteria stated in the objectives of the study: usability, functionality and efficiency.

The respondents included students and walk-in users (127 college students and 20 senior high students) who were randomly selected, as well as faculty and staff members (composed of 20 randomly selected faculty and staff members) who evaluated the system according to the features available to their respective user accounts. The students and walk-in users assessed the self-service ordering functions, while faculty and staff evaluated features related to account access, loyalty points, campus delivery, order tracking and other available functions.

The results are presented according to the respondent groups and the features evaluated in their respective questionnaires. The statistical results and interpretations are based on the collected responses and the appropriate statistical treatment. The findings are intended to identify the strengths of Canteen Express and the areas that may require further improvement.

Students and Walk-In Users

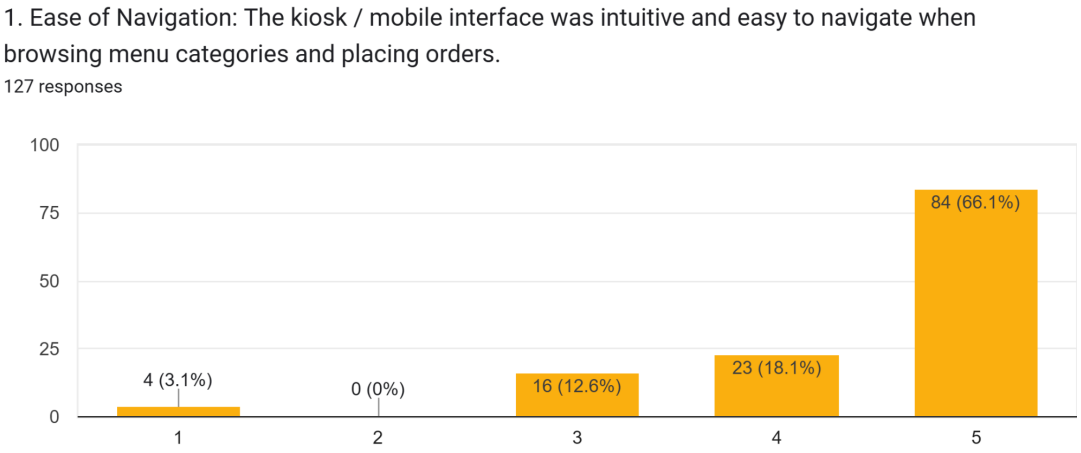
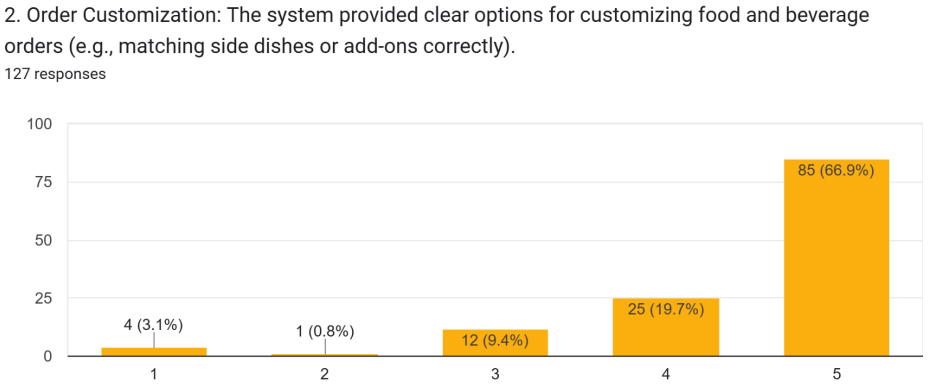


Figure 5.3: Usability and Navigation of Canteen Express

The figure above presents the respondents' assessment of the usability and navigation of Canteen Express. The evaluation focused on whether the mobile kiosk was intuitive and easy to navigate when browsing menu categories and placing orders. Data for this indicator were gathered from online evaluations completed by college students and walk-in users (n=127) as well as physically distributed printed survey questionnaires administered to Senior High School (SHS) students (n=20), yielding a total sample size of 147 respondents.

Based on the combined responses (N=147), 68.0% (n=100) strongly agreed (rating of 5), this comprises 84 online respondents and 17.7% (n=26) agreed (rating of 4) that the mobile kiosk interface was intuitive and easy to navigate. Neutral responses were given by 11.6% (n=17), while 2.7% (n=4) gave a rating of 1 and 0.0% (n=0) gave a rating of 2. The findings indicate that a decisive majority of student and walk-in respondents (85.7% combined positive) found the interface clear, straightforward and accessible across both digital and physical survey modalities. These results provide insight into how students and walk-in users experienced the system's interface when accessing the available menu items and initiating their orders.



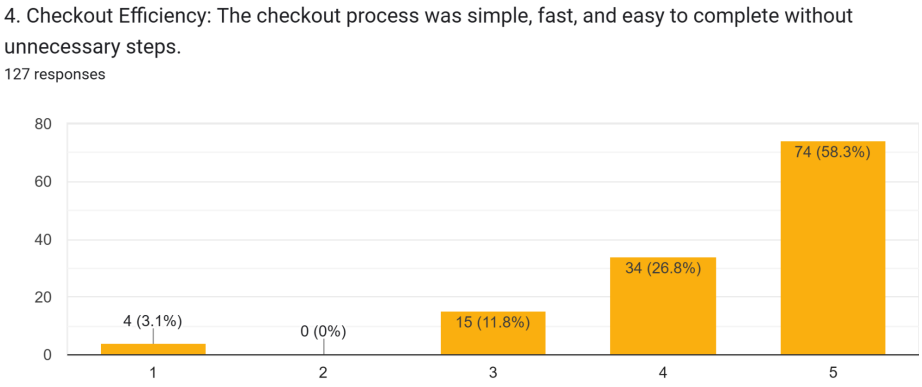
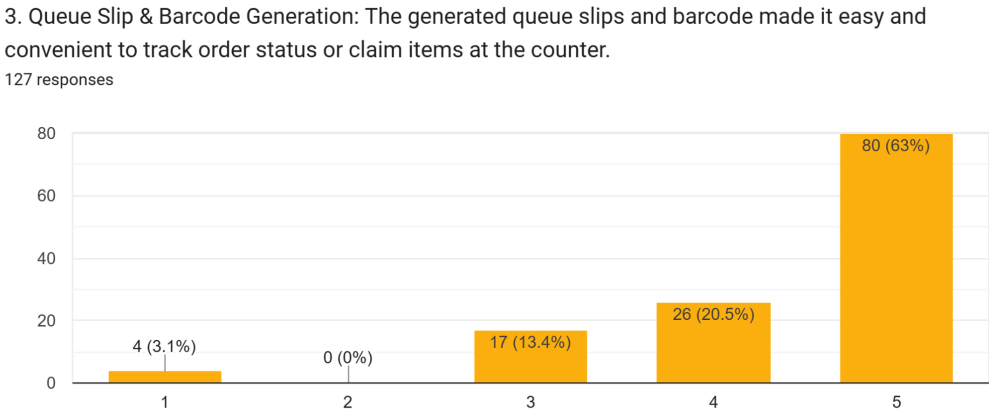


Figure 5.4: Ordering and Checkout Functions of Canteen Express

The figure above presents the respondents' assessment of the ordering functions of Canteen Express. The evaluation covered food and beverage customization, the selection of side dishes and add-ons and the simplicity of the checkout process. Data for these criteria were gathered from online evaluations completed by college students and walk-in users (n=127) as well as physically distributed printed survey questionnaires administered to Senior High School (SHS) students (n=20), yielding a total sample size of 147 respondents.

Based on the collected responses regarding order customization, 67.3% (n=99) strongly agreed (85 online, 14 printed) and 19.0% (n=28) agreed (25 online, 3 printed) that the system provided clear options for customizing food and beverage orders, while 10.2% (n=15) were neutral, (12 online, 3 printed), 0.7% (n=1) rated 2 and 2.7% (n=4) rated 1. Regarding checkout efficiency, 57.1% (n=84) strongly agreed (74 online, 10 printed) and 27.9% (n=41) agreed (34 online, 7 printed) that the checkout process was simple, fast and easy to complete without unnecessary steps, with 12.2% (n=18) remaining neutral (15 online, 3 printed) and 2.7% (n=4) rating 1. The findings indicate that the vast majority of student and walk-in respondents evaluated meal customization (86.4% combined positive) and checkout execution (85.0%

combined positive) as fluid and convenient across both survey modalities. These results help determine whether the ordering process provides users with a clear and convenient way to select their desired meals and complete their transactions.



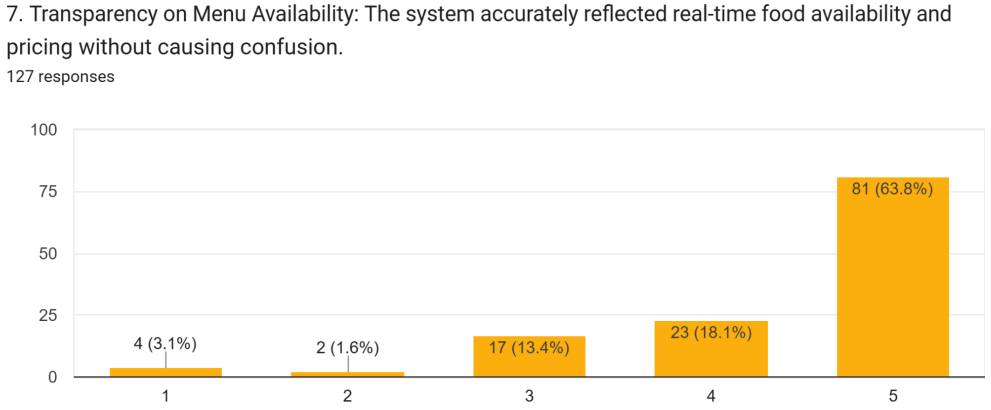
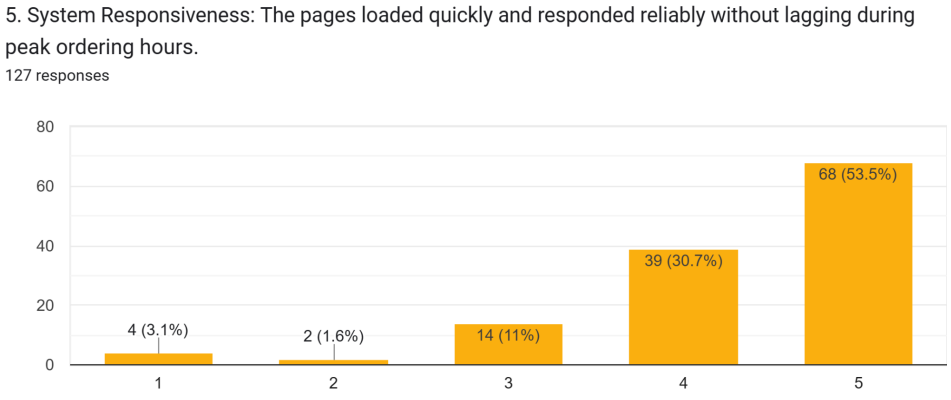


Figure 5.5: Queue Slip, Barcode Generation and Menu Availability

The figure above presents the respondents' assessment of the queue slip and barcode generation features, as well as the accuracy of menu availability and pricing information. These

features are intended to help users identify their orders, claim items at the counter and make informed purchasing decisions based on available menu and prices.

Based on the combined responses ($n = 147$) for queue slip and barcode generation, 62.6% ($n = 92$) strongly agreed (80 online, 12 printed) and 19.7% ($n = 29$) agreed (26 online, 3 printed) that the generated tokens made order status tracking and item claiming easy at the counter, while 15.0% ($n = 22$) responded neutral (17 online, 5 printed) and 2.7% ($n = 4$) rated 1. Furthermore, regarding transparency on menu availability, 63.3% ($n = 93$) strongly agreed (81 online, 12 printed) and 19.0% ($n = 28$) agreed (23 online, 5 printed) that the system accurately reflected real-time food availability and pricing without causing confusion, alongside 13.6% ($n = 20$) neutral (17 online, 3 printed), 1.4% ($n = 2$) rating 2 and 2.7% ($n = 4$) rating 1. The findings indicate that the system reliably communicates live canteen menu availability (82.3% combined positive) and provides accessible digital queue slips (82.3% combined positive) across digital forms and printed surveys. The results help assess whether the system provides clear order identification and reliable information about available food items and their prices.



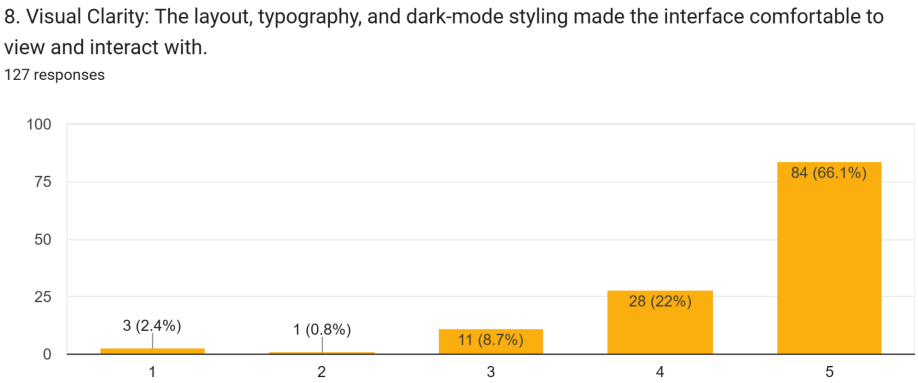


Figure 5.6: System Responsiveness and Visual Clarity

The figure above presents the respondents' assessment of the responsiveness and visual presentation of Canteen Express. The evaluation covered page-loading speed, system reliability during ordering, readability of the interface and the comfort of using the system's layout and dark-mode styling.

Based on the combined responses ($N = 147$) evaluating system responsiveness, 53.7% ($n = 79$) strongly agreed (68 online, 11 printed) and 28.6% ($n = 42$) agreed (39 online, 3 printed) that pages loaded quickly and responded reliably without lagging during peak ordering hours, while 13.6% ($n = 20$) were neutral (14 online, 6 printed), 1.4% ($n = 2$) rated 2 and 2.7% ($n = 4$) rated 1. Regarding visual clarity, 69.4% ($n = 102$) strongly agreed (84 online, 18 printed) and 20.4% ($n = 30$) agreed (28 online, 2 printed) that the layout, typography and dark-mode styling made the interface comfortable to view and interact with, with 7.5% ($n = 11$) neutral (11 online, 0 printed), 0.7% ($n = 1$) rating 2 and 2.0% ($n = 3$) rating 1. The findings indicate that student and walk-in respondents found the dark-mode layout exceptionally clear (89.8% combined positive) and experienced solid system performance and speed (82.3%

combined positive) during evaluations. These results provide information about the system's performance and the ease with which users can view and interact with the interface.

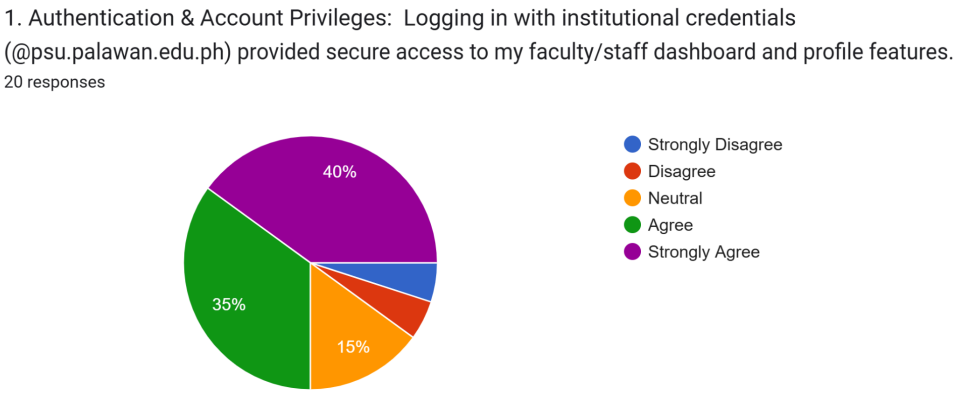
Figure 5.7: Overall Functionality and Recommendation Among Students and Walk-In Users

Figure 5.7 shows the respondents' assessment of the overall functionality of Canteen Express as a self-service ordering tool and their willingness to use the system again for campus canteen transactions.

Based on the combined responses ($N = 147$) regarding overall functionality, 66.0% ($n = 97$) strongly agreed (85 online, 12 printed) and 20.4% ($n = 30$) agreed (24 online, 6 printed) that the system successfully met their expectations as a self-service walk-in ordering tool, with 10.9% ($n = 16$) neutral (14 online, 2 printed), 0.7% ($n = 1$) rating 2 and 2.0% (

$n = 3$) rating 1. On system recommendation, 65.3% ($n = 96$) strongly agreed (83 online, 13 printed) and 23.8% ($n = 35$) agreed (30 online, 5 printed) that they would gladly use the kiosk/web system again, alongside 8.2% ($n = 12$) neutral (10 online, 2 printed) and 2.7% ($n = 4$) rating 1. Additionally, when assessed on understanding walk-in account limitations (Item 6), 50.0% ($n = 10$) of SHS respondents strongly agreed and 25.0% ($n = 5$) agreed that restriction boundaries were clearly conveyed. The findings indicate high user adoption intent and overall system acceptance, as 86.4% affirmed overall functionality and 89.1% expressed willingness to use the platform continuously. The responses provide an overall assessment of whether the system's self-service ordering features meet the expectations of students and walk-in users.

Faculty and Staff Users



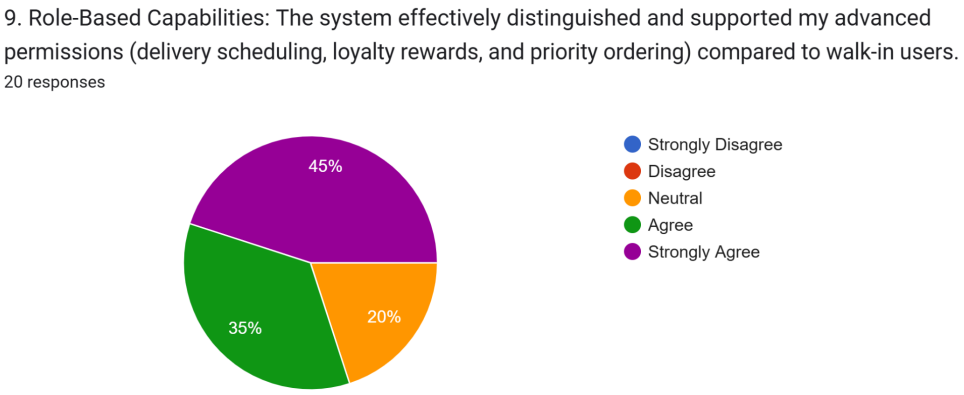


Figure 5.8: Account Access and Role-Based Capabilities

The figure above presents the faculty and staff respondents' assessment of institutional account authentication and role-based system permissions. The evaluation covered access through institutional credentials, profile features and the availability of functions appropriate to their assigned user roles.

Based on the collected responses (n=20) regarding account privileges, 40.0% (n=8) strongly agreed (rating of 5) and 35.0% (n=7) agreed (rating of 4) that logging in with institutional credentials (@psu.palawan.edu.ph) provided secure access to their dashboard and profile features, while 15.0% (n=3) were neutral, 5.0% (n=1) disagreed and 5.0% (n=1) strongly disagreed. Regarding role-based capabilities, 45.0% (n=9) strongly agreed and 35.0% (n=7) agreed that the system effectively distinguished and supported advanced permissions (such as delivery scheduling, loyalty rewards and priority ordering) compared to walk-in users, with 20.0% (n=4) remaining neutral. The findings indicate that a strong majority of faculty and staff evaluated credential security (75.0% combined positive) favorably. These results provide insight into the accessibility and appropriateness of the system features provided to faculty and staff accounts.

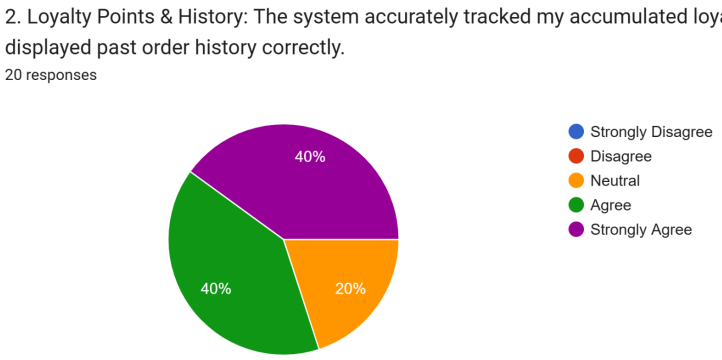


Figure 5.9: Loyalty Points and Order History

The figure above presents the respondents' assessment of the loyalty points and order history features of Canteen Express. The evaluation focused on whether accumulated loyalty points were tracked accurately and whether previous orders were displayed correctly.

Based on the collected responses ($N = 20$), 40.0% ($n = 8$) strongly agreed and 40.0% ($n = 8$) agreed that the system accurately tracked accumulated loyalty points and correctly displayed past order history, while 20.0% ($n = 4$) remained neutral. The findings indicate an overall positive satisfaction rate of 80.0% with zero negative responses, confirming that transaction logging and point balance tracking are dependable. These results help assess the reliability and usefulness of the system's loyalty and order history features for faculty and staff users.

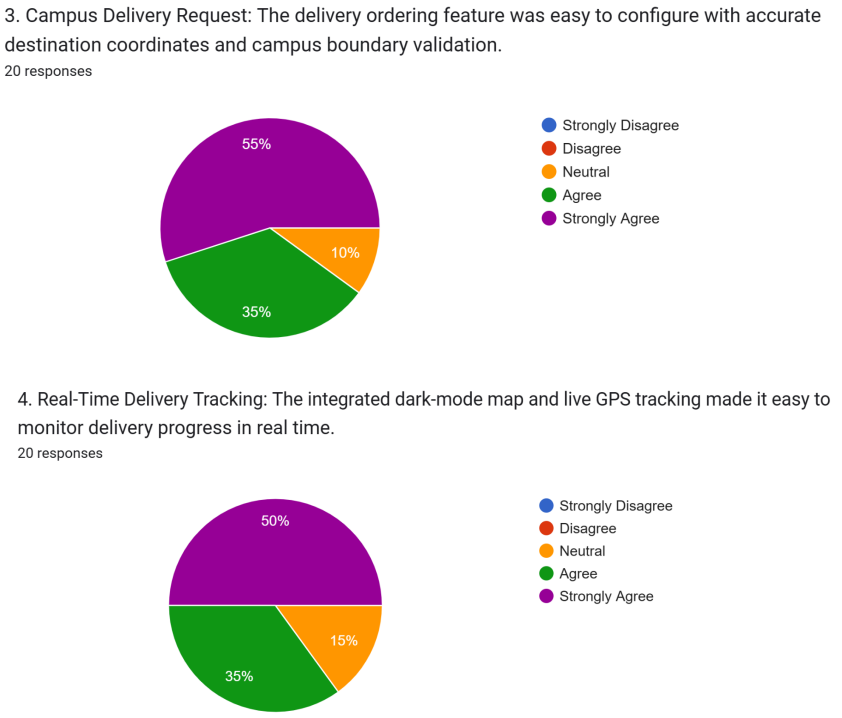


Figure 5.10: Campus Delivery Requests and Real-Time Tracking

The figure above presents the faculty and staff respondents' assessment of the campus delivery functions of Canteen Express. The evaluation covered the configuration of delivery requests, destination coordinates, campus boundary validation and the use of the integrated map and GPS tracking to monitor delivery progress.

Based on the collected responses ($N = 20$) regarding delivery requests, 55.0% ($n = 11$) strongly agreed and 35.0% ($n = 7$) agreed that configuring delivery requests was easy and destination boundaries were validated accurately, while 10.0% ($n = 2$) remained neutral. For real-time delivery tracking, 50.0% ($n = 10$) strongly agreed and 35.0% ($n = 7$) agreed that the integrated dark-mode map and live GPS tracking made monitoring delivery progress easy, alongside 15.0% ($n = 3$) neutral responses. The findings indicate high user satisfaction across both setup (90.0% combined positive) and live tracking (85.0% combined positive). These results provide information about the usability and perceived effectiveness of the system's campus delivery and tracking features.

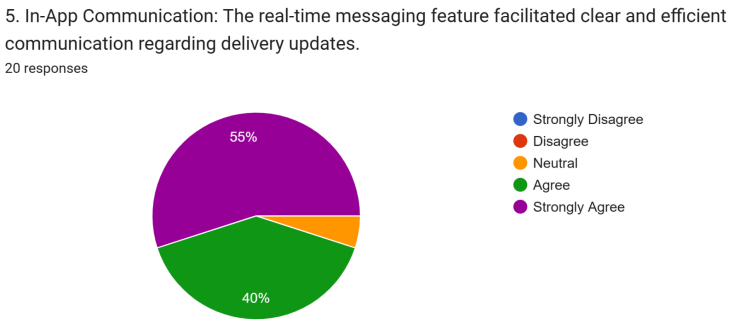


Figure 5.11:In-App Communication for Delivery Updates

The figure above presents the respondents' assessment of the real-time messaging feature of Canteen Express. The evaluation focused on whether the in-app communication function supported clear and efficient communication regarding delivery updates.

Based on the collected responses ($N = 20$), 55.0% ($n = 11$) strongly agreed and 40.0% ($n = 8$) agreed that the real-time messaging feature facilitated clear and efficient communication regarding delivery updates, while 5.0% ($n = 1$) remained neutral. The findings indicate a 95.0% positive evaluation from faculty and staff users with zero dissatisfied responses. These results help determine how respondents experienced the communication feature while using the campus delivery service.

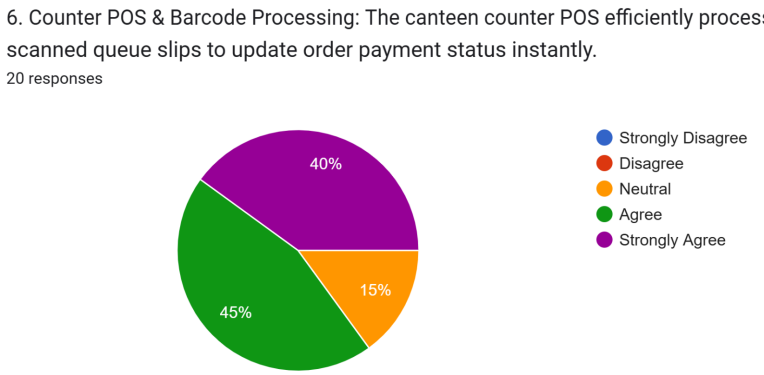


Figure 5.12: Counter POS and Barcode Processing

The figure above presents the faculty and staff respondents' assessment of the counter Point-of-Sale (POS) and barcode processing functions. The evaluation covered the processing of barcodes and the scanning of queue slips to update order payment status.

Based on the collected responses ($N = 20$), 40.0% ($n = 8$) strongly agreed and 45.0% ($n = 9$) agreed that the canteen counter POS efficiently processed barcodes and scanned queue slips to update payment statuses instantly, while 15.0% ($n = 3$) were neutral. The findings indicate that 85.0% of respondents found counter transactions and queue slip processing fast and reliable. These results provide insight into the perceived effectiveness of the system's counter transaction and order identification features.

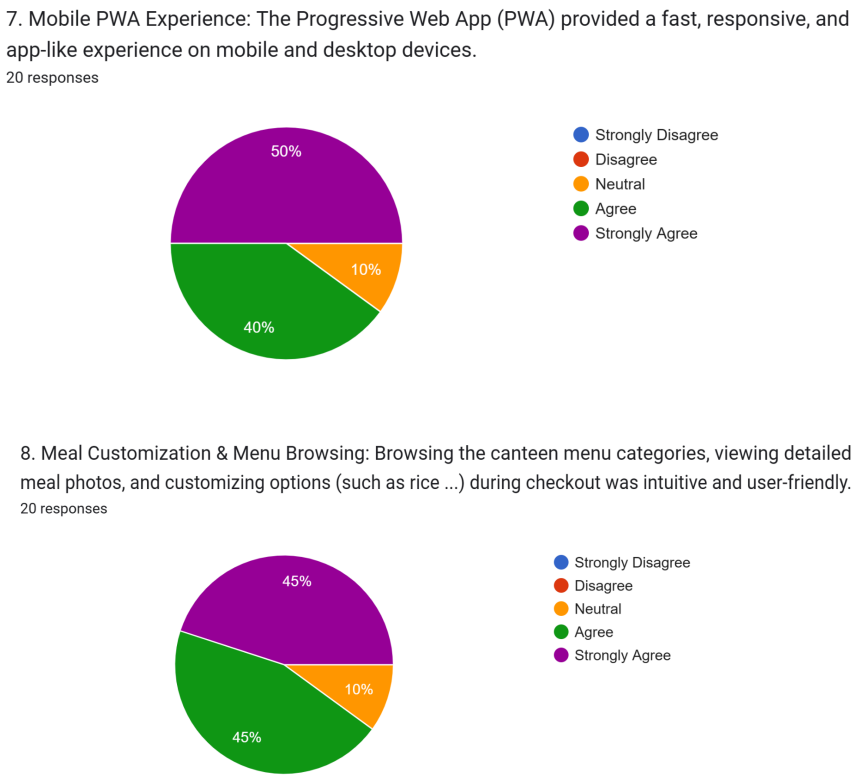


Figure 5.13: Mobile PWA Experience and Menu Browsing

The figure above presents the respondents' assessment of the Progressive Web App (PWA) experience and the customer ordering interface available to faculty and staff. The evaluation covered the responsiveness of the system across mobile and desktop devices, menu browsing, meal photos and customization options such as rice add-ons and beverage choices.

Based on the collected responses ($N = 20$) regarding PWA performance, 50.0% ($n = 10$) strongly agreed and 40.0% ($n = 8$) agreed that the app provided a fast, responsive and app-like experience on mobile and desktop devices, with 10.0% ($n = 2$) neutral. Similarly, for menu browsing and customization, 45.0% ($n = 9$) strongly agreed and 45.0% ($n = 9$) agreed that browsing categories, viewing detailed meal photos and selecting add-ons were intuitive and

user-friendly, with 10.0% ($n = 2$) neutral. The findings indicate high approval for both cross-platform performance (90.0% combined positive) and menu browsing usability (90.0% combined positive). These results help assess the accessibility and ease of using the system's ordering interface across supported devices.

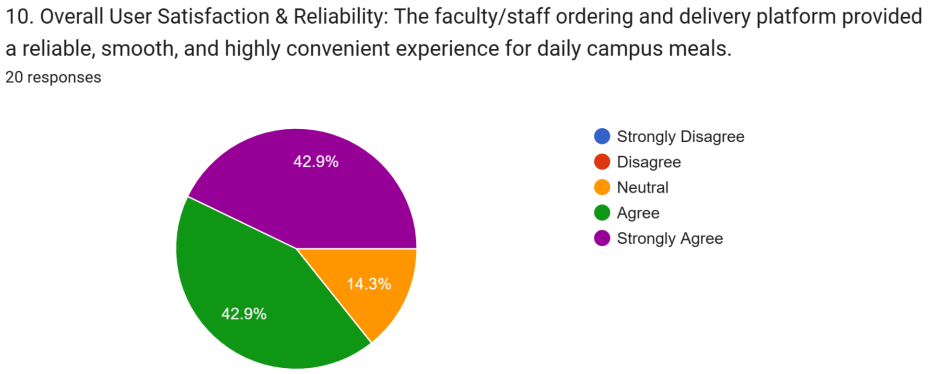


Figure 5.14: Overall User Satisfaction and Reliability Among Faculty and Staff

The figure above presents the faculty and staff respondents' overall assessment of the ordering and delivery platform. The evaluation focused on the reliability, smoothness and convenience of using Canteen Express for campus meal transactions.

Based on the collected responses ($N = 20$), 42.9% ($n = 9$) strongly agreed and 42.9% ($n = 8$) agreed that the faculty/staff ordering and delivery platform provided a reliable, smooth and highly convenient experience for daily campus meals, while 14.3% ($n = 3$) remained neutral. The findings indicate an 85.8% combined positive rating with zero dissatisfied responses. These results summarize the respondents' overall experience with the ordering and delivery features available to faculty and staff.

Overall Evaluation of Canteen Express

The figure above presents the overall evaluation results of Canteen Express based on the responses gathered during the User Acceptance Testing (UAT). Across all participating respondent groups, comprising student and walk-in users (n=147) as well faculty and staff members (n=20) the system achieved a combined positive satisfaction rating of 87.2% (with 87.8% positive feedback among students/walk-in users and 85.8% among faculty and staff). The overall results successfully exceeded the 80.0% target satisfaction rating established for the study. This indicates high user acceptance and demonstrates that the self-service ordering, menu browsing, counter POS processing and campus delivery capabilities effectively met user expectations.

Based on the results of the development, testing, implementation and User Acceptance Testing, Canteen Express was developed to provide an organized web/mobile-based application ordering and delivery platform for the Palawan State University Main Canteen. The development process incorporated the system requirements identified during the initial data gathering and requirements analysis. The resulting system included customer ordering functions, canteen management tools, delivery management features, administrative functions, loyalty and voucher features and supporting functions such as order tracking, menu stocks monitoring, GPS-assisted delivery and reporting.

The system was tested throughout the development process to verify its major functions and identify possible issues. The results of the testing were used to correct and revise the system before it proceeded to implementation and user evaluation. The UAT results provided additional information regarding how the intended users experienced and evaluated the system.

Based on the actual findings, the strongest performing areas included visual clarity and interface presentation (89.8% positive among students, 90.0% positive among faculty/staff), in-app delivery communication (95.0% positive), overall user recommendation (89.1% of students willing to reuse) and campus delivery request configuration (90.0% positive). The respondents also identified slight latency or lagging during peak ordering hours (rated neutral or lower by 17.7% of students) and account privilege clarity (rated neutral or lower by 25.0% of faculty/staff) as minor concerns, which may be considered for further refinement of system infrastructure and user onboarding.

Overall, the results of the development, testing, implementation and User Acceptance Testing provide the basis for determining the extent to which Canteen Express achieved its intended objectives and addressed the identified ordering and operational needs of the Palawan State University Main Canteen.

CHAPTER 6

CONCLUSION AND RECOMMENDATIONS

Conclusion

The Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff and Faculty at Palawan State University Main Canteen was developed to provide a more organized and convenient way of ordering meals within the Palawan State University Main Campus. The system connects Customer, Delivery Personnel, Canteen Staff and Administrator through role-based functions that support ordering, order processing, delivery, menu stocks monitoring, account management and administrative oversight. Through its different features the system was designed to address common concerns related to long queues, manual order processing, order monitoring and on-campus food delivery.

Based on the development, testing, implementation and User Acceptance Testing conducted for the study the following objectives were addressed;

For the first objective which was to develop a responsive web/mobile-based ordering application that enables Student/Walk-In and Faculty/Staff customers to browse available menu items, view recorded nutritional information, select food items and submit orders, while providing account-free counter ordering for Student/Walk-In customers and institutional email and OTP verification for Faculty/Staff customers, the system successfully provided comprehensive menu browsing, nutritional display, meal customization and checkout functions tailored to these designated user roles.

For the second objective, which focused on providing additional ordering options for authenticated Faculty/Staff customers by enabling advance ordering, counter pickup and eligible

on-campus delivery, with cash payment at the counter for pickup or delivery orders, the system successfully implemented the Faculty/Staff ordering workflow. This enabled eligible users to schedule advance orders, choose between counter pickup or on-campus delivery and settle transactions using cash payment methods.

The third objective, which was to develop an order-processing and kitchen-management interface that allows Canteen Staff to retrieve and verify orders, process cash payments through the Counter POS and update order statuses through the Kitchen Board during preparation and fulfillment, the system provided dedicated functions for canteen staff. These tools supported the processing of cash transactions, barcode and order number verification and real-time order monitoring from receipt through final fulfillment.

The fourth objective, which addressed providing delivery-management functions that allow authorized Delivery Personnel to view and accept eligible requests, access GPS location and route information, update delivery statuses, communicate with customers, submit proof-of-delivery photos and review earnings associated with completed deliveries, the system provided the Rider Hub. Through this module, delivery personnel managed assigned orders, utilized GPS route guidance, communicated via in-app chat, updated delivery progress, submitted delivery photos and tracked earnings within an established campus geofence.

The fifth objective, which was to develop administrative functions allowing the Administrator to manage user accounts, menu items, prices, nutritional information, discount vouchers, system settings, reports and analytics based on recorded system data, the system provided comprehensive administrative controls. Through these tools, administrators managed user credentials, configured menu pricing and nutritional data, monitored menu stocks levels,

administered discount vouchers, adjusted operational settings and generated system-wide analytical reports.

The sixth objective focused on providing a loyalty rewards feature for eligible Faculty/Staff customers that awards one point for every ₱100 spent on eligible purchases and allows accumulated points to be redeemed for available discount vouchers according to the system's rules; the system successfully integrated this points-based rewards structure. Through this feature, eligible Faculty and Staff customers accumulated points based on their spending and redeemed them for discount vouchers.

For the seventh objective, regarding the customer evaluation, the system underwent User Acceptance Testing (UAT) among selected Palawan State University students/walk-in users, faculty and staff members based on software quality characteristics from ISO/IEC 25010 and the System Usability Scale (SUS). Canteen Express achieved an overall positive satisfaction rating of 87.2% across all customer respondent groups, comprising an 87.8% positive rating among student and walk-in users (n=147) and an 85.8% positive rating among faculty and staff members (n=20). These results successfully exceeded the 80.0% target satisfaction rating established for the study. Customer respondents highlighted visual clarity and interface presentation (89.8% positive among students, 90.0% positive among faculty/staff), in-app delivery communication (95.0% positive), overall user adoption intent (89.1% of students willing to reuse) and campus delivery request configuration (90.0% positive) as key system strengths.

Continuing under the seventh objective, the technical evaluation conducted by IT experts assessed the system's underlying architecture, security protocols and interface execution. The experts noted key technical concerns during stress testing: first, regarding Security, the experts

identified that sensitive server credentials and debug outputs were temporarily exposed during unhandled system crashes, alongside accessible Django admin views that should be strictly restricted to system administrators; secondly, regarding UI/UX and Server Performance, server response times exhibited latency under high concurrent user loads and minor UI layout responsiveness inconsistencies occurred across certain non-standard screen resolutions. Despite these technical observations, the IT experts concluded that the overall functionality and operational architecture of the system are strong and all identified security vulnerabilities, access controls, server latency issues and responsive display discrepancies were successfully addressed and rectified prior to final deployment.

Overall the Canteen Express was developed as a campus-based digital platform that brings together customer ordering, canteen operations, delivery management and administrative monitoring in one system. The development and testing activities allowed the researchers to identify and address issues during the development process while the implementation and UAT provided an opportunity to evaluate the system using its intended functions and user roles. The findings of the study will serve as a basis for determining the extent to which the system achieved its intended objectives and addressed the ordering and operational needs of the Palawan State University Main Canteen.

Recommendations

Based on the development and evaluation of Canteen Express future researchers may consider further improving the system by expanding its functions and addressing features that were limited by the scope, resource and development period of the current study. One possible improvement is the integration of online payment options. The current system was designed

around cash-based transactions while future versions may consider supporting digital payments methods to provide users with additional payment options and reduce the need for cash transactions.

One primary recommendation is the integration of OAuth-based single sign-on (SSO), specifically a ‘Continue with Google” feature using institutional Google Workspace accounts (@psu.palawan.edu.ph). While the current version utilizes manual credential-based login, implementing Google SSO would streamline the authentication process, eliminate login friction, enhance security compliance and ensure seamless account creation for faculty and staff.

To optimize system performance and prevent server latency during high-volume periods, future researchers should consider upgrading to a high-concurrency cloud hosting environment. While the current platform was deployed on a standard web hosting server, scaling to an enterprise-grade, containerized cloud infrastructure or serverless edge platform would support automated load balancing and dynamic horizontal scaling. Such platforms can automatically allocate server resources in response to traffic spikes, ensuring that the application can efficiently process hundreds of simultaneous database read and write actions, such as concurrent order placements, real-time menu stock updates and live GPS tracking without experiencing page loading delays or server downtime during peak canteen rush hours.

Another possible improvement is the development of a native mobile application for Android and iOS devices. Although Canteen Express was developed as a responsive web/mobile-based platform, a dedicated mobile application may provide additional device specifications capabilities and a more specialized mobile experience for users.

Future researchers may also consider expanding the delivery management features of the system. Additional delivery-related functions may be developed to improve assignment, monitoring, communication and delivery coordination, Further improvements to GPS and location-based functions may also be considered to provide more detailed delivery guidance within the campus.

The loyalty and discount voucher system may also be expanded in future versions. Additional rewards options, promotional campaigns and more flexible voucher management may be considered to provide more ways for users to use their accumulated points. The system may also be enhanced to provide more detailed monitoring of loyalty transactions and reward usage.

Another recommendation is to further improve the analytics and reporting functions. Future versions may provide more detailed reports on sales (that follows the actual format of the main canteen's report of sales), orders, deliveries, menu stock management, customer activity and other operational information. Additional reporting options may help authorized personnel analyze system data and monitor the performance of the canteen more effectively.

Future researchers may also consider expanding the feedback and dispute management functions. Additional tools for categorizing concerns, monitoring their resolution and reviewing customer feedback may help provide a more organized process for handling complaints and issues related to orders, pickup and delivery.

Lastly future researchers are encouraged to conduct additional testing with a wider range of users and over a longer implementation period. This may provide more information about the system performance during different operating conditions particularly during peak hours when

the volume of order and customers may increase. Additional testing may also help identify further improvements that can be incorporated into future versions of Canteen Express.

In conclusion, Canteen Express provides a foundation for a campus-based pre-order and delivery system that integrates customer ordering, canteen operations, delivery management and administrative functions. Future improvements and additional features may further expand the capabilities of the system and support its continued development according to the changing needs of the Palawan State University Main Canteen and its users.

References

Anino Jr., C. T., Comillas, G. J. C., & Malabo, K. Q. (2019). Food ordering system with non-primary ingredients customization. *Advancing Information Technology Research*, 1(1), 45–67.https://asianscientificjournal.com/new/publication/index.php/aitr/article/view/1658

Awang, N. A., Manaf, M., Ahmad, M., Arifin, N., & Wan Ismail, W. S. A. (2025). *Development of an integrated mobile application for food ordering and delivery. Journal of Mathematics and Computing Science,* 11(2), 1–16. https://journal.uitm.edu.my/ojs/index.php/JMCS/article/view/9550/5289

Buenaventura, R. U., Ignacio, A. E., & Laspoña, J. A. S. (2021). *Mobile ordering application for a generic fast food restaurant. International Journal of Multidisciplinary: Applied Business and Education Research, 2*(5), 371–380. http://dx.doi.org/10.11594/ijmaber.02.05.01

Lambino, J., & Murcia, J. V. (2025). *A path analysis of factors that influence acceptance of online food ordering systems among Davao City customers. Business and Organization Studies e-Journal, 3*(1), 1–15. https://ieesjournals.com/index.php/bosej/article/view/221/184

Mollayev, B., & Atabayeva, B. (2024). *Oguz canteen app: Revolutionizing campus dining. International Scientific Journal "Innovative Science*," (12-2-1), 57–59. https://cyberleninka.ru/article/n/oguz-canteen-app-revolutionizing-campus-dining/viewer

Dhruvkumar, V. D., & Thakare, S. W. (2025). FoodGet: College campus food delivery system. *International Journal of Innovative Research in Computer and Communication Engineering*, 13(3). https://www.ijircce.com

Bari, P., Patil, Y., Sathawane, N., Suroshe, T., & Patil, A. (2025). Deploying an intelligent online food ordering system to optimize college canteen operations. *Proceedings of the 3rd International Conference on Futuristic Technology (INCOFT)*, 3, 263–268. https://doi.org/10.5220/0013613700004664

Grubhub. (2021). *How the University of Notre Dame uses Grubhub to provide a comprehensive dining experience*. Grubhub Onsite Blog. https://onsite.grubhub.com/blog/how-the-university-of-notre-dame-uses-grubhub-to-provide-a-comprehensive-dining-experience/

Notre Dame Campus Dining. (2024). *Mobile ordering*. University of Notre Dame. https://dining.nd.edu/about/service-details/delivery-and-mobile-ordering/

Ohio State University Dining Services. (n.d.). *Grubhub campus dining guide*. The Ohio State University. https://dining.osu.edu/documents/GrubHub%20Campus%20Dining%20Flyer.pdf

APPENDICES

APPENDIX A: USER GUIDE

1. INTRODUCTION

Canteen Express is a web/mobile based ordering and delivery system developed for the Palawan State University (PalSU) Main Canteen. It allows customers to browse available food items, place orders, select a fulfillment method and monitor order progress. The system also provides operational tools for Canteen Staff, Delivery Personnel and the Administrator.

This user manual explains the system's main functions and provides guidance on how each user role can access and use its available features.

2. SYSTEM OVERVIEW

Canteen Express provides the following core functionalities:

1. **User Authentication and Role-Based Access** - Provides access to system functions according to the user's assigned role.
2. **Menu Browsing and Ordering** - Allows customers to view available food items, customize meals and add-ons and place orders.
3. **Pickup and On-Campus Delivery** - Allows customers to select an available fulfillment method, subject to the system's delivery restrictions.
4. **Order Status Tracking** - Displays order progress based on status updates from Canteen Staff and Delivery Personnel.
5. **Counter POS and Barcode Processing** - Allows Canteen Staff to scan queue slips and record cash payment confirmation.

6. **Kitchen Board** - Allows Canteen Staff to update order preparation statuses.
7. **GPS Navigation and Delivery Tracking** - Provides map-based routing and location information for on-campus deliveries.
8. **In-App Chat** - Supports communication between customers and Delivery Personnel during delivery.
9. **Loyalty Points and Feedback** - Supports loyalty point accumulation and customer ratings and feedback.
10. **Administrative Management** - Provides functions for managing accounts, menu items, promotions, reports, analytics and activity records.

3. ACCESSING THE APPLICATION

3.1 Opening Canteen Express

1. Open a compatible web browser on your device.
2. Navigate to the Canteen Express application using the URL provided by the project administrator.
3. Open the appropriate user interface or login page.
4. Log in if the selected feature requires an authenticated account.

3.2 Customer Access

1. Faculty/Staff Customers: Log in using their registered account credentials to access account-base ordering and delivery functions.
2. Student/Walk-In Customers: Access the mobile kiosk interface to browse available menu items and place orders, subject to the system's access requirements.

4. Customer User Manual (Faculty and Staff)

4.1 Role Selection and Logging In

1. Launch the Canteen Express web application on your device.
2. Choose your role “Faculty/Staff”.
3. Allow location (for on-campus delivery).
4. Enter your registered institutional email address and password.
5. Submit your login credentials.
6. Once authenticated, access the functions available to your account.
7. If you don’t have an account, click “Sign Up” to register an account.

4.2 Browsing the Menu

1. The menu is shown.
2. Browse the available food items.
3. Use the search or category filters to locate specific items and to know if they are available.
4. Review item descriptions, prices and stock availability.
5. Select a food item to proceed with ordering.

4.3 Customizing Meals and Add-ons

1. Select a food item from the menu.
2. Choose the available customization options or add ons.
3. Review your selections.
4. Add the item to your cart.

4.4 Managing the Cart and Checkout

1. Open your cart to review the selected items.
2. Check the item quantities and order total.
3. Make any necessary changes to the cart.

4. Proceed to checkout.
5. Select the available fulfillment method.

4.5 Selecting a Fulfillment Method

Two fulfillment methods:

1. Pickup: The customer collects the order at the canteen.
2. On-Campus Delivery: The order is delivered to an eligible location within the system's designated delivery boundary.

Delivery checkout requires a valid destination within the configured geofence. Orders with missing destination coordinates or destinations outside campus boundary are rejected.

4.6 Queue Slips and Barcodes

After a customer placed an order, the system generates a digital queue slip containing order information and a barcode, depending on the implemented display.

1. Open the generated queue slip.
2. Review the order details.
3. Keep the slip available for the relevant transaction.
4. For applicable counter transactions, present the slip to Canteen Staff for payment processing.

The queue slip helps staff identify and process the corresponding order.

4.7 Cash Payments

Canteen Express supports cash payments only.

- Pickup orders: Pay at the canteen counter.
- Delivery orders: Follow the applicable cash payment procedure for the delivery transaction.

The Counter POS records payment confirmation and updates the order's processing status as implemented in the system.

4.8 Viewing Order Status

1. Open the order dashboard or order history.
2. Select the relevant order.
3. Review its current status.

4.9 Tracking Delivery

For eligible delivery orders:

1. Open the relevant order or delivery tracking view.
2. View the available map and delivery route information.
3. Check the delivery personnel's available location information and delivery progress.
4. Use the in-app chat if communication with the assigned delivery personnel is necessary.

GPS information and routing depend on device location services, internet connectivity and the availability of the mapping and routing services.

4.10 In-App Chat

1. Open the relevant delivery order.
2. Access chat interface.
3. Send a message to the assigned Delivery Personnel.
4. Review incoming messages in the conversation.

The chat is intended to support communication related to the delivery.

4.11 Loyalty Points

Canteen Express includes a loyalty points function.

1. Place an order through the system.

2. Complete the order according to the applicable process.
3. Check the account or loyalty section for the recorded points, where available.
4. Each (1) point is accumulated when a ₱100 purchase is made. ₱100 = 1 Point.

4.12 Ratings and Feedback

1. Open a completed order.
2. Access the rating or feedback function.
3. Submit your rating and comments.
4. Confirm the submission.

5. Customer User Manual (Students and Walk-In)

5.1 Role Selection

1. Launch the Canteen Express web application on your Mobile Kiosk device.
2. Choose your applicable user role on the welcome screen:
    - Student/Walk-In: Select for direct ordering, Dine-In and Pickup services.

5.2 Starting the Ordering Session

1. View the onboarding splash screen displaying ordering instructions and kiosk details.
2. Click or tap GET STARTED to begin browsing the menu.
3. If you need to switch your account type, select CHANGE ROLE/SELECT ACCOUNT.

5.3 Selecting Dining Mode

1. When prompted by the “How will you be eating today?”modal, select your dining preference:
    - Dine-In: Choose if you plan to eat inside the canteen.
    - Take-Out: Choose if you require your meal packaged for takeaway.

5.4 Browsing and Filtering the Menu

1. Browse categories on the left navigation sidebar (categories: All Items, Ulam, Rice Meal, Pastry, Pasta, Kakanin, etc).
2. Use the top action bar to search for specific food items or view order history and settings.
3. View available item names, descriptions and prices in Philippine Pesos (₱).

5.5 Item Customization and Adding to Cart

1. Tap on an individual food item to view detailed information.
2. Check nutritional information and feature badges such as Halal Friendly, High Protein, or Warm Meal, etc.
3. Adjust item quantity using the - and + buttons.
4. Tap ADD to place the item into your current order cart.

5.6 Managing the Shopping Cart

1. Review the selected items, individual prices and adjusted quantities inside YOUR CART.
2. Confirm your chosen dining mode and review the total order amount.
3. Click + ADD MORE to return to the menu, or select DONE/PAY to finalize your order.

5.7 Generating and Handling the Queue Slip

1. Review your generated Official Mobile Kiosk Queue Slip containing your assigned Queue Number (example: CE-1763), order summary, total amount and validation expiration window.
2. Present the displayed barcode or queue number at the canteen payment counter to complete counter payment within the specified time limit.
3. Choose from the available slip actions:

   DOWNLOAD: Save an electronic copy of your queue slip.

EDIT ORDER: Return to modify items in your order.

CANCEL ORDER: Cancel the created queue slip.

CLOSE: Dismiss the popup while keeping your queue active.

5.8 Cancelling a Queue Slip

1. Tap CANCEL ORDER on the active queue slip.
2. Confirm cancellation when prompted by the system warning modal.
3. Select YES, CANCEL to clear your cart and invalidate the current queue slip, or tap KEEP SLIP to retain it.

5.9 Viewing Order History and Activity

1. Tap the History icon in the main navigation bar to open Order History and Activity.
2. View pending and past orders, including queue numbers, status indicators (example: UNPAID), payment deadlines and timestamps.

5.10 Submitting Rating and Feedback

1. Once an order is completed, open the rating modal.
2. Select a star rating (1 to 5 stars) corresponding to your dining experience.
3. Optionally enter text comments or feedback in the provided text area.
4. Tap SUBMIT to send your feedback, or select SKIP to bypass.

5.11 Downloading Payment Receipt

1. Upon counter payment completion, view your Official Payment Receipt.
2. Confirm your order queue number, payment status badge (PAID), itemized order details and feedback confirmation message.
3. Tap DOWNLOAD RECEIPT to store a copy on your device, or select CLOSE to return to the main interface.

6. Canteen Staff User Manual

Canteen Staff use the system to process counter transactions and manage the preparation of customer orders.

6.1 Accessing the Staff Interface

1. Open the designated Canteen Staff interface.
2. Log in using the authorized account credentials.
3. Open the relevant operational module, such as Counter POS or Kitchen Board.

6.2 Using the Counter POS

The counter POS supports queue-slip number inputting and cash payment processing.

1. Open the Counter POS interface from the Canteen Staff/Admin dashboard.
2. Locate the customer's queue slip.
3. Input the order number using the available input order number function.
4. Check the retrieved order information.
5. Confirm the cash payment according to the canteen's procedure.
6. Verify that the system updates the order's payment and processing status.

The implemented Counter POS process workflow transitions an order from unpaid to pending when payment is confirmed and triggers the corresponding kitchen notification.

6.3 Using the Kitchen Board

The Kitchen Board uses a Kanban workflow to organize orders by preparation status.

The available order statuses are:

- **Pending -** the order is waiting for preparation.
- **Preparing** - order is being prepared.
- **Ready** - the order is ready for pickup or the next fulfillment step.

- **Completed** - the order has reached the completed status.

To update an order:

1. Open the Kitchen Board.
2. Locate the order in its current status column.
3. Select the appropriate status update.
4. Confirm that the order appears under the correct status.

Staff should update statuses according to the actual progress of the order.

7. Delivery Personnel User Manual

Delivery Personnel uses the Rider Hub as a delivery dashboard to manage delivery requests and complete assigned deliveries.

7.1 Accessing the Rider Hub

1. Open the delivery dashboard.
2. Log in using the authorized account credentials.
3. Review the available delivery requests.

7.2 Viewing and Accepting Delivery Requests

1. Open the available delivery requests.
2. Review the order and destination details.
3. Select a delivery request.
4. Accept or reject the request using the available controls.

The system allows Delivery Personnel to accept available requests, it does not automatically assign every delivery to a delivery personnel.

7.3 Viewing Delivery Routes

1. Open an accepted delivery order.

2. Access the map and route guidance.
3. Review the route to the destination.
4. Follow the displayed route information while completing the delivery.

Route guidance uses the available mapping and routing services. Actual navigation may depend on GPS accuracy, internet connectivity and service availability.

7.4 Sharing GPS Location

When location access is enabled and the relevant tracking function is active, the system can share the rider's location with the delivery tracking interface.

1. Allow the required location permission on the device.
2. Open the relevant delivery.
3. Activate or use the available location-sharing function.
4. Verify that location information is displayed as expected.

7.5 Communicating with Customers

1. Open the accepted delivery order.
2. Access the in-app chat.
3. Send messages related to the delivery as needed.
4. Review customer replies.

7.6 Completing a Delivery

1. Navigate to the designated delivery destination.
2. Hand over the order to the customer.
3. Update the delivery status using the available completion control.
4. Take a photo as proof of delivery and mark the order as delivered.

Delivery status should be updated after the corresponding delivery action has actually occurred.

7.7 Viewing Delivery History and Password Change

Delivery Personnel can access the available profile and dashboard functions to review account information and order history.

Password change:

1. Open the dashboard interface.
2. Look for the “key” icon.
3. Type old password, input new password and confirm new password.
4. Save changes by tapping the “Update Password”, or tap “Cancel” if the user has a change of mind.

8. Administrator User Manual

The Administrator oversees system-wide management and monitoring functions.

8.1 Logging In

1. Open the Administrator login page.
2. Enter the authorized credentials.
3. Submit the login form.
4. Access the administrative dashboard.

8.2 Navigating the Dashboard Overview

1. Navigate to Dashboard Overview on the left navigation sidebar.
2. Click Open Kitchen Board at the top right to launch the real-time order tracking display in a new view.
3. Manage operating status under Canteen Status by selecting Automatic, Open Now, or entering a Closing reason and clicking Close.

4. View quick live metrics including Orders Today, Pending Orders, In Kitchen and Ready for Pickup.
5. Monitor live incoming transactions directly inside the Active Orders Queue list and click Refresh Feed to update real-time records.

8.3 Managing Menu Stock

1. Click Menu Management from the sidebar menu.
2. Use the Filter Category dropdown or Search menu items bar and click Search to locate items.
3. Review the Current Menu Items table displaying item thumbnails, names, categories, unit prices, stock levels and toggle statuses.
4. Click Edit beside any entry to modify dish details, or click Delete to remove an item.
5. Use the Availability toggle switch to instantly enable or disable an item on the active menu.

8.4 Managing Food Categories

1. Navigate to Menu Management and click the Categories button.
2. In the Manage Categories modal window, fill out Category Name and optional Description under Add New Category.
3. Click + Add Category to save the new section.
4. Scroll through Existing Categories to view registered food classifications.
5. Click Edit to modify a category name or Delete to remove an existing category.

8.5 Adding New Menu Items

1. Click Menu Management on the sidebar, then select + Add New Item.

2. Input the Item Name, Price (₱) and Initial Stock in their designated input fields.
3. Select the appropriate group from the Category dropdown menu.
4. Provide an item image by pasting an Image URL (optional) or clicking Choose File under or Upload Image File.
5. Type an optional product overview under Description (or leave empty to auto-generate), then click Save Item.

8.6 User Account Management

1. Click User Management on the left sidebar navigation.
2. View the table of registered accounts containing Username, Full Name, Email, Role and current Status.
3. Locate a specific student, faculty, or staff profile from the list.
4. Click Restrict under the Actions column to temporarily limit account privileges.
5. Click Ban to completely revoke account access when necessary.

8.7 Managing Delivery Personnel and Campus Dispatch

1. Click Delivery Personnel on the left menu sidebar.
2. Review Registered Riders/Delivery Personnel to monitor personnel statuses (Offline or Online-Ready).
3. Use the Deactivate, Hold, or Penalize buttons under Actions to manage rider status.
4. Track active deliveries under Live Delivery Dispatches, showing Delivery ID, Order Number, Location, Assigned Rider, Proof Photo and Status.

8.8 Creating Delivery Personnel Accounts

1. Click Delivery Personnel on the sidebar and press + Add Delivery Personnel to register a new rider account.
2. Enter the new personnel's Username and Full Name.
3. Provide an Email Address (Optional) or leave empty for auto-generation.
4. Input a secure Password (click the eye icon to toggle visibility).
5. Enter the personnel's Phone Number and Vehicle/Plate Number, then click Create Account.

8.9 Monitoring Customer Feedback and Ratings

1. Select Feedback and Ratings from the sidebar menu.
2. Check top summary metrics including Total Reviews, Overall Avg, Mobile Kiosk score and Delivery Score.
3. Review ratings separated by channel between Mobile Kiosk/Counter Pick-Up and Delivery.
4. Inspect detailed feedback listed under Mobile Kiosk Reviews for in-person pickups.
5. Inspect feedback under Delivery Reviews to evaluate completed rider drop-offs.

8.10 Viewing Sales Analytics and Reports

1. Open Reports and Sales from the left navigation panel.
2. Review available sales cards for Overall Sales Today, Overall Sales This Week and Overall Sales This Month.
3. Check Daily Sales and Order Trend (Last 7 Days) charts comparing revenue bar charts and order volume lines.
4. Switch chart timelines using the Per Day, Weekly, or Monthly view toggles.
5. Click the Download dropdown button to select and export report files.

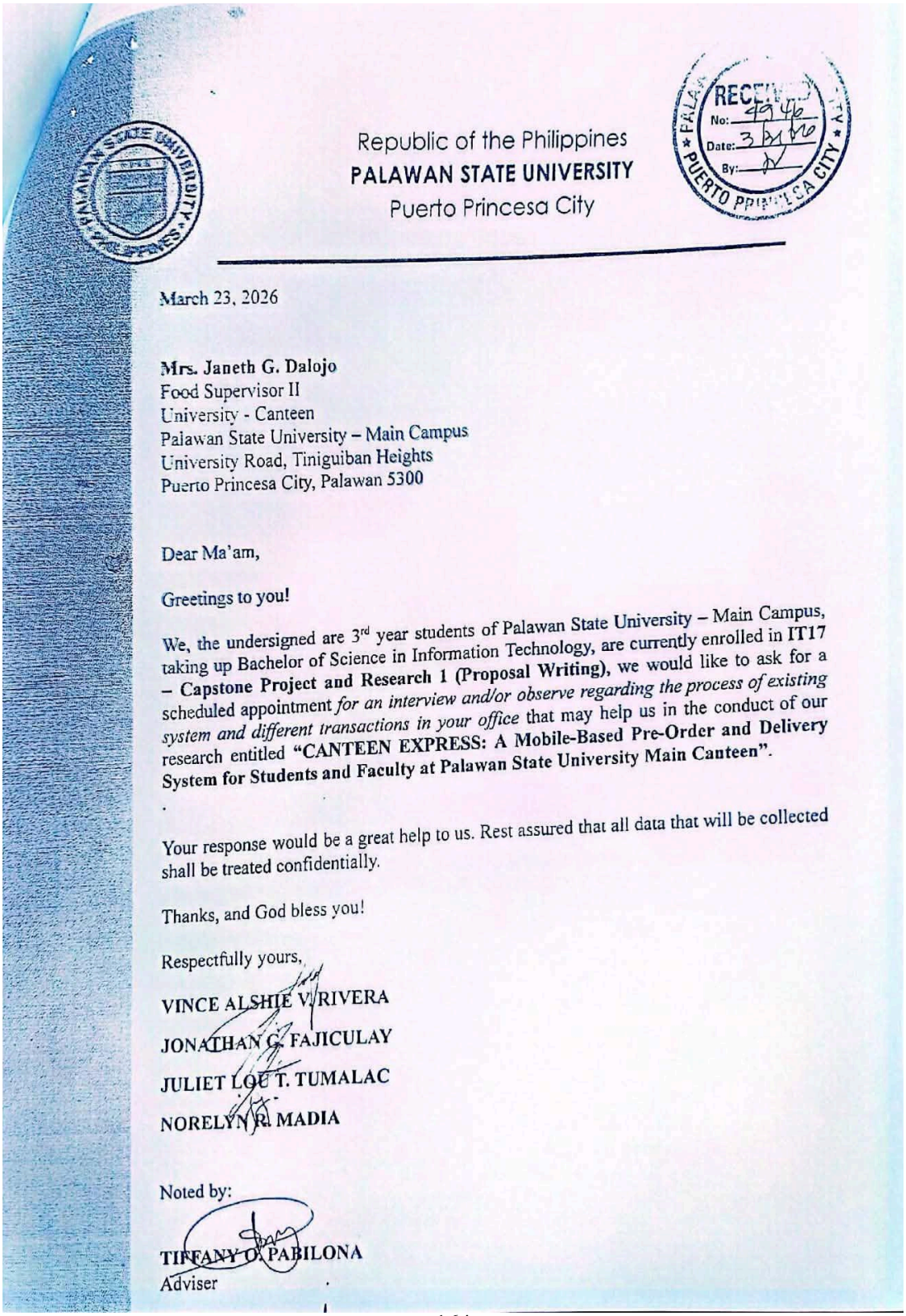
8.11 Monitoring System Activity Logs

1. Click Activity Log on the left navigation bar.
2. Filter entries using the Category, Level and Time Range dropdown menus.
3. Enter specific order numbers, items, or user names into the Search field.
4. Review timestamped records displaying Time, Activity, Actor and event Type.
5. Click Refresh at any time to manually fetch the latest system logs.

8.12 Updating Staff Password

1. Click Change Password located near the bottom of the left sidebar.
2. Type your current login credentials into the Current Password field.
3. Enter your updated password in the New Password field.
4. Re-enter the new password in Confirm New Password.
5. Click Update Password to apply the changes.

APPENDIX B: LETTER AND FORMS SIGNED BY THE FACULTY, DEAN, CLIENT AND OTHERS



Republic of the Philippines
PALAWAN STATE UNIVERSITY
Puerto Princesa City

RECEIVED
No: 4946
Date: 3 24 26
By:

March 23, 2026

Mrs. Janeth G. Dalojo
Food Supervisor II
University - Canteen
Palawan State University – Main Campus
University Road, Tiniguiban Heights
Puerto Princesa City, Palawan 5300

Dear Ma'am,

Greetings to you!

We, the undersigned are 3rd year students of Palawan State University – Main Campus, taking up Bachelor of Science in Information Technology, are currently enrolled in **IT17 – Capstone Project and Research 1 (Proposal Writing)**, we would like to ask for a scheduled appointment *for an interview and/or observe regarding the process of existing system and different transactions in your office* that may help us in the conduct of our research entitled **"CANTEEN EXPRESS: A Mobile-Based Pre-Order and Delivery System for Students and Faculty at Palawan State University Main Canteen"**.

Your response would be a great help to us. Rest assured that all data that will be collected shall be treated confidentially.

Thanks, and God bless you!

Respectfully yours,

VINCE ALSHIE V. RIVERA
JONATHAN G. FAJICULAY
JULIET LOU T. TUMALAC
NORELYN R. MADIA

Noted by:

TIFFANY O. PABILONA
Adviser

Republic of the Philippines
PALAWAN STATE UNIVERSITY
Office of the Vice President for Finance and Administration
Business Affairs and Auxiliary Services Office
Puerto Princesa City

April 6, 2026

MR VINCE ALSHIE V. RIVERA ET.AL
BSIT 3RD Year Students
Palawan State University – Main Campus

Dear Mr. Rivera and Team,

Greetings of peace and service.

This is in response to your request for a scheduled appointment to conduct an interview and/or observation regarding the processes and transactions of our office in support of your Capstone Project entitled ***"CANTEEN EXPRESS: A Mobile-Based Pre-Order and Delivery System for Students and Faculty at Palawan State University Main Canteen."***

We are pleased to inform you that we are open to accommodating your request, subject to the availability of our personnel and operational schedule. Kindly coordinate with our office to finalize the preferred date and time of your visit, as well as the specific areas or processes you wish to observe and discuss. This will allow us to properly prepare and ensure that your research needs are addressed accordingly without disrupting ongoing operations.

Please also coordinate in advance the list of interview questions or data you may require, for our review and approval.

We appreciate your initiative and commend your effort in developing a system that may contribute to improving the efficiency and convenience of services within the university.

Should you have any further inquiries or clarifications, feel free to contact our office Look for Ms Jelly Kate Garibay.

Thank you.

Sincerely,

MELISSA GRACE C. OLIT
Director, BAO-AS

Republic of the Philippines
PALAWAN STATE UNIVERSITY
Puerto Princesa City

PALAWAN STATE UNIVERSITY
RECEIVED
No: 2-4046
Date: 05/12/2026
By:
PUERTO PRINCESA CITY

May 11, 2026

Mrs. Melissa Grace C. Olit
Director BAO-AS
Palawan State University – Main Campus
University Road, Tiniguiban Heights
Puerto Princesa City, Palawan 5300

Dear Ma'am,

Greetings to you!

The proponents are thankful for your response to their request. The proponents would like to conduct an interview with you and Mrs. Janeth Dalojo within this week, in between the dates of May 12 - May 14, 2026 subject to your availability, or if the proponents are in need to be specific regarding the time, we would like to propose within the afternoon hours.

The interview questions will primarily focus on gathering insights and recommendations regarding the proposed delivery operations of the system, particularly the possible involvement of student part-timers as delivery personnel within the campus. The discussion will cover considerations such as operational workflow, supervision, policies, safety measures, accountability, incentives, and other factors necessary to ensure an organized, practical, and efficient implementation of the delivery service for the proposed system.

Thank you, and God bless you!

Respectfully yours,

VINCE ALSHIE V. RIVERA
JONATHAN G. PAJICULAY
JULIET LOU T. TUMALAC
NORELAN R. MADIA

Noted by:

MRS. JANETH G. DALOJO
Food Supervisor II
Palawan State University - Canteen

DR. RONALD EDILBERTO A. ONA

Republic of the Philippines
PALAWAN STATE UNIVERSITY
Business Affairs and Auxiliary Services
Puerto Princesa City

PALAWAN STATE UNIVERSITY
RECEIVED
No. 4446
Date: 05/12/2026
By

APPROVAL TO CONDUCT INTERVIEW
May 12, 2026

This is to formally approve and endorse the conduct of an interview as part of the Capstone Project and Research Proposal entitled:
"CANTEEN EXPRESS: A Mobile-Based Pre-Order and Delivery System for Students and Faculty at Palawan State University Main Canteen"
with proponents Rivera et al., under the advisory of Prof. Tiffany Pabilona.

The said interview is hereby authorized to be conducted within the Palawan State University Canteen premises. It will involve selected personnel, including representatives from the BAOAS Office and designated canteen personnel, as deemed necessary for the completion of the student research study.

The activity aims to gather relevant data, insights, and recommendations essential to the development and completion of the proposed system, particularly in relation to operational procedures, service delivery, and possible system improvements within canteen operations.

The proponents are advised to properly coordinate with the concerned offices and ensure that the conduct of the interview will not disrupt ongoing operations within the premises. All applicable institutional protocols, proper decorum, and ethical standards in data gathering must be strictly observed at all times.

In view thereof, this Office hereby grants approval to conduct the interview, subject to proper scheduling, coordination, and full compliance with university rules and regulations.

Issued this 12th day of May 2026 at the Business Affairs and Auxiliary Services Office, Palawan State University.

MELISSA GRACE C. OLIT
DIRECTOR,BAOAS

Republic of the Philippines
PALAWAN STATE UNIVERSITY
Office of the University President
Puerto Princesa City

OFFICIAL RELEASE
No. 1092
Date 05/13/26

SPECIAL ORDER
NO. 190 ; Series of 2026

To: **PROF ESTELITA C. ELEVENCIONE**
Unit Head, SAS-ISPS

Subject: **Designation as OIC, Office of the Dean, Student Affairs and Services**

Date: May 12, 2026

Effective immediately, Prof. Estelita C. Elevencione is hereby designated Officer-In-Charge, Office of the Dean, Student Affairs and Services (SAS) in the absence of Dr. Carlos Alfonso C. Salvador who is on leave.

As Officer-In-Charge, Prof. Elevencione's authority shall be limited to routinary and non-policy determining matters or as specifically instructed by the University's higher authorities.

This designation shall be in effect on May 12, 2026 upon the return of Dr. Salvador or unless sooner revoked by the University President.

For the information and guidance of all concerned.

MARISSA S. PONTILLAS
Officer-in-Charge
Office of the University President

Cf: OSAS

Republic of the Philippines
PALAWAN STATE UNIVERSITY
Office of the Vice President for Academic Affairs
Office of the Dean for Student Affairs and Services
Puerto Princesa City

PALAWAN STATE UNIVERSITY
RECEIVED
No.: 6595
Date: 05/14/2026
By:
PUERTO PRINCESA CITY

FIRST ENDORSEMENT
May 14, 2026
Transaction No. _____

Respectfully forwarded to the Officer-in-Charge, University President, Prof. Marissa S. Pontillas, through the Vice President for Academic Affairs, Dr. Maila N. Lucero, with a recommendation for approval of the request submitted by Mr. Vince Alshie V. Rivera et al. for the conduct and implementation of their capstone project entitled, *"Canteen Express: A Mobile-Based Pre-Order and Delivery System for Students and Faculty of the Palawan State University Main Canteen,"* in partial fulfillment of the requirements for IT 17 – Capstone Project and Research 1.

The approval of this request is recommended subject to proper coordination with the Canteen management, as well as compliance with all existing University policies, rules, and regulations.

DR. CARLOS ALFONSO C. SALVADOR
Dean of Student Affairs and Services

Republic of the Philippines
PALAWAN STATE UNIVERSITY
Puerto Princesa City

August 12, 2026

MARISSA S. PONTILLAS
Vice President for Academic Affairs
This University

Dear Ma'am,

Greetings!

We, the undersigned 4th-year students of Palawan State University – Main Campus, taking up Bachelor of Science in Information Technology, are currently enrolled in **IT19 – Capstone Project and Research 2 (Data Gathering)**. We are currently conducting our data gathering phase for our research entitled **"CANTEEN EXPRESS: A Mobile-Based Pre-Order and Delivery System for Students and Faculty at Palawan State University Main Canteen."**

In line with this, we would like to humbly request your permission and approval to conduct interviews with faculty, administrative staff, and students across various departments and colleges within the Main Campus. The primary objective is to gather user requirements, feedback, and operational insights to ensure that the proposed system effectively meets the needs of the university community.

To protect the rights and welfare of all participants, our data collection strictly adheres to institutional research ethics guidelines and the **Data Privacy Act of 2012 (Republic Act No. 10173)** under the following conditions:

- **Voluntary Participation:** Participation will be entirely voluntary. Respondents may choose to decline or withdraw from the interview at any point.
- **Confidentiality & Anonymity:** Personal identities and responses will be kept strictly confidential. Recordings and raw notes will be stored securely using multi-factor authentication (MFA) and accessed solely by the research team.
- **Data Disposal:** All collected raw data will be permanently disposed of upon the official completion and acceptance of the capstone project.

Your favorable approval will allow us to proceed with our data collection activities smoothly. Upon approval, we will submit a copy of our interview protocol to your office before the conduct of the interviews.

Thank you very much for your time, support, and consideration. May God bless you!

Respectfully yours,

VINCE ALSHIE V. RIVERA

JONATHAN C. FAJICULAY

JULIET DOU T. TUMALAC

NORELYN R. MADIA

Researchers

Noted by:

MRS. JANETH G. DALOJO
Food Supervisor II
Palawan State University - Canteen

Recommending Approval:

TIFFANY O. PABILONA
Adviser

FLOREDITH JEANNE G. ALCID
Faculty-In-Charge

JENNIFER C. RABANG
Program Head, Information Technology Department
College of Sciences

FLOREDEL DANGAN-GALON, Ph.D.
Dean, College of Sciences

PALAWAN STATE UNIVERSITY
COLLEGE OF SCIENCES

CERTIFICATION

This is to certify that the research project entitled "Canteen Express: A Web/Mobile-Based Pre-Order and Delivery System for Students, Staff, and Faculty at Palawan State University Main Canteen" was conducted in consideration of the ethical principles and guidelines governing research at Palawan State University.

As mandated by the University Research Manual, all research projects are required to undergo review by the University Research Ethics Review Committee (URERC). However, at the time of the conduct of this study, the URERC was under moratorium while awaiting the results of the Level 2 Accreditation.

In view of this circumstance, the researchers sought guidance through consultation regarding the ethical considerations of the study. Prior to the conduct of the research, the researchers complied with the comments, recommendations, and suggestions provided during the consultation to ensure that the research procedures were ethically sound.

This certification is issued to attest to the researchers' consultation and compliance with the ethical guidance provided for the conduct of the study.

Issued this 10th day of October 2026, at Palawan State University, Puerto Princesa City, Palawan, Philippines.

Jean Marie L. Diego

Faculty, Department of Environmental Science
College of Sciences
Palawan State University

APPENDIX C: SCREEN LAYOUTS

Customer (Students and Walk-In)

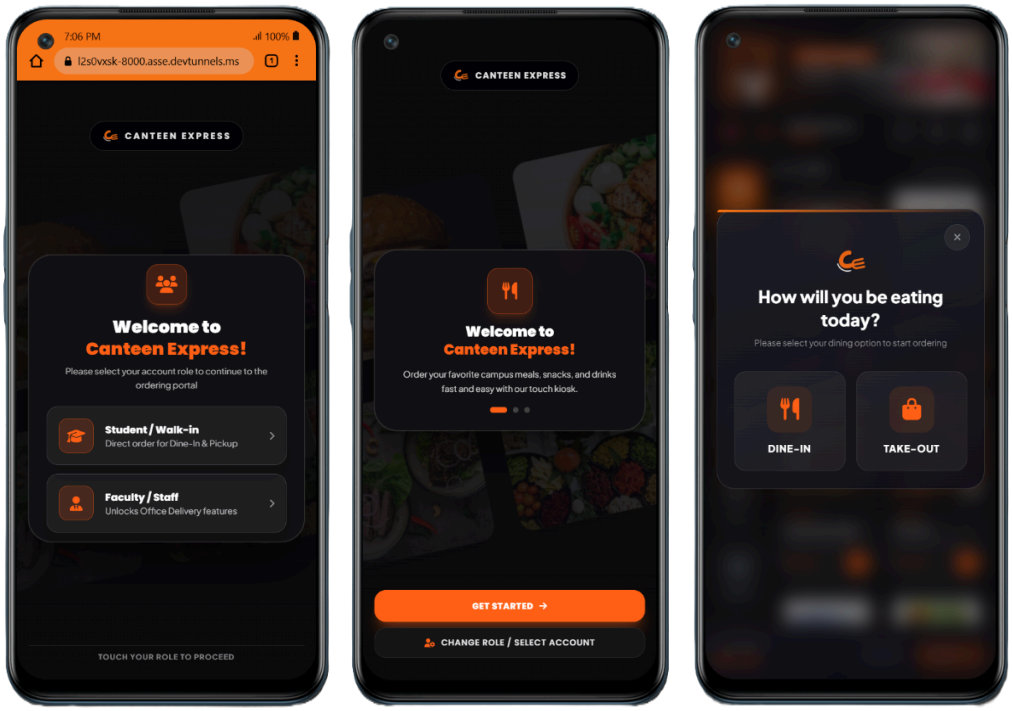


Figure 6.1: Onboarding Screen of Student/Walk-In Users

The figure above shows the initial onboarding and welcoming screens of the Student/Walk-In side within the Canteen Express PWA (Progressive Web App). It begins on the landing role-selection screen where users can choose between "Student/Walk-In" (for direct order for Dine-In and Pick-Up) and "Faculty/Staff" (for office delivery features). Selecting "Student/Walk-In" brings up the app's welcome screen for students/walk-ins featuring a "Get Started" action button and an option to change accounts or roles. Upon tapping "Get Started", an interactive dining preference modal appears prompting the user to select "Dine-In" or "Take-out" before proceeding to browse the menu and place their order.

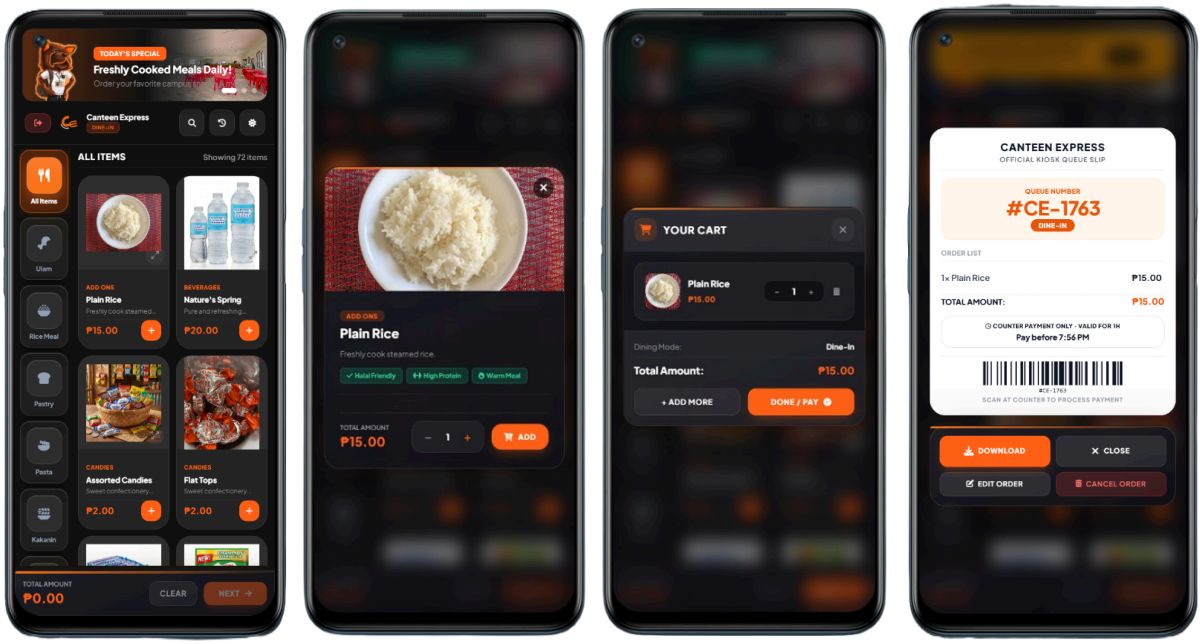


Figure 6.2: Student/Walk-In Menu Interface, Cart and Queue Slip Screens

The figure above shows the Student/Walk-In Menu Browsing Menu interface, Item Customization, Shopping Cart and Digital Queue Slip screens within the student ordering workflow of the Canteen Express PWA. It begins on the Menu screen where users can browse categorized menu items, search products, check live cart totals and select items. Tapping an item opens the Item Customization modal to adjust quantities and add it to the review items, adjust dining mode (Dine-In/Take-Out) and submit the order, which instantly generates the Digital Queue Slip featuring an official queue number, itemized pricing, payment expiration countdown and a unique barcode for canteen counter scanning.

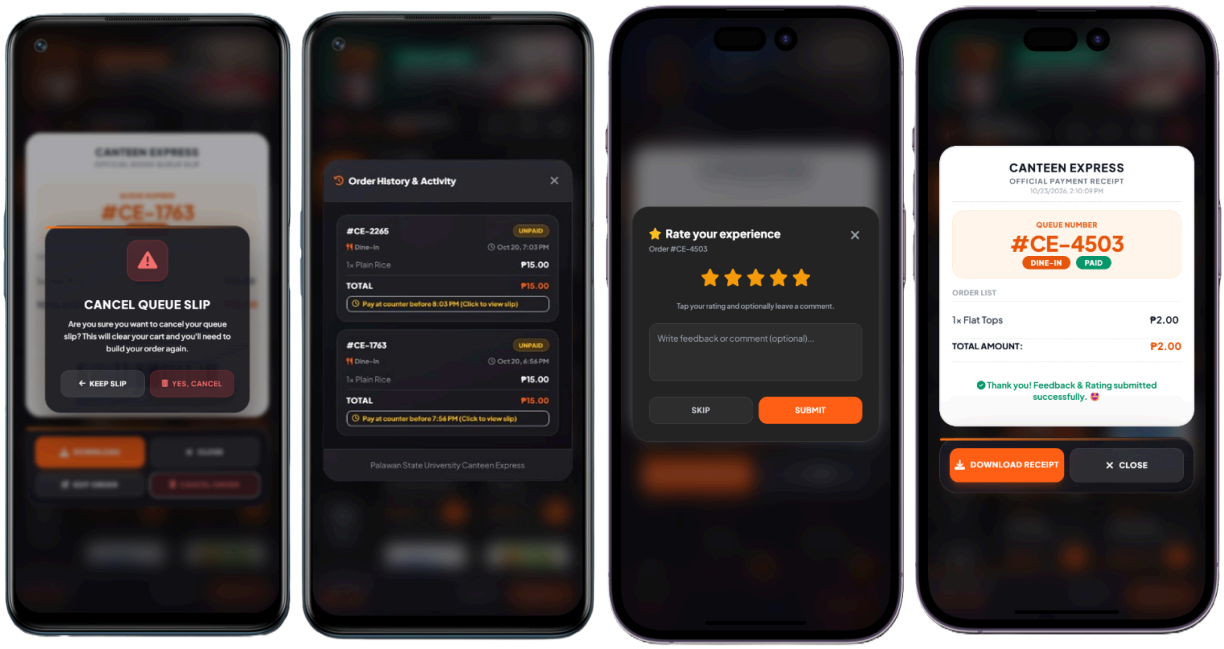


Figure 6.3: Cancel Queue Slip and Order History Screens

The figure above shows the Cancel Queue Slip confirmation dialog and Order History and Activity screens within the Canteen Express Student/Walk-In Interface. Tapping "Cancel Order" on the queue slip prompts a confirmation modal asking the user if they are sure they want to cancel their queue slip and clear their cart. Meanwhile, the Order History and Activity panel displays a chronological list of orders (such as #CE-2265 and #CE-1763), their UNPAID status, dining mode, item summary, total amounts and payment expiration countdowns with quick links to view existing queue slips. After the customer claims the order at the counter, a Feedback Dialog will appear to rate the customer's experience. Upon completing the purchase, an e-receipt will appear along with the details of the order. The e-receipt can be downloaded as a png file.

Customer (Faculty and Staff)

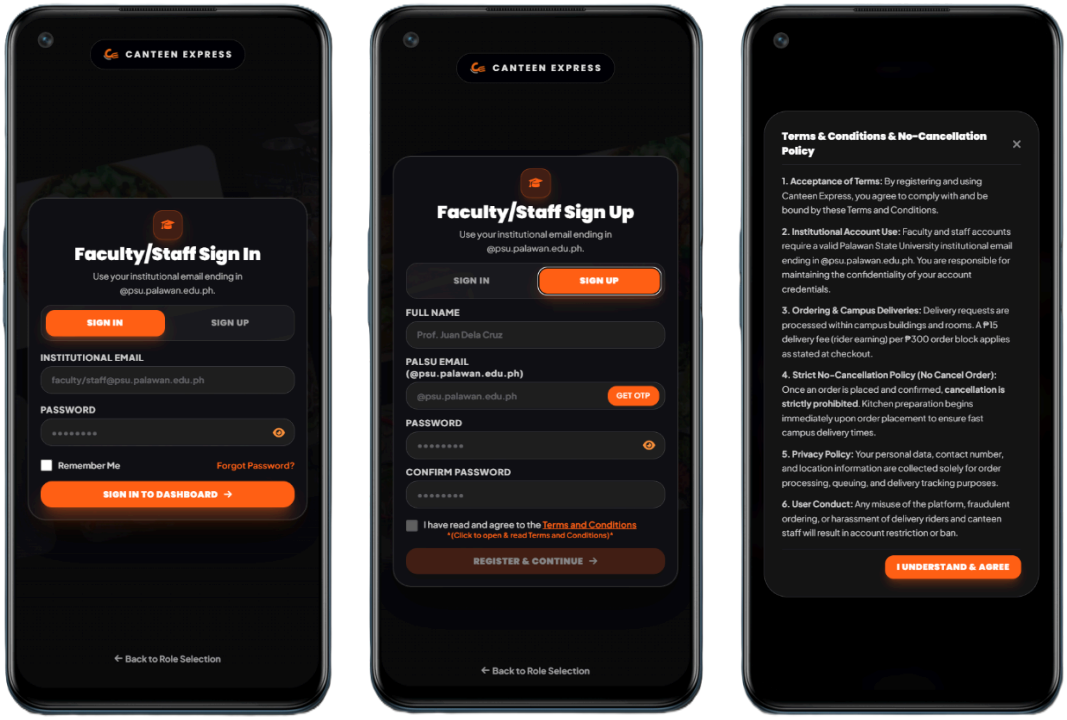


Figure 6.4: Faculty/Staff Sign In, Sign Up and Terms Modal

The figure above shows the Faculty/Staff authentication and registration workflow within the Canteen Express PWA. The Faculty/Staff Sign In screen allows authorized personnel to log in using their @psu.palawan.edu.ph institutional email and password with options for "Remember Me" and password recovery. If new, users can switch to the Faculty/Staff Sign Up screen to register by providing their full name, Palawan State University Email, password and agreement to the terms. Tapping the terms link opens the Terms and Conditions and No-Cancellation Policy modal detailing institutional account rules, delivery fees, strict non-cancellation policies, privacy policies and user conduct guidelines before the user can proceed by tapping "I Understand and Agree".

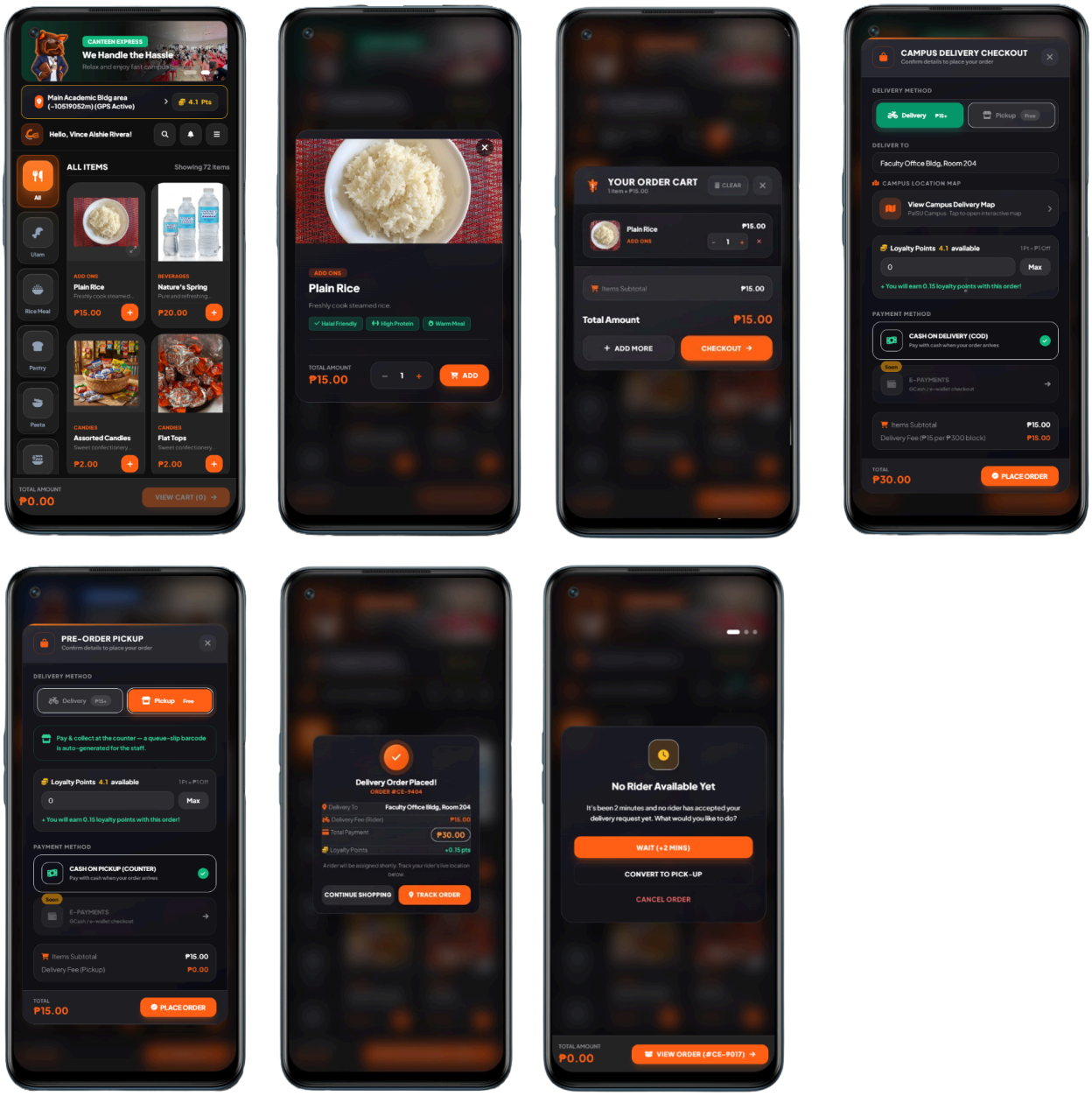


Figure 6.5: Customer Menu Browsing, Cart and Order Checkout Flow

The figures illustrate the end-to-end customer ordering workflow, starting with the Menu Browsing Dashboard for catalog filtering. Users can select any item to view its details, dietary tags and quantities in the Item Detail View, then manage their selections through the Order Cart Summary. Depending on preference, users can proceed to either Campus Delivery Checkout

which enforces office address input, campus geofencing and loyalty point redemption or Pre-Order Pickup Checkout for counter collection with zero delivery fees. If there is no available delivery personnel within the time range of 5 minutes, a dialog box will appear for the user to choose among these options; “”Wait (+2 mins)”, “Convert to Pick-Up” and “Cancel Order”. Finally, successful transactions trigger the Order Placement Confirmation screen, generating an order reference code, payment breakdown, earned loyalty points and direct links to live GPS order tracking.

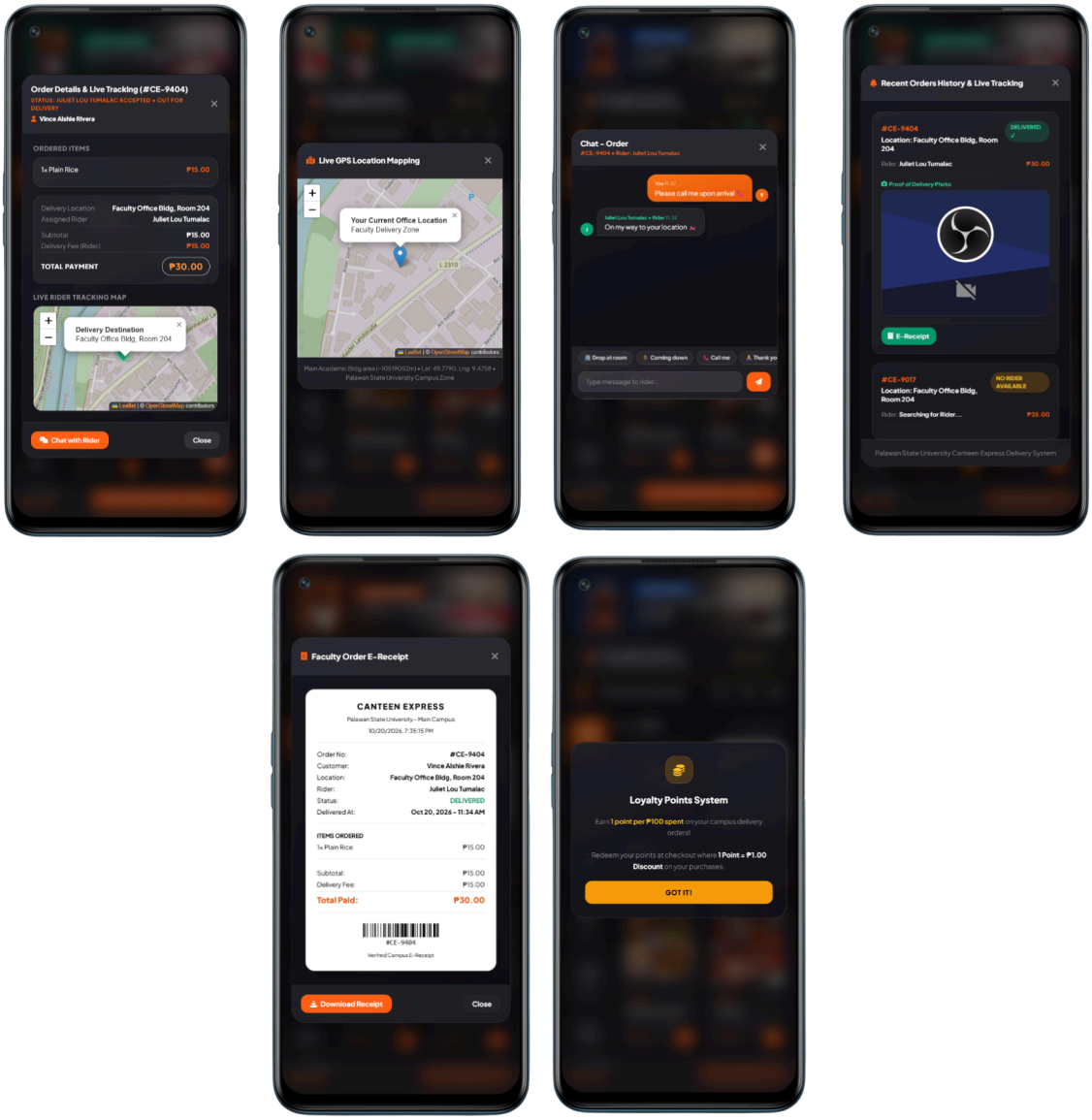


Figure 6.6: Live Tracking, Delivery and Post-Order Features

The figures illustrate the post-order tracking and delivery management workflow, starting with the Order Details and Live Tracking view which displays real-time courier acceptance status, delivery location, assigned rider details, itemized totals and a map preview. Users can expand the Live GPS Location Mapping module to monitor courier movement and destination positioning on an interactive Leaflet map. Direct communication between the customer and delivery personnel is supported via the In-App Order Chat interface equipped with present quick-reply chips. Completed transactions are archived in the Recent Orders History feed, where users can inspect photographic proof of delivery delivery, access itemized Digital E-Receipts complete with scannable barcodes and review accumulated rewards through the Loyalty Points System informational modal.

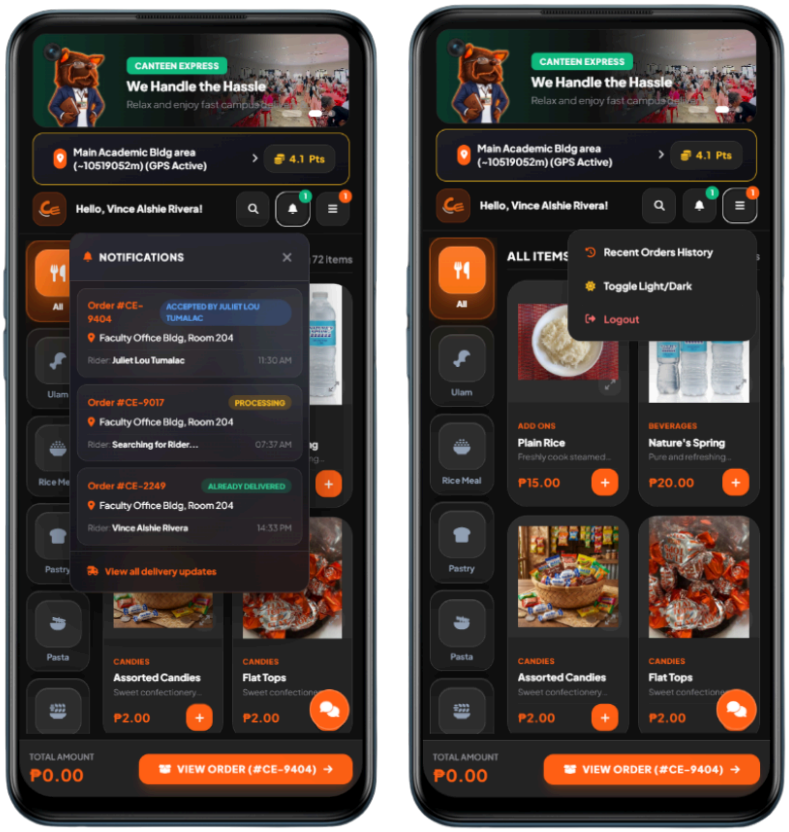


Figure 6.7: Customer Notification System and Account Navigation Menu

These figures illustrate the customer dashboard navigation and notification features, starting with the Notifications Dropdown which displays real-time order status updates, courier acceptance details and delivery progression actress active orders. Additionally, the Navigation Menu provides quick access to Recent Orders History, theme customization via the Light/Dark Mode Toggle and secure user session management through the Logout option.

Delivery Personnel

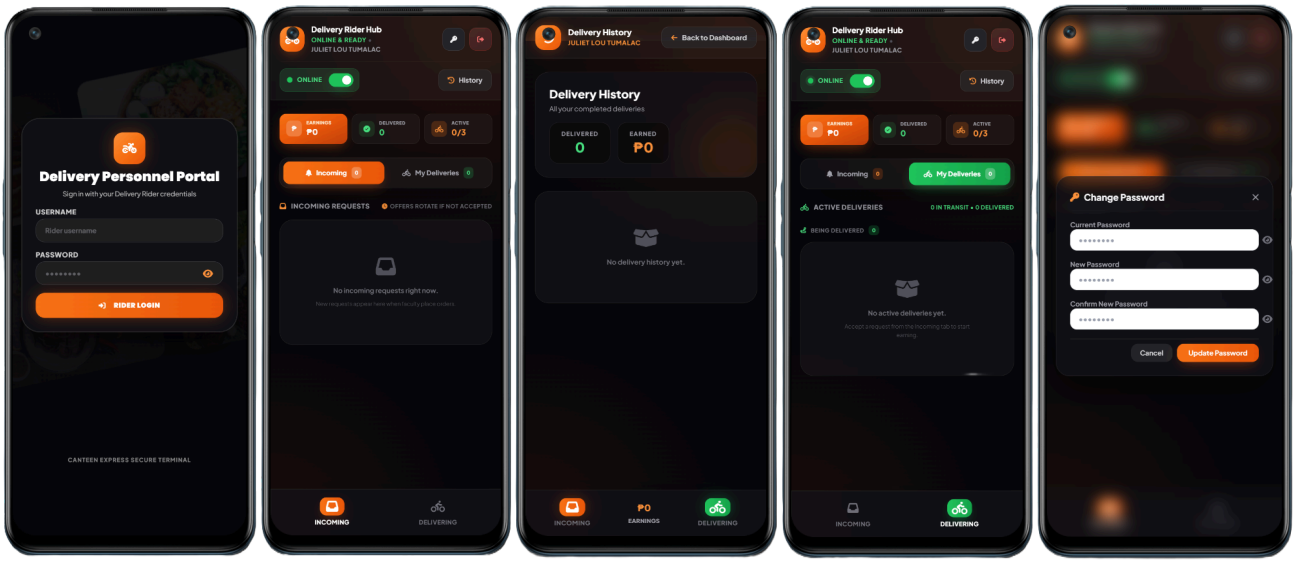


Figure 6.8: Delivery Personnel Portal and Rider Hub Workflow

The figures illustrate the end-to-end delivery rider workflow, starting with the Delivery Personnel Portal Login screen for secure courier authentication. Upon login, couriers access the Delivery Rider Hub, featuring an online status toggle, earnings summary and performance metrics. Couriers can monitor Incoming Requests, review Delivery History logs for completed tasks, manage active shipments under My Deliveries and update account security via the Change Password modal.

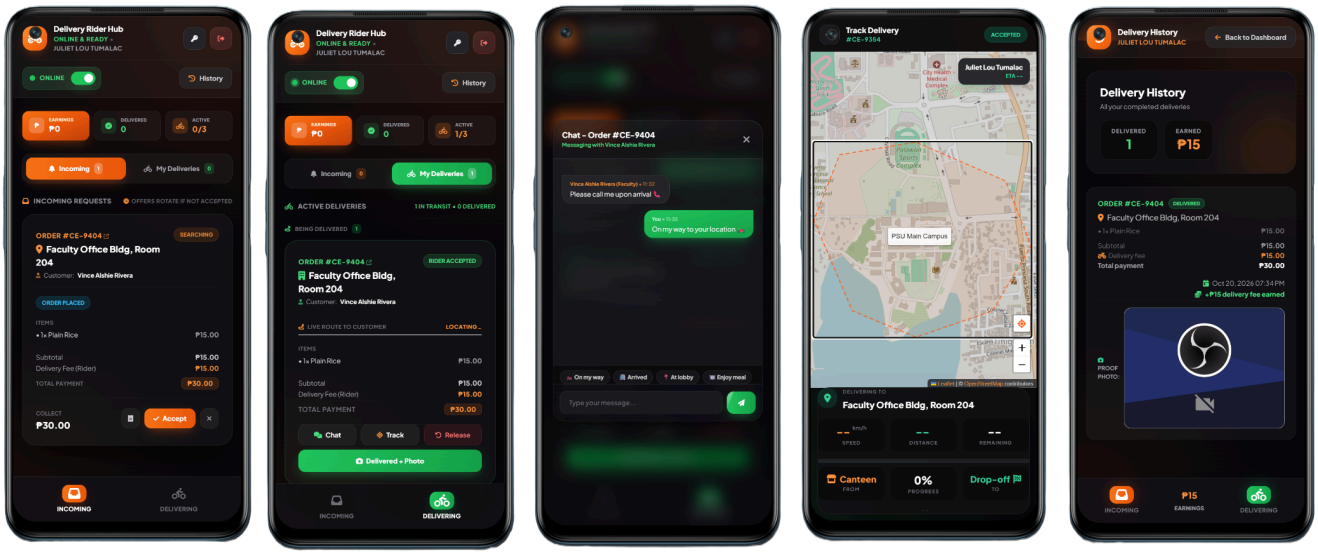


Figure 6.9: Rider Dispatch, Live GPS Tracking and Delivery Fulfillment Workflow

The figures illustrate the rider dispatch and order fulfillment workflow, beginning with the Incoming Requests feed where couriers can review order details and accept delivery assignments. Once accepted, orders transition to Active Deliveries under the My Deliveries tab, granting access to real-time communication via the In-App Rider Chat interface with quick-reply chips. Couriers can also launch the Live GPS Tracking Map featuring campus geofencing overlays, turn-by-turn routing metrics and trip progress tracking. Finally, successful handoffs are finalized using the Delivered + Photo Proof capture feature, updating the Delivery History logs with completed deliveries, earned delivery fees and photographic confirmation records.

Canteen Staff/Admin

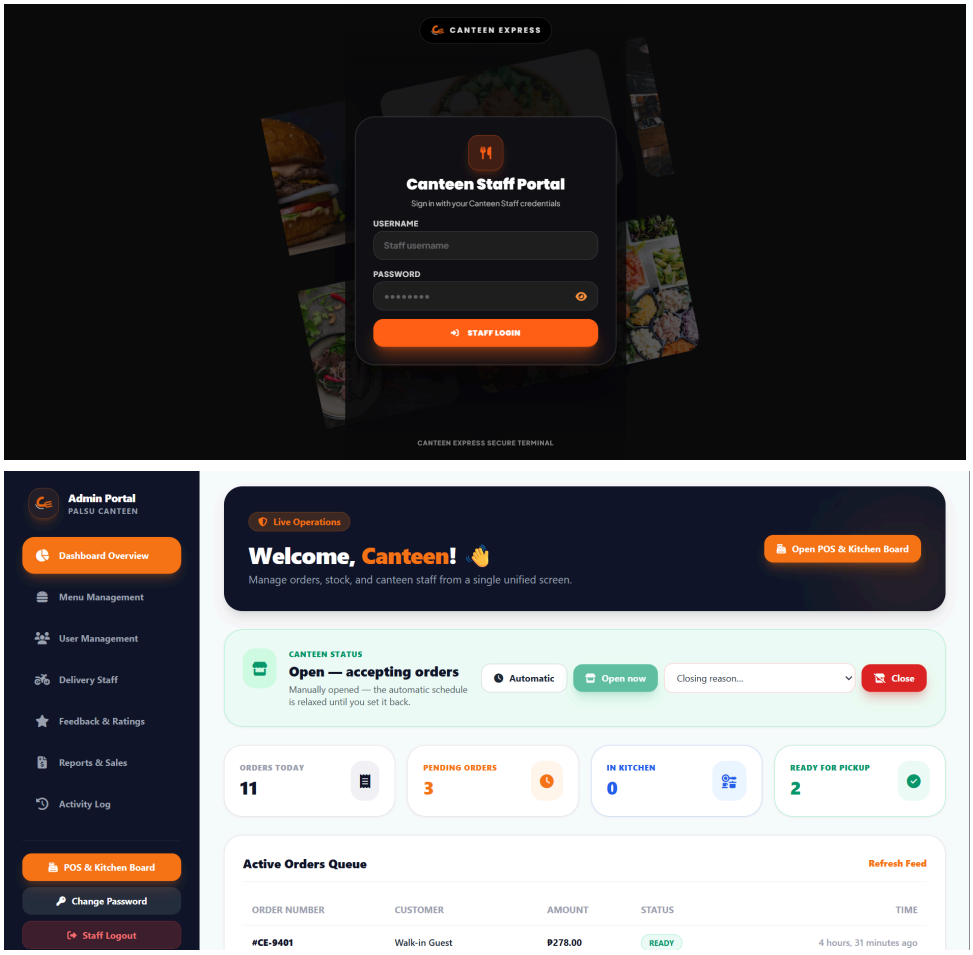


Figure 6.10: Canteen Staff Portal Login and Management Dashboard Overview

The figures illustrate the canteen staff administrative workflow, beginning with the Canteen Staff Portal Login screen for secure authentication. Upon successful login, staff are directed to the Staff Dashboard Overview, which features real-time operational status toggles (open/closed controls), daily order metrics, pending order counters, kitchen status indicators and an Active Orders Queue providing comprehensive oversight of current canteen transactions.

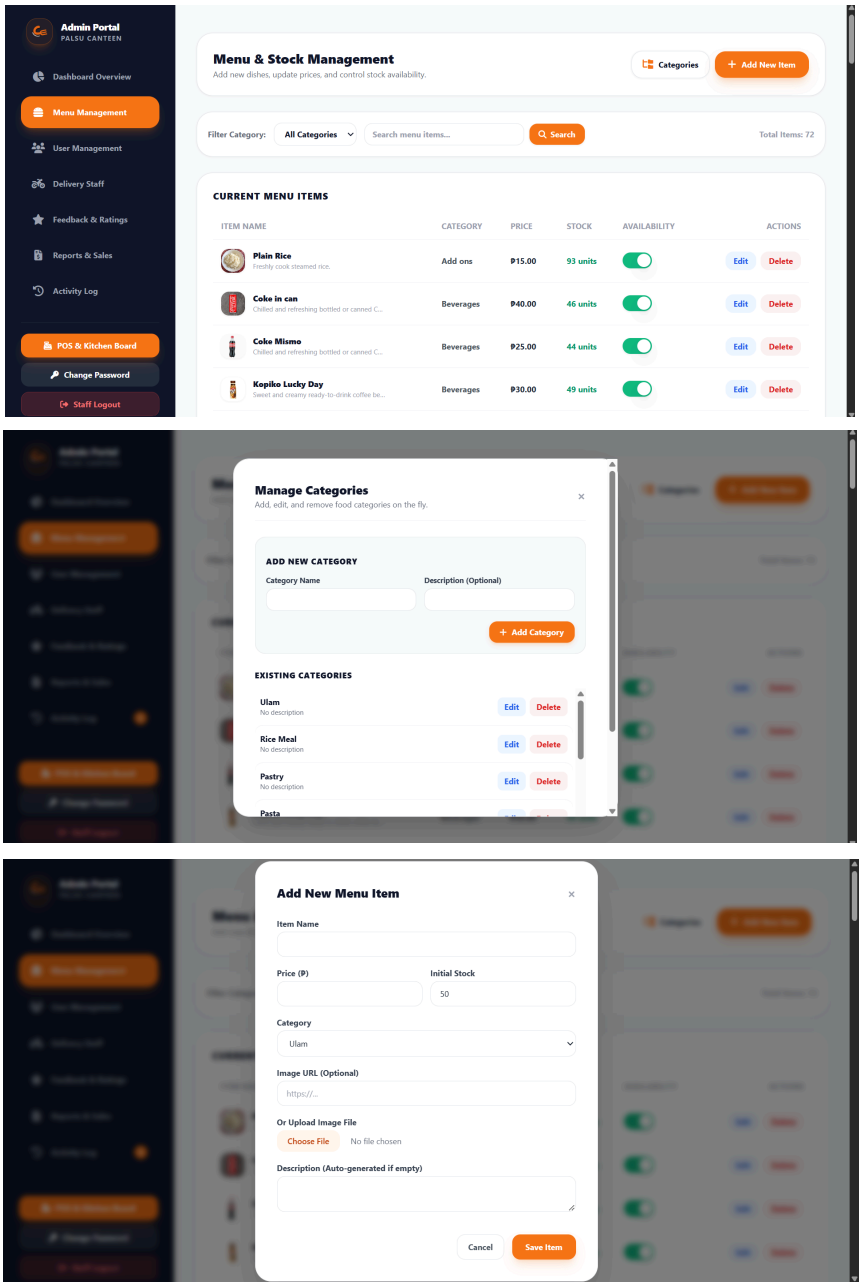


Figure 6.11: Menu and Stock Management Interface

The figures illustrate the canteen menu, featuring the Menu Stock Management Dashboard for searching, filtering categories and updating item pricing, stock units and real-time availability toggles. Additionally, the Manage Categories modal enables staff to add, edit and organize product classifications on the fly.

This figure also illustrates the “Add New Menu Item” modal within the Canteen Staff portal, allowing staff to input item names, set prices, define initial stock levels, assign product categories, provide optional image URLs or upload image files and include detailed descriptions before saving the new menu item to the system.

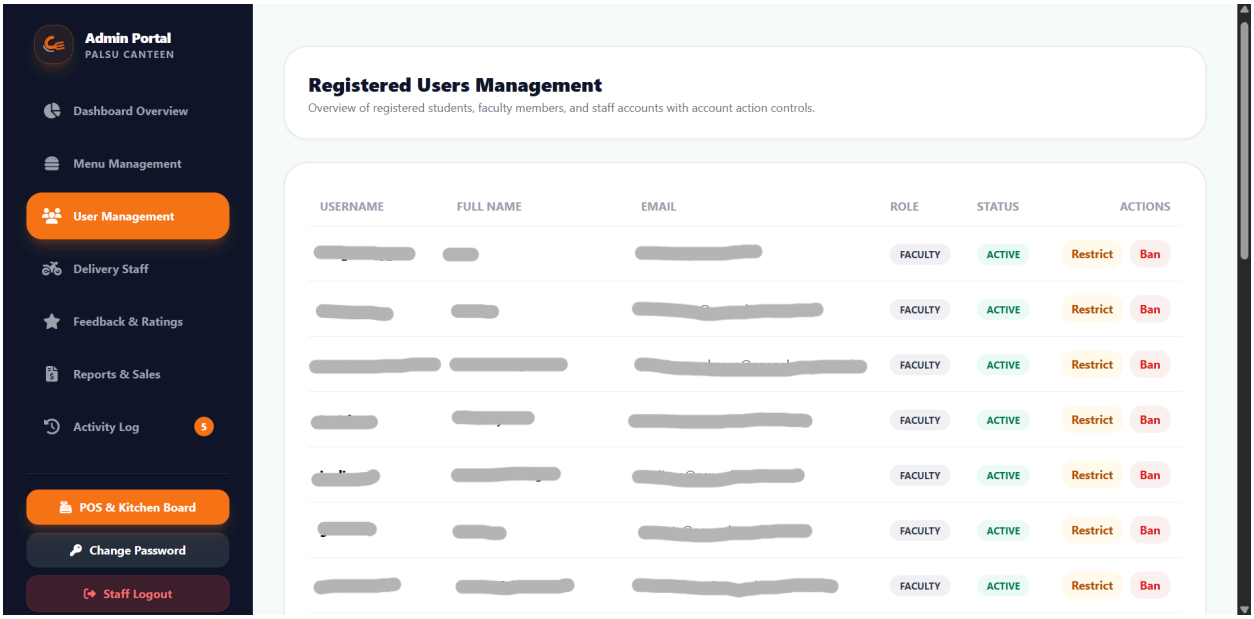


Figure 6.12: Registered Users Management View

The Staff Portal interface displays the registered users management screen, listing student, faculty and staff accounts with their username, full name, email, role, status and administrative action controls (Restrict and Ban).

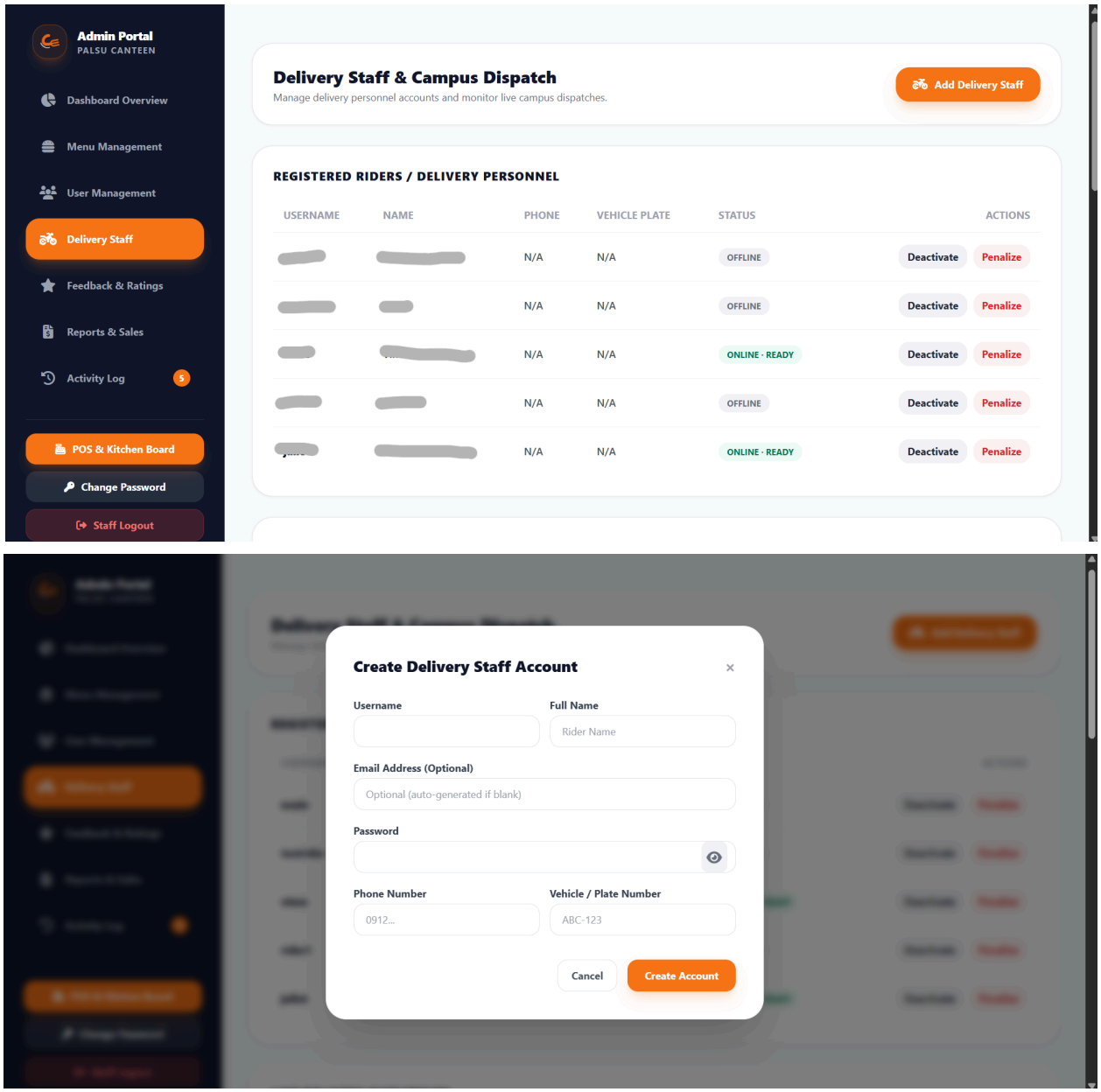


Figure 6.13: Delivery Personnel and Campus Dispatch View with Account Creation Modal

The Staff Portal interface shows the delivery staff management screen with registered delivery personnel list, live delivery dispatches and the “Create Delivery Staff Account” modal dialog user for registering new delivery personnel.

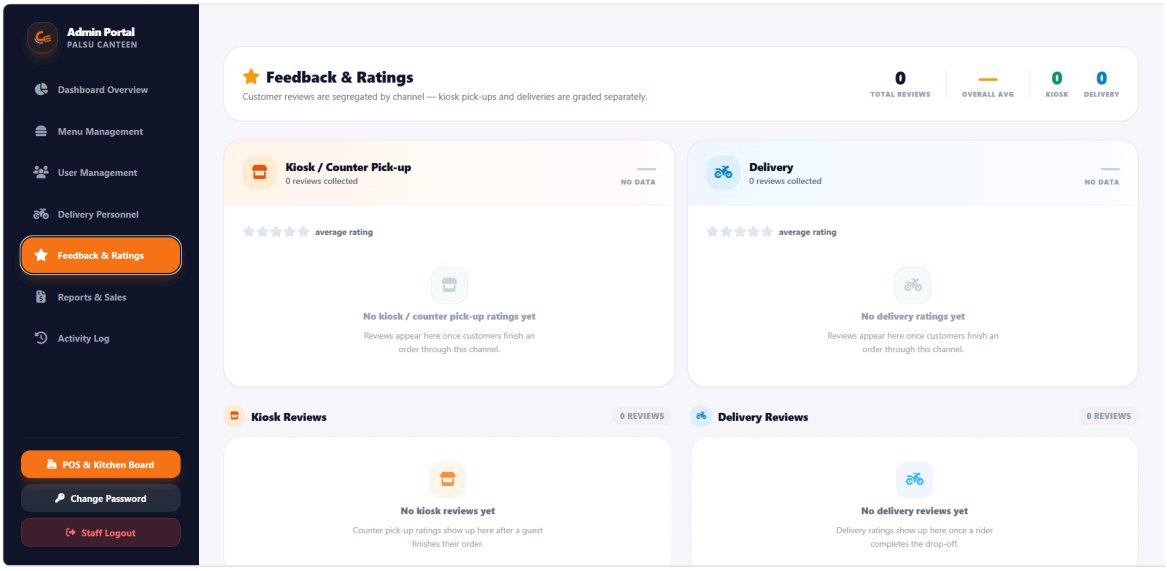
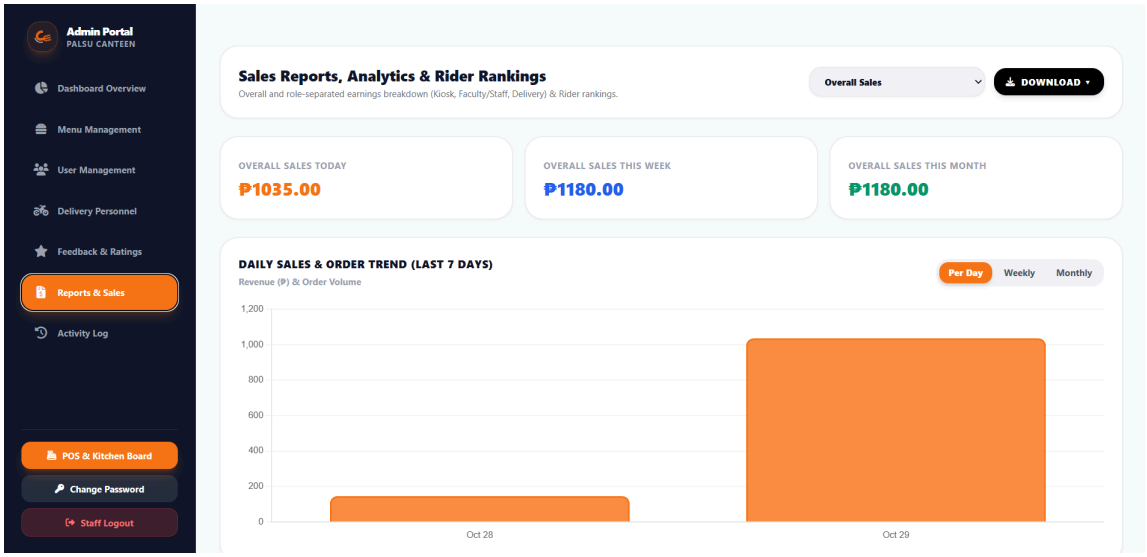


Figure 6.14: Feedback and Ratings Dashboard

The Staff Portal interface displays customer feedback and ratings segregated by order channel (Student/Walk-In purchasing/Counter Pick-up and Delivery), showing rating statistics and review breakdown cards.



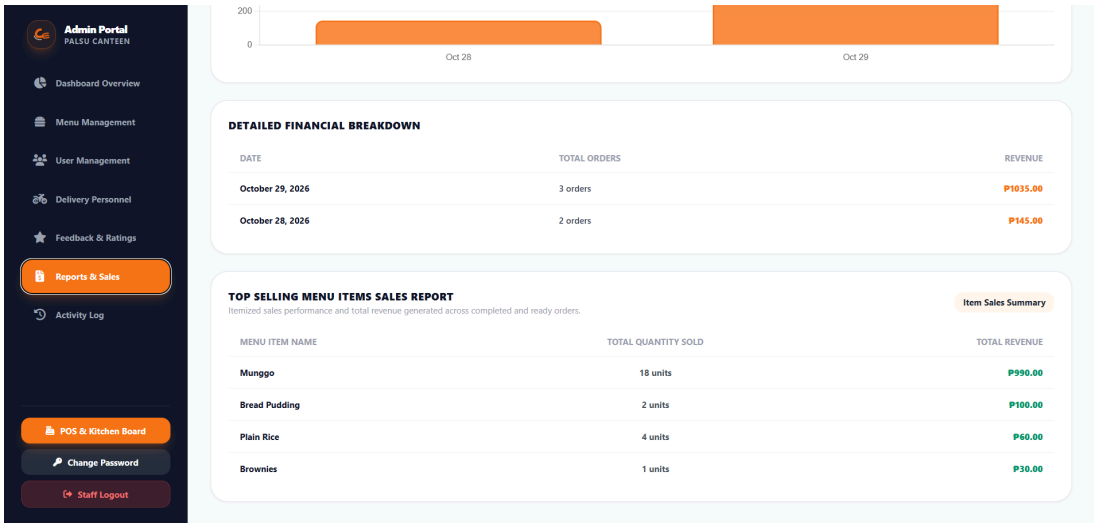


Figure 6.15: Sales Reports, Analytics and Rider Rankings Dashboard

The Staff Portal interface shows sales reports, analytics, overall sales metrics (Today, This Week, This Month), daily sales revenue bar chart and order volume trend charts.

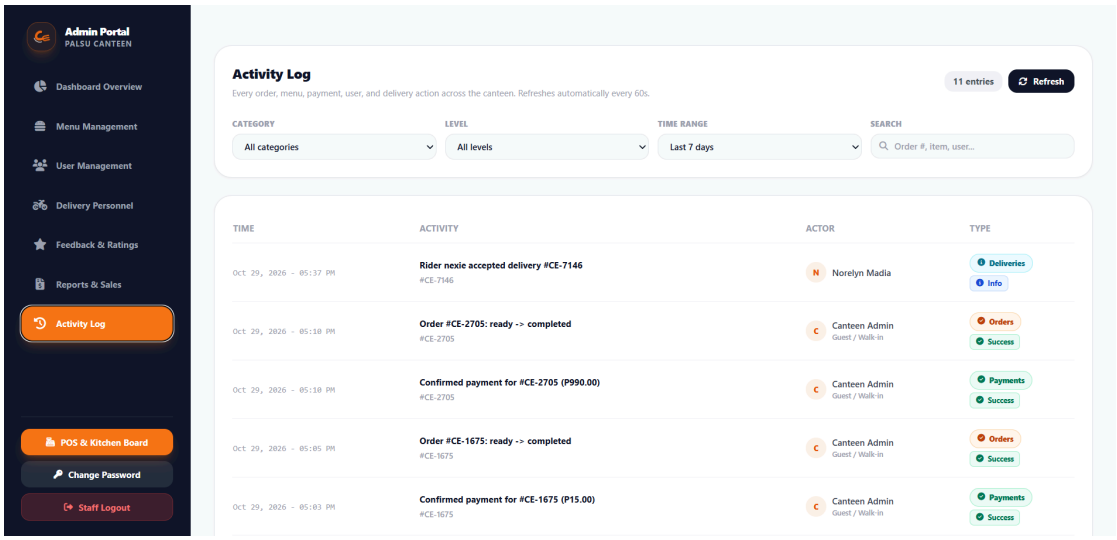


Figure 6.16: Activity Log Page

The Staff Portal interface displays the system activity log tracking orders, menu updates, payments, user actions and delivery events with filtering by category, level and time range.

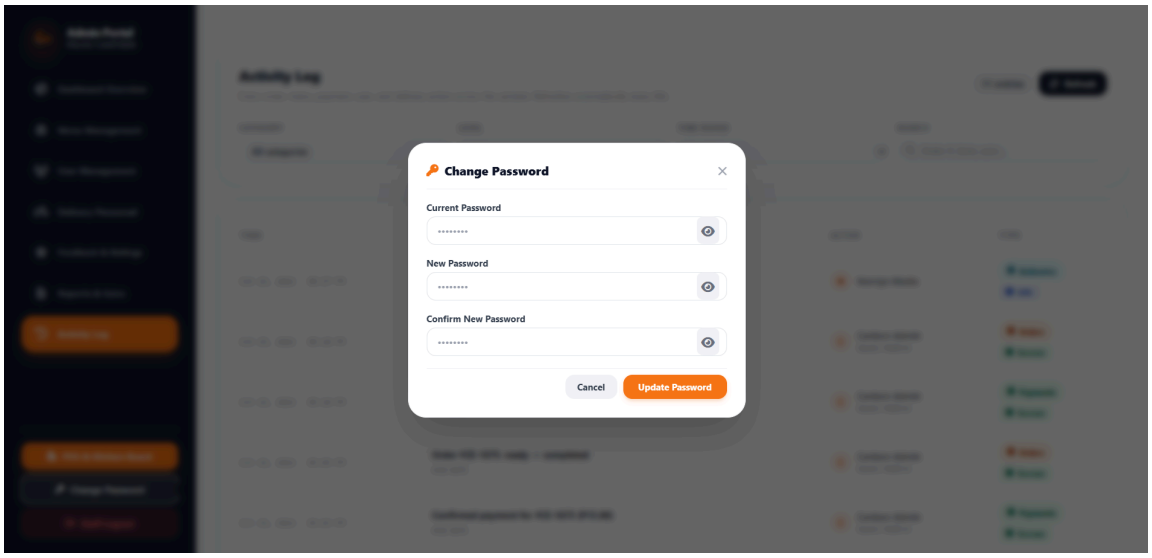


Figure 6.17: Change Password

The Staff Portal interface features the “Change Password” modal dialog overlay, allowing canteen staff to securely update their password by entering their current password, new password and confirmation.

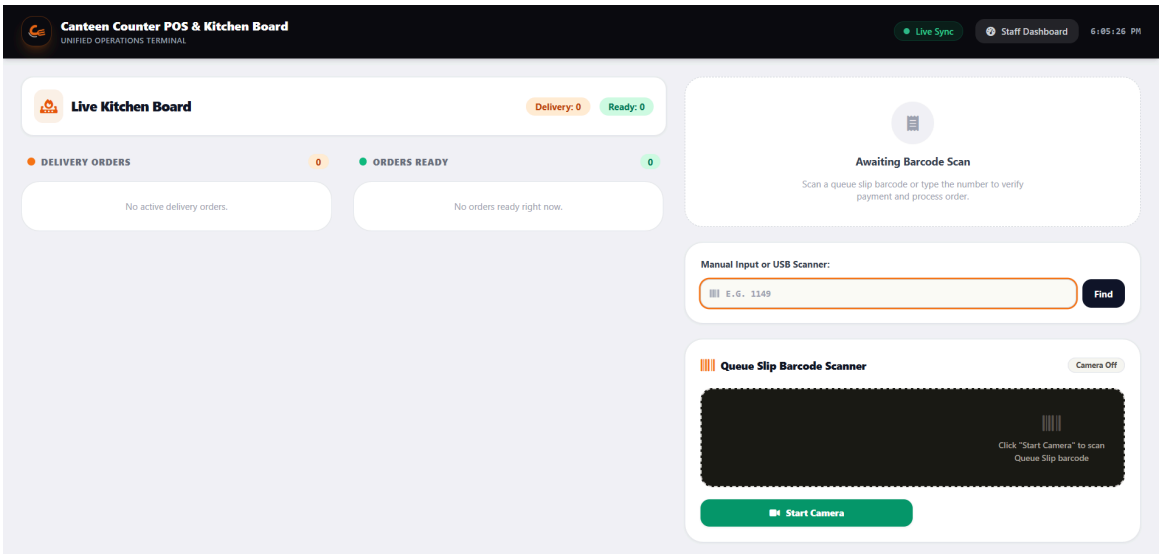


Figure 6.18: Kitchen Order Display and Counter POS Interface

The integrated staff workspace combines the Kitchen Order Display (kanban board for managing preparation status across Student/Walk-In Ordering Interface and Delivery orders) and the Counter POS (queue slip barcode scanner and payment verification module). Its function is to bridge cashier payment verification with real-time kitchen fulfillment by automatically routing scanned and pain queue slips into active preparation workflows.

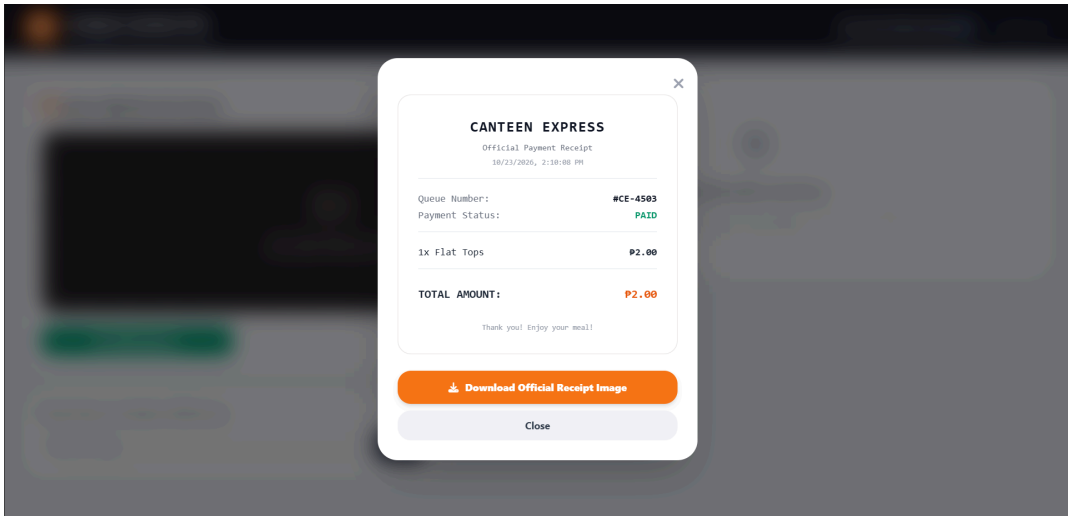


Figure 6.19: E-Receipt Student/Walk-In Customer through Canteen Staff POS

This screenshot displays the official Canteen Express e-receipt modal generated for a student or walk-in customer once canteen staff confirm payment at the POS. It shows the transaction timestamp, queue number (#CE-4503), payment status (PAID), ordered items with pricing (1x Flat Tops - ₱2.00), total amount due and action buttons to download the receipt image or close the modal.

APPENDIX D: GUIDED QUESTIONS FOR TESTING THE SYSTEM

Customer (Student/Walk-In)

Legend:

1 = Strongly Disagree

2 = Disagree

3 = Neutral

4 = Agree

5 = Strongly Agree

Questions	1	2	3	4	5
1. Ease of Navigation: The kiosk / mobile interface was intuitive and easy to navigate when browsing menu categories and placing orders.					
2. Order Customization: The system provided clear options for customizing food and beverage orders (e.g., matching side dishes or add-ons correctly).					
3. Queue Slip & Barcode Generation: The generated queue slips and barcode made it easy and convenient to track order status or claim items at the counter.					
4. Checkout Efficiency: The checkout process was simple, fast and easy to complete without unnecessary steps.					
5. System Responsiveness: The pages loaded quickly and responded					

reliably without lagging during peak ordering hours.					
6. Feature Availability Clarity: The system clearly indicated which features and services were available or restricted for my user account.					
7. Transparency on Menu Availability: The system accurately reflected real-time food availability and pricing without causing confusion.					
8. Visual Clarity: The layout, typography and dark-mode styling made the interface comfortable to view and interact with.					
9. Overall Functionality: The system successfully met my expectations as a self-service walk-in ordering tool.					
10. Recommendation: I would gladly use this kiosk/web system again for ordering meals at the campus canteen.					

Customer (Faculty/Staff)

Legend:

1 = Strongly Disagree

2 = Disagree

3 = Neutral

4 = Agree

5 = Strongly Agree

Questions	1	2	3	4	5
1. Authentication & Account Privileges: Logging in with institutional credentials (@psu.palawan.edu.ph) provided secure access to my faculty/staff dashboard and profile features.					
2. Loyalty Points & History: The system accurately tracked my accumulated loyalty points and displayed past order history correctly.					
3. Campus Delivery Request: The delivery ordering feature was easy to configure with accurate destination coordinates and campus boundary validation.					
4. Real-Time Delivery Tracking: The integrated dark-mode map and live GPS tracking made it easy to monitor delivery progress in real time.					
5. In-App Communication: The real-time messaging feature					

facilitated clear and efficient communication regarding delivery updates.					
6. Counter POS & Barcode Processing: The canteen counter POS efficiently processed barcodes and scanned queue slips to update order payment status instantly.					
7. Mobile PWA Experience: The Progressive Web App (PWA) provided a fast, responsive and app-like experience on mobile and desktop devices.					
8. Meal Customization & Menu Browsing: Browsing the canteen menu categories, viewing detailed meal photos and customizing options (such as rice add-ons and drink choices) during checkout was intuitive and user-friendly.					
9. Role-Based Capabilities: The system effectively distinguished and supported my advanced permissions (delivery scheduling, loyalty rewards and priority ordering) compared to walk-in users.					
10. Overall User Satisfaction & Reliability: The faculty/staff ordering and delivery platform provided a reliable, smooth and highly convenient experience for daily campus meals.					

Client (Palawan State University - Main Canteen)

No.	Module / Feature	Expected Function / Verification Criteria	Tester / Client / Stakeholder Remarks	Status	Signature
1	Account Access	Customers who need an account, canteen staff, delivery personnel, and the administrator can sign in with the correct details. The system does not allow access with an incorrect password.		☐ Passed ☐ Partial ☐ Failed	
2	User Roles and Access	Customers can place and check their orders; canteen staff can handle food orders and payments; delivery personnel can manage assigned deliveries; and the administrator can manage the system. Each role can use only the features meant for that role.		☐ Passed ☐ Partial ☐ Failed	
3	Home Page	Customers can find menu items and check their orders. Canteen staff can access order-handling features, delivery personnel can view delivery requests, and the administrator can open management and report features available to their role.		☐ Passed ☐ Partial ☐ Failed	
4	View Menu	Customers can view food names, descriptions, prices, categories, photos, and whether items are available before ordering.		☐ Passed ☐ Partial ☐ Failed	
5	Search and Filter Menu	Customers can search for food and narrow the menu by category so they can find items more easily.		☐ Passed ☐ Partial ☐ Failed	
6	Manage Menu	Authorized staff or the administrator can add, change, or remove menu items and update prices and photos. Customers see the updated menu.		☐ Passed ☐ Partial ☐ Failed	
7	Food Availability and Stock	Customers cannot order items marked as unavailable. When authorized staff update item availability or stock, the menu shows the current status.		☐ Passed ☐ Partial ☐ Failed	
8	Shopping Cart	Customers can add food to the cart, remove items, change quantities, and check that the total amount updates correctly.		☐ Passed ☐ Partial ☐ Failed	
9	Place an Order	Customers can review their selected food and submit an order. The system records the order so canteen staff can view and prepare it.		☐ Passed ☐ Partial ☐ Failed	
10	Pickup or Campus Delivery	Customers can choose pickup or campus delivery when available. For delivery, the system accepts only a location within the allowed campus area.		☐ Passed ☐ Partial ☐ Failed	
11	Cash Payment	Customers can pay in cash. Canteen staff can scan the order's queue slip or barcode and confirm that payment has been received. The payment status updates after confirmation.		☐ Passed ☐ Partial ☐ Failed	
12	Order Confirmation and Queue Slip	After an order is placed, the customer can view the order details and its queue slip or barcode for staff to check.		☐ Passed ☐ Partial ☐ Failed	
13	Handle Food Orders	Canteen staff can view incoming orders on the Counter POS or Kitchen Board and update the order as it moves from pending to preparing, ready, and completed.		☐ Passed ☐ Partial ☐ Failed	
14	Check Order Status	Customers can check the latest order status. Canteen staff can update food-order progress, and delivery personnel can update delivery progress when handling a delivery.		☐ Passed ☐ Partial ☐ Failed	
15	Delivery Requests and Updates	Delivery personnel can view available delivery requests, accept or reject a request, view the delivery location, follow the map, and update the delivery status. Customers can view the delivery progress.		☐ Passed ☐ Partial ☐ Failed	
16	Order Cancellation	When an order is cancelled, the system updates the order and restores the food stock. Any loyalty points used for that order are returned when applicable.		☐ Passed ☐ Partial ☐ Failed	
17	Order and Delivery Updates	The system shows relevant order updates to customers and staff. Delivery personnel can check the delivery details needed to complete an accepted request.		☐ Passed ☐ Partial ☐ Failed	
18	Order History	Customers can review their previous orders. Canteen staff, delivery personnel, and the administrator can view order or delivery records available to their role.		☐ Passed ☐ Partial ☐ Failed	

APPENDIX E: SAMPLE INPUT/OUTPUT/REPORTS

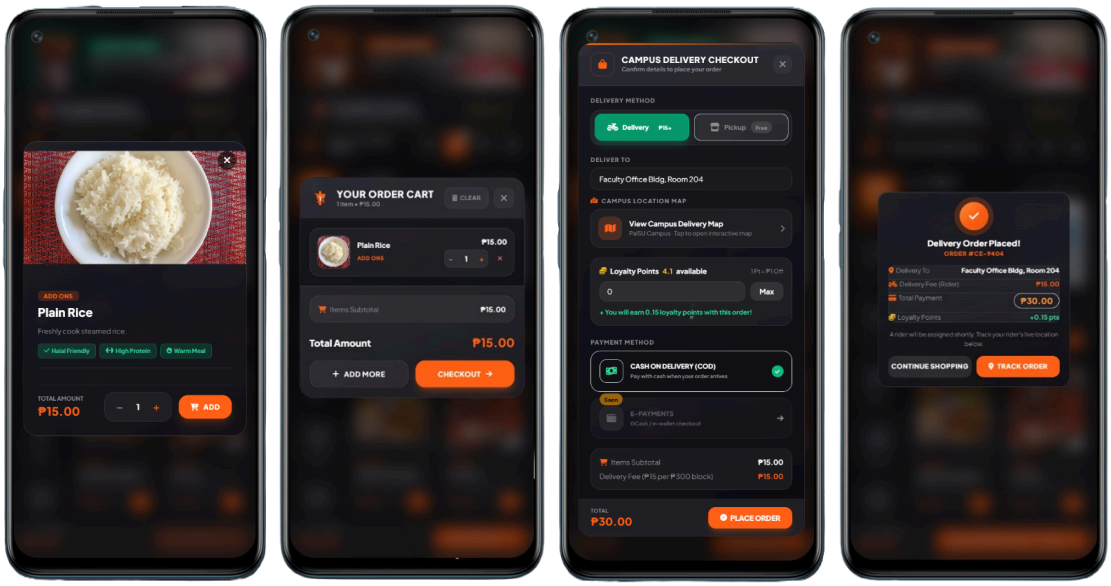


Figure 6.20: Sample Input

Figure 6.20 illustrates the customer ordering and campus delivery workflow, beginning with item customization and cart review, proceeding through the campus delivery checkout screen with address geofencing, loyalty points redemption and payment selection and concluding with an order confirmation screen displaying the assigned order number and options to track live GPS delivery or continue shopping.

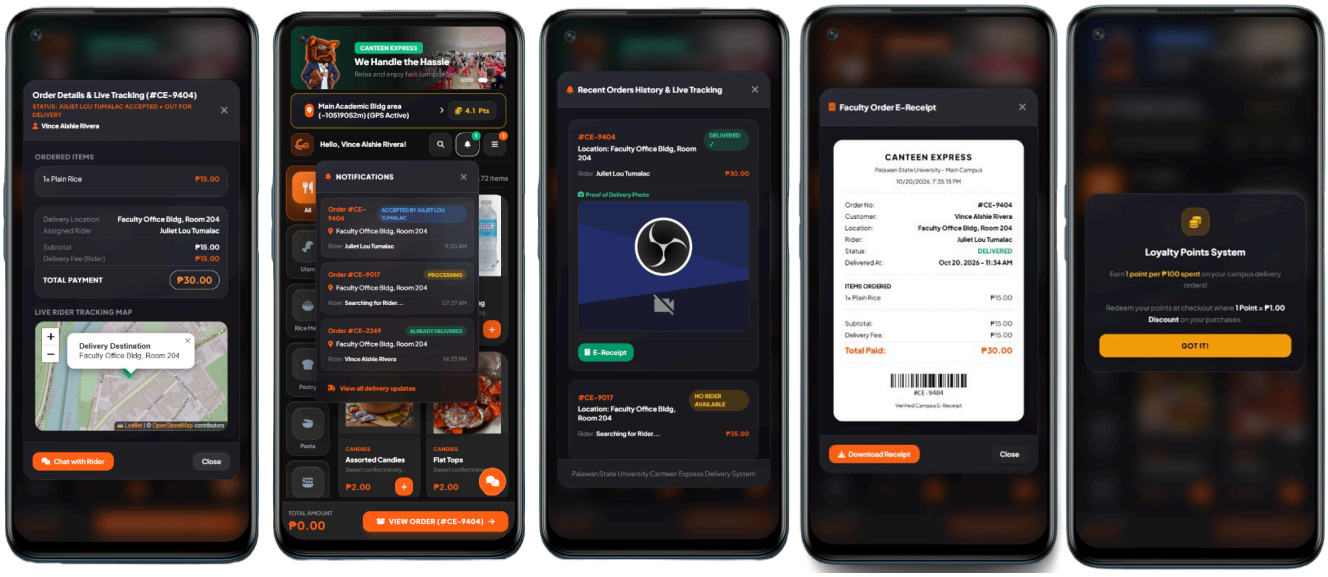


Figure 6.21: Sample Output

Figure 6.21 presents Sample Output, which are the live order tracking and post-transaction output interfaces, featuring real time GPS tracking with road routing and rider communication, the active notification dropdown for order status updates, the recent orders history panel, the digital e-receipt complete with itemized billing, timestamps and scannable barcodes (which have an alternative of inputting order number) and the loyalty points rewards summary modal.

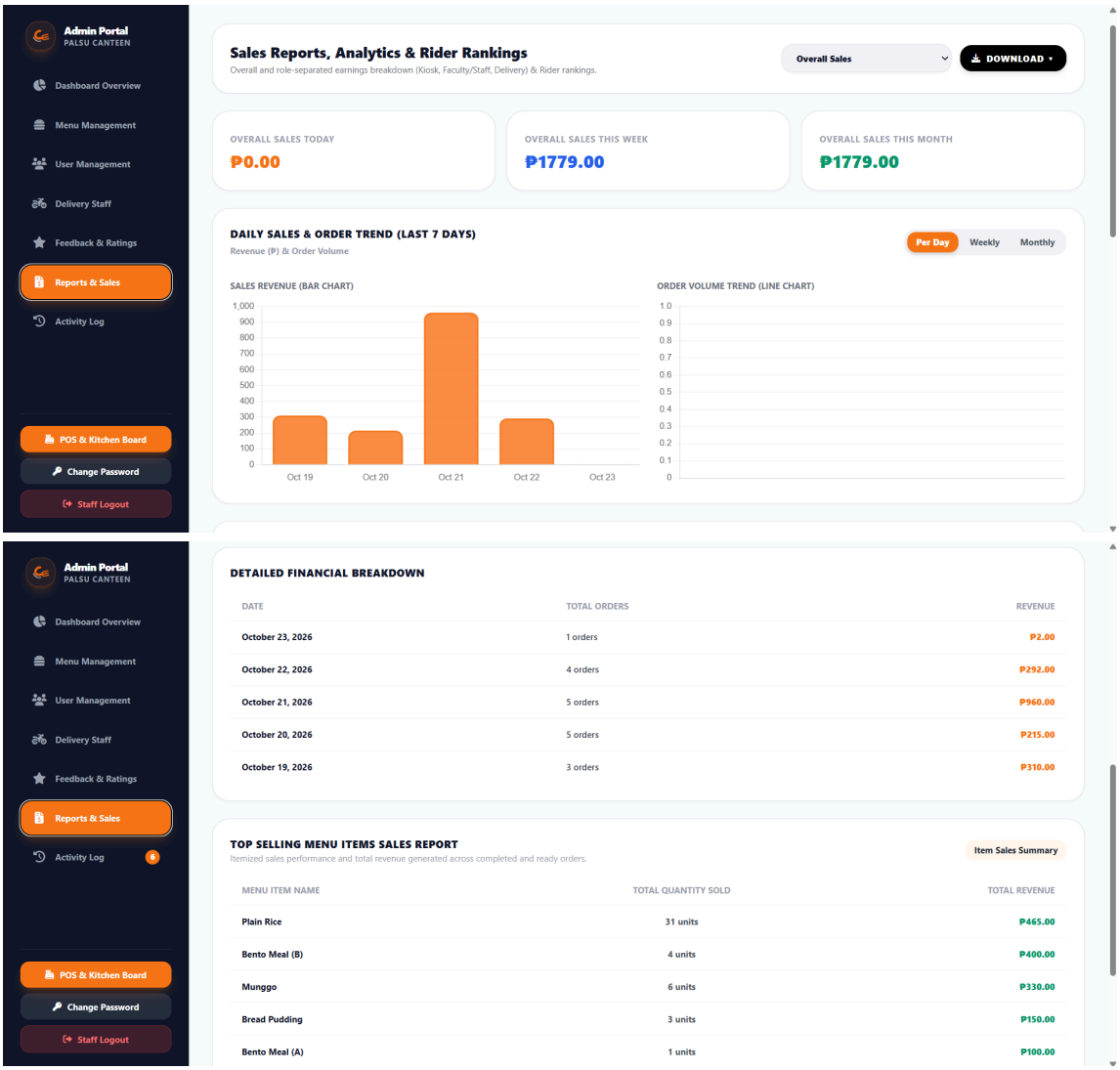
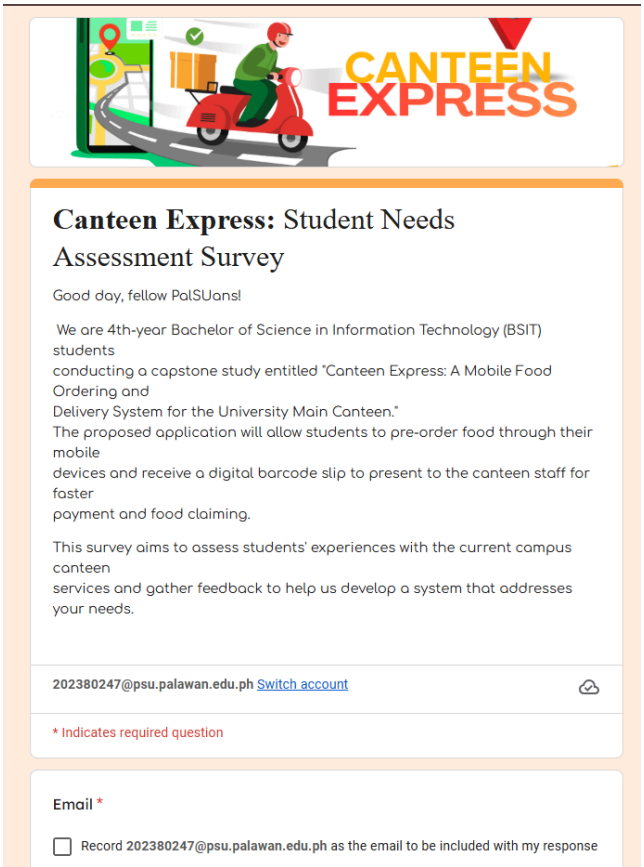


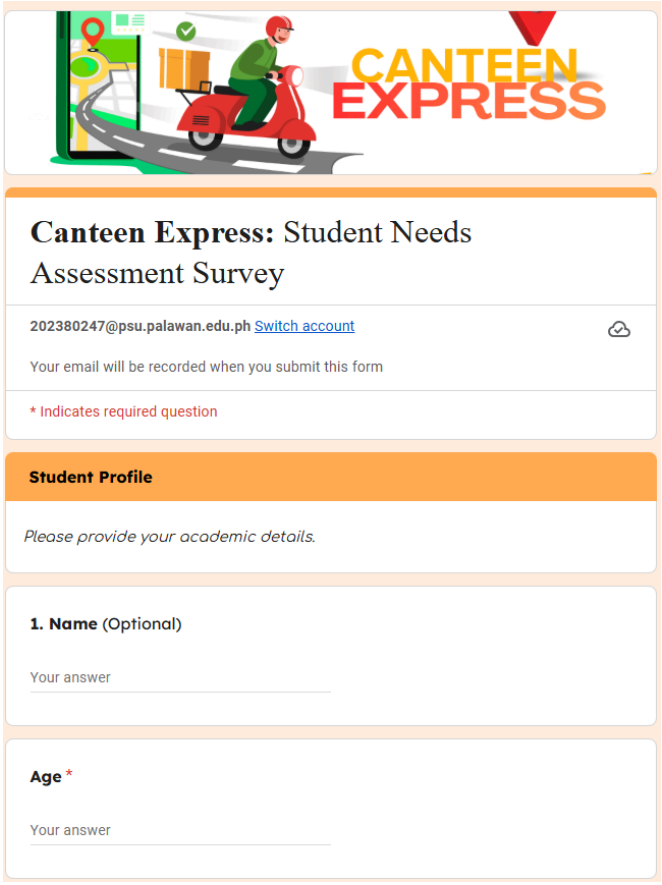
Figure 6.22: Sample Reports

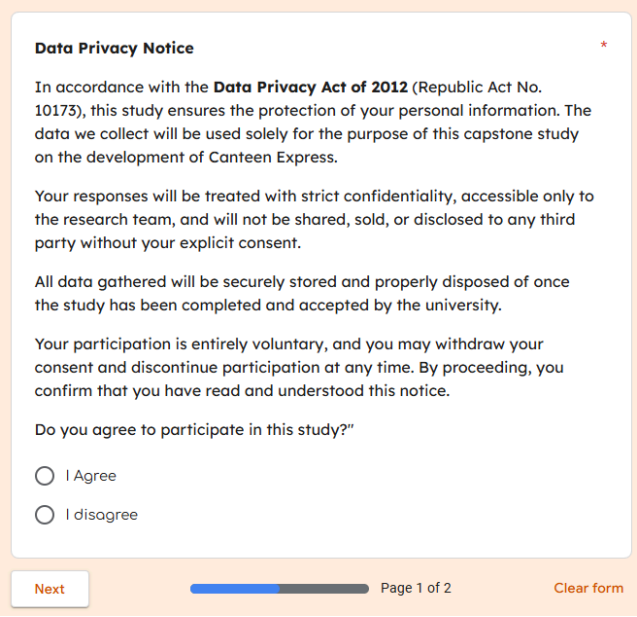
This figure displays the Sample Reports of the web application. In the figures shown are the Admin Portal Sales Reports and Analytics dashboard, featuring summary cards for daily, weekly and monthly sales performance and an interactive chart. While js graphs tracking daily sales revenue trends and order volumes over time, a detailed financial breakdown table by date and an itemized top-selling menu items sales report highlighting total quantity sold and revenue generated.

APPENDIX F: EVALUATION TOOL OR TEST DOCUMENTS

Data Gathering (Students)







2. How often do you buy meals or snacks at the MAIN CANTEEN? *

- ○ Daily
- ○ 2-3 times a week
- ○ Once a week
- ○ Rarely
- ○ No, I never buy at the canteen (If no, please skip to section 3)

3. On average, how much time do you spend waiting in line to order and pay? *

- ○ Daily
- ○ 2-3 times a week
- ○ Once a week
- ○ Rarely
- ○ No, I never buy at the canteen (If no, please skip to section 3)

4. Have you ever found it challenging to manage your time when purchasing food at the canteen before a class or activity? *

- ○ Yes, frequently
- ○ Yes, a few times
- ○ Rarely
- ○ No, never

5. How often do you decide not to purchase food at the canteen because you have limited time before your next class or activity? *

- ○ Always
- ○ Often
- ○ Sometimes
- ○ Rarely
- ○ Never

6. Have you ever walked to the canteen only to find out that the food you wanted was already sold out or unavailable? *

- ○ Yes, frequently
- ○ Yes, a few times
- ○ Rarely
- ○ No, never

7. If an app allowed you to view the real-time menu and item availability before walking to the canteen, would you use it? *

- ○ Yes, definitely
- ○ Yes, if the app is fast and easy to use
- ○ Undecided
- ○ No

8. Would presenting a digital barcode slip at a dedicated "Express Lane" counter help reduce your waiting time? *

- ○ Yes, significantly
- ○ Yes, somewhat
- ○ No, it would take longer
- ○ No difference

9. Which features are most important to you for a campus food app? *

- ☐ Real-time item availability
- ☐ Real-time menu viewing
- ☐ Digital barcode for fast claiming
- ☐ Clean and modern user interface
- ☐ Order-ahead / pre-ordering
- ☐ Order status tracking
- ☐ On-campus delivery
- ☐ Other: _____

10. If an on-campus delivery service is implemented, would you be interested in working as a freelance student delivery courier during your vacant hours to earn extra income? *

- ○ Yes, I am very interested in earning extra income on campus
- ○ Maybe, depending on my class schedule and work flexibility
- ○ No, I am not interested

Do you have any additional suggestions or features you would like to see in Canteen Express? (*Optional*) *

Your answer

A copy of your responses will be emailed to 202380247@psu.palawan.edu.ph.

Back | Submit | Page 2 of 2 | Clear form

Data Gathering (Staff/Faculty)

Canteen Express: Faculty & Staff Needs Assessment

Good day, PalSU Faculty and Staff!

We are 4th-year Information Technology students conducting a capstone study to develop **Canteen Express**, a mobile food-ordering app for our campus.

Designed for your convenience, the app will allow you to pre-order meals and have them delivered directly to your office. This survey aims to understand your current canteen experiences so we can build a system that best supports your needs. Your responses are voluntary and strictly confidential.

Thank you for your time and support!

202380247@psu.palawan.edu.ph Switch account

Not shared

* Indicates required question

Data Privacy Notice *

In accordance with the **Data Privacy Act of 2012** (Republic Act No. 10173), this study ensures the protection of your personal information. The data we collect will be used solely for the purpose of this capstone study on the development of Canteen Express.

Your responses will be treated with strict confidentiality, accessible only to the research team, and will not be shared, sold, or disclosed to any third party without your explicit consent.

All data gathered will be securely stored and properly disposed of once the study has been completed and accepted by the university.

Your participation is entirely voluntary, and you may withdraw your consent and discontinue participation at any time. By proceeding, you confirm that you have read and understood this notice.
Do you agree to participate in this study?

- ◯ I Agree
- ◯ I Do Not Agree

Next Clear form

Canteen Express: Faculty & Staff Needs Assessment

202380247@psu.palawan.edu.ph Switch account

Not shared

* Indicates required question

Part I: User Profile

Please provide your basic information and purchasing habits.

1. College / Department / Office *

Your answer

2. Primary Role *

- ◯ Teaching Faculty / Instructor
- ◯ Non-Teaching Staff / Administration

Back Next Clear form

Part II: Current Situation

A. Time-Related Questions

Please select the option that best estimates your current time spent.

3. Do you buy meals or snacks specifically at the MAIN CANTEEN of the campus? If so, how often? *

- ◯ Yes, Daily
- ◯ 2-3 times a week
- ◯ Yes, Once a week or less
- ◯ No, I rarely or never buy at the main canteen

4. Approximately how long does it take you to walk from your office/classroom to the Main Canteen? *

- ◯ Less than 2 minutes
- ◯ 2-5 minutes
- ◯ 6-10 minutes
- ◯ More than 10 minutes

5. I need to leave my office or assigned workplace to purchase food from the Main Canteen. *

- ◯ Less than 5 minutes
- ◯ 5-10 minutes
- ◯ 11-15 minutes
- ◯ More than 15 minutes

Back Next Clear form

B. Current Experience

Using the scale below, indicate your level of agreement.
(Scale: 5 = Strongly Agree | 4 = Agree | 3 = Neutral | 2 = Disagree | 1 = Strongly Disagree)

6. Walking to the Main Canteen affects my productivity or interrupts my work responsibilities. *

1	2	3	4	5
◯	◯	◯	◯	◯

7. Long queues at the Main Canteen cause me to spend more time purchasing food than I would prefer. *

1	2	3	4	5
◯	◯	◯	◯	◯

8. Because of my work schedule and the time required to purchase food, I sometimes delay or skip meals. *

1 2 3 4 5

○ ○ ○ ○ ○

9. Overall, I am satisfied with the current process of purchasing food from the Main Canteen. *

1 2 3 4 5

○ ○ ○ ○ ○

Back Next Clear form

Part III: Proposed Canteen Express System

Using the scale below, indicate your level of agreement regarding the proposed mobile app.
(Scale: 5 = Strongly Agree | 4 = Agree | 3 = Neutral | 2 = Disagree | 1 = Strongly Disagree)

10. A mobile application that allows me to pre-order food would save me time during office hours or between classes. *

1 2 3 4 5

○ ○ ○ ○ ○

11. Receiving my food through on-campus delivery would reduce the need for me to leave my office or workplace. *

1 2 3 4 5

○ ○ ○ ○ ○

12. Being able to place my order before going to the canteen would reduce waiting time. *

1 2 3 4 5

○ ○ ○ ○ ○

User Testing (Students)

PalSU Students/Walk-Ins Mobile Survey

Good day, PalSU Students and Walk-In Patrons!

We are **4th-year Information Technology students** conducting a capstone research study to develop and evaluate Canteen Express, a web and mobile Progressive Web App (PWA) designed for self-service kiosk ordering, queue slip generation, and counter POS processing. You are invited to participate because your feedback is vital in assessing system requirements, interface navigation, and ordering efficiency.

The purpose of this study is to evaluate the walk-in ordering experience and test system features, including category-based menu customization, QR/barcode queue slip generation, and kiosk responsiveness. Participation involves completing a structured evaluation questionnaire, which entails minimal psychological, physical, or social risks—no greater than everyday mobile or web browsing.

While there is no direct monetary compensation, your participation directly contributes to improving campus canteen queuing workflows, reducing waiting times, and enhancing student self-service ordering.

202380247@psu.palawan.edu.ph Switch account

* Indicates required question

Email *

Record 202380247@psu.palawan.edu.ph as the email to be included with my response

Data Privacy Notice *

In accordance with the **Data Privacy Act of 2012 (Republic Act No. 10173)**, your personal information and responses will be treated with strict confidentiality. Data will be secured through strong authentication mechanisms and secure session management.

Survey responses will be kept strictly **anonymous** with no personal identifiers collected, while any transactional logs prepared for quantitative evaluation will be properly coded and shared solely with the designated project statistician under strict data privacy protocols. All collected participant and system data will be retained temporarily during the evaluation and deployment phase, after which they will be permanently deleted and destroyed by May 2027.

Your participation is entirely voluntary. You may refuse to participate or withdraw your consent and discontinue participation at any time without penalty or loss of benefits.

Do you agree to participate in this study?"

- ○ I Agree
- ○ I Do Not Agree

1. Age *

- ○ Above 18
- ○ Below 18

Next Page 1 of 3 Clear form

PalSU Students/Walk-Ins Mobile Survey

202380247@psu.palawan.edu.ph Switch account

Your email will be recorded when you submit this form

* Indicates required question

Student Profile

Please provide your academic details.

Name (Optional)

Sample

Age

21

College *

Choose

Demographics *(Optional)*

A. User Classification *

This section identifies your current challenges when buying food at the main canteen.

- ○ Student
- ○ Walk-in Visitor / Guest
- ○ Other

B. Frequency of Canteen Ordering *

- ○ First-time user
- ○ 1-2 times a week
- ○ 3-4 times a week
- ○ Daily / Almost daily

User Experience & System Evaluation

Please rate your agreement with each statement below using the scale:

1 = Strongly Disagree | 2 = Disagree | 3 = Neutral | 4 = Agree | 5 = Strongly Agree

	1	2	3	4	5	
1 = Strongly Disagree	○	○	○	○	○	5 = Strongly Agree

1. Ease of Navigation: The kiosk / mobile interface was intuitive and easy to navigate when browsing menu categories and placing orders. *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

2. Order Customization: The system provided clear options for customizing food and beverage orders (e.g., matching side dishes or add-ons correctly). *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

3. Queue Slip & Barcode Generation: The generated queue slips and barcode made it easy and convenient to track order status or claim items at the counter. *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

4. Checkout Efficiency: The checkout process was simple, fast, and easy to complete without unnecessary steps. *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

5. System Responsiveness: The pages loaded quickly and responded reliably without lagging during peak ordering hours. *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

6. Feature Availability Clarity: The system clearly indicated which features and services were available or restricted for my user account. *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

7. Transparency on Menu Availability: The system accurately reflected real-time food availability and pricing without causing confusion. *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

8. Visual Clarity: The layout, typography, and dark-mode styling made the interface comfortable to view and interact with. *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

9. Overall Functionality: The system successfully met my expectations as a self-service walk-in ordering tool. *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

10. Recommendation: I would gladly use this kiosk/web system again for ordering meals at the campus canteen. *

1 2 3 4 5

1 = Strongly Disagree ○ ○ ○ ○ ○ 5 = Strongly Agree

User Testing (Staff/Faculty)

PalSU Staff and Faculty Mobile Ordering Survey

Good day, PalSU Faculty and Staff!

We are **4th-year Information Technology students** conducting a capstone research study to develop and evaluate Canteen Express, a web and mobile Progressive Web App (PWA) designed for automated canteen ordering, counter POS, and campus delivery. You are invited to participate because your feedback is essential in assessing system requirements, usability and operational efficiency.

The purpose of this study is to evaluate your current canteen experience and test advanced system capabilities, including authenticated institutional login (**@psu.palawan.edu.ph**), loyalty points accumulation, delivery scheduling, and real-time GPS tracking. Participation involves completing a **structured evaluation questionnaire, which entails minimal psychological, physical, or social risks no greater than everyday computer usage.**

While there is no direct monetary compensation, your participation directly contributes to improving campus canteen services, automated queuing, and efficient office meal delivery.

202380247@psu.palawan.edu.ph Switch account

* Indicates required question

Email *

☑ Record 202380247@psu.palawan.edu.ph as the email to be included with my response

Data Privacy Notice *

In accordance with the **Data Privacy Act of 2012 (Republic Act No. 10173),** your personal information and responses will be treated with strict confidentiality. Data will be secured through strong authentication mechanisms (such as secure password hashing and role-based session tokens).

Survey responses will be kept **anonymous**, and data prepared for quantitative evaluation will be properly coded and shared solely with the designated project statistician under strict data privacy protocols. All collected participant and system data will be retained temporarily during the evaluation and deployment phase, after which they will be permanently deleted and destroyed by May 2027.

Your participation is entirely voluntary. You may refuse to participate or withdraw your consent and discontinue participation at any time without penalty or loss of benefits.

Do you agree to participate in this study?

◉ I Agree

○ I Do Not Agree

PalSU Staff and Faculty Mobile Ordering Survey

202380247@psu.palawan.edu.ph Switch account

Your email will be recorded when you submit this form

* Indicates required question

Profile

Please provide your academic details.

Name (Optional)

Sample

1. Age *

21

Demographics & Canteen Usage Patterns

Optional background questions to analyze system requirements across university units.

A. College / Office / Department *

Your answer

B. Employment Status *

○ Full-time Faculty

○ Part-time / Visiting Faculty

○ Administrative / Support Staff

○ Utility / Project-based Personnel

C. Canteen & Office Meal Frequency *

○ First-time user

○ 1–2 times a week

○ 3–4 times a week

○ Daily / Almost daily

1. Authentication & Account Privileges: Logging in with institutional credentials (@psu.palawan.edu.ph) provided secure access to my faculty/staff dashboard and profile features. *

- [ ] Strongly Disagree
- [ ] Disagree
- [ ] Neutral
- [ ] Agree
- [ ] Strongly Agree

2. Loyalty Points & History: The system accurately tracked my accumulated loyalty points and displayed past order history correctly. *

- [ ] Strongly Disagree
- [ ] Disagree
- [ ] Neutral
- [ ] Agree
- [ ] Strongly Agree

3. Campus Delivery Request: The delivery ordering feature was easy to configure with accurate destination coordinates and campus boundary validation. *

- [ ] Strongly Disagree
- [ ] Disagree
- [ ] Neutral
- [ ] Agree
- [ ] Strongly Agree

4. Real-Time Delivery Tracking: The integrated dark-mode map and live GPS tracking made it easy to monitor delivery progress in real time. *

- [ ] Strongly Disagree
- [ ] Disagree
- [ ] Neutral
- [ ] Agree
- [ ] Strongly Agree

5. In-App Communication: The real-time messaging feature facilitated clear and efficient communication regarding delivery updates. *

- [ ] Strongly Disagree
- [ ] Disagree
- [ ] Neutral
- [ ] Agree
- [ ] Strongly Agree

6. Counter POS & Barcode Processing: The canteen counter POS efficiently processed barcodes and scanned queue slips to update order payment status instantly. *

- ( ) Strongly Disagree
- ( ) Disagree
- ( ) Neutral
- ( ) Agree
- ( ) Strongly Agree

7. Mobile PWA Experience: The Progressive Web App (PWA) provided a fast, responsive, and app-like experience on mobile and desktop devices. *

- [ ] Strongly Disagree
- [ ] Disagree
- [ ] Neutral
- [ ] Agree
- [ ] Strongly Agree

8. Meal Customization & Menu Browsing: Browsing the canteen menu categories, viewing detailed meal photos, and customizing options (such as rice add-ons and drink choices) during checkout was intuitive and user-friendly. *

- [ ] Strongly Disagree
- [ ] Disagree
- [ ] Neutral
- [ ] Agree
- [ ] Strongly Agree

9. Role-Based Capabilities: The system effectively distinguished and supported my advanced permissions (delivery scheduling, loyalty rewards, and priority ordering) compared to walk-in users. *

- [ ] Strongly Disagree
- [ ] Disagree
- [ ] Neutral
- [ ] Agree
- [ ] Strongly Agree

10. Overall User Satisfaction & Reliability: The faculty/staff ordering and delivery platform provided a reliable, smooth, and highly convenient experience for daily campus meals. *

- [ ] Strongly Disagree
- [ ] Disagree
- [ ] Neutral
- [ ] Agree
- [ ] Strongly Agree

A copy of your responses will be emailed to 202380247@psu.palawan.edu.ph.

Back Submit Page 3 of 3 Clear form

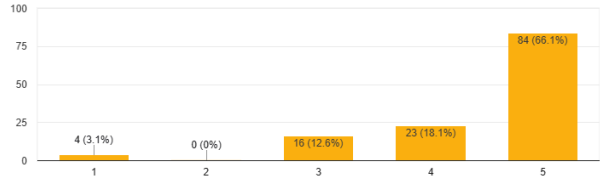
APPENDIX G: TEST RESULTS

Users (Students/Walk-In)

Copy chart

1. Ease of Navigation: The kiosk / mobile interface was intuitive and easy to navigate when browsing menu categories and placing orders.

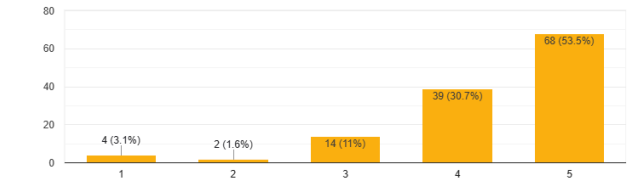
127 responses



Copy chart

5. System Responsiveness: The pages loaded quickly and responded reliably without lagging during peak ordering hours.

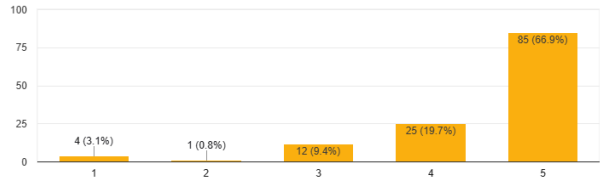
127 responses



Copy chart

2. Order Customization: The system provided clear options for customizing food and beverage orders (e.g., matching side dishes or add-ons correctly).

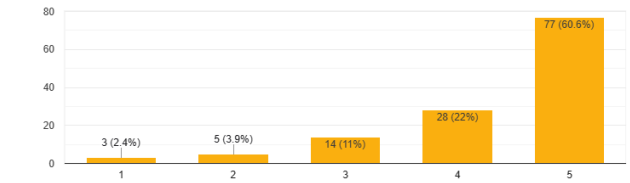
127 responses



Copy chart

6. Feature Availability Clarity: The system clearly indicated which features and services were available or restricted for my user account.

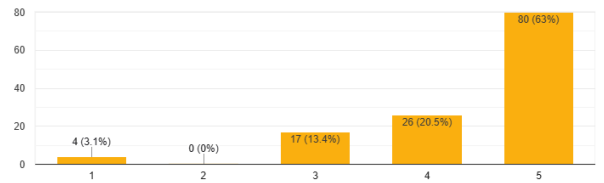
127 responses



Copy chart

3. Queue Slip & Barcode Generation: The generated queue slips and barcode made it easy and convenient to track order status or claim items at the counter.

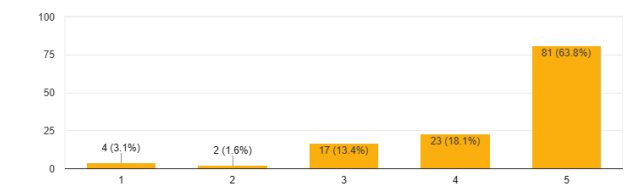
127 responses



Copy chart

7. Transparency on Menu Availability: The system accurately reflected real-time food availability and pricing without causing confusion.

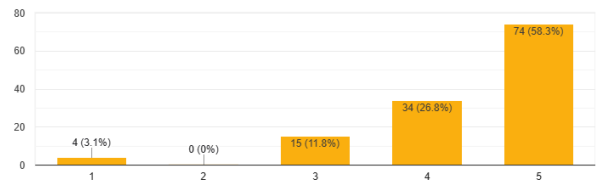
127 responses



Copy chart

4. Checkout Efficiency: The checkout process was simple, fast, and easy to complete without unnecessary steps.

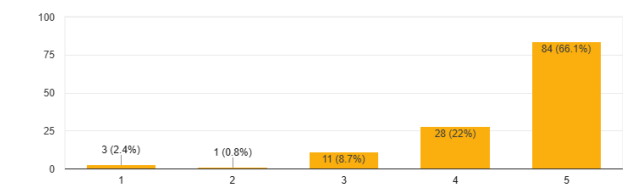
127 responses

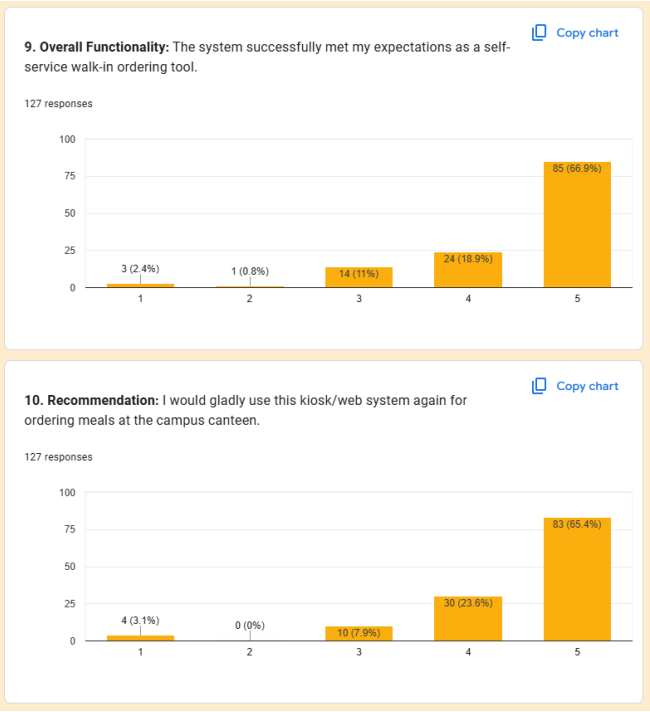


Copy chart

8. Visual Clarity: The layout, typography, and dark-mode styling made the interface comfortable to view and interact with.

127 responses

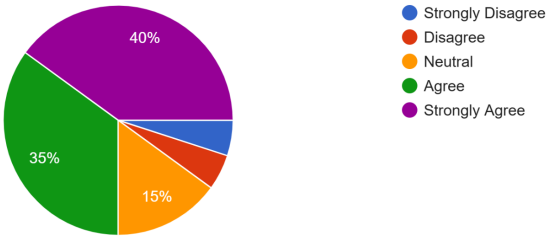




Users (Faculty/Staff)

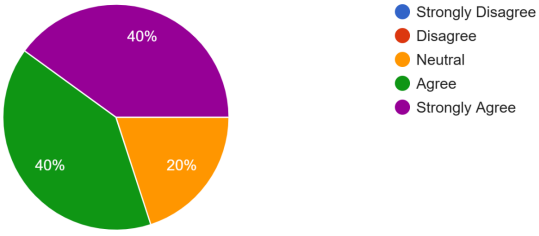
1. Authentication & Account Privileges: Logging in with institutional credentials (@psu.palawan.edu.ph) provided secure access to my faculty/staff dashboard and profile features.

20 responses



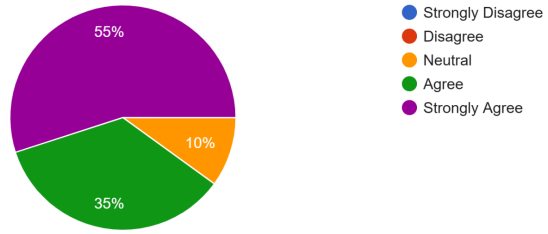
2. Loyalty Points & History: The system accurately tracked my accumulated loyalty points and displayed past order history correctly.

20 responses



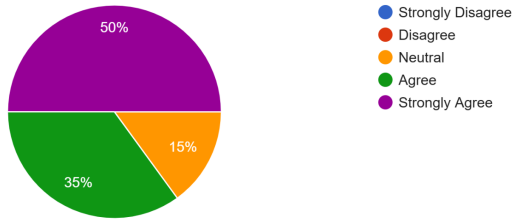
3. Campus Delivery Request: The delivery ordering feature was easy to configure with accurate destination coordinates and campus boundary validation.

20 responses



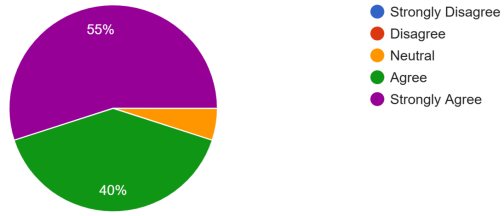
4. Real-Time Delivery Tracking: The integrated dark-mode map and live GPS tracking made it easy to monitor delivery progress in real time.

20 responses



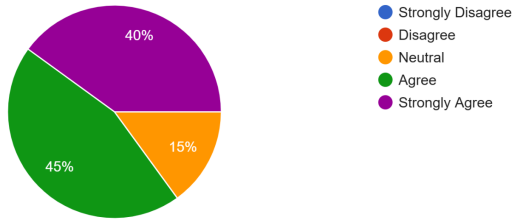
5. In-App Communication: The real-time messaging feature facilitated clear and efficient communication regarding delivery updates.

20 responses



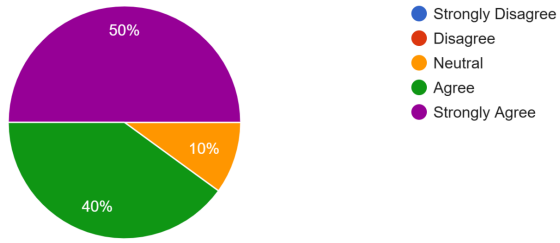
6. Counter POS & Barcode Processing: The canteen counter POS efficiently processed barcodes and scanned queue slips to update order payment status instantly.

20 responses



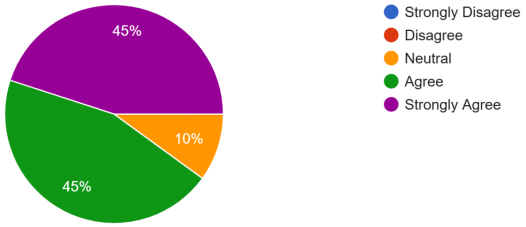
7. Mobile PWA Experience: The Progressive Web App (PWA) provided a fast, responsive, and app-like experience on mobile and desktop devices.

20 responses



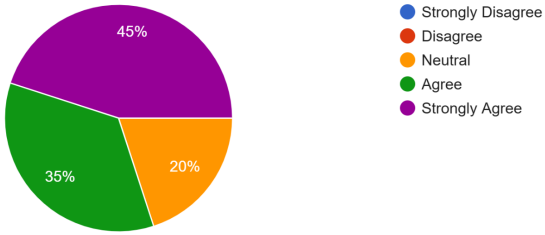
8. Meal Customization & Menu Browsing: Browsing the canteen menu categories, viewing detailed meal photos, and customizing options (such as rice ...) during checkout was intuitive and user-friendly.

20 responses



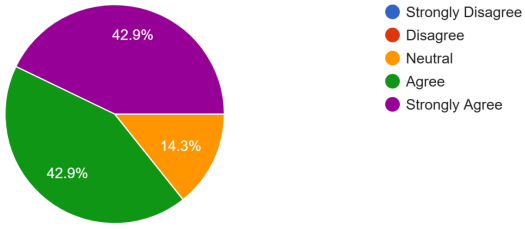
9. Role-Based Capabilities: The system effectively distinguished and supported my advanced permissions (delivery scheduling, loyalty rewards, and priority ordering) compared to walk-in users.

20 responses



10. Overall User Satisfaction & Reliability: The faculty/staff ordering and delivery platform provided a reliable, smooth, and highly convenient experience for daily campus meals.

20 responses



APPENDIX H: PROCESS/DATA/INFORMATION FLOW

Process Flow

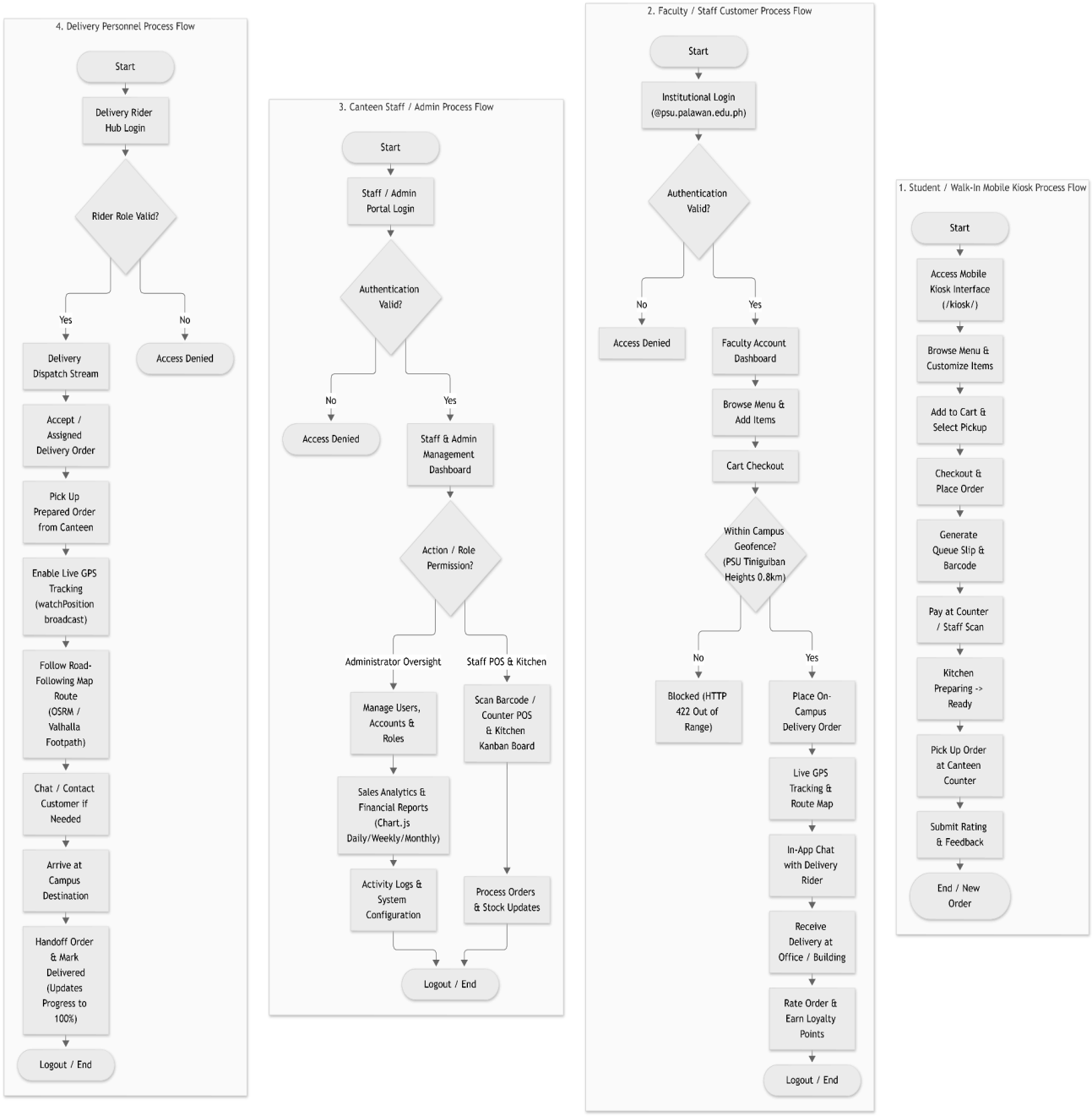


Figure 6.23: Process Flow of Canteen Express

The flowchart above shows the process flow of the Canteen Express system, which illustrates the navigation and order process for the four user roles: Student/Walk-In Mobile Kiosk Customer, Faculty/Staff Customer, Canteen/Admin and Delivery Personnel. While the student/walk-in mobile kiosk starts directly with open self-service menu browsing and queue slip generation without authentication, the institutional, staff/admin and delivery personnel roles start with login authentication and continue to their respective dashboards to access features based on their permissions. The processes conclude with order completion or a logout option once they are done using the app.

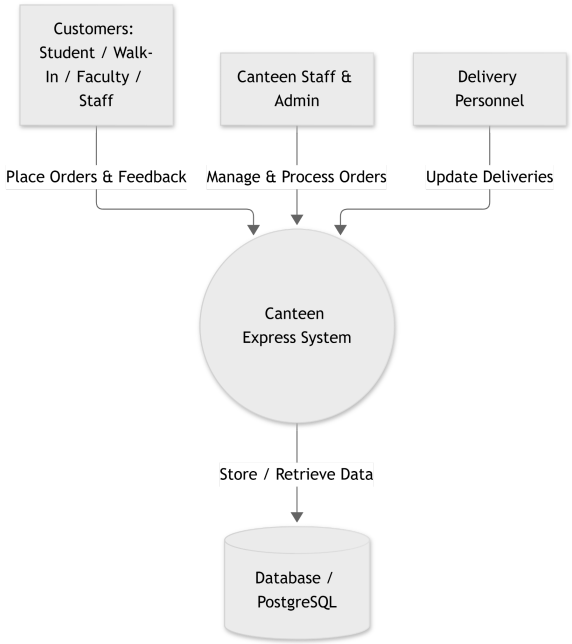


Figure 6.24: Data Flow of Canteen Express

The diagram above shows the data flow of the Canteen Express system, which illustrates how the system interacts with its external entities: Customers (Student/Walk-In/Faculty/Staff), Canteen Staff and Admin, Delivery Personnels and Database. Customers place orders and submit feedback, Canteen Staff and Admin manage and process system operations and orders and

Delivery Personnels handle deliveries. All data and transactions are stored and retrieved from the database to maintain system integrity.

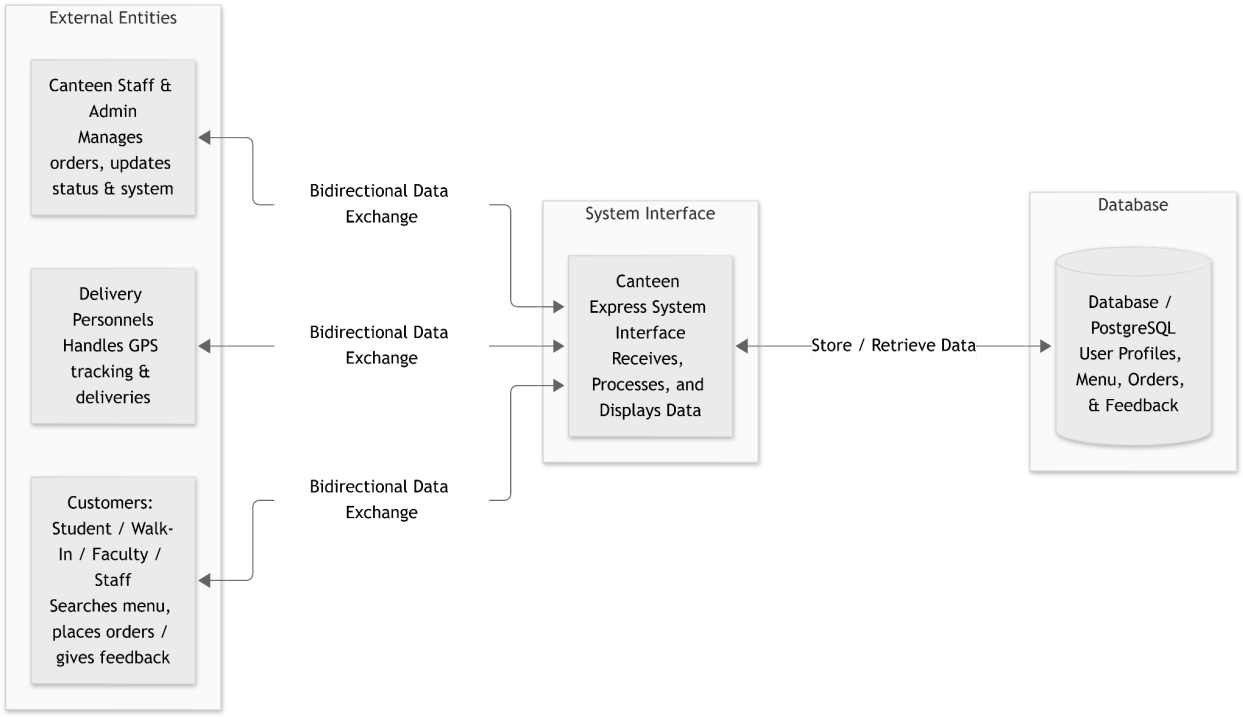


Figure 6.25: Information Flow of Canteen Express

Figure 6.25 shows the information flow of the Canteen Express system, which illustrates how information is exchanged among Delivery Personnels, Customers (Student/Walk-In/Faculty/Staff), Canteen Staff and Admin and the Database through the system interface. Customers can search the menu, place orders and give feedback/reviews. Canteen Staff and Admin manage orders and update status/system operations, while delivery personnel handle GPS tracking and deliveries. The system interface receives, processes and displays data, which are then stored and retrieved in the database containing user profiles, menu items, orders and feedback.

APPENDIX I: SAMPLE GENERATED OUTPUTS

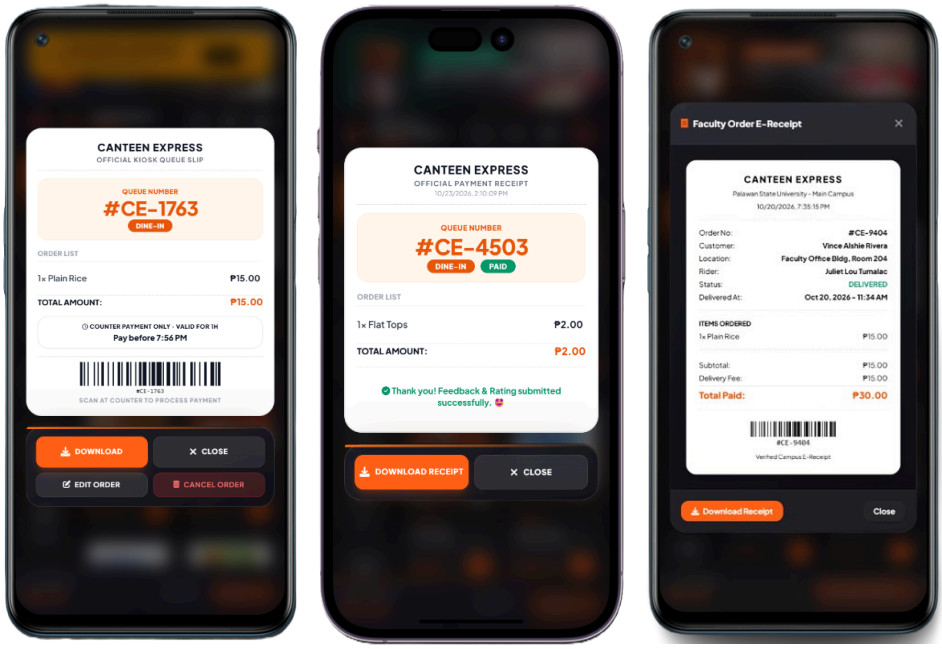


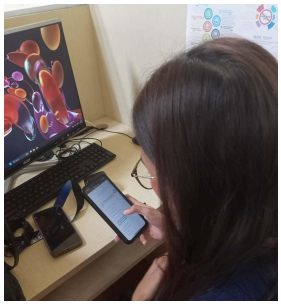
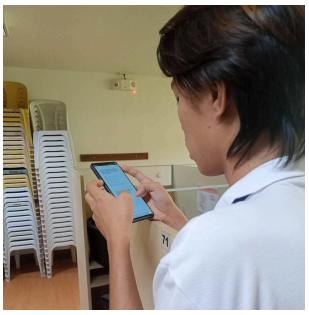
Figure 6.26: Mobile Kiosk Interface and E-Receipt Generation Workflow

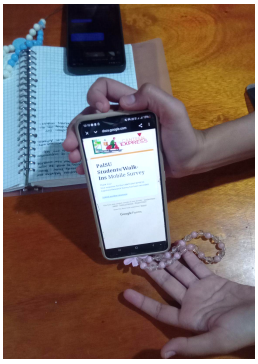
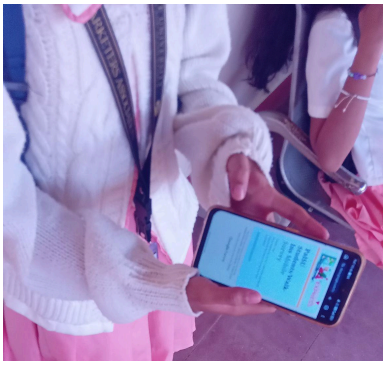
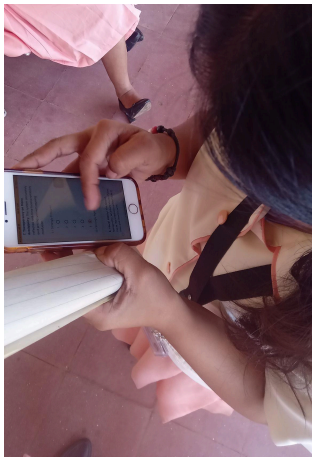
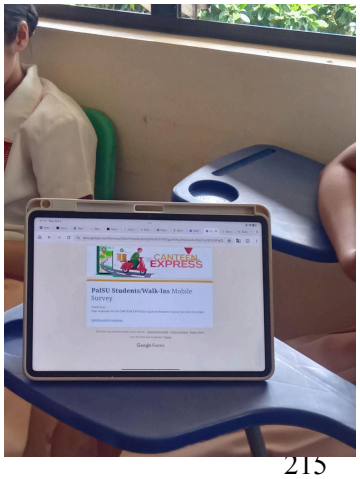
The figure above illustrates the mobile kiosk interface and receipt generation workflow of the Canteen Express system across three progressive stages: an initial unpaid queue slip featuring an order number and scannable barcode for counter cash payment (left), a confirmed payment receipt displaying a successful "PAID" status and feedback integration (middle) and a comprehensive faculty campus delivery e-receipt detailing customer information, delivery location, assigned rider, itemized pricing breakdown and verification barcode (right).

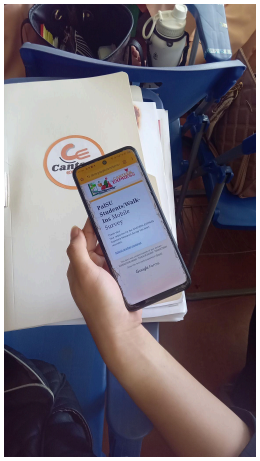
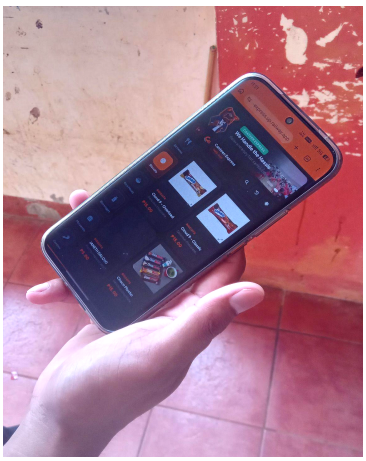
APPENDIX J: PICTURES SHOWCASING THE DATA GATHERING, INVESTIGATION DONE

Meeting with Client (Palawan State University Main Canteen Staff)

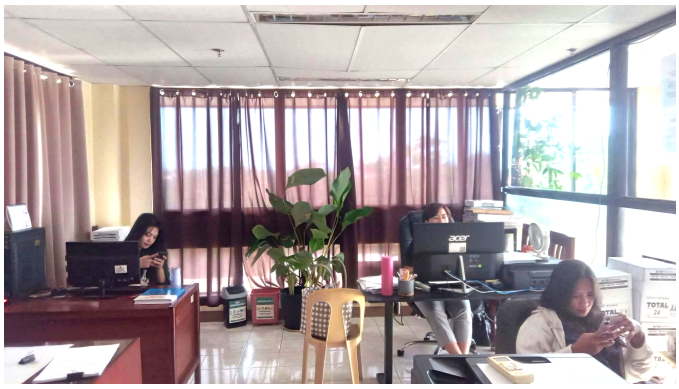
Data Gathering

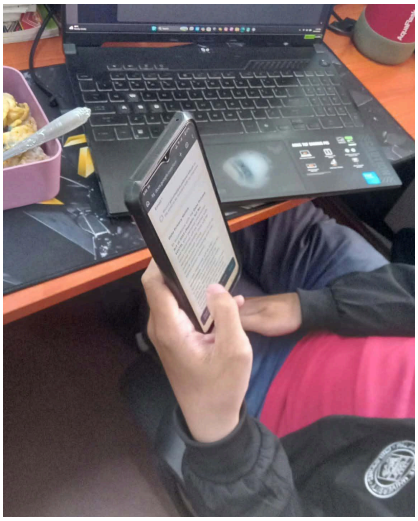



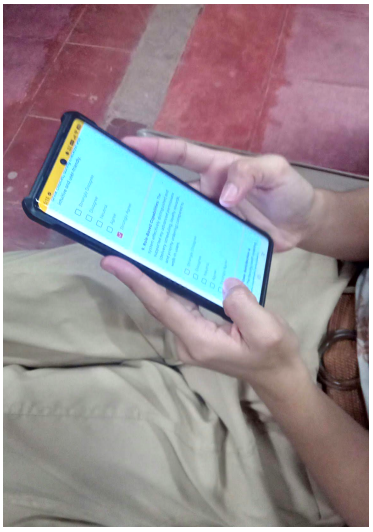





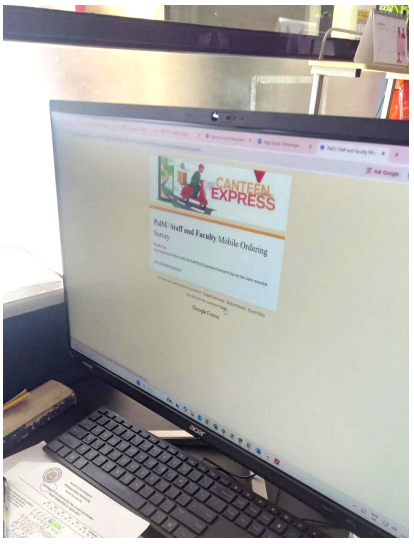



DR. MARYANN X. ARTICUELA
Deputy Director
Quality Assurance Center


acer






CANTEEN EXPRESS
PUP Staff and Faculty Mobile Ordering
Survey

APPENDIX K. RELEVANT SOURCE CODE

Student / Walk-in Customer (Mobile Kiosk Portal)

Allows students and walk-in customers to browse the menu, add items to their cart, check out instantly and generate a digital queue slip with a barcode for self-service ordering.

backend/customer_portal/models.py

```
from django.db import models
from django.conf import settings
import urllib.parse
class MenuItem(models.Model):
    CATEGORY_CHOICES = [
        ('biscuits', 'Biscuits'),
        ('rice', 'Rice Meals'),
        ('beverages', 'Beverages'),
        ('meryenda', 'Meryenda'),
        ('dietary', 'Dietary Plans'),
        ('candies', 'Candies'),
    ]
    name = models.CharField(max_length=120)
    category = models.CharField(max_length=20, choices=CATEGORY_CHOICES, default='rice')
    price = models.DecimalField(max_digits=8, decimal_places=2)
    description = models.TextField(blank=True, null=True)
    badge = models.CharField(max_length=50, blank=True, null=True, help_text="hal. Popular, Bestseller")
    # Suporta sa image file o Image URL
    image = models.ImageField(upload_to='menu_images/', blank=True, null=True)
    image_url = models.URLField(max_length=500, blank=True, null=True, help_text="URL galing sa internet")
    is_siomai = models.BooleanField(default=False, verbose_name="Is Siomai Customizer?")
    is_available = models.BooleanField(default=True, verbose_name="Available in Kiosk?")
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    def __str__(self):
        return f"{self.name} - ₱{self.price}"
    @property
    def get_image_src(self):
        if self.image_url and self.image_url.strip():
            return self.image_url
        if self.image:
            try:
                if self.image.storage.exists(self.image.name):
                    return self.image.url
            except Exception:
                pass
        name_lower = self.name.lower()
        cat_name = str(self.category).lower() if self.category else ''
        if 'coke' in name_lower or 'sprite' in name_lower or 'royal' in name_lower or 'pepsi' in name_lower or
'drink' in name_lower or 'beverage' in name_lower or 'coffee' in name_lower or 'energen' in name_lower:
            return "https://images.unsplash.com/photo-1554866585-cd94860890b7?auto=format&fit=crop&w=600&q=80"
        elif 'rice' in name_lower or 'silog' in name_lower or 'adobo' in name_lower or 'sinigang' in name_lower or
'menudo' in name_lower or 'curry' in name_lower or 'tinola' in name_lower or 'chicken' in name_lower or 'pork' in
name_lower or 'beef' in name_lower or 'fish' in name_lower or 'ulam' in cat_name:
            return "https://images.unsplash.com/photo-1555396273-367ea4eb4db5?auto=format&fit=crop&w=600&q=80"
        elif 'cookie' in name_lower or 'biscuit' in name_lower or 'crinkle' in name_lower or 'brownie' in
name_lower:
            return "https://images.unsplash.com/photo-1558961363-fa8fdf82db35?auto=format&fit=crop&w=600&q=80"
        elif 'turon' in name_lower or 'bread' in name_lower or 'pastry' in name_lower or 'meryenda' in name_lower
or 'snack' in name_lower or 'puto' in name_lower or 'suman' in name_lower:
            return "https://images.unsplash.com/photo-1509440159596-0249088772ff?auto=format&fit=crop&w=600&q=80"
        return "https://images.unsplash.com/photo-1546069901-ba9599a7e63c?auto=format&fit=crop&w=600&q=80"
```

```
class Order(models.Model):
    STATUS_CHOICES = [
        ('unpaid', 'Unpaid'),
        ('pending', 'Pending (Paid)'),
        ('preparing', 'Preparing'),
        ('ready', 'Ready'),
        ('completed', 'Completed'),
        ('cancelled', 'Cancelled'),
    ]
    order_number = models.CharField(max_length=20, unique=True, db_index=True)
    customer = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL, null=True, blank=True,
related_name='portal_orders', db_index=True)
    total_amount = models.DecimalField(max_digits=10, decimal_places=2)
    delivery_fee = models.DecimalField(max_digits=10, decimal_places=2, default=0.00)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='unpaid', db_index=True)
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
    points_redeemed = models.DecimalField(max_digits=8, decimal_places=2, default=0.00)
    points_earned = models.DecimalField(max_digits=8, decimal_places=2, default=0.00)
    guest_name = models.CharField(max_length=150, blank=True, null=True)
    guest_phone = models.CharField(max_length=30, blank=True, null=True)
    def __str__(self):
        return f"Order {self.order_number} - ₱{self.total_amount}"
    @property
    def total_payment(self):
        return self.total_amount + self.delivery_fee
class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name='items')
    item_name = models.CharField(max_length=150)
    price = models.DecimalField(max_digits=8, decimal_places=2)
    quantity = models.PositiveIntegerField(default=1)
    def __str__(self):
        return f"{self.quantity}x {self.item_name}"
    @property
    def total_price(self):
        return self.price * self.quantity
    @property
    def product(self):
        class DummyProduct:
            def __init__(self, name):
                self.name = name
        return DummyProduct(self.item_name)
class OrderFeedback(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name='feedbacks', null=True, blank=True)
    customer = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL, null=True, blank=True)
    rating = models.IntegerField(default=5)
    comment = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    class Meta:
        ordering = ['-created_at']
        constraints = [
            models.UniqueConstraint(
                fields=['order', 'customer'],
                name='uniq_feedback_order_customer',
            ),
        ]
    def __str__(self):
        return f"Feedback {self.rating}★ for Order {self.order.order_number if self.order else 'N/A'}"
```

backend/customer_portal/views.py

```
import json
import random
from decimal import Decimal
from django.db import IntegrityError, transaction
```

```python
from django.db.models import F
from django.utils import timezone
from django.core.cache import cache
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib import messages
from django.http import JsonResponse
from django.views.decorators.http import require_POST
from canteen_menu.models import MenuItem, Category
from .models import Order, OrderItem

class CheckoutError(Exception):
    pass
def _get_formatted_menu():
    cache_key = 'formatted_menu_active_kiosk'
    cached = cache.get(cache_key)
    if cached is not None:
        return cached
    db_items = MenuItem.objects.select_related('category').filter(is_available=True).order_by('-id')
    formatted_menu = []
    for item in db_items:
        img_url = ''
        if hasattr(item, 'get_image_src'):
            attr = getattr(item, 'get_image_src')
            img_url = attr() if callable(attr) else attr
        elif hasattr(item, 'image') and item.image:
            try:
                img_url = item.image.url
            except ValueError:
                img_url = ''

        category_str = 'General'
        if hasattr(item, 'category') and item.category:
            category_str = item.category.name if hasattr(item.category, 'name') else str(item.category)

        formatted_menu.append({
            'id': item.id,
            'name': item.name,
            'category': category_str,
            'price': float(item.price) if item.price else 0.0,
            'desc': getattr(item, 'description', '') or getattr(item, 'desc', '') or '',
            'badge': getattr(item, 'badge', '') or '',
            'isSiomai': getattr(item, 'is_siomai', False) or getattr(item, 'isSiomai', False),
            'stock': getattr(item, 'stock', None),
            'img': img_url
        })
    cache.set(cache_key, formatted_menu, 60)
    return formatted_menu
# 1. CUSTOMER SIDE - Landing Page
def kiosk_welcome(request):
    return render(request, 'customer_portal/kiosk_welcome.html')
# 2. CUSTOMER SIDE - Main Menu Page
def kiosk_menu(request):
    formatted_menu = _get_formatted_menu()
    categories = Category.objects.all().order_by('-name')
    # Add user context to kiosk
    context = {
        'menu_data_json': json.dumps(formatted_menu),
        'categories': categories,
        'user': request.user
    }
    return render(request, 'customer_portal/kiosk_menu.html', context)
# 3. CUSTOMER SIDE - Live Sync API
def get_kiosk_menu_api(request):
    formatted_menu = _get_formatted_menu()
    return JsonResponse({'status': 'success', 'menu': formatted_menu})
```

```
# 4. CUSTOMER SIDE - Checkout Endpoint
@require_POST
def process_checkout(request):
    Try:
        from core_app.system_status import get_status, is_accepting_orders
        if not is_accepting_orders():
            status = get_status()
            reason = status['message'] or (
                'The canteen is not accepting orders right now. '
                'Please try again later.'
            )
            return JsonResponse({
                'success': False,
                'message': reason,
                'system_closed': True,
                'mode': status['mode'],
            }, status=422)
        data = json.loads(request.body)
        cart_items = data.get('cart', [])
        is_delivery = data.get('is_delivery', False)
        delivery_location = data.get('delivery_location', 'Faculty Office Building')
        if not cart_items:
            return JsonResponse({'success': False, 'message': 'Cart is empty.'}, status=400)
        user = request.user if request.user.is_authenticated else None
        lines = {}  # {menu_item_id: {'menu_item': item, 'qty': qty}}
        for item in cart_items:
            try:
                qty = int(item.get('qty', 0))
            except (TypeError, ValueError):
                return JsonResponse({'success': False, 'message': 'Invalid cart item quantity.'}, status=400)
            if qty <= 0 or qty > 99:
                return JsonResponse({'success': False, 'message': 'Invalid cart item quantity.'}, status=400)

            item_id = item.get('id')
            item_name = str(item.get('name', '')).strip()
            menu_item = None
            if item_id:
                menu_item = MenuItem.objects.filter(id=item_id).first()
            if menu_item is None and item_name:
                menu_item = MenuItem.objects.filter(name__iexact=item_name).first()
            if menu_item is None:
                return JsonResponse({
                    'success': False,
                    'message': f'"{item_name or item_id}" is no longer on the menu. Please refresh and try again.'
                }, status=400)
            if menu_item.id in lines:
                lines[menu_item.id]['qty'] += qty
            else:
                lines[menu_item.id] = {'menu_item': menu_item, 'qty': qty}
        subtotal = round(sum(
            float(line['menu_item'].price) * line['qty']
            for line in lines.values()
        ), 2)
        if subtotal <= 0 or subtotal > 99999999.99:
            return JsonResponse({'success': False, 'message': 'Invalid cart total.'}, status=400)
        if is_delivery:
            from deliveries.utils import is_within_campus
            from django.conf import settings
            enforce_geofence = getattr(settings, 'ENFORCE_GEOFENCE', True)
            if enforce_geofence and not is_within_campus(data.get('dest_lat'), data.get('dest_lng')):
                return JsonResponse({
                    'success': False,
                    'message': 'Delivery is only available within the Palawan State University campus. Please make
sure your location is inside the campus and try again.'
                }, status=422)
```

```
        from deliveries.utils import delivery_fee_for_order
        delivery_fee = delivery_fee_for_order(subtotal) if is_delivery else 0.0
        points_earned = round(subtotal / 100.0, 2)
        try:
            points_redeemed = float(data.get('points_redeemed', 0) or 0)
        except (TypeError, ValueError):
            points_redeemed = 0.0
        if not user:
            points_redeemed = 0.0
        elif points_redeemed < 0:
            return JsonResponse({'success': False, 'message': 'Invalid loyalty points.'}, status=400)
        elif points_redeemed > float(user.loyalty_points or 0):
            return JsonResponse({'success': False, 'message': 'Insufficient loyalty points.'}, status=400)
        else:
            points_redeemed = min(points_redeemed, subtotal)
        total_amount = round(subtotal - points_redeemed, 2)

        initial_status = 'pending' if is_delivery else 'unpaid'
        # Guest delivery contact info (optional) so riders can reach a guest.
        guest_name = str(data.get('guest_name', '') or '').strip()[:150]
        guest_phone = str(data.get('guest_phone', '') or '').strip()[:30]
        def _create_order_with_unique_number(**order_kwargs):
            for _ in range(50):
                candidate = f"#CE-{random.randint(1000, 9999)}"
                try:
                    with transaction.atomic():
                        return Order.objects.create(order_number=candidate, **order_kwargs)
                except IntegrityError:
                    continue
            raise CheckoutError('Could not allocate a unique order number. Please try again.')
        with transaction.atomic():
            order = _create_order_with_unique_number(
                total_amount=total_amount,
                delivery_fee=delivery_fee,
                status=initial_status,
                customer=user,
                points_earned=points_earned,
                points_redeemed=points_redeemed,
                guest_name=guest_name or None,
                guest_phone=guest_phone or None,
            )
            if user and points_redeemed > 0:
                from accounts.models import CustomUser
                spent = CustomUser.objects.filter(
                    pk=user.pk, loyalty_points__gte=points_redeemed
                ).update(loyalty_points=F('loyalty_points') - Decimal(str(points_redeemed)))
                if not spent:
                    raise CheckoutError('Insufficient loyalty points.')
            for line in lines.values():
                menu = MenuItem.objects.select_for_update().get(pk=line['menu_item'].pk)
                qty = line['qty']
                if menu.stock is not None and menu.stock < qty:
                    raise CheckoutError(
                        f'Sorry, only {menu.stock} left for "{menu.name}". '
                        'Please reduce the quantity.'
                    )
                if menu.stock is not None:
                    menu.stock -= qty
                    if menu.stock <= 0:
                        menu.is_available = False
                        menu.save(update_fields=['stock', 'is_available'])
                    else:
                        menu.save(update_fields=['stock'])
            # Stock changed -> the kiosk's cached menu (60s TTL) must refresh now.
            cache.delete('formatted_menu_active_kiosk')
```

```
            from queuing.models import DigitalQueueSlip
            try:
                DigitalQueueSlip.objects.get_or_create(
                    order=order,
                    defaults={'queue_number': order.order_number}
                )
            except Exception:
                pass
            for menu_item_id, line in lines.items():
                menu = line['menu_item']
                OrderItem.objects.create(
                    order=order,
                    item_name=menu.name,
                    price=float(menu.price),
                    quantity=line['qty'],
                )
            if is_delivery:
                from deliveries.models import DeliveryRequest
                dest_lat = data.get('dest_lat')
                dest_lng = data.get('dest_lng')
                delivery_req = DeliveryRequest.objects.create(
                    order=order,
                    delivery_location=delivery_location,
                    status=DeliveryRequest.RequestStatus.SEARCHING,
                    assigned_to=None,
                    assigned_at=None,
                    dest_lat=dest_lat,
                    dest_lng=dest_lng,
                )
        new_points = 0.0
        if user:
            user.refresh_from_db()
            new_points = float(user.loyalty_points or 0)
        return JsonResponse({
            'success': True,
            'order_number': order.order_number,
            'order_id': order.id,
            'is_delivery': is_delivery,
            'delivery_id': delivery_req.id if is_delivery else None,
            'delivery_fee': float(order.delivery_fee),
            'total_payment': float(order.total_payment),
            'points_earned': points_earned,
            'points_redeemed': points_redeemed,
            'new_points': new_points,
            'expires_at': slip_expires_at(order),
            'expiry_hours': KIOSK_SLIP_EXPIRY_HOURS,
        })

    except CheckoutError as e:
        return JsonResponse({'success': False, 'message': str(e)}, status=400)
    except Exception as e:
        return JsonResponse({'success': False, 'message': str(e)}, status=500)
def credit_points_for_order(order):
    if not order or order.status != 'completed' or not order.customer_id:
        return
    if order.points_earned and order.points_earned > 0:
        from accounts.models import CustomUser
        CustomUser.objects.filter(pk=order.customer_id).update(
            loyalty_points=F('loyalty_points') + Decimal(str(order.points_earned))
        )
def refund_points_for_cancel(order):
    if not order or not order.customer_id:
        return
    if order.points_redeemed and order.points_redeemed > 0:
        from accounts.models import CustomUser
```

```
        CustomUser.objects.filter(pk=order.customer_id).update(
            loyalty_points=F('loyalty_points') + Decimal(str(order.points_redeemed))
        )


def restore_stock_for_order(order):
    if not order:
        return
    matched_ids = []
    for oi in order.items.all():
        menu = MenuItem.objects.filter(name__iexact=oi.item_name).first()
        qty = oi.quantity or 0
        if menu and qty:
            menu.stock = (menu.stock or 0) + qty
            if menu.stock > 0:
                menu.is_available = True
            menu.save(update_fields=['stock', 'is_available'])
            matched_ids.append(menu.id)
    if matched_ids:
        cache.delete('formatted_menu_active_kiosk')
KIOSK_SLIP_EXPIRY_HOURS = 1
_KIOSK_EXPIRY_SWEEP_KEY = 'kiosk_unpaid_expiry_sweep'
_KIOSK_EXPIRY_SWEEP_TTL = 60
from datetime import timedelta
def slip_expires_at(order):
    if order.status != 'unpaid':
        return None
    from datetime import timedelta
    return timezone.localtime(order.created_at + timedelta(hours=KIOSK_SLIP_EXPIRY_HOURS)).isoformat()
def expire_stale_unpaid_orders():
    if not cache.add(_KIOSK_EXPIRY_SWEEP_KEY, '1', _KIOSK_EXPIRY_SWEEP_TTL):
        return 0
    try:
        cutoff = timezone.now() - timedelta(hours=KIOSK_SLIP_EXPIRY_HOURS)
        stale_ids = list(
            Order.objects.filter(status='unpaid', created_at__lt=cutoff)
            .order_by('created_at')
            .values_list('id', flat=True)[:200]
        )
        count = 0
        for oid in stale_ids:
            with transaction.atomic():
                order = Order.objects.select_for_update() \
                    .filter(pk=oid, status='unpaid').first()
                if not order:
                    continue
                order.status = 'cancelled'
                order.save(update_fields=['status'])
                try:
                    refund_points_for_cancel(order)
                    restore_stock_for_order(order)
                except Exception:
                    pass
                from activity_log.models import ActivityLog
                from activity_log.services import log_activity
                log_activity(
                    None,
                    action=f"Auto-cancelled expired queue slip {order.order_number} "
                           f"(P{float(order.total_amount):.2f})",
                    category=ActivityLog.Category.ORDER,
                    level=ActivityLog.Level.WARNING,
                    target=order.order_number,
                    is_guest=order.customer_id is None,
                    metadata={
                        'order_id': order.id,
```

```
                        'expiry_hours': KIOSK_SLIP_EXPIRY_HOURS,
                    },
                )
                count += 1
        return count
    finally:
        cache.delete(_KIOSK_EXPIRY_SWEEP_KEY)
def find_order_by_number(raw):
    from django.db.models import Q
    raw = str(raw or '').strip()
    import re
    match = re.search(r'#?\s*CE-?\s*(\d+)', raw, re.IGNORECASE)
    digits = match.group(1) if match else None
    if digits is None:
        digits = re.sub(r'[^0-9]', '', raw) or None
    numbers = {raw}
    if digits:
        numbers.update({f'#CE-{digits}', f'CE-{digits}', digits})
    query = Q()
    for n in numbers:
        if n:
            query |= Q(order_number__iexact=n)
    return Order.objects.filter(query).first()
def check_order_status_api(request, order_num):
    expire_stale_unpaid_orders()
    order = find_order_by_number(order_num)
    if order is None:
        return JsonResponse({'exists': False})
    return JsonResponse({
        'exists': True,
        'status': order.status,
        'expires_at': slip_expires_at(order),
        'expired': order.status == 'cancelled',
        'expiry_hours': KIOSK_SLIP_EXPIRY_HOURS,
    })

def _feedback_gate(order):
    from deliveries.models import DeliveryRequest
    delivery = DeliveryRequest.objects.filter(order=order).select_related().first()
    if delivery is None:
        if order.status != 'completed':
            return False, 'Rating is only available after the order has been completed.'
        return True, ''
    if delivery.status != DeliveryRequest.RequestStatus.DELIVERED:
        return False, 'Rating opens once the rider marks this delivery as delivered.'
    if not delivery.proof_photo:
        return False, 'Rating opens once the rider attaches the proof-of-delivery photo.'
    return True, ''
@require_POST
def submit_feedback_api(request):
    try:
        data = json.loads(request.body)
        order_number = data.get('order_number')
        rating = int(data.get('rating', 5))
        comment = data.get('comment', '')

        order = find_order_by_number(order_number) if order_number else None
        if order is None:
            return JsonResponse(
                {'success': False, 'message': 'Order not found. Check the order number and try again.'},
                status=404)
        customer = request.user if request.user.is_authenticated else None
        if order.customer_id is not None and customer is not None and order.customer_id != customer.pk:
            return JsonResponse(
                {'success': False, 'message': 'You can only rate your own order.'},
```

```
                status=403)
        if order.customer_id is not None and customer is None:
            return JsonResponse(
                {'success': False, 'message': 'Please log in to rate this order.'},
                status=403)
        ok, reason = _feedback_gate(order)
        if not ok:
            return JsonResponse({'success': False, 'message': reason}, status=400)
        from .models import OrderFeedback
        rating = max(1, min(5, int(rating or 5)))
        comment = str(comment or '')[:2000]
        if customer is not None:
            feedback = OrderFeedback.objects.filter(order=order, customer=customer).first()
        else:
            feedback = OrderFeedback.objects.filter(order=order, customer__isnull=True).first()
        if feedback:
            feedback.rating = rating
            feedback.comment = comment
            feedback.save(update_fields=['rating', 'comment'])
        else:
            OrderFeedback.objects.create(
                order=order,
                customer=customer,
                rating=rating,
                comment=comment
            )
        return JsonResponse({'success': True, 'message': 'Feedback submitted successfully'})
    except Exception as e:
        return JsonResponse({'success': False, 'message': str(e)}, status=400)
```

backend/customer_portal/urls.py

```
# customer_portal/urls.py
from django.urls import path
from . import views
app_name = 'customer_portal'
urlpatterns = [
    path('', views.kiosk_welcome, name='kiosk_welcome'),
    path('menu/', views.kiosk_menu, name='kiosk_menu'),
    path('api/menu/', views.get_kiosk_menu_api, name='get_kiosk_menu_api'),
    path('api/checkout/', views.process_checkout, name='process_checkout'),
    path('api/order-status/<str:order_num>/', views.check_order_status_api, name='check_order_status_api'),
    path('api/submit-feedback/', views.submit_feedback_api, name='submit_feedback_api'),
]
```

backend/customer_portal/templates/customer_portal/kiosk_menu.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <link rel="icon" href="{% static 'customer_portal/images/canteen-express-logo.png' %}" type="image/png" />
    <link rel="manifest" href="{% static 'manifest-kiosk.json' %}">
    <meta name="theme-color" content="#f97316">
    <script>
        if ('serviceWorker' in navigator) {
            window.addEventListener('load', () => {
                navigator.serviceWorker.register("{% static 'sw.js' %}").catch(err => console.log('SW error:',
err));
            });
        }
    </script>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no,
viewport-fit=cover">
    <meta name="mobile-web-app-capable" content="yes">
    <meta name="apple-mobile-web-app-capable" content="yes">
    <meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
```

```
    <title>Today's Menu - Canteen Express Kiosk</title>
    <!-- Tailwind CSS CDN -->
    <link rel="stylesheet" href="{% static 'css/dashboard.css' %}">
    <link rel="stylesheet" href="{% static 'vendor/fonts/plus-jakarta-sans.css' %}">
    <!-- FontAwesome Icons (self-hosted) -->
    <link rel="stylesheet" href="{% static 'vendor/fontawesome/css/all.min.css' %}" />
    <script src="{% static 'vendor/jsbarcode/JsBarcode.all.min.js' %}"></script>
    <script src="{% static 'vendor/html2canvas/html2canvas.min.js' %}"></script>
</head>
<body class="bg-brand-dark text-brand-light h-screen h-[100dvh] flex flex-col justify-between overflow-hidden
selection:bg-brand-orange selection:text-white relative">
<!-- MANUAL CLOSURE OVERLAY: rendered server-side state, kept live by polling -->
<div id="system-closed-overlay" style="display:none; position:fixed; inset:0; z-index:9999;
background:rgba(6,6,7,0.94); -webkit-backdrop-filter:blur(10px); backdrop-filter:blur(10px); align-items:center;
justify-content:center; padding:24px;">
    <div style="max-width:430px; width:100%; text-align:center; background:linear-gradient(160deg,
rgba(32,32,41,0.95), rgba(18,18,24,0.92)); border:1px solid rgba(249,115,22,0.35); border-radius:26px; padding:36px
28px; box-shadow:0 28px 70px rgba(0,0,0,0.6);">
        <div style="width:72px; height:72px; margin:0 auto 18px; border-radius:22px; display:flex;
align-items:center; justify-content:center; background:rgba(249,115,22,0.14); border:1px solid
rgba(249,115,22,0.4); font-size:30px; color:#f97316;">
            <i class="fa-solid fa-store-slash"></i>
        </div>
        <h2 style="margin:0 0 8px; font-size:22px; font-weight:800; color:#ffffff;">Canteen is Currently
Closed</h2>
        <p style="margin:0; font-size:13.5px; line-height:1.6; color:#a1a1aa;">We are not accepting new orders
right now. Please check back shortly.</p>
        <p id="system-closed-reason" style="display:none; margin:18px 0 0; padding:12px 14px; border-radius:14px;
background:rgba(249,115,22,0.10); border:1px solid rgba(249,115,22,0.28); color:#fdba74; font-size:13px;
font-weight:600; line-height:1.5;"></p>
        <button type="button" onclick="refreshSystemStatus()" style="margin-top:22px; width:100%;
background:#f97316; color:#ffffff; border:0; border-radius:14px; padding:13px; font-size:13px; font-weight:800;
cursor:pointer;">Check Again</button>
        <button type="button" onclick="dismissClosedOverlay()" style="margin-top:10px; width:100%;
background:rgba(255,255,255,0.06); color:#e4e4e7; border:1px solid rgba(255,255,255,0.12); border-radius:14px;
padding:13px; font-size:13px; font-weight:700; cursor:pointer;">View My Existing Order</button>
    </div>
</div>
    <!-- ================= FLOATING TOAST NOTIFICATION (AUTO-DISMISSING) ================= -->
    <div id="pending-order-banner" class="hidden fixed top-5 left-1/2 -translate-x-1/2 z-50 bg-amber-500 text-black
px-5 py-3 rounded-2xl shadow-2xl border border-amber-300 text-xs font-bold transition-all duration-300 transform
-translate-y-4 opacity-0 max-w-xl w-[90%] sm:w-auto flex items-center justify-between gap-4">
        <div class="flex items-center gap-2.5">
            <i class="fa-solid fa-triangle-exclamation text-base shrink-0"></i>
            <span class="leading-tight">You have an active unpaid order! Settle it at the counter or edit it before
placing a new one.</span>
        </div>
        <div class="flex items-center gap-2 shrink-0 border-l border-black/20 pl-3">
            <button onclick="openQueueSlipModal()" class="bg-black text-amber-400 hover:bg-black/80 px-3 py-1.5
rounded-xl font-black transition active:scale-95 text-[11px] uppercase tracking-wider">
                View
            </button>
            <button onclick="hidePendingBanner()" class="text-black/60 hover:text-black p-1 transition">
                <i class="fa-solid fa-xmark text-sm"></i>
            </button>
        </div>
    </div>
    <!-- ================= 1. DINE-IN / TAKE-OUT SELECTION POPUP MODAL ================= -->
    <div id="dining-option-modal" class="fixed inset-0 z-50 glass-backdrop flex items-center justify-center p-4">
        <div class="glass-panel glass-panel-accent rounded-3xl p-5 sm:p-6 max-w-md w-full text-center space-y-4
modal-pop relative overflow-hidden">
            <a href="{% url 'customer_portal:kiosk_welcome' %}" class="absolute right-4 top-4 w-8 h-8 rounded-full
bg-white/5 hover:bg-red-500/20 text-brand-slate hover:text-red-400 border border-white/10 flex items-center
justify-center transition active:scale-95" title="Exit Kiosk">
                <i class="fa-solid fa-xmark text-xs font-black"></i>
```

```
            </a>
            <div class="space-y-1.5">
                <img src="{% static 'customer_portal/images/canteen-express-logo.png' %}" alt="Logo" class="h-10
w-auto mx-auto mb-1 object-contain">
                <h2 class="text-xl sm:text-2xl font-black text-white">How will you be eating today?</h2>
                <p class="text-[11px] sm:text-xs text-brand-slate">Please select your dining option to start
ordering</p>
            </div>
            <div class="grid grid-cols-2 gap-3 pt-1">
                <button onclick="setDiningOption('Dine-In')" class="group p-4 rounded-2xl bg-white/5
hover:bg-brand-orange/20 border border-white/10 hover:border-brand-orange transition active:scale-95 flex flex-col
items-center justify-center gap-2.5">
                    <div class="w-12 h-12 rounded-xl bg-brand-orange/10 text-brand-orange flex items-center
justify-center text-2xl group-hover:bg-brand-orange group-hover:text-white transition">
                        <i class="fa-solid fa-utensils"></i>
                    </div>
                    <span class="font-black text-xs sm:text-sm text-white group-hover:text-brand-orange transition
uppercase tracking-wider">Dine-In</span>
                </button>
                <button onclick="setDiningOption('Take-Out')" class="dining-takeout-btn group p-4 rounded-2xl
bg-white/5 border border-white/10 transition active:scale-95 flex flex-col items-center justify-center gap-2.5">
                    <div class="dining-takeout-icon w-12 h-12 rounded-xl flex items-center justify-center text-2xl
transition">
                        <i class="fa-solid fa-bag-shopping"></i>
                    </div>
                    <span class="dining-takeout-label font-black text-xs sm:text-sm transition uppercase
tracking-wider">Take-Out</span>
                </button>
            </div>
        </div>
    </div>
    <!-- ================= HEADER ================= -->
    <header class="bg-brand-dark border-b border-white/10 shrink-0 space-y-3 pb-3">
        <div class="w-full px-3 sm:px-6 md:px-8 pt-3">
            <!-- Carousel Container -->
            <div class="relative w-full h-28 sm:h-36 md:h-44 rounded-2xl overflow-hidden border border-white/10
bg-brand-dark shadow-xl group">
                <div id="promo-carousel" class="flex h-full transition-transform duration-500 ease-in-out"
style="width: 300%;">
                    <!-- Slide 1 -->
                    <div class="w-1/3 h-full shrink-0 bg-gradient-to-r from-brand-orange/30 via-brand-dark to-black
p-4 sm:p-6 md:p-8 flex items-center justify-between relative overflow-hidden">
                        <div class="space-y-1 max-w-xl z-10 flex items-center gap-4 sm:gap-6">
                            <img src="{% static 'Student.png' %}" alt="Student" class="h-24 sm:h-32 md:h-40 w-auto
object-contain shrink-0 drop-shadow-lg -mb-4 sm:-mb-6 md:-mb-8">
                            <div class="space-y-1">
                                <span class="bg-brand-orange text-white text-[10px] sm:text-xs font-black uppercase
px-2.5 py-1 rounded-md tracking-wider">Today's Special</span>
                                <h2 class="text-base sm:text-xl md:text-3xl font-black text-white
leading-tight">Freshly Cooked Meals Daily!</h2>
                                <p class="text-xs sm:text-sm md:text-base text-brand-slate line-clamp-1">Order your
favorite campus snacks & meals in seconds.</p>
                            </div>
                        </div>
                        <div class="absolute right-0 top-0 bottom-0 w-1/2 sm:w-1/3 overflow-hidden">
                            <img src="{% static 'main-canteen 2.jpg' %}" alt="Main Canteen 2" class="w-full h-full
object-cover opacity-80">
                            <div class="absolute inset-y-0 left-0 w-28 bg-gradient-to-r from-brand-dark
via-brand-dark/70 to-transparent"></div>
                        </div>
                    </div>
                    <!-- Slide 2 -->
                    <div class="w-1/3 h-full shrink-0 bg-gradient-to-r from-emerald-600/30 via-brand-dark to-black
p-4 sm:p-6 md:p-8 flex items-center justify-between relative overflow-hidden">
                        <div class="space-y-1 max-w-xl z-10 flex items-center gap-4 sm:gap-6">
```

```
                            <img src="{% static 'Prof.png' %}" alt="Professor" class="h-24 sm:h-32 md:h-40 w-auto
object-contain shrink-0 drop-shadow-lg -mb-4 sm:-mb-6 md:-mb-8">
                            <div class="space-y-1">
                                <span class="bg-emerald-500 text-white text-[10px] sm:text-xs font-black uppercase
px-2.5 py-1 rounded-md tracking-wider">Canteen Express</span>
                                <h2 class="text-base sm:text-xl md:text-3xl font-black text-white leading-tight">We
Handle the Hassle</h2>
                                <p class="text-xs sm:text-sm md:text-base text-brand-slate line-clamp-1">Relax and
enjoy fast campus delivery & smooth ordering.</p>
                            </div>
                        </div>
                        <div class="absolute right-0 top-0 bottom-0 w-1/2 sm:w-1/3 overflow-hidden">
                            <img src="{% static 'main-canteen 4.jpg' %}" alt="Main Canteen 4" class="w-full h-full
object-cover opacity-80">
                            <div class="absolute inset-y-0 left-0 w-28 bg-gradient-to-r from-brand-dark
via-brand-dark/70 to-transparent"></div>
                        </div>
                    </div>
                    <!-- Slide 3 -->
                    <div class="w-1/3 h-full shrink-0 bg-gradient-to-r from-blue-600/30 via-brand-dark to-black p-4
sm:p-6 md:p-8 flex items-center justify-between relative overflow-hidden">
                        <div class="space-y-1 max-w-xl z-10 flex items-center gap-4 sm:gap-6">
                            <img src="{% static 'Rider.png' %}" alt="Rider" class="h-24 sm:h-32 md:h-40 w-auto
object-contain shrink-0 drop-shadow-lg -mb-4 sm:-mb-6 md:-mb-8">
                            <div class="space-y-1">
                                <span class="bg-blue-500 text-white text-[10px] sm:text-xs font-black uppercase
px-2.5 py-1 rounded-md tracking-wider">Join Our Team</span>
                                <h2 class="text-base sm:text-xl md:text-3xl font-black text-white
leading-tight">Looking for a part-time inside campus?</h2>
                                <p class="text-xs sm:text-sm md:text-base text-brand-slate line-clamp-1">Come and
join Can-Ex Delivery!</p>
                            </div>
                        </div>
                        <div class="absolute right-0 top-0 bottom-0 w-1/2 sm:w-1/3 overflow-hidden">
                            <img src="{% static 'main-canteen.jpg' %}" alt="Main Canteen" class="w-full h-full
object-cover opacity-80">
                            <div class="absolute inset-y-0 left-0 w-28 bg-gradient-to-r from-brand-dark
via-brand-dark/70 to-transparent"></div>
                        </div>
                    </div>
                </div>
                <!-- Carousel Indicators / Dots -->
                <div class="absolute bottom-3 right-4 z-20 flex items-center gap-2">
                    <button onclick="setCarouselSlide(0)" id="carousel-dot-0" class="w-6 h-2 rounded-full bg-white
transition-all shadow-md"></button>
                    <button onclick="setCarouselSlide(1)" id="carousel-dot-1" class="w-2 h-2 rounded-full
bg-white/40 transition-all shadow-md"></button>
                    <button onclick="setCarouselSlide(2)" id="carousel-dot-2" class="w-2 h-2 rounded-full
bg-white/40 transition-all shadow-md"></button>
                </div>
            </div>
        </div>
        <div class="max-w-7xl mx-auto px-4 flex items-center justify-between gap-3">
            <div class="flex items-center gap-3">
                <div class="w-10 h-10 rounded-xl bg-brand-orange/20 border border-brand-orange/30 p-1 flex
items-center justify-center overflow-hidden shadow">
                    <img src="{% static 'customer_portal/images/canteen-express-logo.png' %}" alt="Logo"
class="w-full h-full object-contain">
                </div>
                <div id="brand-text-container" class="transition-all duration-300">
                    <h1 class="text-xs sm:text-base font-black text-white leading-tight">Welcome to Canteen
Express!</h1>
                    <p class="text-[8px] sm:text-xs text-brand-orange font-bold uppercase tracking-wider flex
items-center gap-1.5">
```

```
                        <span id="dining-mode-badge" onclick="toggleDiningOption()"
onkeydown="if(event.key==='Enter'||event.key===' '){event.preventDefault();toggleDiningOption();}" role="button"
tabindex="0" title="Switch between Dine-In and Take-Out" aria-label="Dining option: Dine-In. Activate to switch."
class="cursor-pointer inline-flex items-center gap-1 px-2 py-0.5 rounded-md border transition active:scale-95
bg-brand-orange/20 text-brand-orange border-brand-orange/30"><span id="dining-mode-label">Dine-In</span><i
class="fa-solid fa-right-left text-[8px] opacity-80" aria-hidden="true"></i></span>
                    </p>
                </div>
            </div>
            <div class="flex items-center gap-2" id="search-container">
                <!-- Expanding Search Wrapper (Expands Left) -->
                <!-- Expanding Search Wrapper (Expands Left) -->
                <div id="search-wrapper" class="flex items-center overflow-hidden w-10 transition-all duration-300
rounded-xl bg-brand-card border border-white/10">
                    <button onclick="toggleSearch(event)" id="search-toggle-btn" type="button" class="w-10 h-10
shrink-0 flex items-center justify-center text-white transition active:scale-95" title="Search Menu">
                        <i class="fa-solid fa-magnifying-glass text-xs"></i>
                    </button>
                    <input
                        type="text"
                        id="search-input"
                        oninput="filterItems()"
                        placeholder="Search menu or price..."
                        class="flex-1 min-w-0 bg-transparent border-0 px-1 py-2 text-xs text-white
placeholder-brand-slate focus:outline-none focus:ring-0"
                    >
                    <button type="button" onclick="clearSearchInput(event)" id="search-clear-btn" style="display:
none;" class="w-6 h-6 shrink-0 items-center justify-center bg-white/15 hover:bg-white/30 text-white rounded-full
transition mr-2 cursor-pointer" title="Clear search">
                        <i class="fa-solid fa-xmark text-[10px]"></i>
                    </button>
                </div>
                <!-- Order History / Activity Log -->
                <button onclick="openOrderHistory()" id="history-toggle-btn" class="w-10 h-10 shrink-0 rounded-xl
bg-brand-card hover:bg-white/10 border border-white/10 flex items-center justify-center text-white transition
active:scale-95" title="Order History & Activity">
                    <i class="fa-solid fa-clock-rotate-left text-xs"></i>
                </button>
                <!-- Light / Dark Mode Toggle Button -->
                <button onclick="toggleTheme()" id="theme-toggle-btn" class="w-10 h-10 shrink-0 rounded-xl
bg-brand-card hover:bg-white/10 border border-white/10 flex items-center justify-center text-white transition
active:scale-95" title="Toggle Light/Dark Mode">
                    <i class="fa-solid fa-sun text-xs" id="theme-icon"></i>
                </button>
                <!-- Exit Kiosk -->
                <a href="{% url 'customer_portal:kiosk_welcome' %}" class="bg-red-500/20 hover:bg-red-500/30 border
border-red-500/40 text-red-400 font-extrabold px-3.5 h-10 rounded-xl text-xs flex items-center gap-2 transition
active:scale-95 shrink-0" title="Exit Kiosk">
                    <i class="fa-solid fa-right-from-bracket"></i>
                    <span class="hidden sm:inline font-black uppercase tracking-wider">Exit</span>
                </a>
            </div>
        </div>
    </header>
    <!-- ================ MAIN LAYOUT ================ -->
    <div class="flex-1 max-w-7xl w-full mx-auto flex overflow-hidden min-h-0 p-2 sm:p-3 gap-3">

        <aside class="w-20 sm:w-28 flex-shrink-0 flex flex-col gap-2 overflow-y-auto no-scrollbar pr-1"
id="category-sidebar">
            <button onclick="selectCategory('all', this)" class="category-btn active-cat cat-pod flex-1 flex
flex-col items-center justify-center text-center cursor-pointer min-h-[64px] px-1 py-2.5 rounded-2xl">
                <span class="pod-icon text-xl flex items-center justify-center w-14 h-14 rounded-2xl"><i
class="fa-solid fa-utensils"></i></span>
                <span class="pod-label text-[10px] font-black leading-tight mt-1.5">All Items</span>
            </button>
```

```
            {% for cat in categories %}
            <button onclick="selectCategory('{{ cat.name|escapejs }}', this)" class="category-btn cat-pod flex-1
flex flex-col items-center justify-center text-center cursor-pointer min-h-[64px] px-1 py-2.5 rounded-2xl">
                <span class="pod-icon text-xl flex items-center justify-center w-14 h-14 rounded-2xl">
                    {% if 'ulam' in cat.name|lower %}
                        <i class="fa-solid fa-drumstick-bite"></i>
                    {% elif 'pasta' in cat.name|lower or 'noodle' in cat.name|lower %}
                        <svg class="w-6 h-6 text-current" viewBox="0 0 24 24" fill="currentColor">
                            <path d="M3.5 11h17a0.5 0.5 0 0 1 0.5 0.5 8 8 0 0 1-8 8h-3a8 8 0 0 1-8-8 0.5 0.5 0 0 1
0.5-0.5z"/>
                            <rect x="9" y="5" width="1.5" height="6" rx="0.75"/>
                            <rect x="12" y="4" width="1.5" height="7" rx="0.75"/>
                            <rect x="15" y="6" width="1.5" height="5" rx="0.75"/>
                            <line x1="6" y1="5" x2="20" y2="8" stroke="currentColor" stroke-width="2"
stroke-linecap="round"/>
                            <line x1="6" y1="7" x2="19" y2="10" stroke="currentColor" stroke-width="2"
stroke-linecap="round"/>
                        </svg>
                    {% elif 'instant' in cat.name|lower %}
                        <i class="fa-solid fa-mug-hot"></i>
                    {% elif 'pastry' in cat.name|lower or 'bakery' in cat.name|lower %}
                        <i class="fa-solid fa-bread-slice"></i>
                    {% elif 'kakanin' in cat.name|lower %}
                        <i class="fa-solid fa-plate-wheat"></i>
                    {% elif 'candies' in cat.name|lower %}
                        <i class="fa-solid fa-candy-cane"></i>
                    {% elif 'add' in cat.name|lower or 'addon' in cat.name|lower or 'extra' in cat.name|lower %}
                        <i class="fa-solid fa-circle-plus"></i>
                    {% elif 'rice' in cat.name|lower %}
                        <i class="fa-solid fa-bowl-rice"></i>
                    {% elif 'biscuit' in cat.name|lower %}
                        <i class="fa-solid fa-cookie"></i>
                    {% elif 'beverage' in cat.name|lower or 'drink' in cat.name|lower %}
                        <i class="fa-solid fa-bottle-water"></i>
                    {% elif 'meryenda' in cat.name|lower %}
                        <i class="fa-solid fa-burger"></i>
                    {% elif 'diet' in cat.name|lower %}
                        <i class="fa-solid fa-heart-pulse"></i>
                    {% else %}
                        <i class="fa-solid fa-bowl-food"></i>
                    {% endif %}
                </span>
                <span class="pod-label text-[10px] font-bold leading-tight mt-1.5 line-clamp-1">{{ cat.name
}}</span>
            </button>
            {% endfor %}
        </aside>
        <main class="flex-1 flex flex-col min-w-0 overflow-y-auto no-scrollbar space-y-3 pr-1">
            <div class="flex items-center justify-between border-b border-white/10 pb-1.5">
                <h2 id="category-heading" class="text-sm sm:text-lg font-black text-white uppercase
tracking-wider">All Items</h2>
                <span id="items-count-indicator" class="text-xs text-brand-slate font-bold">Showing items</span>
            </div>
            <section id="menu-grid" class="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-4 gap-2.5 sm:gap-3 pb-12">
            </section>
        </main>

    </div>
    <!-- ================= FOOTER BAR ================= -->
    <footer id="kiosk-footer" class="hidden shrink-0 glass-panel glass-panel-accent border-t border-white/10 p-3
sm:p-4 pb-[max(env(safe-area-inset-bottom),12px)] shadow-2xl z-20">
        <div class="max-w-7xl mx-auto flex items-center justify-between gap-4">

            <div class="text-left">
                <span class="text-[10px] sm:text-xs text-brand-slate uppercase font-bold block">Total Amount</span>
```

```
                <span id="cart-total" class="text-xl sm:text-3xl font-black text-brand-orange">₱0.00</span>
            </div>
            <div class="flex items-center gap-2.5">
                {% csrf_token %}
                <button onclick="clearCart()" class="px-4 py-2.5 sm:py-3 rounded-xl bg-white/5 hover:bg-red-500/20
text-brand-slate hover:text-red-400 font-bold text-xs sm:text-sm uppercase tracking-wider transition border
border-white/10 active:scale-95">
                    Clear
                </button>

                <button onclick="openCartDrawer()" id="next-btn" disabled class="bg-brand-orange
disabled:opacity-40 disabled:cursor-not-allowed hover:bg-orange-600 active:scale-95 text-white font-extrabold
text-xs sm:text-sm px-6 sm:px-8 py-2.5 sm:py-3 rounded-xl shadow-lg shadow-brand-orange/25 uppercase tracking-wider
transition flex items-center gap-2">
                    <span>Next</span>
                    <i class="fa-solid fa-arrow-right"></i>
                </button>
            </div>

        </div>
    </footer>
    <!-- ================= YOUR CART MODAL (centered, glassy) ================= -->
    <div id="cart-drawer-backdrop" class="fixed inset-0 z-50 glass-backdrop hidden flex items-center justify-center
p-4">
        <div class="cart-sheet glass-panel glass-panel-accent rounded-3xl max-w-md w-full overflow-hidden flex
flex-col max-h-[85vh]">
            <!-- Header -->
            <div class="shrink-0 glass-panel-head border-b border-white/10 p-4 flex items-center justify-between
pt-[max(env(safe-area-inset-top),16px)] sm:pt-4">
                <div class="flex items-center gap-2.5">
                    <div class="w-9 h-9 rounded-xl bg-orange-500/15 text-brand-orange flex items-center
justify-center text-base shrink-0">
                        <i class="fa-solid fa-cart-shopping"></i>
                    </div>
                    <h3 class="text-base font-black text-white uppercase tracking-wider">Your Cart</h3>
                </div>
                <button onclick="closeCartDrawer()" class="w-9 h-9 rounded-xl bg-white/5 hover:bg-white/10
text-brand-slate hover:text-white flex items-center justify-center transition shrink-0">
                    <i class="fa-solid fa-xmark"></i>
                </button>
            </div>
            <!-- Cart items -->
            <div id="drawer-cart-list" class="flex-1 overflow-y-auto no-scrollbar p-4 space-y-3 min-h-0
glass-scroll">
            </div>
            <!-- Footer bar -->
            <div class="shrink-0 glass-panel-foot border-t border-white/10 p-4
pb-[max(env(safe-area-inset-bottom),16px)] space-y-4">
                <div class="space-y-2 text-xs text-brand-slate">
                    <div class="flex justify-between">
                        <span>Dining Mode:</span>
                        <span id="drawer-dining-mode" class="text-white font-bold">Dine-In</span>
                    </div>
                    <div class="flex justify-between text-base font-black text-white pt-2 border-t border-white/5">
                        <span>Total Amount:</span>
                        <span id="drawer-total-amount" class="text-brand-orange">₱0.00</span>
                    </div>
                </div>
                <div class="grid grid-cols-2 gap-3">
                    <button onclick="closeCartDrawer()" class="w-full bg-white/5 hover:bg-white/10 text-white
font-bold py-3 rounded-xl uppercase tracking-wider text-xs border border-white/10 transition active:scale-95">
                        + Add More
                    </button>
```

```
                    <button onclick="checkoutOrder()" id="drawer-pay-btn" class="w-full bg-brand-orange
hover:bg-orange-600 text-white font-extrabold py-3 rounded-xl uppercase tracking-wider text-xs shadow-glow-orange
transition flex items-center justify-center gap-2 active:scale-95">
                        <span>Done / Pay</span>
                        <i class="fa-solid fa-circle-check"></i>
                    </button>
                </div>
            </div>
        </div>
    </div>
    <!-- ================ ORDER HISTORY / ACTIVITY LOG MODAL ================ -->
    <div id="order-history-modal" class="fixed inset-0 z-50 glass-backdrop hidden flex items-center justify-center
p-4">
        <div class="glass-panel rounded-3xl max-w-lg w-full overflow-hidden shadow-2xl flex flex-col max-h-[80vh]
modal-pop">
            <div class="p-4 border-b border-white/10 flex justify-between items-center glass-panel-head">
                <h3 class="font-black text-white text-sm flex items-center gap-2">
                    <i class="fa-solid fa-clock-rotate-left text-brand-orange"></i> Order History & Activity
                </h3>
                <button onclick="closeOrderHistory()" class="text-brand-slate hover:text-white p-1"><i
class="fa-solid fa-xmark text-lg"></i></button>
            </div>
            <div id="order-history-container" class="p-5 overflow-y-auto space-y-3 flex-1 glass-scroll">
                <!-- Populated dynamically -->
            </div>
            <div class="p-4 glass-panel-foot border-t border-white/10 flex items-center justify-between gap-3">
                <span class="text-[10px] sm:text-xs text-brand-slate">Palawan State University Canteen
Express</span>
                <button onclick="clearOrderHistory()" id="clear-history-btn" title="Clear finished orders"
class="shrink-0 inline-flex items-center gap-1.5 px-3 py-1.5 rounded-lg border border-white/10 text-brand-slate
hover:text-red-400 hover:bg-white/10 font-bold text-[10px] sm:text-xs transition active:scale-95">
                    <i class="fa-solid fa-trash-can text-[10px]"></i>
                    <span>Clear finished</span>
                </button>
            </div>
        </div>
    </div>
    <!-- ================ CLEAR HISTORY CONFIRM ================ -->
    <div id="clear-history-confirm" class="fixed inset-0 z-[80] glass-backdrop hidden flex items-center
justify-center p-4">
        <div class="glass-panel glass-panel-accent rounded-3xl p-5 max-w-xs w-full text-center space-y-4 modal-pop
relative overflow-hidden">
            <div class="w-12 h-12 rounded-2xl bg-white/5 text-red-400 border border-white/10 flex items-center
justify-center mx-auto text-xl">
                <i class="fa-solid fa-trash-can"></i>
            </div>
            <div class="space-y-1">
                <h3 class="text-white font-black text-sm">Clear finished orders?</h3>
                <p class="text-[11px] text-brand-slate leading-relaxed">Only completed, delivered, and cancelled
orders will be removed. Active orders stay on this kiosk.</p>
            </div>
            <div class="flex gap-2">
                <button onclick="closeClearHistoryConfirm()" class="flex-1 py-2.5 rounded-xl bg-white/5
hover:bg-white/10 text-white font-bold text-[11px] uppercase tracking-wider border border-white/10 transition
active:scale-95">Cancel</button>
                <button onclick="confirmClearHistory()" class="flex-1 py-2.5 rounded-xl bg-red-500 text-white
font-bold text-[11px] uppercase tracking-wider transition active:scale-95">Clear</button>
            </div>
        </div>
    </div>
    <!-- ================ QUEUE SLIP MODAL (DOWNLOADABLE & WITH BARCODE) ================ -->
    <div id="queue-slip-modal" class="fixed inset-0 z-50 glass-backdrop hidden flex items-center justify-center p-4
overflow-y-auto">
        <div class="max-w-sm w-full space-y-4 my-auto">
            <!-- Target Printable Queue Slip Container -->
```

```
            <div id="queue-slip-card" class="bg-white text-gray-900 rounded-3xl p-4 shadow-2xl space-y-3
text-center border-2 border-gray-100">
                <div class="border-b border-dashed border-gray-300 pb-2 space-y-0.5">
                    <h3 class="text-base font-black text-gray-900 uppercase tracking-wider">CANTEEN EXPRESS</h3>
                    <p class="text-[10px] text-gray-500 font-bold uppercase tracking-widest"
id="slip-title-label">Official Mobile Kiosk Queue Slip</p>
                    <p class="text-[9px] text-gray-400 hidden" id="slip-receipt-date"></p>
                </div>
                <!-- Queue Number & Mode -->
                <div class="bg-orange-50 rounded-2xl p-3 border border-orange-100 space-y-0.5">
                    <span class="text-[10px] font-black uppercase tracking-wider text-orange-600 block">Queue
Number</span>
                    <span id="slip-queue-number" class="text-3xl font-black text-orange-600
tracking-tight">#CE-0000</span>
                    <div class="pt-0.5 flex items-center justify-center gap-2">
                        <span id="slip-dining-mode" class="inline-block bg-orange-600 text-white text-[10px]
font-extrabold px-3 py-0.5 rounded-full uppercase tracking-wider">DINE-IN</span>
                        <span id="slip-payment-badge" class="hidden inline-block bg-emerald-600 text-white
text-[10px] font-extrabold px-3 py-0.5 rounded-full uppercase tracking-wider">PAID</span>
                    </div>
                </div>
                <!-- Order Summary Items List -->
                <div class="text-left space-y-2 text-xs">
                    <span class="text-[10px] font-bold text-gray-400 uppercase tracking-wider block border-b
border-gray-100 pb-1">Order List</span>
                    <div id="slip-items-list" class="space-y-1 max-h-28 overflow-y-auto pr-1">
                        <!-- Dynamic Item Lines -->
                    </div>
                    <div class="border-t border-dashed border-gray-300 pt-1.5 flex justify-between items-center
text-xs font-black text-gray-900">
                        <span>TOTAL AMOUNT:</span>
                        <span id="slip-total-amount" class="text-orange-600 text-sm">₱0.00</span>
                    </div>
                </div>
                <!-- Expiry Callout: pay at the counter before this time -->
                <div id="slip-expiry-line" class="hidden rounded-2xl bg-amber-50 border border-amber-200 px-3 py-2
text-center">
                    <span class="text-[9px] font-black uppercase tracking-wider text-amber-700 block"><i
class="fa-regular fa-clock"></i> Counter payment only · valid for <span id="slip-expiry-hours">1</span>h</span>
                    <span class="text-xs font-black text-amber-800 block mt-0.5">Pay before <span
id="slip-expires-at">--:--</span></span>
                </div>
                <!-- Barcode Container / Footer -->
                <div class="border-t border-dashed border-gray-300 pt-2 flex flex-col items-center justify-center"
id="slip-footer-container">
                    <svg id="slip-barcode" class="w-full h-12"></svg>
                    <span class="text-[9px] text-gray-400 font-bold uppercase tracking-widest mt-1"
id="slip-footer-text">Scan at Counter to Process Payment</span>
                </div>
            </div>
            <!-- Action Buttons Below Queue Slip -->
            <div class="glass-panel glass-panel-accent rounded-3xl p-3.5 space-y-2 modal-pop">
                <!-- Icon actions: Save Slip + Close (one container, professional) -->
                <div class="grid grid-cols-2 gap-2">
                    <button onclick="downloadQueueSlip()" id="download-slip-btn" class="w-full h-12 rounded-xl
bg-brand-orange hover:bg-orange-600 text-white shadow-lg shadow-brand-orange/25 transition flex items-center
justify-center gap-2 active:scale-95 cursor-pointer">
                        <i class="fa-solid fa-download text-sm"></i>
                        <span id="slip-download-text" class="font-extrabold text-[11px] uppercase
tracking-wider">Download</span>
                    </button>
                    <button onclick="closeQueueSlipModal()" id="close-slip-btn" class="w-full h-12 rounded-xl
bg-white/10 hover:bg-white/20 text-white border border-white/10 transition flex items-center justify-center gap-2
active:scale-95 cursor-pointer">
                        <i class="fa-solid fa-xmark text-sm"></i>
```

```
                        <span class="font-extrabold text-[11px] uppercase tracking-wider">Close</span>
                    </button>
                </div>
                <div class="grid grid-cols-2 gap-2" id="slip-actions-grid">
                    <button onclick="editPendingOrder()" class="w-full bg-white/10 hover:bg-white/20 text-white
font-bold py-2.5 rounded-xl uppercase tracking-wider text-[11px] transition border border-white/10 flex
items-center justify-center gap-1.5">
                        <i class="fa-solid fa-pen-to-square"></i>
                        <span>Edit Order</span>
                    </button>
                    <button onclick="cancelPendingOrder()" class="w-full bg-red-500/20 hover:bg-red-500/30
text-red-400 font-bold py-2.5 rounded-xl uppercase tracking-wider text-[11px] transition border border-red-500/30
flex items-center justify-center gap-1.5">
                        <i class="fa-solid fa-trash-can"></i>
                        <span>Cancel Order</span>
                    </button>
                </div>
            </div>

        </div>
    </div>
    <!-- ================= OPTIONAL FEEDBACK MODAL (after e-receipt) ================= -->
    <div id="feedback-modal" class="fixed inset-0 z-[80] glass-backdrop hidden flex items-center justify-center
p-4">
        <div class="bg-brand-card border border-white/10 rounded-3xl max-w-sm w-full p-6 space-y-4 shadow-2xl
modal-pop">
            <div class="flex items-start justify-between gap-2">
                <div>
                    <h3 class="font-black text-white text-base"><i class="fa-solid fa-star text-amber-400
mr-1.5"></i>Rate your experience</h3>
                    <p class="text-[10px] text-gray-400 font-semibold mt-0.5" id="feedback-order-label"></p>
                </div>
                <button onclick="skipFeedback()" class="text-gray-400 hover:text-white text-lg px-1 shrink-0"
aria-label="Close feedback form">
                    <i class="fa-solid fa-xmark"></i>
                </button>
            </div>
            <div class="flex justify-center gap-1.5 text-2xl text-amber-500" id="feedback-star-container">
                <button type="button" onclick="setRating(1)" class="star-btn text-amber-500" data-star="1"><i
class="fa-solid fa-star"></i></button>
                <button type="button" onclick="setRating(2)" class="star-btn text-amber-500" data-star="2"><i
class="fa-solid fa-star"></i></button>
                <button type="button" onclick="setRating(3)" class="star-btn text-amber-500" data-star="3"><i
class="fa-solid fa-star"></i></button>
                <button type="button" onclick="setRating(4)" class="star-btn text-amber-500" data-star="4"><i
class="fa-solid fa-star"></i></button>
                <button type="button" onclick="setRating(5)" class="star-btn text-amber-500" data-star="5"><i
class="fa-solid fa-star"></i></button>
            </div>
            <p class="text-[10px] text-gray-400 text-center font-semibold">Tap your rating and optionally leave a
comment.</p>
            <textarea id="feedback-comment-input" placeholder="Write feedback or comment (optional)..."
class="w-full bg-white/5 border border-white/10 text-white placeholder:text-gray-500 rounded-xl p-3 text-xs
focus:outline-none focus:border-brand-orange resize-none h-20"></textarea>
            <p id="feedback-note" class="hidden text-[10px] font-bold text-amber-400 text-center
leading-relaxed"></p>
            <div class="grid grid-cols-2 gap-2">
                <button onclick="skipFeedback()" class="w-full bg-white/5 hover:bg-white/10 text-gray-300 border
border-white/10 font-bold py-2.5 rounded-xl uppercase tracking-wider text-[11px] transition
active:scale-95">Skip</button>
                <button onclick="submitKioskFeedback()" class="w-full bg-brand-orange hover:bg-orange-600
text-white font-bold py-2.5 rounded-xl uppercase tracking-wider text-[11px] transition
active:scale-95">Submit</button>
            </div>
        </div>
```

```
    </div>
    <!-- ================ USER ACCOUNT MODAL ================ -->
    <div id="user-account-modal" class="fixed inset-0 z-[80] hidden flex items-center justify-center p-4"
style="background: rgba(8, 8, 12, 0.65); -webkit-backdrop-filter: blur(8px); backdrop-filter: blur(8px);">
        <div class="bg-brand-card rounded-3xl p-6 max-w-sm w-full border border-white/10 shadow-2xl relative">
            <button onclick="toggleUserModal()" class="absolute top-4 right-4 text-brand-slate hover:text-white"><i
class="fa-solid fa-xmark"></i></button>
            <h2 class="text-white font-black text-lg mb-4">Account Details</h2>
            <div class="space-y-4">
                <div class="flex items-center gap-3 bg-white/5 p-3 rounded-xl border border-white/5">
                    <div class="w-10 h-10 rounded-full bg-brand-orange/20 flex items-center justify-center"><i
class="fa-solid fa-user text-brand-orange"></i></div>
                    <div>
                        <p class="text-white text-sm font-bold">{{ user.first_name }} {{ user.last_name }}</p>
                        <p class="text-brand-slate text-xs">{{ user.username }}</p>
                    </div>
                </div>
                <div class="space-y-2 text-xs">
                    <p class="text-brand-slate font-bold uppercase tracking-wider">Email</p>
                    <p class="text-white bg-white/5 p-2 rounded-lg border border-white/5">{{ user.email }}</p>
                </div>
                <div class="space-y-2 text-xs">
                    <p class="text-brand-slate font-bold uppercase tracking-wider">Role</p>
                    <p class="text-white bg-white/5 p-2 rounded-lg border border-white/5">{{ user.get_role_display
}}</p>
                </div>
            </div>
        </div>
    </div>

    <div id="queue-cancel-modal" class="fixed inset-0 z-[70] hidden flex items-center justify-center p-4"
style="background: rgba(8, 8, 12, 0.35); -webkit-backdrop-filter: blur(3px); backdrop-filter: blur(3px);">
        <div class="glass-panel glass-panel-accent rounded-3xl p-6 sm:p-7 max-w-xs w-full text-center space-y-5
modal-pop relative">
            <div class="w-16 h-16 rounded-2xl bg-red-500/20 border border-red-500/30 text-red-400 flex items-center
justify-center mx-auto text-3xl shadow-lg shadow-red-500/10 animate-bounce">
                <i class="fa-solid fa-triangle-exclamation"></i>
            </div>
            <div class="space-y-1.5">
                <h3 class="text-lg font-black text-white uppercase tracking-wider">Cancel Queue Slip</h3>
                <p class="text-xs text-white/70 font-semibold leading-relaxed">
                    Are you sure you want to cancel your queue slip? This will clear your cart and you'll need to
build your order again.
                </p>
            </div>
            <div class="grid grid-cols-2 gap-2 pt-1">
                <button onclick="closeQueueCancelModal()" id="queue-cancel-back-btn"
                    class="w-full bg-white/10 hover:bg-white/20 text-white font-bold py-3 rounded-xl uppercase
tracking-wider text-[11px] transition border border-white/10 flex items-center justify-center gap-1.5
cursor-pointer">
                    <i class="fa-solid fa-arrow-left"></i>
                    <span>Keep Slip</span>
                </button>
                <button onclick="confirmCancelPendingOrder()" id="queue-cancel-confirm-btn"
                    class="w-full bg-red-500/20 hover:bg-red-500/30 text-red-400 font-bold py-3 rounded-xl
uppercase tracking-wider text-[11px] transition border border-red-500/30 flex items-center justify-center gap-1.5
cursor-pointer shadow-lg shadow-red-500/10">
                    <i class="fa-solid fa-trash-can"></i>
                    <span>Yes, Cancel</span>
                </button>
            </div>
        </div>
    </div>
    <!-- ================ CHECKOUT ERROR MODAL (Matching Image 1) ================ -->
```

```
    <div id="checkout-error-modal" class="fixed inset-0 z-[90] hidden flex items-center justify-center p-4"
style="background: rgba(8, 8, 12, 0.65); -webkit-backdrop-filter: blur(8px); backdrop-filter: blur(8px);">
        <div class="glass-panel glass-panel-accent rounded-3xl p-6 sm:p-7 max-w-xs sm:max-w-sm w-full text-center
space-y-5 modal-pop relative shadow-2xl">
            <button onclick="closeCheckoutErrorModal()" class="absolute top-4 right-4 text-gray-400
hover:text-white bg-white/5 hover:bg-white/10 w-8 h-8 rounded-full flex items-center justify-center transition
cursor-pointer" aria-label="Close error modal">
                <i class="fa-solid fa-xmark text-sm"></i>
            </button>
            <div class="w-16 h-16 rounded-2xl bg-red-500/20 border border-red-500/30 text-red-400 flex items-center
justify-center mx-auto text-2xl shadow-lg shadow-red-500/10">
                <i class="fa-solid fa-triangle-exclamation"></i>
            </div>
            <div class="space-y-1.5">
                <h3 class="text-lg font-black text-white tracking-wide">Checkout Error</h3>
                <p id="checkout-error-message" class="text-xs text-gray-300 font-medium leading-relaxed">
                    Canteen is currently closed. Operating hours are Monday to Friday, 8:00 AM to 5:00 PM.
                </p>
            </div>
            <button onclick="closeCheckoutErrorModal()" class="w-full bg-brand-orange hover:bg-orange-600
text-white font-extrabold py-3.5 rounded-2xl uppercase tracking-wider text-xs shadow-lg shadow-brand-orange/30
transition active:scale-95 cursor-pointer">
                GOT IT
            </button>
        </div>
    </div>
    <!-- ================ ITEM EXPAND / DETAILS MODAL ================ -->
    <div id="item-details-modal" class="fixed inset-0 z-50 glass-backdrop hidden flex items-center justify-center
p-4 overflow-y-auto">
        <div class="glass-panel glass-panel-accent rounded-3xl max-w-md w-full overflow-hidden modal-pop relative
my-8">
            <button onclick="closeItemModal()" class="absolute right-4 top-4 z-10 w-8 h-8 rounded-full bg-black/60
text-white flex items-center justify-center hover:bg-black transition">
                <i class="fa-solid fa-xmark"></i>
            </button>
            <div class="relative h-48 sm:h-56 bg-white/5">
                <img id="modal-item-img" src="" alt="Item Image"
onerror="this.src='https://images.unsplash.com/photo-1546069901-ba9599a7e63c?auto=format&fit=crop&w=600&q=80'"
class="w-full h-full object-cover">
            </div>
            <div class="p-6 space-y-4">
                <div>
                    <span id="modal-item-category" class="bg-brand-orange/20 text-brand-orange px-2.5 py-0.5
rounded-md text-[10px] font-black uppercase tracking-wider">Category</span>
                    <h3 id="modal-item-name" class="text-xl font-black text-white mt-1">Item Name</h3>
                    <p id="modal-item-desc" class="text-xs text-brand-slate mt-2 leading-relaxed">Description goes
here...</p>
                    <!-- Dietary & Health Tags Below Description -->
                    <div id="modal-dietary-tags" class="flex flex-wrap gap-1.5 mt-3">
                        <!-- Dynamic Badges -->
                    </div>
                </div>
                <!-- ADD-ONS / CUSTOMIZERS -->
                <div class="space-y-4 pt-3 border-t border-white/10" id="modal-addons-container">
                   <!-- Size / Volume Selector for Nature's Spring & Bottled Water -->
                    <div class="space-y-1.5" id="modal-size-container" style="display: none;">
                        <label class="text-[11px] font-bold text-brand-slate uppercase tracking-wider block flex
items-center gap-1.5">
                            <i class="fa-solid fa-bottle-water text-brand-orange"></i> Select Size / Volume
                        </label>
                        <select id="modal-size-select" onchange="updateModalTotal()" class="w-full bg-brand-dark
border border-white/10 text-white rounded-xl px-3 py-2.5 text-xs font-semibold focus:outline-none
focus:border-brand-orange">
                            <!-- Populated dynamically -->
                        </select>
```

```
                    </div>
                </div>
                <div class="flex items-center justify-between pt-3 border-t border-white/10">
                    <div>
                        <span class="text-[10px] text-brand-slate uppercase font-bold block">Total Amount</span>
                        <span id="modal-item-price" class="text-xl sm:text-2xl font-black
text-brand-orange">₱0.00</span>
                    </div>
                    <div class="flex items-center gap-3">
                        <!-- Quantity Counter [ - qty + ] -->
                        <div class="flex items-center gap-2 bg-white/5 px-3 py-2 rounded-2xl border border-white/10
backdrop-blur-md">
                            <button type="button" onclick="changeItemModalQty(-1)" class="text-brand-slate
hover:text-white font-black px-2 py-1 transition cursor-pointer"><i class="fa-solid fa-minus text-xs"></i></button>
                            <input id="modal-item-qty" type="text" inputmode="numeric" value="1"
aria-label="Quantity"
                                oninput="onModalQtyInput(this)" onchange="onModalQtyCommit(this)"
onblur="onModalQtyCommit(this)"
                                class="w-10 bg-transparent text-sm font-black text-white text-center
focus:outline-none focus:ring-0">
                            <button type="button" onclick="changeItemModalQty(1)" class="text-brand-orange
hover:text-orange-500 font-black px-2 py-1 transition cursor-pointer"><i class="fa-solid fa-plus
text-xs"></i></button>
                        </div>
                        <button id="modal-add-btn" onclick="addCustomizedItemToCart()" class="bg-brand-orange
hover:bg-orange-600 text-white font-extrabold px-5 py-3 rounded-2xl uppercase tracking-wider text-xs shadow-lg
shadow-brand-orange/25 transition flex items-center gap-2 active:scale-95 cursor-pointer">
                            <i class="fa-solid fa-cart-plus"></i> Add
                        </button>
                    </div>
                </div>
            </div>
        </div>
    </div>
    <script src="{% static 'js/ui-loading.js' %}"></script>
</body>
</html>
```

Faculty / Staff Customer Portal

Institutional customer authentication and portal for faculty and staff (@psu.palawan.edu.ph email requirement with username validation). Enables institutional users to place pickup/office-delivery orders, track loyalty points/rewards, and manage account settings and order history.

backend/accounts/views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth import get_user_model
from django.views.decorators.csrf import ensure_csrf_cookie, csrf_protect
from django.http import JsonResponse
from django.core.cache import cache
from django.core.mail import send_mail
from django.core.exceptions import ValidationError
from django.contrib.auth.password_validation import validate_password
from django.conf import settings
import random
import json
import os
import re
import time
import hashlib
```

```
import hmac
from customer_portal.views import _get_formatted_menu
User = get_user_model()
def role_portal(user):
    role = getattr(user, 'role', '') or ''
    if user.is_superuser or user.is_staff or role in ('STAFF', 'ADMIN'):
        return 'canteen_menu:staff_dashboard'
    if role in ('DELIVERY', 'RIDER'):
        return 'deliveries:dashboard'
    if role == 'FACULTY':
        return 'accounts:dashboard'
    return 'customer_portal:kiosk_menu'
_EMAIL_RE = re.compile(r'^[^@\s]+@[^@\s]+\.[^@\s]+$')
def _is_valid_email(email):
    return bool(email and _EMAIL_RE.match(email))
def _otp_rate_allowed(session, prefix, limit=5, window=600, cooldown=60):
    now = int(time.time())
    last = session.get(f'{prefix}_sent_at', 0)
    count = session.get(f'{prefix}_sent_count', 0)
    if now - last < cooldown:
        return 'cooldown'
    if now - last >= window:
        session[f'{prefix}_sent_at'] = now
        session[f'{prefix}_sent_count'] = 1
        return ''
    if count >= limit:
        return 'limit'
    session[f'{prefix}_sent_at'] = now
    session[f'{prefix}_sent_count'] = count + 1
    return ''
def _otp_hash(code):
    return hashlib.sha256(str(code).encode('utf-8')).hexdigest()
def _otp_verify(entered, stored_hash):
    return bool(stored_hash) and hmac.compare_digest(_otp_hash(entered), stored_hash)
def _send_otp_email(to_email, subject, message):
    import urllib.request
    import urllib.error
    try:
        send_mail(
            subject=subject,
            message=message,
            from_email=None,
            recipient_list=[to_email],
            fail_silently=False,
        )
        return True
    except Exception as smtp_err:
        print(f"SMTP send failed: {smtp_err}")
    api_key = getattr(settings, 'EMAIL_HOST_PASSWORD', '') or os.getenv('EMAIL_HOST_PASSWORD', '')
    from_email = getattr(settings, 'DEFAULT_FROM_EMAIL', '') or os.getenv('DEFAULT_FROM_EMAIL', 'Canteen Express
<onboarding@resend.dev>')
    if api_key and (api_key.startswith('re_') or 'resend' in str(getattr(settings, 'EMAIL_HOST', '')).lower()):
        try:
            url = 'https://api.resend.com/emails'
            payload = {
                "from": from_email,
                "to": [to_email],
                "subject": subject,
                "html": f"<div
style='font-family:sans-serif;font-size:16px;line-height:1.5;color:#333;'><p>{message.replace(chr(10),
'<br>')}</p></div>"
            }
            data = json.dumps(payload).encode('utf-8')
            req = urllib.request.Request(
                url,
```

```
                data=data,
                headers={
                    "Authorization": f"Bearer {api_key}",
                    "Content-Type": "application/json",
                    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/120.0.0.0 Safari/537.36"
                },
                method="POST"
            )
            with urllib.request.urlopen(req, timeout=10) as response:
                if response.status in (200, 201):
                    print(f"Resend HTTP API send OK -> {to_email}")
                    return True
        except urllib.error.HTTPError as he:
            err_body = he.read().decode('utf-8') if he.fp else ''
            print(f"Resend HTTP API HTTPError {he.code}: {err_body}")
        except Exception as resend_err:
            print(f"Resend HTTP API fallback failed: {resend_err}")

    return False

def _client_ip(request):
    fwd = request.META.get('HTTP_X_FORWARDED_FOR', '')
    if fwd:
        return fwd.split(',')[0].strip()
    return request.META.get('REMOTE_ADDR', '')
def _ip_otp_rate_allowed(request, prefix, limit=5, window=600, cooldown=60):
    now = int(time.time())
    ip = _client_ip(request)
    base = f'otp_ip_{prefix}_{ip}'
    last = cache.get(base + '_sent_at', 0)
    count = cache.get(base + '_count', 0)
    if now - last < cooldown:
        return 'cooldown'
    if now - last >= window:
        cache.set(base + '_sent_at', now, timeout=window)
        cache.set(base + '_count', 1, timeout=window)
        return ''
    if count >= limit:
        return 'limit'
    cache.set(base + '_sent_at', now, timeout=window)
    cache.set(base + '_count', count + 1, timeout=window)
    return ''
def _login_locked(request, key):
    ip = _client_ip(request)
    lock_until = cache.get(f'login_lock_until_{key}_{ip}')
    if lock_until:
        remaining = int(lock_until - time.time())
        if remaining > 0:
            return remaining
        else:
            cache.delete(f'login_lock_until_{key}_{ip}')
            cache.delete(f'login_fail_{key}_{ip}')
    return 0
def _login_attempt_failed(request, key):
    ip = _client_ip(request)
    counter = cache.get(f'login_fail_{key}_{ip}', 0) + 1
    cache.set(f'login_fail_{key}_{ip}', counter, timeout=900)
    if counter >= 5:
        cache.set(f'login_lock_until_{key}_{ip}', time.time() + 60, timeout=900)
def _login_clear(request, key):
    ip = _client_ip(request)
    cache.delete(f'login_fail_{key}_{ip}')
    cache.delete(f'login_lock_until_{key}_{ip}')
def landing_view(request):
```

```
    if request.user.is_authenticated:
        return redirect(role_portal(request.user))
    return render(request, 'accounts/landing.html')
def access_denied_view(request):
    user_role = getattr(request.user, 'role', '')
    referer = request.META.get('HTTP_REFERER', '')
    login_url = 'accounts:landing'
    if 'faculty' in referer or 'dashboard' in referer or user_role == 'FACULTY':
        login_url = 'accounts:faculty_auth'
    elif 'staff' in referer or 'canteen' in referer or user_role in ('STAFF', 'ADMIN') or request.user.is_staff:
        login_url = 'accounts:staff_login'
    elif 'delivery' in referer or 'rider' in referer or user_role in ('DELIVERY', 'RIDER'):
        login_url = 'accounts:delivery_login'
    elif user_role == 'STUDENT':
        login_url = 'accounts:landing'
    else:
        if user_role == 'FACULTY':
            login_url = 'accounts:faculty_auth'
        elif user_role in ('STAFF', 'ADMIN') or request.user.is_staff:
            login_url = 'accounts:staff_login'
        elif user_role in ('DELIVERY', 'RIDER'):
            login_url = 'accounts:delivery_login'
    return render(request, 'accounts/access_denied.html', {'login_url': login_url})
# STEP 1: Faculty Location Check
@ensure_csrf_cookie
def faculty_location_view(request):
    if request.method == 'POST':
        lat = request.POST.get('latitude')
        lng = request.POST.get('longitude')
        request.session['faculty_lat'] = lat
        request.session['faculty_lng'] = lng
        return redirect('accounts:faculty_auth')
    return render(request, 'accounts/faculty_location.html')
# STEP 2: Faculty Auth (Login or Signup with Database Saving & Staff Section Reflection)
@ensure_csrf_cookie
@csrf_protect
def faculty_auth_view(request):
    error = None
    mode = request.GET.get('mode', 'login')
    if request.method == 'POST':
        action = request.POST.get('action')
        if action == 'signup':
            if request.POST.get('website'):
                return redirect('accounts:landing')
            email = request.POST.get('email', '').strip().lower()
            name = request.POST.get('name', '').strip()
            password = request.POST.get('password')
            confirm_password = request.POST.get('confirm_password')
            local_part = email.split('@')[0] if '@' in email else ''
            if not (_is_valid_email(email) and email.endswith('@psu.palawan.edu.ph')):
                error = "Please enter a valid @psu.palawan.edu.ph email address."
                mode = 'signup'
            elif not any(c.isalpha() for c in local_part):
                error = "Email username before @psu.palawan.edu.ph must contain letters (cannot be numbers only)."
                mode = 'signup'
            elif password != confirm_password:
                error = "Passwords do not match."
                mode = 'signup'
            elif not request.session.get('otp_verified') or request.session.get('signup_email') != email:
                error = "Please verify your email with the OTP code first."
                mode = 'signup'
            elif User.objects.filter(email=email).exists():
                error = "An account with this email already exists. Please sign in."
                mode = 'login'
            else:
```

```
                try:
                    validate_password(
                        password,
                        user=User(username=(email.split('@')[0] or 'user'), email=email, first_name=name),
                    )
                except ValidationError as e:
                    error = ' '.join(e.messages)
                    mode = 'signup'
                    return render(request, 'accounts/faculty_auth.html', {'error': error, 'mode': mode})
                try:
                    username = email.split('@')[0]
                    if User.objects.filter(username=username).exists():
                        username = f"{username}_{User.objects.count()}"

                    user = User.objects.create_user(
                        username=username,
                        email=email,
                        password=password,
                        first_name=name,
                        role='FACULTY',
                        is_email_verified=True,
                        college_department='',
                        department=''
                    )
                    logout(request)
                    login(request, user)
                    request.session['faculty_email'] = email
                    return redirect('accounts:dashboard')
                except Exception as e:
                    error = f"Registration error: {str(e)}"
                    mode = 'signup'
        elif action == 'login':
            email = request.POST.get('email', '').strip().lower()
            password = request.POST.get('password')
            remember_me = request.POST.get('remember_me')

            if _login_locked(request, 'faculty'):
                error = "Too many failed attempts. Please try again in a few minutes."
                mode = 'login'
            else:
                try:
                    user_obj = User.objects.filter(email=email).first()
                    if user_obj:
                        if not user_obj.is_email_verified:
                            error = "Please verify your email first before signing in."
                            mode = 'login'
                        elif not user_obj.is_active or user_obj.account_status in ('banned', 'restricted', 'held',
'penalized'):
                            status_lbl = user_obj.account_status.upper() if user_obj.account_status else
'BANNED/RESTRICTED'
                            reason_txt = f"\nReason: {user_obj.status_reason}" if user_obj.status_reason else ""
                            error = f"⚠️ Account Access Denied!\nYour account has been
{status_lbl}.{reason_txt}\nPlease contact canteen administration."
                            mode = 'login'
                        else:
                            user = authenticate(request, username=user_obj.username, password=password)
                            if user is not None:
                                logout(request)
                                login(request, user)
                                request.session.cycle_key()
                                if remember_me:
                                    request.session.set_expiry(1209600)
                                else:
                                    request.session.set_expiry(0)
                                request.session['faculty_email'] = email
```

```
                                _login_clear(request, 'faculty')
                                return redirect(role_portal(user))
                            else:
                                _login_attempt_failed(request, 'faculty')
                                error = "Invalid password."
                                mode = 'login'
                    else:
                        error = "No account found with this email. Please sign up first."
                        mode = 'signup'
                except Exception as e:
                    error = f"Login error: {str(e)}"
                    mode = 'login'

    return render(request, 'accounts/faculty_auth.html', {'error': error, 'mode': mode})
# STEP 3: Faculty Dashboard
def faculty_dashboard_view(request, token=None):
    if not request.user.is_authenticated:
        return redirect('accounts:faculty_auth')
    user_role = getattr(request.user, 'role', '')
    if user_role not in ('FACULTY', 'STAFF', 'ADMIN') and not request.user.is_staff and not
request.user.is_superuser:
        return redirect('accounts:access_denied')
    import uuid
    from django.urls import reverse
    session_token = request.session.get('faculty_secure_token')
    if not session_token:
        session_token = uuid.uuid4().hex[:12]
        request.session['faculty_secure_token'] = session_token
    if not token or token != session_token:
        query_string = request.META.get('QUERY_STRING', '')
        redirect_url = reverse('accounts:dashboard_hashed', kwargs={'token': session_token})
        if query_string:
            redirect_url += f'?{query_string}'
        return redirect(redirect_url)
    from canteen_menu.models import MenuItem, Category
    import json
    menu_items = MenuItem.objects.select_related('category').filter(is_available=True)
    categories = Category.objects.all().order_by('-name')
    email = request.session.get('faculty_email', '') or getattr(request.user, 'email', '')
    if request.user.is_authenticated and (request.user.get_full_name() or request.user.first_name):
        faculty_display_name = request.user.get_full_name() or request.user.first_name
    elif request.user.is_authenticated and request.user.username:
        faculty_display_name = request.user.username
    elif email:
        faculty_display_name = email.split('@')[0]
    else:
        faculty_display_name = 'User'
    formatted_menu = _get_formatted_menu()
    from deliveries.models import DeliveryRequest
    from deliveries.utils import serialize_delivery, prefetch_delivery_relations
    if request.user.is_authenticated:
        ongoing_deliveries = prefetch_delivery_relations(DeliveryRequest.objects.filter(
            order__customer=request.user).order_by('-requested_at')[:5])
    else:
        ongoing_deliveries = []
    ongoing_data = [serialize_delivery(d) for d in ongoing_deliveries]
    context = {
        'menu_items': menu_items,
        'categories': categories,
        'menu_data_json': json.dumps(formatted_menu),
        'faculty_display_name': faculty_display_name,
        'ongoing_deliveries_json': json.dumps(ongoing_data),
        'user_points': float(request.user.loyalty_points) if request.user.is_authenticated else 0.0,
        'enforce_geofence': getattr(settings, 'ENFORCE_GEOFENCE', True),
    }
```

```
    return render(request, 'accounts/dashboard.html', context)
# Separate Staff Login View
@ensure_csrf_cookie
@csrf_protect
def staff_login_view(request):
    error = None
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        if _login_locked(request, 'staff'):
            error = "Too many failed attempts. Please try again in a few minutes."
        else:
            user_obj = User.objects.filter(username=username).first()
            if user_obj and (user_obj.is_staff or getattr(user_obj, 'role', '') in ['STAFF', 'ADMIN']):
                if not user_obj.is_active or user_obj.account_status in ('banned', 'restricted', 'held',
'penalized'):
                    status_lbl = user_obj.account_status.upper() if user_obj.account_status else 'BANNED/INACTIVE'
                    reason_txt = f"\nReason: {user_obj.status_reason}" if user_obj.status_reason else ""
                    error = f"⚠️ Account Access Denied!\nYour account has been {status_lbl}.{reason_txt}\nPlease
contact canteen administration."
                else:
                    user = authenticate(request, username=username, password=password)
                    if user is not None:
                        logout(request)
                        login(request, user)
                        request.session.cycle_key()
                        _login_clear(request, 'staff')
                        return redirect('canteen_menu:staff_dashboard')
                    else:
                        _login_attempt_failed(request, 'staff')
                        error = "Invalid canteen staff credentials."
            else:
                _login_attempt_failed(request, 'staff')
                error = "Invalid canteen staff credentials."
    return render(request, 'accounts/staff_login.html', {'error': error})
# Separate Delivery Personnel Login View
@ensure_csrf_cookie
@csrf_protect
def delivery_login_view(request):
    error = None
    lock_secs = _login_locked(request, 'rider')
    if request.method == 'POST':
        if lock_secs > 0:
            error = f"Too many failed attempts. Please try again in {lock_secs} seconds."
        else:
            username = request.POST.get('username')
            password = request.POST.get('password')
            user_obj = User.objects.filter(username=username).first()
            if user_obj and (getattr(user_obj, 'role', '') in ['DELIVERY', 'RIDER'] or user_obj.is_staff):
                if not user_obj.is_active or user_obj.account_status in ('banned', 'restricted', 'held',
'penalized'):
                    status_lbl = user_obj.account_status.upper() if user_obj.account_status else 'BANNED/INACTIVE'
                    reason_txt = f"\nReason: {user_obj.status_reason}" if user_obj.status_reason else ""
                    error = f"⚠️ Account Access Denied!\nYour account has been {status_lbl}.{reason_txt}\nPlease
contact canteen administration."
                else:
                    user = authenticate(request, username=username, password=password)
                    if user is not None:
                        logout(request)
                        login(request, user)
                        request.session.cycle_key()
                        _login_clear(request, 'rider')
                        return redirect('deliveries:dashboard')
                    else:
                        _login_attempt_failed(request, 'rider')
```

```
                        lock_secs = _login_locked(request, 'rider')
                        error = f"Invalid delivery personnel credentials. {f'Locked for {lock_secs}s.' if lock_secs
> 0 else ''}"
            else:
                _login_attempt_failed(request, 'rider')
                lock_secs = _login_locked(request, 'rider')
                error = f"Invalid delivery personnel credentials. {f'Locked for {lock_secs}s.' if lock_secs > 0
else ''}"
    return render(request, 'accounts/delivery_login.html', {'error': error, 'lock_seconds': lock_secs})
# Role-specific Logout Views
def faculty_logout_view(request):
    request.session.flush()
    logout(request)
    return redirect('accounts:faculty_auth')
def staff_logout_view(request):
    logout(request)
    return redirect('accounts:staff_login')
def delivery_logout_view(request):
    logout(request)
    return redirect('accounts:delivery_login')
@ensure_csrf_cookie
def send_signup_otp(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            email = data.get('email', '').strip().lower()
            local_part = email.split('@')[0] if '@' in email else ''
            otp_code = f"{random.randint(100000, 999999)}"
            if not (_is_valid_email(email) and email.endswith('@psu.palawan.edu.ph')):
                return JsonResponse({'success': False, 'error': 'Please enter a valid @psu.palawan.edu.ph email
address.'}, status=400)
            if not any(c.isalpha() for c in local_part):
                return JsonResponse({'success': False, 'error': 'Email username before @psu.palawan.edu.ph must
contain letters (cannot be numbers only).'}, status=400)
            rate_result = _otp_rate_allowed(request.session, 'signup')
            ip_rate_result = _ip_otp_rate_allowed(request, 'signup')
            if rate_result or ip_rate_result:
                return JsonResponse({'success': False, 'error': 'Please wait a minute before requesting another
code.' if (rate_result or ip_rate_result) == 'cooldown' else 'Too many OTP requests. Please wait 10 minutes.'},
status=429)
            request.session['signup_otp'] = _otp_hash(otp_code)
            request.session['signup_email'] = email
            request.session['otp_verified'] = False
            request.session['signup_otp_created_at'] = int(time.time())
            request.session['signup_otp_attempts'] = 0
            request.session.save()
            email_sent = _send_otp_email(
                to_email=email,
                subject='Canteen Express - Your OTP Verification Code',
                message=(
                    f'Hello,\n\n'
                    f'Your Canteen Express verification OTP code is: {otp_code}\n\n'
                    f'This code expires in 10 minutes. If you did not request this, please ignore this email.\n\n'
                    f'- Canteen Express Team'
                )
            )
            if email_sent:
                return JsonResponse({
                    'success': True,
                    'email_sent': True,
                    'message': 'OTP sent to your email.'
                })
            for _key in ('signup_otp', 'signup_email', 'signup_otp_created_at', 'signup_otp_attempts'):
                request.session.pop(_key, None)
```

```
            return JsonResponse({
                'success': True,
                'email_sent': False,
                'message': 'Email delivery failed. Use the on-screen OTP instead.',
                'otp_code': otp_code,
            })
        except Exception as e:
            return JsonResponse({'success': False, 'error': str(e)}, status=400)
    return JsonResponse({'success': False}, status=405)
@ensure_csrf_cookie
def verify_signup_otp(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            entered_otp = data.get('otp', '').strip()
            session_otp = request.session.get('signup_otp')
            created_at = request.session.get('signup_otp_created_at', 0)
            attempts = request.session.get('signup_otp_attempts', 0)
            if not session_otp or not created_at or (int(time.time()) - created_at) > 600:
                return JsonResponse({'success': False, 'error': 'OTP expired. Please request a new code.'},
status=400)
            if attempts >= 5:
                return JsonResponse({'success': False, 'error': 'Too many incorrect attempts. Please request a new
code.'}, status=400)
            if _otp_verify(entered_otp, session_otp):
                request.session['otp_verified'] = True
                request.session['signup_otp'] = None
                request.session.save()
                return JsonResponse({'success': True})
            request.session['signup_otp_attempts'] = attempts + 1
            request.session.save()
            return JsonResponse({'success': False, 'error': 'Invalid or expired OTP code.'}, status=400)
        except Exception as e:
            return JsonResponse({'success': False, 'error': str(e)}, status=400)
    return JsonResponse({'success': False}, status=405)
@ensure_csrf_cookie
def send_password_reset_otp(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            email = data.get('email', '').strip().lower()
            otp_code = f"{random.randint(100000, 999999)}"
            if not (_is_valid_email(email) and email.endswith('@psu.palawan.edu.ph')):
                return JsonResponse({'success': False, 'error': 'Please enter a valid @psu.palawan.edu.ph email
address.'}, status=400)
            if not User.objects.filter(email=email).exists():
                return JsonResponse({'success': False, 'error': 'No account found with this email.'}, status=400)
            rate_result = _otp_rate_allowed(request.session, 'reset')
            ip_rate_result = _ip_otp_rate_allowed(request, 'reset')
            if rate_result or ip_rate_result:
                return JsonResponse({'success': False, 'error': 'Please wait a minute before requesting another
code.' if (rate_result or ip_rate_result) == 'cooldown' else 'Too many OTP requests. Please wait 10 minutes.'},
status=429)
            request.session['reset_otp'] = _otp_hash(otp_code)
            request.session['reset_email'] = email
            request.session['reset_otp_verified'] = False
            request.session['reset_otp_created_at'] = int(time.time())
            request.session['reset_otp_attempts'] = 0
            email_sent = _send_otp_email(
                to_email=email,
                subject='Canteen Express - Your Password Reset OTP',
                message=(
                    f'Hello,\n\n'
                    f'Your Canteen Express password reset OTP code is: {otp_code}\n\n'
                    f'This code expires in 10 minutes. If you did not request this, please ignore this email.\n\n'
```

```
                    f'- Canteen Express Team'
                )
            )
            if email_sent:
                return JsonResponse({
                    'success': True,
                    'email_sent': True,
                    'message': 'OTP sent to your email.'
                })
            request.session.pop('reset_otp', None)
            request.session.pop('reset_email', None)
            request.session.pop('reset_otp_verified', None)
            request.session.pop('reset_otp_created_at', None)
            request.session.pop('reset_otp_attempts', None)
            return JsonResponse({
                'success': True,
                'email_sent': False,
                'message': 'Email delivery failed. Use the on-screen OTP instead.',
                'otp_code': otp_code,
            })
        except Exception as e:
            return JsonResponse({'success': False, 'error': str(e)}, status=400)
    return JsonResponse({'success': False}, status=405)


@ensure_csrf_cookie
def verify_and_reset_password(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            email = data.get('email', '').strip().lower()
            entered_otp = data.get('otp', '').strip()
            new_password = data.get('new_password')
            session_otp = request.session.get('reset_otp')
            session_email = request.session.get('reset_email')
            created_at = request.session.get('reset_otp_created_at', 0)
            attempts = request.session.get('reset_otp_attempts', 0)
            if not session_email or session_email != email or not session_otp or not created_at:
                return JsonResponse({'success': False, 'error': 'Invalid or expired OTP code.'}, status=400)
            if (int(time.time()) - created_at) > 600:
                return JsonResponse({'success': False, 'error': 'OTP expired. Please request a new code.'},
status=400)
            if attempts >= 5:
                return JsonResponse({'success': False, 'error': 'Too many incorrect attempts. Please request a new
code.'}, status=400)
            if not _otp_verify(entered_otp, session_otp):
                request.session['reset_otp_attempts'] = attempts + 1
                return JsonResponse({'success': False, 'error': 'Invalid or expired OTP code.'}, status=400)

            user = User.objects.filter(email=email).first()
            if not user:
                return JsonResponse({'success': False, 'error': 'User not found.'}, status=400)
            try:
                validate_password(new_password, user=user)
            except ValidationError as e:
                return JsonResponse({'success': False, 'error': ' '.join(e.messages)}, status=400)
            user.set_password(new_password)
            user.save()
            request.session.cycle_key()
            request.session.pop('reset_otp', None)
            request.session.pop('reset_email', None)
            request.session.pop('reset_otp_verified', None)
            return JsonResponse({'success': True, 'message': 'Password successfully reset.'})
        except Exception as e:
            return JsonResponse({'success': False, 'error': str(e)}, status=400)
```

```
    return JsonResponse({'success': False}, status=405)
@ensure_csrf_cookie
@csrf_protect
def change_password_view(request):
    if not request.user.is_authenticated:
        return JsonResponse({'success': False, 'error': 'Unauthorized'}, status=401)
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            current_password = data.get('current_password', '')
            new_password = data.get('new_password', '')
            confirm_password = data.get('confirm_password', '')

            if not new_password or not confirm_password:
                return JsonResponse({'success': False, 'error': 'New password and confirmation are required.'},
status=400)
            if new_password != confirm_password:
                return JsonResponse({'success': False, 'error': 'New passwords do not match.'}, status=400)
            user = request.user
            if user.has_usable_password() and current_password:
                if not user.check_password(current_password):
                    return JsonResponse({'success': False, 'error': 'Incorrect current password.'}, status=400)
            elif user.has_usable_password() and not current_password:
                return JsonResponse({'success': False, 'error': 'Current password is required.'}, status=400)
            try:
                validate_password(new_password, user=user)
            except ValidationError as e:
                return JsonResponse({'success': False, 'error': ' '.join(e.messages)}, status=400)
            user.set_password(new_password)
            user.save()
            from django.contrib.auth import update_session_auth_hash
            update_session_auth_hash(request, user)
            return JsonResponse({'success': True, 'message': 'Password successfully updated.'})
        except Exception as e:
            return JsonResponse({'success': False, 'error': str(e)}, status=400)
    return JsonResponse({'success': False, 'error': 'Method not allowed'}, status=405)
```

backend/accounts/urls.py

```
# accounts/urls.py
from django.urls import path
from django.contrib.auth import views as auth_views
from . import views
app_name = 'accounts'
urlpatterns = [
    path('', views.landing_view, name='landing'),
    path('access-denied/', views.access_denied_view, name='access_denied'),
    path('login/', auth_views.LoginView.as_view(template_name='accounts/login.html'), name='login'),
    path('staff-login/', views.staff_login_view, name='staff_login'),
    path('delivery-login/', views.delivery_login_view, name='delivery_login'),
    # Specific Role Logouts & Fallback Logout
    path('logout/', views.faculty_logout_view, name='logout'),
    path('logout/faculty/', views.faculty_logout_view, name='faculty_logout'),
    path('logout/staff/', views.staff_logout_view, name='staff_logout'),
    path('logout/delivery/', views.delivery_logout_view, name='delivery_logout'),
    path('faculty/location/', views.faculty_location_view, name='faculty_location'),
    path('faculty/auth/', views.faculty_auth_view, name='faculty_auth'),
    path('faculty/dashboard/', views.faculty_dashboard_view, name='dashboard'),
    path('faculty/<str:token>/dashboard/', views.faculty_dashboard_view, name='dashboard_hashed'),
    path('send-otp/', views.send_signup_otp, name='send_signup_otp'),
    path('verify-otp/', views.verify_signup_otp, name='verify_signup_otp'),
    path('send-password-reset-otp/', views.send_password_reset_otp, name='send_password_reset_otp'),
    path('reset-password-confirm/', views.verify_and_reset_password, name='reset_password_confirm'),
    path('change-password/', views.change_password_view, name='change_password'),
]
```

backend/accounts/templates/accounts/faculty_auth.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link rel="preconnect" href="https://cdnjs.cloudflare.com">
    <link rel="icon" href="{% static 'customer_portal/images/canteen-express-logo.png' %}" type="image/png" />
    <link rel="manifest" href="{% static 'manifest-faculty.json' %}">
    <meta name="theme-color" content="#f97316">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
    <title>Faculty/Staff Portal - Login & Sign Up</title>
    <!-- Tailwind CSS CDN -->
    <link rel="stylesheet" href="{% static 'css/auth.css' %}">
    <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600;700;800;900&family=Plus+Jakarta+Sans:wght@4
00;500;600;700;800;900&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" />
</head>
<body class="bg-brand-dark text-white flex flex-col justify-between py-2 px-4 max-w-md mx-auto w-full relative
select-none"
      x-data="facultyAuth()"
      x-init="initLocation()">

    <!-- ================= DIAGONAL BACKGROUND FOOD CAROUSEL ================= -->
    <div class="absolute inset-0 z-0 pointer-events-none opacity-35 overflow-hidden flex flex-col justify-center
gap-10 diagonal-wrapper">
        <div class="animate-scroll-1 flex gap-6">
            <div class="flex gap-6 shrink-0">
                <img
src="https://images.unsplash.com/photo-1568901346375-23c9450c58cd?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1546069901-ba9599a7e63c?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1555396273-367ea4eb4db5?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1565299624946-b28f40a0ae38?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
            </div>
            <div class="flex gap-6 shrink-0">
                <img
src="https://images.unsplash.com/photo-1568901346375-23c9450c58cd?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1546069901-ba9599a7e63c?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1555396273-367ea4eb4db5?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1565299624946-b28f40a0ae38?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
            </div>
        </div>
        <div class="animate-scroll-2 flex gap-6">
            <div class="flex gap-6 shrink-0">
                <img
src="https://images.unsplash.com/photo-1512621776951-a57141f2eefd?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
```

```
                <img
src="https://images.unsplash.com/photo-1504674900247-0877df9cc836?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1498837167922-ddd27525d352?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1540189549336-e6e99c3679fe?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
            </div>
            <div class="flex gap-6 shrink-0">
                <img
src="https://images.unsplash.com/photo-1512621776951-a57141f2eefd?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1504674900247-0877df9cc836?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1498837167922-ddd27525d352?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
                <img
src="https://images.unsplash.com/photo-1540189549336-e6e99c3679fe?auto=format&fit=crop&w=500&q=80" class="w-64 h-40
object-cover rounded-2xl shadow-xl border border-white/10">
            </div>
        </div>
    </div>
    <!-- Dark Gradient Overlay -->
    <div class="absolute inset-0 bg-gradient-to-b from-brand-dark/80 via-brand-dark/75 to-brand-dark/90 z-0"></div>
    <!-- Header Brand (Image 2) -->
    <header class="flex justify-center pt-10 relative z-10 shrink-0 animate-page-in">
        <div class="flex items-center gap-2 bg-black/60 px-4 py-2 rounded-full border border-white/15
backdrop-blur-md">
            <img src="{% static 'customer_portal/images/canteen-express-logo.png' %}" alt="Logo" class="h-6 w-auto
object-contain">
            <span class="text-xs font-bold uppercase tracking-[0.2em] text-white/90 font-poppins">CANTEEN
EXPRESS</span>
        </div>
    </header>
    <!-- Main Card Container (Larger & Roomier Form) -->
    <main class="w-full max-w-md mx-auto relative z-10 shrink-0 my-auto animate-rise-in">
        <div class="bg-[#121215]/90 border border-white/15 rounded-[24px] p-4 sm:p-5 space-y-2.5 backdrop-blur-2xl
shadow-[0_25px_60px_-15px_rgba(0,0,0,0.9)] ring-1 ring-white/10 text-white">

            <div class="text-center space-y-1">
                <div class="w-10 h-10 bg-brand-orange/20 border border-brand-orange/40 text-brand-orange rounded-xl
flex items-center justify-center mx-auto shadow-lg shadow-brand-orange/20 text-sm">
                    <i class="fa-solid fa-graduation-cap"></i>
                </div>
                <h2 class="text-2xl font-black text-white font-poppins" x-text="mode === 'login' ? 'Faculty/Staff
Sign In' : 'Faculty/Staff Sign Up'"></h2>
                <p class="text-xs text-white/70 leading-relaxed max-w-xs mx-auto">Use your institutional email
ending in @psu.palawan.edu.ph.</p>
            </div>
            {% if error %}
            <div class="p-3 bg-red-500/10 border border-red-500/30 text-red-400 rounded-xl text-xs font-bold flex
items-center gap-2 backdrop-blur-md">
                <i class="fa-solid fa-triangle-exclamation"></i>
                <span>{{ error }}</span>
            </div>
            {% endif %}
            <!-- Tab Toggle Buttons -->
            <div class="grid grid-cols-2 gap-2 bg-white/5 border border-white/10 p-1.5 rounded-2xl">
                <button type="button" @click="mode = 'login'" :class="mode === 'login' ? 'bg-brand-orange
text-white font-black shadow-lg shadow-brand-orange/30' : 'text-white/60 font-bold hover:text-white'" class="py-2.5
rounded-xl text-xs transition uppercase tracking-wider cursor-pointer font-poppins">
```

```
                    Sign In
                </button>
                <button type="button" @click="mode = 'signup'" :class="mode === 'signup' ? 'bg-brand-orange
text-white font-black shadow-lg shadow-brand-orange/30' : 'text-white/60 font-bold hover:text-white'" class="py-2.5
rounded-xl text-xs transition uppercase tracking-wider cursor-pointer font-poppins">
                    Sign Up
                </button>
            </div>
            <!-- LOGIN FORM -->
            <form method="POST" onsubmit="return handleLoginSubmit(this)" class="space-y-3.5" x-show="mode ===
'login'">
                {% csrf_token %}
                <input type="hidden" name="action" value="login">
                <input type="hidden" name="latitude" x-model="lat">
                <input type="hidden" name="longitude" x-model="lng">
                <div>
                    <label class="block text-xs font-extrabold text-white/80 uppercase tracking-wider
mb-1">Institutional Email</label>
                    <input type="email" name="email" required placeholder="faculty/staff@psu.palawan.edu.ph"
                        class="w-full px-3.5 py-2.5 bg-white/5 border border-white/15 rounded-2xl focus:ring-2
focus:ring-brand-orange focus:outline-none text-xs text-white font-bold placeholder-white/30 backdrop-blur-md">
                </div>
                <div>
                    <label class="block text-xs font-extrabold text-white/80 uppercase tracking-wider
mb-1">Password</label>
                    <div class="relative">
                        <input :type="showLoginPass ? 'text' : 'password'" name="password" required
placeholder="••••••••"
                            class="w-full px-3.5 py-2.5 pr-11 bg-white/5 border border-white/15 rounded-2xl
focus:ring-2 focus:ring-brand-orange focus:outline-none text-xs text-white font-bold placeholder-white/30
backdrop-blur-md">
                        <button type="button" @click="showLoginPass = !showLoginPass" class="absolute right-3.5
top-1/2 -translate-y-1/2 text-orange-400 hover:text-orange-300 cursor-pointer text-sm hover:scale-110
active:scale-90 transition">
                            <i :class="showLoginPass ? 'fa-solid fa-eye-slash' : 'fa-solid fa-eye'"></i>
                        </button>
                    </div>
                </div>
                <!-- Remember Me & Forgot Password -->
                <div class="flex items-center justify-between text-xs pt-1">
                    <label class="flex items-center gap-2 cursor-pointer select-none">
                        <input type="checkbox" name="remember_me" class="w-4 h-4 rounded border-white/20 bg-white/5
text-brand-orange focus:ring-brand-orange cursor-pointer">
                        <span class="font-bold text-white/80 text-xs">Remember Me</span>
                    </label>
                    <button type="button" @click="showForgotModal = true" class="text-brand-orange font-bold
hover:underline cursor-pointer text-xs">
                        Forgot Password?
                    </button>
                </div>
                <button type="submit" class="w-full py-2.5 bg-brand-orange hover:bg-orange-600 disabled:opacity-50
disabled:cursor-not-allowed text-white font-black rounded-2xl shadow-xl shadow-brand-orange/35 transition text-xs
uppercase tracking-wider flex items-center justify-center gap-2 cursor-pointer relative z-20 font-poppins">
                    <template x-if="loginSubmitting">
                        <span class="flex items-center gap-2">
                            <i class="fa-solid fa-spinner fa-spin"></i>
                            <span>Signing In...</span>
                        </span>
                    </template>
                    <template x-if="!loginSubmitting">
                        <span class="flex items-center gap-2">
                            <span>Sign In to Dashboard</span>
                            <i class="fa-solid fa-arrow-right text-xs"></i>
                        </span>
                    </template>
```

```
                </button>
            </form>
            <!-- SIGNUP FORM -->
            <form method="POST" onsubmit="return handleSignupSubmit(this)" class="space-y-2.5" x-show="mode ===
'signup'" style="display: none;">
                {% csrf_token %}
                <input type="hidden" name="action" value="signup">
                <input type="hidden" name="latitude" x-model="lat">
                <input type="hidden" name="longitude" x-model="lng">
                <input type="text" name="website" value="" tabindex="-1" autocomplete="off" aria-hidden="true"
class="hidden">
                <div>
                    <label class="block text-xs font-extrabold text-white/80 uppercase tracking-wider mb-1">Full
Name</label>
                    <input type="text" name="name" required placeholder="Prof. Juan Dela Cruz"
                        class="w-full px-3.5 py-2.5 bg-white/5 border border-white/15 rounded-2xl focus:ring-2
focus:ring-brand-orange focus:outline-none text-xs text-white font-bold placeholder-white/30 backdrop-blur-md">
                </div>
                <div>
                    <label class="block text-xs font-extrabold text-white/80 uppercase tracking-wider mb-1">PalSU
Email <p class="lowercase">(@psu.palawan.edu.ph)</p></label>
                    <div class="relative flex items-center">
                        <input type="email" name="email" x-model="signupEmail" required
placeholder="@psu.palawan.edu.ph"
                            class="w-full px-3.5 py-2.5 pr-28 bg-white/5 border border-white/15 rounded-2xl
focus:ring-2 focus:ring-brand-orange focus:outline-none text-xs text-white font-bold placeholder-white/30
backdrop-blur-md">
                        <button type="button" @click="getOtp()" :disabled="otpCountdown > 0 || otpSending"
                            class="absolute right-2 px-3 py-1.5 bg-brand-orange hover:bg-orange-600
disabled:opacity-50 disabled:cursor-not-allowed text-white font-bold rounded-xl text-[10px] uppercase
tracking-wider transition cursor-pointer inline-flex items-center gap-1.5">
                            <i class="fa-solid fa-spinner fa-spin text-[9px]" x-show="otpSending"></i>
                            <span x-show="!otpSending" x-text="otpVerified ? 'Verified ✓' : (otpCountdown > 0 ?
'Resend (' + otpCountdown + 's)' : 'Get OTP')"></span>
                            <span x-show="otpSending">Sending...</span>
                        </button>
                    </div>
                    <span class="text-[10px] text-emerald-400 font-bold mt-1 block" x-show="otpVerified">✓ OTP
Verified Successfully</span>
                </div>
                <div>
                    <label class="block text-xs font-extrabold text-white/80 uppercase tracking-wider
mb-1">Password</label>
                    <div class="relative">
                        <input :type="showSignupPass ? 'text' : 'password'" name="password" x-model="password"
required placeholder="••••••••"
                            class="w-full px-3.5 py-2.5 pr-11 bg-white/5 border border-white/15 rounded-2xl
focus:ring-2 focus:ring-brand-orange focus:outline-none text-xs text-white font-bold placeholder-white/30
backdrop-blur-md">
                        <button type="button" @click="showSignupPass = !showSignupPass" class="absolute right-3.5
top-1/2 -translate-y-1/2 text-orange-400 hover:text-orange-300 cursor-pointer text-sm hover:scale-110
active:scale-90 transition">
                            <i :class="showSignupPass ? 'fa-solid fa-eye-slash' : 'fa-solid fa-eye'"></i>
                        </button>
                    </div>
                    <!-- Password Strength & Requirements Checklist -->
                    <div class="mt-2 px-1 space-y-1.5" x-show="password">
                        <div class="flex items-center justify-between">
                            <span class="text-[10px] text-white/60 font-bold">Password Strength:</span>
                            <span class="text-[10px] font-black uppercase px-2 py-0.5 rounded-md tracking-wider"
                                  :class="{
                                      'bg-red-500/20 text-red-400 border border-red-500/30': passwordStrength ===
'Weak',
                                      'bg-amber-500/20 text-amber-400 border border-amber-500/30': passwordStrength
=== 'Medium',
```

```
                                    'bg-emerald-500/20 text-emerald-400 border border-emerald-500/30':
passwordStrength === 'Strong'
                                }"
                                x-text="passwordStrength"></span>
                        </div>
                        <ul class="text-[10px] space-y-1 bg-white/5 p-2 rounded-xl border border-white/10
font-medium">
                            <li :class="passwordCriteria.length ? 'text-emerald-400' : 'text-white/50'" class="flex
items-center gap-1.5">
                                <i :class="passwordCriteria.length ? 'fa-solid fa-check text-emerald-400' :
'fa-solid fa-circle text-[4px]'"></i>
                                At least 8 characters
                            </li>
                            <li :class="passwordCriteria.hasLetter ? 'text-emerald-400' : 'text-white/50'"
class="flex items-center gap-1.5">
                                <i :class="passwordCriteria.hasLetter ? 'fa-solid fa-check text-emerald-400' :
'fa-solid fa-circle text-[4px]'"></i>
                                Contains letters (A-Z)
                            </li>
                            <li :class="passwordCriteria.hasNumber ? 'text-emerald-400' : 'text-white/50'"
class="flex items-center gap-1.5">
                                <i :class="passwordCriteria.hasNumber ? 'fa-solid fa-check text-emerald-400' :
'fa-solid fa-circle text-[4px]'"></i>
                                Contains numbers (0-9)
                            </li>
                            <li :class="passwordCriteria.hasSpecial ? 'text-emerald-400' : 'text-white/50'"
class="flex items-center gap-1.5">
                                <i :class="passwordCriteria.hasSpecial ? 'fa-solid fa-check text-emerald-400' :
'fa-solid fa-circle text-[4px]'"></i>
                                Contains unique/special character (!@#$...)
                            </li>
                        </ul>
                    </div>
                </div>

                <div>
                    <label class="block text-xs font-extrabold text-white/80 uppercase tracking-wider mb-1">Confirm
Password</label>
                    <div class="relative">
                        <input :type="showSignupPass ? 'text' : 'password'" name="confirm_password"
x-model="confirmPassword" required placeholder="••••••••"
                            class="w-full px-3.5 py-2.5 pr-11 bg-white/5 border border-white/15 rounded-2xl
focus:ring-2 focus:ring-brand-orange focus:outline-none text-xs text-white font-bold placeholder-white/30
backdrop-blur-md">
                    </div>
                    <p class="text-[10px] text-red-400 font-bold mt-1" x-show="confirmPassword && password !==
confirmPassword">Passwords do not match</p>
                </div>

                <!-- Terms and Conditions Agreement -->
                <div class="space-y-1 pt-1">
                    <div class="flex items-start gap-2">
                        <input type="checkbox" id="terms_check" x-model="agreedTerms" :disabled="!termsOpened"
required
                            @click="if (!termsOpened) { $event.preventDefault(); showTermsModal = true;
termsOpened = true; }"
                            class="mt-0.5 w-4 h-4 rounded border-white/20 bg-white/5 text-brand-orange
focus:ring-brand-orange cursor-pointer disabled:opacity-50 disabled:cursor-not-allowed">
                        <label for="terms_check" @click.prevent="showTermsModal = true; termsOpened = true;"
class="text-[10px] text-white/80 font-medium select-none leading-snug cursor-pointer">
                            I have read and agree to the
                            <span class="text-brand-orange font-bold underline hover:text-orange-400 text-center">
                                Terms and Conditions
                            </span>
```

```
                            <span class="block text-[10px] text-brand-orange font-bold text-center"
x-show="!termsOpened">*(Click to open & read Terms and Conditions)*</span>
                        </label>
                    </div>
                </div>
                <button type="submit"
                        :disabled="!termsOpened || !agreedTerms || !otpVerified || password !== confirmPassword ||
!isPasswordStrong"
                        :class="(!termsOpened || !agreedTerms || !otpVerified || password !== confirmPassword ||
!isPasswordStrong) ? 'opacity-50 cursor-not-allowed bg-brand-orange/40' : 'bg-brand-orange hover:bg-orange-600
shadow-brand-orange/30 cursor-pointer'"
                        class="w-full py-2.5 text-white font-black rounded-2xl shadow-lg transition text-xs
uppercase tracking-wider flex items-center justify-center gap-2 relative z-20 font-poppins">
                    <span>Register & Continue</span>
                    <i class="fa-solid fa-arrow-right text-xs"></i>
                </button>
            </form>
        </div>
    </main>
    <!-- Footer / Back Link -->
    <footer class="pb-1 text-center relative z-10 shrink-0 animate-page-in">
        <a href="{% url 'accounts:landing' %}" class="text-xs font-bold text-white/60 hover:text-white transition
inline-flex items-center gap-1">
            <i class="fa-solid fa-arrow-left text-xs"></i> Back to Role Selection
        </a>
    </footer>
    <!-- Location Permission Modal -->
    <div x-show="showLocationModal" class="fixed inset-0 z-50 flex items-center justify-center bg-black/85
backdrop-blur-xl p-4" style="display: none;">
        <div class="bg-[#141416] border border-white/20 w-full max-w-sm rounded-[32px] p-6 text-center space-y-4
shadow-2xl">
            <div class="w-14 h-14 bg-brand-orange/20 border border-brand-orange/40 text-brand-orange rounded-2xl
flex items-center justify-center mx-auto text-xl shadow-lg shadow-brand-orange/20">
                <i class="fa-solid fa-location-crosshairs"></i>
            </div>
            <h3 class="text-base font-black text-white font-poppins">Allow Location Access</h3>
            <p class="text-xs text-white/70 leading-relaxed">
                Canteen Express needs your office location to enable seamless campus deliveries. Please allow
location access.
            </p>
            <div class="space-y-2 pt-2">
                <button type="button" @click="detectLocation()" class="w-full py-3 bg-brand-orange
hover:bg-orange-600 text-white font-black rounded-xl text-xs uppercase tracking-wider cursor-pointer shadow-lg
shadow-brand-orange/30 font-poppins flex items-center justify-center gap-2">
                    <i class="fa-solid fa-crosshairs"></i> Allow & Detect Location
                </button>
            </div>
        </div>
    </div>
    <div x-show="showTermsModal" class="fixed inset-0 z-50 flex items-center justify-center bg-black/80
backdrop-blur-xl p-4" style="display: none;">
        <div @click.away="showTermsModal = false" class="bg-[#141416] border border-white/20 w-full max-w-lg
rounded-[32px] p-6 shadow-2xl space-y-3 max-h-[80vh] flex flex-col text-white">
            <div class="flex justify-between items-center border-b border-white/10 pb-2">
                <h3 class="text-sm font-black text-white font-poppins">Terms & Conditions & No-Cancellation
Policy</h3>
                <button @click="showTermsModal = false" class="text-white/50 hover:text-white text-base font-bold
cursor-pointer">
                    <i class="fa-solid fa-xmark"></i>
                </button>
            </div>
            <div class="overflow-y-auto text-xs text-white/80 space-y-2.5 leading-relaxed pr-2">
                <p><strong>1. Acceptance of Terms:</strong> By registering and using Canteen Express, you agree to
comply with and be bound by these Terms and Conditions.</p>
```

```
                <p><strong>2. Institutional Account Use:</strong> Faculty and staff accounts require a valid
Palawan State University institutional email ending in <strong>@psu.palawan.edu.ph</strong>. You are responsible
for maintaining the confidentiality of your account credentials.</p>
                <p><strong>3. Ordering & Campus Deliveries:</strong> Delivery requests are restricted strictly
within the PSU Tiniguiban Heights campus geofenced area. A ₱15 delivery fee per ₱300 order block applies as stated
at checkout.</p>
                <p><strong>4. Cancellation Policy & Smart Timeout:</strong> Once an order is confirmed and under
kitchen preparation, cancellation is restricted. However, if no rider accepts your order within 2 minutes (120
seconds) while searching, a smart timeout modal will prompt you with options to <strong>Convert to Pick-up</strong>
or <strong>Cancel Order</strong> (which automatically restores menu stock and refunds redeemed loyalty points).</p>
                <p><strong>5. Loyalty Rewards:</strong> Earn 1 loyalty point per ₱100 spent on campus delivery
orders. Points are credited upon order completion and can be redeemed at checkout (1 Point = ₱1.00 discount).</p>
                <p><strong>6. Privacy & User Conduct:</strong> Personal data and location info are used solely for
order processing and campus delivery tracking. Any misuse, fraudulent ordering, or harassment will result in
immediate account restriction or ban.</p>
            </div>
            <div class="pt-2 border-t border-white/10 flex justify-end">
                <button type="button" @click="showTermsModal = false; termsOpened = true; agreedTerms = true;"
class="px-4 py-2 bg-brand-orange hover:bg-orange-600 text-white font-black rounded-xl text-xs uppercase
tracking-wider cursor-pointer shadow-lg shadow-brand-orange/30 font-poppins">
                    I Understand & Agree
                </button>
            </div>
        </div>
    </div>
    <!-- Forgot Password Modal -->
    <div x-show="showForgotModal" class="fixed inset-0 z-50 flex items-center justify-center bg-black/85
backdrop-blur-xl p-4" style="display: none;">
        <div @click.away="showForgotModal = false" class="bg-[#141416] border border-white/25 w-full max-w-sm
rounded-[32px] p-6 text-center space-y-4 shadow-2xl text-white">
            <div class="w-14 h-14 bg-brand-orange/20 border border-brand-orange/40 text-brand-orange rounded-2xl
flex items-center justify-center mx-auto text-xl shadow-lg shadow-brand-orange/20">
                <i class="fa-solid fa-key"></i>
            </div>
            <h3 class="text-base font-black text-white font-poppins">Reset Faculty/Staff Password</h3>
            <p class="text-xs text-white/70 leading-relaxed">
                Enter your registered corporate email (@psu.palawan.edu.ph) and new password, then verify via OTP.
            </p>
            <div class="space-y-3 text-left">
                <div>
                    <label class="block text-[11px] font-extrabold text-white/80 uppercase tracking-wider
mb-1">Corporate Email</label>
                    <input type="email" x-model="forgotEmail" placeholder="faculty/staff@psu.palawan.edu.ph"
                           class="w-full px-3.5 py-2.5 bg-white/5 border border-white/15 rounded-xl focus:ring-2
focus:ring-brand-orange focus:outline-none text-xs text-white font-bold placeholder-white/30">
                </div>
                <div>
                    <label class="block text-[11px] font-extrabold text-white/80 uppercase tracking-wider mb-1">New
Strong Password</label>
                    <div class="relative">
                        <input :type="showForgotNewPass ? 'text' : 'password'" x-model="forgotNewPassword"
placeholder="••••••••"
                               class="w-full px-3.5 py-2.5 pr-11 bg-white/5 border border-white/15 rounded-xl
focus:ring-2 focus:ring-brand-orange focus:outline-none text-xs text-white font-bold placeholder-white/30">
                        <button type="button" @click="showForgotNewPass = !showForgotNewPass" class="absolute
right-3.5 top-1/2 -translate-y-1/2 text-orange-400 hover:text-orange-300 cursor-pointer text-sm hover:scale-110
active:scale-90 transition">
                            <i :class="showForgotNewPass ? 'fa-solid fa-eye-slash' : 'fa-solid fa-eye'"></i>
                        </button>
                    </div>
                    <div class="flex items-center justify-between mt-1 px-0.5" x-show="forgotNewPassword">
                        <span class="text-[10px] text-white/60 font-bold">Strength:</span>
                        <span class="text-[10px] font-black uppercase px-2 py-0.5 rounded-md tracking-wider"
                              :class="{
```

```
                                'bg-red-500/20 text-red-400 border border-red-500/30': forgotPasswordStrength ===
'Weak',
                                'bg-amber-500/20 text-amber-400 border border-amber-500/30':
forgotPasswordStrength === 'Medium',
                                'bg-emerald-500/20 text-emerald-400 border border-emerald-500/30':
forgotPasswordStrength === 'Strong'
                            }"
                            x-text="forgotPasswordStrength"></span>
                    </div>
                </div>

                <div>
                    <label class="block text-[11px] font-extrabold text-white/80 uppercase tracking-wider
mb-1">Confirm New Password</label>
                    <div class="relative">
                        <input :type="showForgotConfirmPass ? 'text' : 'password'" x-model="forgotConfirmPassword"
placeholder="••••••••"
                             class="w-full px-3.5 py-2.5 pr-11 bg-white/5 border border-white/15 rounded-xl
focus:ring-2 focus:ring-brand-orange focus:outline-none text-xs text-white font-bold placeholder-white/30">
                        <button type="button" @click="showForgotConfirmPass = !showForgotConfirmPass"
class="absolute right-3.5 top-1/2 -translate-y-1/2 text-orange-400 hover:text-orange-300 cursor-pointer text-sm
hover:scale-110 active:scale-90 transition">
                            <i :class="showForgotConfirmPass ? 'fa-solid fa-eye-slash' : 'fa-solid fa-eye'"></i>
                        </button>
                    </div>
                    <p class="text-[10px] text-red-400 font-bold mt-1" x-show="forgotConfirmPassword &&
forgotNewPassword !== forgotConfirmPassword">Passwords do not match</p>
                </div>
                <div x-show="forgotOtpSent">
                    <label class="block text-[11px] font-extrabold text-white/80 uppercase tracking-wider
mb-1">Enter 6-Digit OTP</label>
                    <input type="text" x-model="forgotOtp" placeholder="######" maxlength="6"
                           class="w-full px-3.5 py-2.5 bg-white/5 border border-white/15 rounded-xl focus:ring-2
focus:ring-brand-orange focus:outline-none text-xs text-white font-bold text-center tracking-widest text-lg
placeholder-white/30">
                </div>
            </div>
            <div class="space-y-2 pt-2">
                <button type="button" @click="getForgotOtp()" x-show="!forgotOtpSent" :disabled="forgotOtpCountdown
> 0 || forgotSendingOtp"
                        class="w-full py-3 bg-brand-orange hover:bg-orange-600 disabled:opacity-50
disabled:cursor-not-allowed text-white font-black rounded-xl text-xs uppercase tracking-wider cursor-pointer
shadow-lg shadow-brand-orange/30 font-poppins inline-flex items-center justify-center gap-2">
                    <i class="fa-solid fa-spinner fa-spin" x-show="forgotSendingOtp"></i>
                    <span x-show="!forgotSendingOtp" x-text="forgotOtpCountdown > 0 ? 'Resend OTP in ' +
forgotOtpCountdown + 's' : 'Get OTP Verification'"></span>
                    <span x-show="forgotSendingOtp">Sending OTP...</span>
                </button>
                <button type="button" @click="submitPasswordReset()" x-show="forgotOtpSent"
:disabled="forgotSubmitting"
                        class="w-full py-3 bg-brand-orange hover:bg-orange-600 disabled:opacity-50
disabled:cursor-not-allowed text-white font-black rounded-xl text-xs uppercase tracking-wider cursor-pointer
shadow-lg shadow-brand-orange/30 font-poppins inline-flex items-center justify-center gap-2">
                    <i class="fa-solid fa-spinner fa-spin" x-show="forgotSubmitting"></i>
                    <span x-show="!forgotSubmitting">Submit Changes & Reset</span>
                    <span x-show="forgotSubmitting">Resetting Password...</span>
                </button>
                <button type="button" @click="showForgotModal = false"
                        class="w-full py-2.5 bg-white/10 hover:bg-white/20 text-white font-bold rounded-xl text-xs
uppercase tracking-wider cursor-pointer border border-white/10 font-poppins">
                    Cancel
                </button>
            </div>
        </div>
    </div>
```

```
    <!-- OTP Verification Modal Popup -->
    <div x-show="showOtpModal" class="fixed inset-0 z-50 flex items-center justify-center bg-black/85
backdrop-blur-xl p-4" style="display: none;">
        <div class="bg-[#141416] border border-white/20 w-full max-w-sm rounded-[32px] p-6 text-center space-y-4
shadow-2xl text-white">
            <div class="w-14 h-14 bg-brand-orange/20 border border-brand-orange/40 text-brand-orange rounded-2xl
flex items-center justify-center mx-auto text-xl shadow-lg shadow-brand-orange/20">
                <i class="fa-solid fa-envelope"></i>
            </div>
            <h3 class="text-base font-black text-white font-poppins">Enter OTP Verification Code</h3>
            <p class="text-xs text-white/70 leading-relaxed">
                We've sent a 6-digit verification code to <span class="text-brand-orange font-bold"
x-text="signupEmail"></span>.
            </p>
            <div>
                <input type="text" x-model="enteredOtp" maxlength="6" placeholder="######"
                    class="w-full px-4 py-3 text-center text-lg tracking-widest font-black bg-white/5 border
border-white/15 rounded-2xl focus:ring-2 focus:ring-brand-orange focus:outline-none text-white
placeholder-white/30">
            </div>
            <div class="space-y-2 pt-2">
                <button type="button" @click="verifyOtp()" :disabled="otpVerifying" class="w-full py-3
bg-brand-orange hover:bg-orange-600 disabled:opacity-50 disabled:cursor-not-allowed text-white font-black
rounded-xl text-xs uppercase tracking-wider cursor-pointer shadow-lg shadow-brand-orange/30 font-poppins
inline-flex items-center justify-center gap-2">
                    <i class="fa-solid fa-spinner fa-spin" x-show="otpVerifying"></i>
                    <span x-show="!otpVerifying">Verify OTP Code</span>
                    <span x-show="otpVerifying">Verifying...</span>
                </button>
                <button type="button" @click="getOtp()" :disabled="otpCountdown > 0 || otpSending" class="w-full
py-2 bg-white/10 hover:bg-white/25 disabled:opacity-50 disabled:cursor-not-allowed text-white font-bold rounded-xl
text-xs uppercase tracking-wider cursor-pointer border border-white/10 font-poppins">
                    <span x-text="otpCountdown > 0 ? 'Resend OTP in ' + otpCountdown + 's' : 'Resend OTP'"></span>
                </button>
                <button type="button" @click="showOtpModal = false" class="w-full py-2 bg-white/5 hover:bg-white/10
text-white/70 hover:text-white font-bold rounded-xl text-xs uppercase tracking-wider cursor-pointer border
border-white/10 font-poppins">
                    Cancel
                </button>
            </div>
        </div>
    </div>
    <script src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js" defer></script>
    <div id="page-fade-overlay"></div>
    <div id="logo-loading-overlay" class="fixed inset-0 z-[100] bg-[#0A0A0C] flex items-center justify-center
hidden opacity-0 transition-opacity duration-300">
        <div class="relative flex items-center justify-center">
            <div class="absolute w-44 h-44 bg-brand-orange/25 rounded-full blur-3xl animate-ping"></div>
            <img src="{% static 'customer_portal/images/canteen-express-logo.png' %}" alt="Canteen Express"
class="w-36 h-36 sm:w-44 sm:h-44 object-contain relative z-10 animate-logo-glow">
        </div>
    </div>
    <div x-show="showToast"
         x-transition:enter="transition ease-out duration-300"
         x-transition:enter-start="opacity-0 -translate-y-4"
         x-transition:enter-end="opacity-100 translate-y-0"
         x-transition:leave="transition ease-in duration-200"
         x-transition:leave-start="opacity-100 translate-y-0"
         x-transition:leave-end="opacity-0 -translate-y-4"
         class="fixed top-6 left-1/2 transform -translate-x-1/2 z-50 w-11/12 max-w-sm px-4 py-3 rounded-2xl
shadow-2xl border flex items-center gap-3 backdrop-blur-md"
         :class="{
            'bg-emerald-950/90 border-emerald-500/30 text-emerald-200': toastType === 'success',
            'bg-rose-950/90 border-rose-500/30 text-rose-200': toastType === 'error',
            'bg-blue-950/90 border-blue-500/30 text-blue-200': toastType === 'info'
```

```
         }"
         style="display: none;">
        <div class="w-8 h-8 rounded-full flex items-center justify-center shrink-0"
             :class="{
                'bg-emerald-500/20 text-emerald-400': toastType === 'success',
                'bg-rose-500/20 text-rose-400': toastType === 'error',
                'bg-blue-500/20 text-blue-400': toastType === 'info'
             }">
            <i class="fas" :class="{
                'fa-check': toastType === 'success',
                'fa-exclamation-triangle': toastType === 'error',
                'fa-info-circle': toastType === 'info'
            }"></i>
        </div>
        <div class="text-sm font-semibold flex-1" x-text="toastMessage"></div>
        <button @click="showToast = false" class="text-white/60 hover:text-white p-1 cursor-pointer">
            <i class="fas fa-times text-xs"></i>
        </button>
    </div>
    <script src="{% static 'js/ui-loading.js' %}"></script>
</body>
</html>
```

Canteen Staff, Kitchen Display & Counter POS (Staff / Admin Portal)

Used by canteen staff to scan queue slip barcodes, confirm cash payments, update order statuses, manage menu items and stock levels, and review sales analytics reports.

backend/canteen_menu/views.py

```
import json

import time

from django.http import JsonResponse, StreamingHttpResponse

from django.shortcuts import render, redirect, get_object_or_404

from django.urls import reverse

from django.contrib import messages

from django.core.cache import cache

from django.views.decorators.csrf import csrf_exempt

from accounts.decorators import role_required

from activity_log.models import ActivityLog

from activity_log.services import log_activity

from .models import MenuItem, Category

from customer_portal.models import OrderItem

try:

    from .forms import MenuItemForm, CategoryForm

except ImportError:

    from customer_portal.forms import MenuItemForm

    CategoryForm = None

@role_required(allowed_roles=['STAFF', 'ADMIN'])
```

```
def staff_menu_list(request):
    items = MenuItem.objects.all().select_related('category').order_by('category__name', 'name')
    categories = Category.objects.all().order_by('-name')
    return render(request, 'canteen_menu/menu_list.html', {'items': items, 'categories': categories})
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def staff_menu_toggle_availability(request, pk):
    item = get_object_or_404(MenuItem, pk=pk)
    item.is_available = not item.is_available
    item.save()
    cache.delete('formatted_menu_active_kiosk')
    now_available = item.is_available
    messages.success(request, f"Updated availability for {item.name}")
    log_activity(
        request,
        action=f"{'Made available' if now_available else 'Marked unavailable'}: {item.name}",
        category=ActivityLog.Category.MENU,
        level=ActivityLog.Level.INFO,
        target=item.name,
        metadata={'item_id': item.pk, 'is_available': now_available, 'stock': item.stock},
    )
    return redirect('canteen_menu:staff_menu_list')
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def category_create(request):
    if request.method == 'POST':
        form = CategoryForm(request.POST)
        if form.is_valid():
            category = form.save()
            messages.success(request, "New category added successfully!")
            log_activity(
                request,
                action=f"Created category: {category.name}",
                category=ActivityLog.Category.MENU,
                level=ActivityLog.Level.SUCCESS,
                target=category.name,
                metadata={'category_id': category.pk},
            )
```

```
    return redirect(reverse('canteen_menu:staff_dashboard') + '?tab=menu')

@role_required(allowed_roles=['STAFF', 'ADMIN'])
def category_update(request, pk):
    category = get_object_or_404(Category, pk=pk)
    if request.method == 'POST':
        form = CategoryForm(request.POST, instance=category)
        if form.is_valid():
            previous_name = category.name
            form.save()
            messages.success(request, "Category updated successfully!")
            log_activity(
                request,
                action=f"Updated category: {previous_name}",
                category=ActivityLog.Category.MENU,
                level=ActivityLog.Level.INFO,
                target=category.name,
                metadata={
                    'category_id': category.pk,
                    'previous_name': previous_name,
                    'new_name': category.name,
                },
            )
    return redirect(reverse('canteen_menu:staff_dashboard') + '?tab=menu')
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def category_delete(request, pk):
    category = get_object_or_404(Category, pk=pk)
    if request.method == 'POST':
        name = category.name
        affected_items = category.items.count()
        category_id = category.pk
        category.delete()
        messages.success(request, "Category deleted successfully!")
        log_activity(
            request,
            action=f"Deleted category: {name} ({affected_items} menu item(s) removed)",
            category=ActivityLog.Category.MENU,
```

```
            level=ActivityLog.Level.DANGER,
            target=name,
            metadata={'category_id': category_id, 'items_removed': affected_items},
        )
    return redirect(reverse('canteen_menu:staff_dashboard') + '?tab=menu')
import os
import shutil
import urllib.parse
from django.conf import settings
from django.utils.text import slugify
def _auto_sync_menu_image(item):
    if item.image:
        try:
            if item.image.storage.exists(item.image.name):
                return
        except Exception:
            pass

    if item.image_url:
        return
    name_lower = item.name.lower()
    cat_name = str(item.category).lower() if item.category else ''
    if 'coke' in name_lower or 'sprite' in name_lower or 'royal' in name_lower or 'pepsi' in name_lower or 'drink'
in name_lower or 'beverage' in name_lower or 'coffee' in name_lower or 'energen' in name_lower:
       item.image_url = "https://images.unsplash.com/photo-1554866585-cd94860890b7?auto=format&fit=crop&w=600&q=80"
    elif 'rice' in name_lower or 'silog' in name_lower or 'adobo' in name_lower or 'sinigang' in name_lower or
'menudo' in name_lower or 'curry' in name_lower or 'tinola' in name_lower or 'chicken' in name_lower or 'pork' in
name_lower or 'beef' in name_lower or 'fish' in name_lower or 'ulam' in cat_name:
        item.image_url =
"https://images.unsplash.com/photo-1555396273-367ea4eb4db5?auto=format&fit=crop&w=600&q=80"
    elif 'cookie' in name_lower or 'biscuit' in name_lower or 'crinkle' in name_lower or 'brownie' in name_lower:
        item.image_url =
"https://images.unsplash.com/photo-1558961363-fa8fdf82db35?auto=format&fit=crop&w=600&q=80"
    elif 'turon' in name_lower or 'bread' in name_lower or 'pastry' in name_lower or 'meryenda' in name_lower or
'snack' in name_lower or 'puto' in name_lower or 'suman' in name_lower:
        item.image_url =
"https://images.unsplash.com/photo-1509440159596-0249088772ff?auto=format&fit=crop&w=600&q=80"
    else:
        item.image_url =
"https://images.unsplash.com/photo-1546069901-ba9599a7e63c?auto=format&fit=crop&w=600&q=80"
```

```
    item.image = None
def upload_to_supabase_storage(image_file):
    if not image_file:
        return None
    try:
        project_ref = getattr(settings, 'SUPABASE_PROJECT_REF', '')
        bucket_name = "menu-images"
        anon_key = getattr(settings, 'SUPABASE_ANON_KEY', '')
        if not project_ref or not anon_key:
            return None
        filename = image_file.name.replace(' ', '_')
        url = f"https://{project_ref}.supabase.co/storage/v1/object/{bucket_name}/{filename}"
        file_data = image_file.read()
        content_type = getattr(image_file, 'content_type', 'image/jpeg')
        req = urllib.request.Request(url, data=file_data, method='POST')
        req.add_header('Authorization', f'Bearer {anon_key}')
        req.add_header('Content-Type', content_type)
        req.add_header('x-upsert', 'true')
        with urllib.request.urlopen(req, timeout=0.8) as response:
            if response.status in [200, 201]:
                return
f"https://{project_ref}.supabase.co/storage/v1/object/public/{bucket_name}/{urllib.parse.quote(filename)}"
    except Exception as e:
        print(f"Supabase upload error: {e}")
    return None
def _diff_menu_fields(before, after):
    return {
        field: {'from': before[field], 'to': after[field]}
        for field in after
        if before.get(field) != after[field]
    }
def _generate_auto_desc(name):
    name_lower = name.lower()
    if 'turon' in name_lower or 'banana' in name_lower or 'meryenda' in name_lower:
        return f"Crispy and sweet golden {name}, freshly fried and glazed for the ultimate campus snack."
    elif 'pork' in name_lower or 'chicken' in name_lower or 'beef' in name_lower or 'rice' in name_lower:
```

```
        return f"Savory and hearty {name} served piping hot, rich in protein, flavor, and cooked with authentic
canteen recipe."
    elif 'coke' in name_lower or 'sprite' in name_lower or 'beverage' in name_lower or 'juice' in name_lower:
        return f"Ice-cold refreshing {name} to complement your meal."
    return f"Freshly prepared {name}, cooked daily with quality ingredients."
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def staff_menu_create(request):
    if request.method == 'POST':
        form = MenuItemForm(request.POST, request.FILES)
        if form.is_valid():
            item = form.save(commit=False)
            if not item.description:
                item.description = _generate_auto_desc(item.name)
            if request.FILES.get('image'):
                pub_url = upload_to_supabase_storage(request.FILES.get('image'))
                if pub_url:
                    item.image_url = pub_url
                    item.image = None
                else:
                    item.image_url = ''
            elif item.image_url:
                item.image = None

            if not item.image and not item.image_url:
                _auto_sync_menu_image(item)
            item.save()
            cache.delete('formatted_menu_active_kiosk')
            messages.success(request, f"Menu item '{item.name}' added successfully with auto-synced image!")
            log_activity(
                request,
                action=f"Added menu item: {item.name} (P{float(item.price):.2f}, stock {item.stock})",
                category=ActivityLog.Category.MENU,
                level=ActivityLog.Level.SUCCESS,
                target=item.name,
                metadata={
                    'item_id': item.pk,
                    'price': float(item.price),
```

```
                    'stock': item.stock,
                    'category': item.category.name if item.category else None,
                },
            )
            return redirect('canteen_menu:staff_menu_list')
        else:
            messages.error(request, "Please correct the errors in the form.")
    else:
        form = MenuItemForm()
    return render(request, 'canteen_menu/menu_form.html', {'form': form, 'action_title': 'Add New Menu Item'})
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def staff_menu_update(request, pk):
    item = get_object_or_404(MenuItem, pk=pk)
    if request.method == 'POST':
        form = MenuItemForm(request.POST, request.FILES, instance=item)
        if form.is_valid():
            # Snapshot the fields whose change is worth explaining in the log.
            before = {
                'name': item.name,
                'price': float(item.price),
                'stock': item.stock,
                'is_available': item.is_available,
            }
            menu_item = form.save(commit=False)
            if request.FILES.get('image'):
                pub_url = upload_to_supabase_storage(request.FILES.get('image'))
                if pub_url:
                    menu_item.image_url = pub_url
                    menu_item.image = None
                else:
                    menu_item.image_url = ''
            elif menu_item.image_url:
                menu_item.image = None
            if not menu_item.image and not menu_item.image_url:
                _auto_sync_menu_image(menu_item)
            menu_item.save()
```

```
            cache.delete('formatted_menu_active_kiosk')
            messages.success(request, "Menu item updated successfully!")
            log_activity(
                request,
                action=f"Edited menu item: {menu_item.name}",
                category=ActivityLog.Category.MENU,
                level=ActivityLog.Level.INFO,
                target=menu_item.name,
                metadata={
                    'item_id': menu_item.pk,
                    'changes': _diff_menu_fields(before, {
                        'name': menu_item.name,
                        'price': float(menu_item.price),
                        'stock': menu_item.stock,
                        'is_available': menu_item.is_available,
                    }),
                },
            )
            return redirect('canteen_menu:staff_menu_list')
        else:
            messages.error(request, "Error updating item. Please check the form.")
    else:
        form = MenuItemForm(instance=item)

    return render(request, 'canteen_menu/menu_form.html', {'form': form, 'action_title': 'Edit Menu Item', 'item':
item})
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def staff_menu_delete(request, pk):
    item = get_object_or_404(MenuItem, pk=pk)
    if request.method == 'POST':
        name = item.name
        price = float(item.price)
        item_id = item.pk
        item.delete()
        cache.delete('formatted_menu_active_kiosk')
        messages.success(request, "Menu item deleted successfully!")
        log_activity(
```

```
            request,
            action=f"Deleted menu item: {name} (P{price:.2f})",
            category=ActivityLog.Category.MENU,
            level=ActivityLog.Level.DANGER,
            target=name,
            metadata={'item_id': item_id, 'price': price},
        )
    return redirect(reverse('canteen_menu:staff_dashboard') + '?tab=menu')
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def counter_board(request):
    from customer_portal.models import Order
    from django.utils import timezone
    incoming_orders = Order.objects.filter(status='pending').order_by('created_at')
    preparing_orders = Order.objects.filter(status='preparing').order_by('created_at')
    ready_orders = Order.objects.filter(status='ready').order_by('created_at')
    today = timezone.now().date()
    done_today_count = Order.objects.filter(status='completed', created_at__date=today).count()
    context = {
        'incoming_orders': incoming_orders,
        'preparing_orders': preparing_orders,
        'ready_orders': ready_orders,
        'done_today_count': done_today_count,
    }
    return render(request, 'canteen_menu/counter_board.html', context)
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def counter_live_stream(request):
    import hashlib
    from django.utils import timezone
    from customer_portal.models import Order
    def state():
        pending_ids = list(Order.objects.filter(status='pending').order_by('created_at').values_list('id',
flat=True))
        preparing_ids = list(Order.objects.filter(status='preparing').order_by('created_at').values_list('id',
flat=True))
        ready_ids = list(Order.objects.filter(status='ready').order_by('created_at').values_list('id', flat=True))
        return {
            'pending_count': len(pending_ids),
```

```
            'preparing_count': len(preparing_ids),
            'ready_count': len(ready_ids),
            'pending_ids': pending_ids,
            'preparing_ids': preparing_ids,
            'ready_ids': ready_ids,
        }
    def event_stream():
        last_hash = None
        while True:
            try:
                payload = {'success': True, 'orders': state()}
                body = json.dumps(payload)
                digest = hashlib.md5(body.encode('utf-8')).hexdigest()
                if digest != last_hash:
                    last_hash = digest
                    yield f"data: {body}\n\n"
            except Exception:
                pass
            yield ": ping\n\n"
            time.sleep(2)
    return StreamingHttpResponse(
        event_stream(),
        content_type='text/event-stream',
        headers={
            'Cache-Control': 'no-cache',
            'X-Accel-Buffering': 'no',
        },
    )
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def staff_dashboard(request, token=None):
    import uuid
    session_token = request.session.get('staff_secure_token')
    if not session_token:
        session_token = uuid.uuid4().hex[:12]
        request.session['staff_secure_token'] = session_token
```

```
    if not token or token != session_token:
        query_string = request.META.get('QUERY_STRING', '')
        redirect_url = reverse('canteen_menu:staff_dashboard_hashed', kwargs={'token': session_token})
        if query_string:
            redirect_url += f'?{query_string}'
        return redirect(redirect_url)
    from django.utils import timezone
    from customer_portal.models import Order
    from deliveries.models import DeliveryRequest
    from django.db.models import Sum, Count, Q
    from django.db.models.functions import TruncDate, TruncWeek, TruncMonth
    from django.contrib.auth import get_user_model
    User = get_user_model()
    today = timezone.now().date()
    day_start = timezone.now().replace(hour=0, minute=0, second=0, microsecond=0)
    day_end = day_start + timezone.timedelta(days=1)
    week_start = day_start - timezone.timedelta(days=7)
    month_start = day_start.replace(day=1)
    today_orders = Order.objects.filter(
        created_at__gte=day_start, created_at__lt=day_end
    ).exclude(status='unpaid')
    total_orders_today = today_orders.count()
    pending_count = Order.objects.filter(status='pending').count()
    preparing_count = Order.objects.filter(status='preparing').count()
    ready_count = Order.objects.filter(status='ready').count()
    recent_orders = Order.objects.exclude(status__in=['completed', 'unpaid',
'cancelled']).select_related('customer', 'delivery_info', 'delivery_info__rider').order_by('-created_at')[:5]
    menu_items = MenuItem.objects.all().select_related('category').order_by('category__name', 'name')
    categories = Category.objects.all().order_by('-name')
    users_list = User.objects.exclude(role__in=['DELIVERY', 'RIDER']).order_by('-date_joined')[:30] if
hasattr(User, 'date_joined') else User.objects.exclude(role__in=['DELIVERY', 'RIDER'])[:30]
    delivery_staff_list = User.objects.filter(role__in=['RIDER', 'DELIVERY'])
    try:
        delivery_requests = DeliveryRequest.objects.all().order_by('-requested_at')[:20]
    except Exception:
        delivery_requests = []
    valid_orders = Order.objects.filter(status__in=['ready', 'completed'])
```

```
    overall_orders = valid_orders

    kiosk_orders = valid_orders.filter(customer__isnull=True)

    faculty_orders = valid_orders.filter(customer__role='FACULTY')

    from deliveries.models import DeliveryRequest

    delivery_order_ids = DeliveryRequest.objects.values('order_id')

    delivery_orders = valid_orders.filter(id__in=delivery_order_ids)

    def compute_stats(qs):

        s_today = qs.filter(created_at__gte=day_start,
created_at__lt=day_end).aggregate(Sum('total_amount'))['total_amount__sum'] or 0

        s_week = qs.filter(created_at__gte=week_start).aggregate(Sum('total_amount'))['total_amount__sum'] or 0

        s_month = qs.filter(created_at__gte=month_start).aggregate(Sum('total_amount'))['total_amount__sum'] or 0

        c_today = qs.filter(created_at__gte=day_start, created_at__lt=day_end).count()

        c_week = qs.filter(created_at__gte=week_start).count()

        c_month = qs.filter(created_at__gte=month_start).count()

        return {'today': s_today, 'week': s_week, 'month': s_month, 'count_today': c_today, 'count_week': c_week,
'count_month': c_month}

    def compute_delivery_stats(qs):

        base = compute_stats(qs)

        d_fees_today = qs.filter(created_at__gte=day_start,
created_at__lt=day_end).aggregate(Sum('delivery_fee'))['delivery_fee__sum'] or 0

        d_fees_week = qs.filter(created_at__gte=week_start).aggregate(Sum('delivery_fee'))['delivery_fee__sum'] or
0

        d_fees_month = qs.filter(created_at__gte=month_start).aggregate(Sum('delivery_fee'))['delivery_fee__sum']
or 0

        base['delivery_fees'] = {'today': d_fees_today, 'week': d_fees_week, 'month': d_fees_month}

        return base

    overall_stats = compute_stats(overall_orders)

    kiosk_stats = compute_stats(kiosk_orders)

    faculty_stats = compute_stats(faculty_orders)

    delivery_stats = compute_delivery_stats(delivery_orders)

    def get_rankings(date_filter=None):

        f = Q(assigned_deliveries__status='DELIVERED')

        if date_filter == 'daily':

            f &= Q(assigned_deliveries__order__created_at__gte=day_start,
assigned_deliveries__order__created_at__lt=day_end)

        elif date_filter == 'weekly':

            f &= Q(assigned_deliveries__order__created_at__gte=week_start)

        elif date_filter == 'monthly':

            f &= Q(assigned_deliveries__order__created_at__gte=month_start)
```

```
        return list(
            User.objects.filter(role__in=['RIDER', 'DELIVERY'])
            .annotate(
                total_delivered=Count('assigned_deliveries', filter=f),
                total_earnings=Sum('assigned_deliveries__order__delivery_fee', filter=f)
            )
            .order_by('-total_delivered')
        )
    rider_rankings = get_rankings(None)
    from customer_portal.models import OrderItem
    from django.db.models import F
    top_selling_items = list(
        OrderItem.objects.filter(order__status__in=['ready', 'completed'])
        .values('item_name')
        .annotate(
            total_qty=Sum('quantity'),
            total_revenue=Sum(F('quantity') * F('price'))
        )
        .order_by('-total_revenue')[:10]
    )
    daily_sales = list(
        valid_orders
        .annotate(period=TruncDate('created_at'))
        .values('period')
        .annotate(total=Sum('total_amount'), count=Count('id'))
        .order_by('-period')[:7]
    )
    weekly_sales = list(
        valid_orders
        .annotate(period=TruncWeek('created_at'))
        .values('period')
        .annotate(total=Sum('total_amount'), count=Count('id'))
        .order_by('-period')[:4]
    )
    monthly_sales = list(
        valid_orders
```

```
        .annotate(period=TruncMonth('created_at'))
        .values('period')
        .annotate(total=Sum('total_amount'), count=Count('id'))
        .order_by('-period')[:6]
    )
    daily_chart_data = {
        'labels': [row['period'].strftime('%b %d') if row['period'] else '' for row in reversed(daily_sales)],
        'data': [float(row['total']) if row['total'] else 0.0 for row in reversed(daily_sales)],
    }
    weekly_chart_data = {
        'labels': [f"Week of {row['period'].strftime('%b %d')}" if row['period'] else '' for row in
reversed(weekly_sales)],
        'data': [float(row['total']) if row['total'] else 0.0 for row in reversed(weekly_sales)],
    }
    monthly_chart_data = {
        'labels': [row['period'].strftime('%b %Y') if row['period'] else '' for row in reversed(monthly_sales)],
        'data': [float(row['total']) if row['total'] else 0.0 for row in reversed(monthly_sales)],
    }
    sales_today = overall_stats['today']
    sales_week = overall_stats['week']
    sales_month = overall_stats['month']
    pie_labels = []
    pie_data = []
    for cat in Category.objects.all():
        cat_items = cat.items.values_list('name', flat=True)
        cat_rev = OrderItem.objects.filter(order__status__in=['ready', 'completed'],
item_name__in=cat_items).aggregate(total=Sum('price'))['total'] or 0
        if cat_rev > 0:
            pie_labels.append(cat.name)
            pie_data.append(float(cat_rev))
    if not pie_labels:
        pie_labels = ['General Meals']
        pie_data = [float(sales_today or 100)]

    pie_chart_data = {'labels': pie_labels, 'data': pie_data}
    line_chart_data = {
        'labels': daily_chart_data['labels'],
```

```
        'data': [row['count'] for row in reversed(daily_sales)] if daily_sales else [1, 2, 3]
    }
    if request.method == 'POST' and 'add_menu_item' in request.POST:
        name = request.POST.get('name')
        price = request.POST.get('price')
        stock = request.POST.get('stock', 50)
        category_id = request.POST.get('category')
        category = get_object_or_404(Category, id=category_id) if category_id else Category.objects.first()
        image_url = request.POST.get('image_url', '')
        image = request.FILES.get('image')
        if image:
            pub_url = upload_to_supabase_storage(image)
            if pub_url:
                image_url = pub_url
                image = None
        description = request.POST.get('description', '')
        if not description:
            description = _generate_auto_desc(name)
        item = MenuItem(
            name=name,
            category=category,
            price=price,
            stock=stock,
            image_url=image_url,
            image=image,
            description=description,
            is_available=True
        )
        _auto_sync_menu_image(item)
        item.save()
        messages.success(request, "New menu item added successfully with auto-synced image!")
        log_activity(
            request,
            action=f"Added menu item: {item.name} (P{float(item.price):.2f}, stock {item.stock})",
            category=ActivityLog.Category.MENU,
            level=ActivityLog.Level.SUCCESS,
```

```
            target=item.name,
            metadata={
                'item_id': item.pk,
                'price': float(item.price),
                'stock': item.stock,
                'category': category.name if category else None,
                'source': 'dashboard',
            },
        )
        return redirect(reverse('canteen_menu:staff_dashboard') + '?tab=menu')
    staff_name = request.user.first_name if request.user.is_authenticated and request.user.first_name else
(request.user.username if request.user.is_authenticated else 'Staff')
    from customer_portal.feedback_context import build_feedback_context
    feedback_ctx = build_feedback_context()
    context = {
        'total_orders_today': total_orders_today,
        'pending_count': pending_count,
        'preparing_count': preparing_count,
        'ready_count': ready_count,
        'recent_orders': recent_orders,
        'menu_items': menu_items,
        'categories': categories,
        'users_list': users_list,
        'delivery_staff_list': delivery_staff_list,
        'delivery_requests': delivery_requests,
        'staff_name': staff_name,
        'daily_sales': daily_sales,
        'weekly_sales': weekly_sales,
        'monthly_sales': monthly_sales,
        'daily_chart_data': daily_chart_data,
        'weekly_chart_data': weekly_chart_data,
        'monthly_chart_data': monthly_chart_data,
        'sales_today': sales_today,
        'sales_week': sales_week,
        'sales_month': sales_month,
        'pie_chart_data': pie_chart_data,
        'line_chart_data': line_chart_data,
```

```
        'pie_chart_data_json': json.dumps(pie_chart_data),
        'line_chart_data_json': json.dumps(line_chart_data),
        'feedbacks': feedback_ctx['feedbacks'],
        'fb_kiosk': feedback_ctx['fb_kiosk'],
        'fb_delivery': feedback_ctx['fb_delivery'],
        'fb_kiosk_stats': feedback_ctx['fb_kiosk_stats'],
        'fb_delivery_stats': feedback_ctx['fb_delivery_stats'],
        'fb_total': feedback_ctx['fb_total'],
        'fb_overall': feedback_ctx['fb_overall'],
        'overall_stats': overall_stats,
        'kiosk_stats': kiosk_stats,
        'faculty_stats': faculty_stats,
        'delivery_stats': delivery_stats,
        'rider_rankings': rider_rankings,
        'top_selling_items': top_selling_items,
    }
    return render(request, 'canteen_menu/staff_dashboard.html', context)
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def export_report_view(request, format_type):
    from django.http import HttpResponse
    from customer_portal.models import Order
    from django.utils import timezone
    from django.db.models import Sum
    valid_orders = Order.objects.filter(status__in=['ready', 'completed'])
    total_rev = valid_orders.aggregate(Sum('total_amount'))['total_amount__sum'] or 0
    total_count = valid_orders.count()
    if format_type == 'excel':
        def _csv_safe(value):
            value = str(value).replace('\r', ' ').replace('\n', ' ')
            if value[:1] in ('=', '+', '-', '@'):
                return "'" + value
            return value
        response = HttpResponse(content_type='text/csv')
        response['Content-Disposition'] = 'attachment; filename="canteen_overall_sales_report.csv"'
        response.write("Canteen Express - Overall Sales & Financial Report\n")
        response.write(f"Generated On,{timezone.now().strftime('%Y-%m-%d %H:%M')}\n\n")
```

```
        response.write(f"Total Completed/Ready Orders,{total_count}\n")

        response.write(f"Total Revenue (PHP),{total_rev:.2f}\n\n")

        response.write("Order ID,Customer,Total Amount,Status,Date\n")

        for o in valid_orders.order_by('-created_at'):

            cust = o.customer.username if o.customer else 'Guest'
response.write(f"{_csv_safe(o.order_number)},{_csv_safe(cust)},{o.total_amount},{_csv_safe(o.status)},{o.created_at
.strftime('%Y-%m-%d %H:%M')}\n")

        return response

    elif format_type == 'docx':

        html_content = f"""

        <html>

        <head><meta charset="utf-8"><title>Canteen Express Overall Report</title></head>

        <body style="font-family: Arial, sans-serif; padding: 20px;">

            <h1 style="color: #FF6117; text-align: center;">Canteen Express - Overall Report</h1>

            <p style="text-align: center; color: #666;">Generated on {timezone.now().strftime('%B %d, %Y
%H:%M')}</p>

            <hr/>

            <h2>Executive Summary</h2>

            <p><strong>Total Revenue:</strong> ₱{total_rev:,.2f}</p>

            <p><strong>Total Completed Orders:</strong> {total_count}</p>

            <hr/>

            <h2>Recent Completed/Ready Orders</h2>

            <table border="1" cellpadding="8" cellspacing="0" style="width:100%; border-collapse: collapse;">

                <tr style="background-color: #f2f2f2;"><th>Order
#</th><th>Customer</th><th>Amount</th><th>Status</th><th>Date</th></tr>

        """

        for o in valid_orders.order_by('-created_at')[:50]:

            cust = o.customer.username if o.customer else 'Guest'

            html_content +=
f"<tr><td>{o.order_number}</td><td>{cust}</td><td>₱{o.total_amount:,.2f}</td><td>{o.status}</td><td>{o.created_at.s
trftime('%Y-%m-%d %H:%M')}</td></tr>"

        html_content += "</table></body></html>"

        response = HttpResponse(html_content, content_type='application/msword')

        response['Content-Disposition'] = 'attachment; filename="canteen_overall_report.doc"'

        return response

    elif format_type == 'pdf':

        html_content = f"""

        <html>

        <head><meta charset="utf-8"><title>Canteen Express Overall Report PDF</title></head>
```

```
        <body style="font-family: Helvetica, Arial, sans-serif; padding: 30px; color: #333;">

            <div style="text-align: center; margin-bottom: 20px;">

                <h1 style="color: #FF6117; margin: 0;">CANTEEN EXPRESS</h1>

                <p style="font-size: 14px; color: #555; margin: 5px 0;">Official Overall Sales & Analytics
Report</p>

                <p style="font-size: 11px; color: #888;">Generated on {timezone.now().strftime('%B %d, %Y
%H:%M')}</p>

            </div>

            <hr style="border: 1px solid #ddd; margin-bottom: 20px;"/>

            <div style="background: #f9f9f9; padding: 15px; border-radius: 8px; margin-bottom: 20px;">

                <h3 style="margin-top: 0; color: #111;">Summary Metrics</h3>

                <p><strong>Total Revenue:</strong> ₱{total_rev:,.2f}</p>

                <p><strong>Total Successful Orders:</strong> {total_count}</p>

            </div>

            <h3>Detailed Orders List</h3>

            <table border="1" cellpadding="6" cellspacing="0" style="width:100%; border-collapse: collapse;
font-size: 12px;">

                <tr style="background-color: #333; color: white;"><th>Order
#</th><th>Customer</th><th>Amount</th><th>Status</th><th>Date</th></tr>

        """

        for o in valid_orders.order_by('-created_at')[:50]:

            cust = o.customer.username if o.customer else 'Guest'

            html_content +=
f"<tr><td>{o.order_number}</td><td>{cust}</td><td>₱{o.total_amount:,.2f}</td><td>{o.status}</td><td>{o.created_at.s
trftime('%Y-%m-%d %H:%M')}</td></tr>"

        html_content += """

            </table>

            <script>window.onload = function() { window.print(); }</script>

        </body></html>

        """

        response = HttpResponse(html_content, content_type='text/html')

        return response

    return redirect('canteen_menu:staff_dashboard')

@role_required(allowed_roles=['STAFF', 'ADMIN'])

def staff_menu_toggle_ajax(request, pk):

    item = get_object_or_404(MenuItem, pk=pk)

    item.is_available = not item.is_available

    item.save()

    cache.delete('formatted_menu_active_kiosk')
```

```
    now_available = item.is_available
    log_activity(
        request,
        action=f"{'Made available' if now_available else 'Marked unavailable'}: {item.name}",
        category=ActivityLog.Category.MENU,
        level=ActivityLog.Level.WARNING if not now_available else ActivityLog.Level.SUCCESS,
        target=item.name,
        metadata={'item_id': item.pk, 'is_available': now_available, 'stock': item.stock},
    )
    return JsonResponse({'success': True, 'is_available': item.is_available, 'stock': item.stock})
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def staff_menu_edit_ajax(request):
    if request.method == 'POST':
        try:
            item_id = request.POST.get('item_id')
            item = get_object_or_404(MenuItem, pk=item_id)
            before = {
                'name': item.name,
                'price': float(item.price),
                'stock': item.stock,
            }
            item.name = request.POST.get('name', item.name)
            item.price = request.POST.get('price', item.price)
            try:
                stock_value = int(request.POST.get('stock', item.stock))
                item.stock = max(0, stock_value)
            except (TypeError, ValueError):
                item.stock = item.stock
            if request.POST.get('description') is not None:
                item.description = request.POST.get('description')
            category_id = request.POST.get('category')
            if category_id:
                item.category = get_object_or_404(Category, pk=category_id)
            new_image_url = request.POST.get('image_url')
            if new_image_url is not None:
                item.image_url = new_image_url
```

```
                if new_image_url.strip():
                    item.image = None
            if request.FILES.get('image'):
                pub_url = upload_to_supabase_storage(request.FILES.get('image'))
                if pub_url:
                    item.image_url = pub_url
                    item.image = None
                else:
                    item.image = request.FILES.get('image')
                    item.image_url = ''

            if not item.image and not item.image_url:
                _auto_sync_menu_image(item)
            item.save()
            cache.delete('formatted_menu_active_kiosk')
            messages.success(request, f"Updated {item.name} successfully!")
            changes = _diff_menu_fields(before, {
                'name': item.name,
                'price': float(item.price),
                'stock': item.stock,
            })
            log_activity(
                request,
                action=f"Edited {item.name}" + (f" ({', '.join(sorted(changes))})" if changes else ' (no
changes)'),
                category=ActivityLog.Category.MENU,
                level=ActivityLog.Level.INFO,
                target=item.name,
                metadata={'item_id': item.pk, 'changes': changes},
            )
        except Exception as e:
            messages.error(request, f"Error updating item: {str(e)}")
    return redirect(reverse('canteen_menu:staff_dashboard') + '?tab=menu')
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def create_delivery_staff_view(request):
    if request.method == 'POST':
        username = request.POST.get('username')
```

```
        email = request.POST.get('email')
        password = request.POST.get('password')
        phone = request.POST.get('phone')
        vehicle_plate = request.POST.get('vehicle_plate')
        from django.contrib.auth import get_user_model
        User = get_user_model()

        if not username or not password:
            messages.error(request, "Username and password are required.")
        elif User.objects.filter(username=username).exists():
            messages.error(request, f"Username '{username}' already exists.")
        elif email and User.objects.filter(email=email).exists():
            messages.error(request, f"Email '{email}' already exists.")
        elif len(password) < 6:
            messages.error(request, "Password must be at least 6 characters long.")
        else:
            final_email = email if email else f"{username.lower()}@canteen.express"
            try:
                created = User.objects.create_user(
                    username=username,
                    email=final_email,
                    password=password,
                    first_name=request.POST.get('first_name', 'Rider'),
                    role='DELIVERY',
                    phone=phone,
                    vehicle_plate=vehicle_plate,
                    account_status='active',
                    is_active=True
                )
                messages.success(request, f"Delivery staff '{username}' created successfully!")
                log_activity(
                    request,
                    action=f"Created rider account: {username}",
                    category=ActivityLog.Category.USER,
                    level=ActivityLog.Level.SUCCESS,
                    target=username,
```

```
                    metadata={
                        'user_id': created.pk,
                        'role': 'DELIVERY',
                        'vehicle_plate': vehicle_plate or None,
                    },
                )
            except Exception as e:
                messages.error(request, f"Error creating account: {str(e)}")
    return redirect(reverse('canteen_menu:staff_dashboard') + '?tab=deliveries')
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def update_delivery_staff_status_view(request, pk):
    if request.method == 'POST':
        from django.contrib.auth import get_user_model
        User = get_user_model()
        rider = get_object_or_404(User, pk=pk, role='DELIVERY')
        action = request.POST.get('action')
        reason = request.POST.get('reason', '').strip()
        before_status = rider.account_status
        if action == 'penalize':
            rider.account_status = 'penalized'
            rider.status_reason = reason or "Penalized by canteen staff"
            messages.success(request, f"Rider {rider.username} penalized.")
        elif action == 'toggle_active':
            rider.is_active = not rider.is_active
            rider.account_status = 'active' if rider.is_active else 'inactive'
            if rider.is_active:
                rider.status_reason = ''
            messages.success(request, f"Rider {rider.username} status toggled.")
        rider.save()
        if action in ('penalize', 'toggle_active'):
            verb = {
                'penalize': 'Penalized',
                'toggle_active': ('Reactivated' if rider.is_active else 'Deactivated'),
            }[action]
            log_activity(
                request,
```

```
                action=f"{verb} rider account: {rider.username}",
                category=ActivityLog.Category.USER,
                level=ActivityLog.Level.DANGER if action == 'penalize' else ActivityLog.Level.WARNING,
                target=rider.username,
                metadata={
                    'user_id': rider.pk,
                    'action': action,
                    'reason': reason,
                    'from_status': before_status,
                    'to_status': rider.account_status,
                },
            )
    return redirect(reverse('canteen_menu:staff_dashboard') + '?tab=deliveries')
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def update_user_status_view(request, pk):
    if request.method == 'POST':
        from django.contrib.auth import get_user_model
        User = get_user_model()
        user = get_object_or_404(User, pk=pk)
        action = request.POST.get('action')
        reason = request.POST.get('reason', '').strip()
        before_status = user.account_status
        if action == 'ban':
            user.is_active = False
            user.account_status = 'banned'
            user.status_reason = reason or "Banned by canteen administration"
            messages.success(request, f"User {user.username} banned.")
        elif action == 'restrict':
            user.account_status = 'restricted'
            user.status_reason = reason or "Restricted by canteen administration"
            messages.success(request, f"User {user.username} restricted.")
        elif action == 'activate':
            user.is_active = True
            user.account_status = 'active'
            user.status_reason = ''
            messages.success(request, f"User {user.username} activated.")
```

```
        user.save()
        if action in ('ban', 'restrict', 'activate'):
            verb = {'ban': 'Banned', 'restrict': 'Restricted', 'activate': 'Reactivated'}[action]
            log_activity(
                request,
                action=f"{verb} user account: {user.username}",
                category=ActivityLog.Category.USER,
                level=ActivityLog.Level.DANGER if action == 'ban' else ActivityLog.Level.WARNING,
                target=user.username,
                metadata={
                    'user_id': user.pk,
                    'action': action,
                    'reason': reason,
                    'target_role': getattr(user, 'role', None),
                    'from_status': before_status,
                    'to_status': user.account_status,
                },
            )
    return redirect(reverse('canteen_menu:staff_dashboard') + '?tab=users')
@csrf_exempt
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def process_barcode_api(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
        except Exception:
            return JsonResponse({'status': 'error', 'message': 'Invalid JSON body'}, status=400)
        action = data.get('action', 'fetch')
        raw_order_id = str(data.get('order_id', '')).strip()
        if not raw_order_id:
            return JsonResponse({'status': 'error', 'message': 'Order ID missing'}, status=400)
        from customer_portal.views import expire_stale_unpaid_orders, slip_expires_at, find_order_by_number
        expire_stale_unpaid_orders()
        from customer_portal.models import Order as KioskOrder
        clean_code = raw_order_id.replace('#', '').strip()
        order = find_order_by_number(raw_order_id)
```

```
        if not order:
            return JsonResponse({'status': 'error', 'message': f'Queue Slip #{clean_code} not found or already
processed!'}, status=404)
        if action == 'confirm_payment':
            if order.status != 'unpaid':
                if order.status == 'cancelled':
                    return JsonResponse({
                        'status': 'error',
                        'message': 'This queue slip was cancelled. Please reorder.'
                    }, status=400)
                return JsonResponse({
                    'status': 'success',
                    'already_paid': True,
                    'message': f'Order already paid ({order.get_status_display()}). Receipt re-issued.',
                })
            order.status = 'ready'
            order.save()
            log_activity(
                request,
                action=f"Confirmed payment for {order.order_number} (P{float(order.total_amount):.2f})",
                category=ActivityLog.Category.PAYMENT,
                level=ActivityLog.Level.SUCCESS,
                target=order.order_number,
                is_guest=order.customer_id is None,
                metadata={
                    'order_id': order.id,
                    'total': float(order.total_amount),
                    'delivery_fee': float(order.delivery_fee),
                    'item_count': sum(i.quantity for i in order.items.all()),
                },
            )
            return JsonResponse({
                'status': 'success',
                'message': f'Order {order.order_number} verified and moved directly to Orders Ready.'
            })
        items = []
```

```
        for item in order.items.all():
           items.append({
                'name': item.item_name,
                'qty': item.quantity,
                'price': float(item.price)
            })
        order_id_clean = order.order_number.replace('#', '')
        return JsonResponse({
            'status': 'success',
            'order': {
                'orderId': order_id_clean,
                'orderNumber': order.order_number,
                'type': 'DINE-IN',
                'status': order.status,
                'total': float(order.total_amount),
                'items': items,
                'expires_at': slip_expires_at(order),
            }
        })
return JsonResponse({'status': 'error', 'message': 'Invalid request method'}, status=405)
@role_required(allowed_roles=['STAFF', 'ADMIN'])
def counter_pos_view(request):
    from customer_portal.models import Order
    from deliveries.models import DeliveryRequest
    from django.utils import timezone
    ready_orders = Order.objects.filter(status='ready').prefetch_related('items').order_by('-created_at')
       accepted_delivery_order_ids =
DeliveryRequest.objects.filter(status=DeliveryRequest.RequestStatus.ACCEPTED).values_list('order_id', flat=True)
    delivery_orders = Order.objects.filter(status__in=['pending', 'preparing'],
id__in=accepted_delivery_order_ids).prefetch_related('items').order_by('-created_at')
    today = timezone.now().date()
    done_today_count = Order.objects.filter(status='completed', created_at__date=today).count()
    context = {
        'ready_orders': ready_orders,
        'delivery_orders': delivery_orders,
        'done_today_count': done_today_count,
    }
```

```
    return render(request, 'canteen_menu/counter_pos.html', context)

@role_required(allowed_roles=['STAFF', 'ADMIN'])
def staff_dashboard_stats_api(request):
    from customer_portal.models import Order
    from deliveries.models import DeliveryRequest
    from django.utils import timezone
    from django.utils.timesince import timesince
    today = timezone.now().date()
    day_start = timezone.now().replace(hour=0, minute=0, second=0, microsecond=0)
    day_end = day_start + timezone.timedelta(days=1)
    today_orders = Order.objects.filter(
        created_at__gte=day_start, created_at__lt=day_end
    ).exclude(status='unpaid')
    total_orders_today = today_orders.count()
    pending_count = Order.objects.filter(status='pending').count()
    preparing_count = Order.objects.filter(status='preparing').count()
    ready_count = Order.objects.filter(status='ready').count()
    recent_qs = Order.objects.exclude(status__in=['completed', 'unpaid', 'cancelled']).select_related('customer',
'delivery_info', 'delivery_info__rider').order_by('-created_at')[:5]
    orders_data = []
    for o in recent_qs:
        cust_name = "Walk-in Guest"
        if o.customer:
            cust_name = o.customer.get_full_name() or o.customer.username
        del_info = getattr(o, 'delivery_info', None)
        del_status = del_info.status if del_info else None
        status_display = o.status
        if del_status == 'ACCEPTED':
            status_display = 'Rider Accepted'
        elif del_status == 'SEARCHING':
            status_display = 'Finding Rider'
        orders_data.append({
            'order_number': o.order_number,
            'customer_name': cust_name,
            'total_amount': f"{o.total_amount:.2f}",
            'status': status_display,
            'raw_status': o.status,
```

```
            'delivery_status': del_status,
            'time_ago': timesince(o.created_at) + " ago",
        })
    return JsonResponse({
        'success': True,
        'total_orders_today': total_orders_today,
        'pending_count': pending_count,
        'preparing_count': preparing_count,
        'ready_count': ready_count,
        'recent_orders': orders_data,
    })
```

backend/canteen_menu/templates/canteen_menu/counter_pos.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <link rel="icon" href="{% static 'customer_portal/images/canteen-express-logo.png' %}" type="image/png" />
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Counter POS & Kitchen Board - Canteen Express</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" />
    <script src="https://unpkg.com/html5-qrcode@2.3.8/html5-qrcode.min.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/html2canvas/1.4.1/html2canvas.min.js"></script>
    </head>
<body class="bg-[#f0f2f5] text-gray-800 font-sans min-h-screen flex flex-col">
    <!-- Top Header Navigation -->
    <header class="bg-[#0d0d12] text-white px-6 py-4 flex items-center justify-between shadow-md">
        <div class="flex items-center gap-3">
            <div class="w-10 h-10 rounded-2xl bg-brand-500/10 border border-orange-500/30 flex items-center
justify-center p-1.5 shadow-lg shadow-orange-500/20">
                <img src="{% static 'customer_portal/images/canteen-express-logo.png' %}" alt="Canteen Express"
class="w-full h-full object-contain">
            </div>
            <div>
                <h1 class="font-black text-base">Canteen Counter POS & Kitchen Board</h1>
```

```
                <p class="text-[10px] text-gray-400 uppercase tracking-wider font-semibold">Unified Operations
Terminal</p>

            </div>

        </div>

        <div class="flex items-center gap-4 text-xs font-semibold">

            <span class="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-emerald-500/10 border
border-emerald-500/20 text-emerald-400">

                <span class="w-2 h-2 rounded-full bg-emerald-400 animate-pulse"></span> Live Sync

            </span>

            <a href="{% url 'canteen_menu:staff_dashboard' %}" class="bg-white/10 hover:bg-white/20 px-3.5 py-2
rounded-xl text-gray-300 transition flex items-center gap-2">

                <i class="fa-solid fa-gauge-high"></i> Staff Dashboard

            </a>

            <span class="text-gray-400 font-mono" id="pos-clock">00:00:00 PM</span>

        </div>

    </header>

    <!-- Status Error Dialog -->

    <div id="status-dialog" class="hidden fixed inset-0 z-50 flex items-center justify-center p-4">

        <div class="absolute inset-0 bg-dark-900/70 backdrop-blur-sm" onclick="closeStatusDialog()"></div>

        <div class="relative bg-white rounded-3xl border border-gray-200/80 shadow-2xl max-w-sm w-full p-6
text-center">

            <div class="mx-auto w-14 h-14 rounded-2xl bg-red-50 border border-red-100 flex items-center
justify-center text-red-500 text-2xl mb-4">

                <i class="fa-solid fa-triangle-exclamation"></i>

            </div>

            <h3 class="text-lg font-black text-gray-900 tracking-tight">Cannot update order</h3>

            <p id="status-dialog-order" class="text-xs font-extrabold text-gray-400 uppercase tracking-wider
mt-1"></p>

            <p id="status-dialog-message" class="text-sm text-gray-600 mt-3 leading-relaxed"></p>

            <button onclick="closeStatusDialog()" class="mt-5 w-full bg-slate-900 hover:bg-black text-white
rounded-xl px-4 py-2.5 text-sm font-bold transition active:scale-[0.98]">

                Got it

            </button>

        </div>

    </div>

    <!-- Main Layout: Left = Kitchen Board (7 Cols), Right = POS (5 Cols) -->

    <main class="flex-1 max-w-[1700px] w-full mx-auto p-4 sm:p-6 grid grid-cols-1 xl:grid-cols-12 gap-6
items-start">

        <!-- LEFT SIDE: KITCHEN BOARD (7 Cols) -->

        <div class="xl:col-span-7 space-y-6">
```

```
            <div class="flex items-center justify-between bg-white p-4 rounded-2xl border border-gray-200
shadow-sm">

                <div class="flex items-center gap-2.5">

                    <span class="bg-orange-500/10 text-orange-600 p-2 rounded-xl text-lg font-bold">

                        <i class="fa-solid fa-fire-burner"></i>

                    </span>

                    <h2 class="text-lg font-black text-gray-900 tracking-tight">Live Kitchen Board</h2>

                </div>

                <div class="flex items-center gap-3 text-xs font-bold">

                    <span class="bg-orange-100 text-orange-700 px-3 py-1 rounded-full">

                        Delivery: <span id="count-delivery">{{ delivery_orders|length }}</span>

                    </span>

                    <span class="bg-emerald-100 text-emerald-700 px-3 py-1 rounded-full">

                        Ready: <span id="count-ready">{{ ready_orders|length }}</span>

                    </span>

                </div>

            </div>

            <!-- 2 Columns: Delivery & Orders Ready -->

            <div class="grid grid-cols-1 md:grid-cols-2 gap-6">

                <!-- Column 1: Delivery Orders -->

                <div class="space-y-4">

                    <div class="flex items-center justify-between px-2">

                        <h3 class="text-xs font-extrabold text-gray-500 uppercase tracking-wider flex items-center
gap-2">

                            <span class="w-2.5 h-2.5 rounded-full bg-orange-500"></span> Delivery Orders

                        </h3>

                        <span class="count-pill text-xs font-bold bg-orange-100 text-orange-700 px-2.5 py-0.5
rounded-full" data-count-for="delivery">{{ delivery_orders|length }}</span>

                    </div>

                    <div data-board="delivery" class="space-y-4 custom-scroll max-h-[70vh] overflow-y-auto pr-1">

                        {% for order in delivery_orders %}

                        <div class="bg-white rounded-3xl p-5 border-2 border-orange-500 shadow-md space-y-3
relative" id="order-card-{{ order.id }}" data-order-row="{{ order.id }}">

                            <div class="flex justify-between items-start border-b border-gray-100 pb-2.5">

                                <div>

                                    <span class="status-title text-xs font-black text-orange-600 uppercase
tracking-wider">

                                        Order #{{ order.id }} ({{ order.order_number }})
```

```
                                    </span>

                                    <h4 class="text-sm font-black text-gray-900">{% if order.customer %}{{
order.customer.get_full_name|default:order.customer.username }}{% else %}Guest{% endif %}</h4>

                                </div>

                                <span class="text-[10px] bg-orange-100 text-orange-800 font-extrabold px-2 py-0.5
rounded-lg border border-orange-300 flex items-center gap-1">

                                    <i class="fa-solid fa-motorcycle"></i> Delivery

                                </span>

                            </div>

                            <div class="space-y-1.5 text-xs text-gray-700 font-medium">

                                {% for item in order.items.all %}

                                <div class="flex justify-between items-center">

                                    <span>{{ item.quantity }}x {{ item.item_name|default:item.product.name
}}</span>

                                    <span class="font-bold text-gray-900">₱{{ item.total_price }}</span>

                                </div>

                                {% empty %}

                                <p class="text-xs text-gray-400">No items found.</p>

                                {% endfor %}

                            </div>

                            <div class="pt-2 border-t border-gray-100 flex items-center justify-between text-xs">

                                <span class="text-gray-400 font-medium">

                                    <i class="fa-regular fa-clock mr-1"></i> {{ order.created_at|time:"H:i" }}

                                </span>

                                <div class="flex gap-1.5">

                                    {% if order.status == 'pending' %}

                                    <button data-order-id="{{ order.id }}" data-status="preparing"
class="status-btn bg-orange-500 hover:bg-orange-600 text-white font-bold px-3 py-1.5 rounded-xl text-xs
transition">

                                        Preparing

                                    </button>

                                    {% endif %}

                                    <button data-order-id="{{ order.id }}" data-status="ready" class="status-btn
bg-emerald-600 hover:bg-emerald-700 text-white font-bold px-3 py-1.5 rounded-xl text-xs transition flex
items-center gap-1">

                                        <i class="fa-solid fa-check"></i> Ready

                                    </button>

                                </div>

                            </div>
```

```
                        </div>

                        {% empty %}

                        <div class="empty-state bg-white rounded-3xl p-6 border border-gray-200 text-center
text-gray-400 text-xs">

                            No active delivery orders.

                        </div>

                        {% endfor %}

                    </div>

                </div>

                <!-- Column 2: Orders Ready (Pickup & Delivery Ready) -->

                <div class="space-y-4">

                    <div class="flex items-center justify-between px-2">

                        <h3 class="text-xs font-extrabold text-gray-500 uppercase tracking-wider flex items-center
gap-2">

                            <span class="w-2.5 h-2.5 rounded-full bg-emerald-500"></span> Orders Ready

                        </h3>

                        <span class="count-pill text-xs font-bold bg-emerald-100 text-emerald-700 px-2.5 py-0.5
rounded-full" data-count-for="ready">{{ ready_orders|length }}</span>

                    </div>

                    <div data-board="ready" class="space-y-4 custom-scroll max-h-[70vh] overflow-y-auto pr-1">

                        {% for order in ready_orders %}

                        <div class="bg-gray-50 rounded-3xl p-5 border-2 border-emerald-500 shadow-sm space-y-3
relative" id="order-card-{{ order.id }}" data-order-row="{{ order.id }}">

                            <div class="flex justify-between items-start border-b border-gray-200 pb-2.5">

                                <div>

                                    <span class="status-title text-xs font-black text-emerald-600 uppercase
tracking-wider">

                                        Order #{{ order.id }} ({{ order.order_number }})

                                    </span>

                                    <h4 class="text-sm font-black text-gray-900">{% if order.customer %}{{
order.customer.get_full_name|default:order.customer.username }}{% else %}Walk-in / Kiosk{% endif %}</h4>

                                </div>

                                {% if order.is_delivery %}

                                <span class="text-[10px] bg-orange-100 text-orange-800 font-extrabold px-2 py-0.5
rounded-lg border border-orange-300 flex items-center gap-1">

                                    <i class="fa-solid fa-motorcycle"></i> Delivery Ready

                                </span>

                                {% else %}

                                <span class="text-[10px] bg-emerald-100 text-emerald-800 font-extrabold px-2 py-0.5
rounded-lg border border-emerald-300 flex items-center gap-1">
```

```
                                    <i class="fa-solid fa-store"></i> Counter Pickup
                                </span>
                                {% endif %}
                            </div>

                            <div class="space-y-1.5 text-xs text-gray-700 font-medium">
                                {% for item in order.items.all %}
                                <div class="flex justify-between items-center">
                                    <span>{{ item.quantity }}x {{ item.item_name|default:item.product.name
}}</span>
                                    <span class="font-bold text-gray-900">₱{{ item.total_price }}</span>
                                </div>
                                {% empty %}
                                <p class="text-xs text-gray-400">No items found.</p>
                                {% endfor %}
                            </div>
                            <p class="text-[11px] text-gray-500 font-medium">
                                {% if order.is_delivery %}
                                    Awaiting assigned delivery rider.
                                {% else %}
                                {% endif %}
                            </p>
                            <div class="pt-2 border-t border-gray-200 flex items-center justify-between text-xs">
                                <span class="text-gray-400 font-medium">
                                    <i class="fa-regular fa-clock mr-1"></i> {{ order.created_at|time:"H:i" }}
                                </span>
                                <button data-order-id="{{ order.id }}" data-status="completed" class="status-btn
bg-slate-900 hover:bg-black text-white font-bold px-3.5 py-2 rounded-xl text-xs transition flex items-center
gap-1.5">
                                    <i class="fa-solid fa-box-archive"></i> Complete
                                </button>
                            </div>
                        </div>
                        {% empty %}
                        <div class="empty-state bg-white rounded-3xl p-6 border border-gray-200 text-center
text-gray-400 text-xs">
                            No orders ready right now.
                        </div>
```

```
                        {% endfor %}

                    </div>

                </div>

            </div>

        </div>

        <div class="xl:col-span-5 space-y-5">

            <div id="no-order-placeholder" class="bg-white rounded-3xl p-8 border border-dashed border-gray-300
text-center space-y-3">

                <div class="w-14 h-14 rounded-full bg-gray-100 text-gray-400 flex items-center justify-center
text-xl mx-auto">

                    <i class="fa-solid fa-receipt"></i>

                </div>

                <h3 class="font-bold text-gray-700 text-sm">Awaiting Barcode Scan</h3>

                <p class="text-xs text-gray-400 max-w-xs mx-auto">Scan a queue slip barcode or type the number to
verify payment and process order.</p>

            </div>

            <div id="active-order-card" class="hidden bg-white rounded-3xl border-2 border-orange-400 shadow-xl
overflow-hidden space-y-5">

                <div class="bg-slate-900 text-white p-4 flex items-center justify-between">

                    <div>

                        <span class="text-[10px] uppercase font-extrabold text-orange-400 tracking-wider">QUEUE
SLIP DETAILS</span>

                        <h3 class="text-2xl font-black text-white" id="order-number-display">#CE-1149</h3>

                    </div>

                    <span id="order-type-badge" class="bg-orange-500 text-white text-[10px] font-black px-3 py-1
rounded-full uppercase">

                        PICKUP

                    </span>

                </div>

                <div class="p-5 space-y-4">

                    <div>

                        <h4 class="text-[11px] font-extrabold uppercase text-gray-400 mb-2">ITEMS</h4>

                        <div class="border rounded-2xl overflow-hidden divide-y text-xs font-semibold"
id="items-list-container">

                            <!-- Items -->

                        </div>

                    </div>

                    <div class="bg-stone-900 text-white p-4 rounded-2xl flex items-center justify-between">

                        <div>
```

```
                            <p class="text-[10px] text-stone-400 font-bold uppercase">TOTAL AMOUNT DUE</p>

                            <p class="text-[10px] text-emerald-400 font-bold">Paid at Counter</p>

                        </div>

                        <span class="text-2xl font-black text-orange-400" id="total-price-display">₱0.00</span>

                    </div>


                    <div class="flex gap-2 pt-1">

                        <button onclick="cancelScan()" class="w-1/3 bg-gray-100 hover:bg-gray-200 text-gray-700
font-bold py-3 rounded-xl text-xs transition">

                            Cancel

                        </button>

                        <button onclick="confirmPaymentAndSendToKitchen()" class="w-2/3 bg-emerald-500
hover:bg-emerald-600 text-white font-black py-3 rounded-xl text-xs tracking-wider uppercase shadow-md
shadow-emerald-500/20 transition flex items-center justify-center gap-2">

                            <i class="fa-solid fa-circle-check"></i> Confirm Payment & Send

                        </button>

                    </div>

                </div>

            </div>

            <div class="bg-white p-5 rounded-3xl border border-gray-200 shadow-sm space-y-3">

                <label class="block text-xs font-bold text-gray-700">Manual Input or USB Scanner:</label>

                <form onsubmit="handleManualSearch(event)" class="flex gap-2">

                    <div class="relative flex-1">

                        <i class="fa-solid fa-barcode absolute left-3 top-1/2 -translate-y-1/2 text-gray-400
text-xs"></i>

                        <input

                            type="text"

                            id="manual-input"

                            inputmode="numeric"

                            maxlength="6"

                            placeholder="e.g. 1149"

                            title="Type only the 4-digit code - CE is added automatically"

                            class="w-full bg-stone-50 border border-gray-300 pl-8 pr-3 py-2.5 rounded-xl text-xs
font-mono font-bold text-gray-900 focus:outline-none focus:ring-2 focus:ring-orange-500 uppercase"

                            autofocus

                        >

                    </div>

                    <button type="submit" class="bg-slate-900 hover:bg-black text-white font-bold px-4 py-2.5
rounded-xl text-xs transition">
```

```
                        Find
                    </button>
                </form>
            </div>


            <!-- Camera Scanner Container Box -->
            <div class="bg-white p-5 rounded-3xl border border-gray-200 shadow-sm space-y-4">
                <div class="flex items-center justify-between">
                    <div class="flex items-center gap-2 text-gray-900 font-extrabold text-sm">
                        <i class="fa-solid fa-barcode text-orange-500 text-lg"></i>
                        <span>Queue Slip Barcode Scanner</span>
                    </div>
                    <span id="camera-status" class="text-[10px] bg-stone-100 text-stone-600 font-bold px-2.5 py-0.5
rounded-full border">
                        Camera Off
                    </span>
                </div>
                <div class="relative bg-stone-900 rounded-2xl overflow-hidden min-h-[130px] flex items-center
justify-center border-2 border-dashed border-stone-300">
                    <div id="reader" class="w-full"></div>
                    <div id="camera-placeholder" class="text-center p-4 space-y-1">
                        <i class="fa-solid fa-barcode text-2xl text-stone-600"></i>
                        <p class="text-[11px] text-stone-400 font-semibold">Click "Start Camera" to scan Queue Slip
barcode</p>
                    </div>
                </div>
                <div class="flex gap-2">
                    <button id="btn-start-cam" onclick="startCameraScanner()" class="w-1/2 bg-emerald-600
hover:bg-emerald-700 text-white font-black py-2.5 rounded-xl text-xs shadow-sm transition flex items-center
justify-center gap-2">
                        <i class="fa-solid fa-video"></i> Start Camera
                    </button>
                    <button id="btn-stop-cam" onclick="stopCameraScanner()" class="w-1/2 bg-rose-600
hover:bg-rose-700 text-white font-black py-2.5 rounded-xl text-xs shadow-sm transition flex items-center
justify-center gap-2 hidden">
                        <i class="fa-solid fa-video-slash"></i> Stop Camera
                    </button>
                </div>
            </div>
```

```
        </div>

    </main>

    <div id="receipt-modal" class="fixed inset-0 glass-backdrop hidden items-center justify-center p-4 z-50
overflow-y-auto">

        <div class="max-w-sm w-full bg-white rounded-3xl p-6 shadow-2xl space-y-5 relative my-8">

            <button onclick="closeReceiptModal()" class="absolute right-4 top-4 text-gray-400 hover:text-gray-600
text-lg">

                <i class="fa-solid fa-xmark"></i>

            </button>

            <div id="printable-receipt" class="bg-white p-6 border border-stone-200 rounded-2xl space-y-4
text-center font-mono">

                <div class="border-b pb-3 space-y-1">

                    <h2 class="font-black text-lg text-gray-950 tracking-wider">CANTEEN EXPRESS</h2>

                    <p class="text-[10px] text-gray-500">Official Payment Receipt</p>

                    <p class="text-[9px] text-gray-400" id="receipt-date"></p>

                </div>

                <div class="text-left text-xs space-y-1 border-b pb-3">

                    <p class="flex justify-between"><span class="text-gray-500">Queue Number:</span> <span
class="font-bold text-gray-900" id="rcpt-order-id">#CE-1149</span></p>

                    <p class="flex justify-between"><span class="text-gray-500">Payment Status:</span> <span
class="font-bold text-emerald-600">PAID</span></p>

                </div>

                <div class="text-left text-xs space-y-2 border-b pb-3" id="rcpt-items"></div>

                <div class="flex justify-between text-sm font-black pt-1">

                    <span>TOTAL AMOUNT:</span>

                    <span class="text-orange-600" id="rcpt-total">₱0.00</span>

                </div>

                <div class="pt-2 text-[9px] text-gray-400">

                    <p>Thank you! Enjoy your meal!</p>

                </div>

            </div>

            <div class="space-y-2">

                <button onclick="downloadReceiptImage()" class="w-full bg-orange-500 hover:bg-orange-600 text-white
font-extrabold py-3 rounded-2xl text-xs shadow-md transition flex items-center justify-center gap-2">

                    <i class="fa-solid fa-download"></i> Download Receipt Image

                </button>

                <button onclick="closeReceiptModal()" class="w-full bg-gray-100 text-gray-700 font-bold py-2.5
rounded-2xl text-xs">

                    Close
```

```
                </button>
            </div>
        </div>
    </div>
</body>
</html>
```

Delivery Personnel (Rider Hub & GPS Tracking)

Empowers delivery riders to accept delivery requests, navigate road-following routes to campus offices, and stream real-time GPS coordinates using HTML5 geolocation.

backend/deliveries/models.py

```
from django.db import models
from django.conf import settings
from customer_portal.models import Order
class DeliveryRequest(models.Model):
    class RequestStatus(models.TextChoices):
        SEARCHING = 'SEARCHING', 'Searching for Rider'
        ACCEPTED = 'ACCEPTED', 'Rider Accepted'
        DELIVERED = 'DELIVERED', 'Delivered'
        REJECTED = 'REJECTED', 'Rejected'
        TIMEOUT = 'TIMEOUT', 'No Rider Available (5-Min Timeout)'
    order = models.OneToOneField(Order, on_delete=models.CASCADE, related_name='delivery_info')
    rider = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='deliveries',
        db_index=True
    )
    assigned_to = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='assigned_deliveries',
        db_index=True
    )
```

```
    assigned_at = models.DateTimeField(null=True, blank=True)
    delivery_location = models.CharField(max_length=255, help_text="Building & Room Number")
    status = models.CharField(max_length=20, choices=RequestStatus.choices, default=RequestStatus.SEARCHING,
db_index=True)
    requested_at = models.DateTimeField(auto_now_add=True)
    accepted_at = models.DateTimeField(null=True, blank=True)
    delivered_at = models.DateTimeField(null=True, blank=True)
    rider_lat = models.FloatField(null=True, blank=True)
    rider_lng = models.FloatField(null=True, blank=True)
    rider_acc = models.FloatField(null=True, blank=True)
    location_updated_at = models.DateTimeField(null=True, blank=True)
    dest_lat = models.FloatField(null=True, blank=True)
    dest_lng = models.FloatField(null=True, blank=True)
    proof_photo = models.ImageField(upload_to='delivery_proofs/', null=True, blank=True)
    @property
    def raw_status(self):
        return self.status
    @property
    def earning(self):
        if self.order is None or self.order.delivery_fee is None:
            return 0.0
        return float(self.order.delivery_fee)
class RiderLocationPoint(models.Model):
    delivery = models.ForeignKey(DeliveryRequest, on_delete=models.CASCADE, related_name='location_points')
    rider = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, null=True, blank=True)
    lat = models.FloatField()
    lng = models.FloatField()
    speed_kmh = models.FloatField(default=0)
    accuracy = models.FloatField(null=True, blank=True)
    timestamp = models.DateTimeField(auto_now_add=True)
    class Meta:
        ordering = ['timestamp']
    def __str__(self):
        return f"{self.lat:.5f}, {self.lng:.5f} @ {self.speed_kmh:.1f} km/h"
class DeliveryMessage(models.Model):
    delivery = models.ForeignKey(DeliveryRequest, on_delete=models.CASCADE, related_name='messages')
    sender = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
```

```
    message = models.TextField()
    is_read = models.BooleanField(default=False)
    timestamp = models.DateTimeField(auto_now_add=True)
    class Meta:
        ordering = ['timestamp', 'id']
    def __str__(self):
        return f"{self.sender.username}: {self.message[:30]}"
```

backend/deliveries/routing.py

```
import json
import sys
import urllib.parse
import urllib.request
from django.conf import settings
from django.core.cache import cache
from .utils import haversine_km
_MISS = object()
_CACHE_PRECISION = 4
_FAIL_TTL = 60
_MAX_POINTS = 600
def _routing_enabled():
    if not getattr(settings, 'ROUTING_ENABLED', True):
        return False
    return 'test' not in sys.argv
def _coord_pair(lat, lng):
    return f'{round(float(lat), _CACHE_PRECISION)},{round(float(lng), _CACHE_PRECISION)}'
def _get_json(url, timeout):
    req = urllib.request.Request(url, headers={'User-Agent': 'CanteenExpress/1.0'})
    with urllib.request.urlopen(req, timeout=timeout) as response:
        return json.loads(response.read().decode('utf-8'))
def decode_polyline(encoded, precision=6):
    coordinates = []
    index = 0
    lat = 0
    lng = 0
    factor = 10 ** precision
    while index < len(encoded):
```

```
        for axis in ('lat', 'lng'):
            shift = 0
            result = 0
            while True:
                if index >= len(encoded):
                    raise ValueError('Truncated polyline payload')
                byte = ord(encoded[index]) - 63
                index += 1
                result |= (byte & 0x1F) << shift
                shift += 5
                if byte < 0x20:
                    break
            delta = ~(result >> 1) if result & 1 else (result >> 1)
            if axis == 'lat':
                lat += delta
            else:
                lng += delta
        coordinates.append((lat / factor, lng / factor))
    return coordinates
def _osrm_route(origin_lat, origin_lng, dest_lat, dest_lng):
    """Ask OSRM for a driving route. Returns (path, distance_km, minutes)."""
    base = (getattr(settings, 'OSRM_BASE_URL', '') or 'https://router.project-osrm.org').rstrip('/')
    coordinates = f'{origin_lng:.6f},{origin_lat:.6f};{dest_lng:.6f},{dest_lat:.6f}'
    url = f'{base}/route/v1/driving/{coordinates}?overview=full&geometries=geojson'
    data = _get_json(url, getattr(settings, 'ROUTING_TIMEOUT', 4))
    if data.get('code') != 'Ok' or not data.get('routes'):
        return None
    route = data['routes'][0]
    path = [(point[1], point[0]) for point in route.get('geometry', {}).get('coordinates', [])]
    if len(path) < 2:
        return None
    return path, float(route['distance']) / 1000.0, float(route['duration']) / 60.0
def _valhalla_route(origin_lat, origin_lng, dest_lat, dest_lng):
    """Ask Valhalla for a walking route (covers campus footpaths a car profile
    cannot reach). Returns (path, distance_km, minutes)."""
    base = (getattr(settings, 'VALHALLA_BASE_URL', '') or 'https://valhalla1.openstreetmap.de').rstrip('/')
```

```
    payload = {
        'locations': [
            {'lat': origin_lat, 'lon': origin_lng},
            {'lat': dest_lat, 'lon': dest_lng},
        ],
        'costing': 'pedestrian',
        'directions_options': {'units': 'kilometers'},
    }
    url = f'{base}/route?json={urllib.parse.quote(json.dumps(payload))}'
    data = _get_json(url, getattr(settings, 'ROUTING_TIMEOUT', 4))
    trip = data.get('trip') or {}
    legs = trip.get('legs') or []
    if not legs or not legs[0].get('shape'):
        return None
    path = decode_polyline(legs[0]['shape'])
    if len(path) < 2:
        return None
    summary = trip.get('summary') or {}
    return path, float(summary.get('length', 0.0)), float(summary.get('time', 0.0)) / 60.0
_PROVIDERS = (_osrm_route, _valhalla_route)
def get_road_route(origin_lat, origin_lng, dest_lat, dest_lng):
    if not _routing_enabled():
        return None
    if None in (origin_lat, origin_lng, dest_lat, dest_lng):
        return None
    key = 'roadroute:{}|{}'.format(
        _coord_pair(origin_lat, origin_lng), _coord_pair(dest_lat, dest_lng))
    cached = cache.get(key, _MISS)
    if cached is not _MISS:
        return cached
    found = None
    for provider in _PROVIDERS:
        name = provider.__name__.lstrip('_').split('_')[0]
        try:
            result = provider(origin_lat, origin_lng, dest_lat, dest_lng)
        except Exception:
```

```
            result = None
        if not result:
            continue
        path, distance_km, duration_min = result
        path = path[:_MAX_POINTS]
        origin = (origin_lat, origin_lng)
        dest = (dest_lat, dest_lng)
        lead = haversine_km(origin[0], origin[1], path[0][0], path[0][1])
        tail = haversine_km(path[-1][0], path[-1][1], dest[0], dest[1])
        if lead > 0.002:  # ~0.2 m: already on the road
            path.insert(0, origin)
        if tail > 0.002:
            path.append(dest)
        found = {
            'path': [[round(lat, 6), round(lng, 6)] for lat, lng in path],
            'distance_km': distance_km + lead + tail,
            'duration_min': duration_min,
            'provider': name,
        }
        break
    cache.set(
        key, found,
        getattr(settings, 'ROUTING_CACHE_SECONDS', 600) if found else _FAIL_TTL,
    )
    return found
```

backend/deliveries/utils.py

```
import math
from django.db.models import Case, When, Value, IntegerField
from django.utils import timezone
from .models import DeliveryRequest
def pending_pool_for_rider(user):
    return (DeliveryRequest.objects
            .filter(status=DeliveryRequest.RequestStatus.SEARCHING)
            .annotate(
                pool_rank=Case(
                    When(assigned_to=user, then=Value(0)),
```

```
                    When(assigned_to__isnull=True, then=Value(1)),
                    default=Value(2),
                    output_field=IntegerField(),
                )
            )
            .order_by('pool_rank', 'requested_at'))
def _clean_amount(amount):
    import math
    if amount is None:
        return 0.0
    return round(float(amount), 2)
def delivery_fee_for_order(total_amount):
    import math
    amount = _clean_amount(total_amount)
    if amount is None:
        return 15.0
    if amount <= 0:
        return 15.0
    return float(max(15, math.ceil((amount - 1e-6) / 300) * 15))
delivery_earning_for_order = delivery_fee_for_order
def prefetch_delivery_relations(qs):
    return qs.select_related(
        'order', 'order__customer', 'rider', 'assigned_to'
    ).prefetch_related('order__items')
def serialize_delivery(d):
    return {
        'id': d.id,
        'order_number': d.order.order_number,
        'status': d.get_status_display(),
        'raw_status': d.status,
        'location': d.delivery_location,
        'rider_name': d.rider.get_full_name() or d.rider.username if d.rider else 'Searching for Rider...',
        'assigned_rider_name': (d.assigned_to.get_full_name() or d.assigned_to.username
                                if d.assigned_to else None),
        'total_amount': float(d.order.total_amount),
        'delivery_fee': float(d.order.delivery_fee),
```

```
        'total_payment': float(d.order.total_amount) + float(d.order.delivery_fee),
        'customer_name': d.order.customer.get_full_name() or d.order.customer.username if d.order.customer else
'Guest',
        'created_at': d.requested_at.strftime('%H:%M %p'),
        'rider_lat': d.rider_lat,
        'rider_lng': d.rider_lng,
        'dest_lat': d.dest_lat,
        'dest_lng': d.dest_lng,
        'location_updated_at': d.location_updated_at.strftime('%H:%M %p') if d.location_updated_at else None,
        'delivered_at': d.delivered_at.strftime('%b %d, %Y - %I:%M %p') if d.delivered_at else None,
        'items': [{'name': i.item_name, 'qty': i.quantity, 'price': float(i.price)} for i in d.order.items.all()],
        'proof_photo': d.proof_photo.url if d.proof_photo else None
    }
CAMPUS_CENTER_LAT = 9.77778
CAMPUS_CENTER_LNG = 118.73333
CAMPUS_RADIUS_KM = 0.8
def haversine_km(lat1, lng1, lat2, lng2):
    """Great-circle distance between two coordinates in kilometers."""
    if lat1 is None or lng1 is None or lat2 is None or lng2 is None:
        return 0.0
    R = 6371.0
    dlat = math.radians(lat2 - lat1)
    dlng = math.radians(lng2 - lng1)
    a = (math.sin(dlat / 2) ** 2
         + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(dlng / 2) ** 2)
    return 2 * R * math.asin(math.sqrt(a))
def is_within_campus(lat, lng):
    if lat is None or lng is None:
        return False
    return haversine_km(lat, lng, CAMPUS_CENTER_LAT, CAMPUS_CENTER_LNG) <= CAMPUS_RADIUS_KM
def distance_from_points(points):
    total = 0.0
    for i in range(1, len(points)):
        total += haversine_km(points[i - 1][0], points[i - 1][1], points[i][0], points[i][1])
    return total
TRAIL_MIN_GAP_KM = 0.006
TRAIL_MAX_POINTS = 180
```

```
def thin_trail(points, min_gap_km=TRAIL_MIN_GAP_KM, max_points=TRAIL_MAX_POINTS):
    pts = [(float(lat), float(lng)) for lat, lng in points]
    if len(pts) <= 2:
        return pts

    kept = [pts[0]]
    for point in pts[1:-1]:
        prev = kept[-1]
        if haversine_km(prev[0], prev[1], point[0], point[1]) >= min_gap_km:
            kept.append(point)
    if kept[-1] != pts[-1]:
        kept.append(pts[-1])
    if len(kept) > max_points:
        step = (len(kept) - 1) / float(max_points - 1)
        sampled = []
        for i in range(max_points):
            point = kept[min(int(round(i * step)), len(kept) - 1)]
            if not sampled or sampled[-1] != point:
                sampled.append(point)
        sampled[-1] = pts[-1]
        kept = sampled
    return kept
def compute_speed_kmh(lat1, lng1, t1, lat2, lng2, t2):
    if lat1 is None or lat2 is None:
        return 0.0
    if t1 is None or t2 is None:
        return 0.0
    dt_sec = (t2 - t1).total_seconds()
    if dt_sec <= 0:
        return 0.0
    d_km = haversine_km(lat1, lng1, lat2, lng2)
    return (d_km / dt_sec) * 3600.0
def compute_bearing(lat1, lng1, lat2, lng2):
    if lat1 is None or lng1 is None or lat2 is None or lng2 is None:
        return None
    dlat = math.radians(lat2 - lat1)
```

```
    dlon = math.radians(lng2 - lng1)
    lat1r = math.radians(lat1)
    lat2r = math.radians(lat2)
    y = math.sin(dlon) * math.cos(lat2r)
    x = (math.cos(lat1r) * math.sin(lat2r)
         - math.sin(lat1r) * math.cos(lat2r) * math.cos(dlon))
    bearing = math.degrees(math.atan2(y, x))
    return (bearing + 360) % 360
```

backend/templates/delivery_dashboard.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <link rel="icon" href="{% static 'customer_portal/images/canteen-express-logo.png' %}" type="image/png" />
    <link rel="manifest" href="{% static 'manifest-rider.json' %}">
    <meta name="theme-color" content="#FF6117">
    <script>
        if ('serviceWorker' in navigator) {
            window.addEventListener('load', () => {
                navigator.serviceWorker.register("{% static 'sw.js' %}").catch(err => console.log('SW error:',
err));
            });
        }
    </script>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Delivery Rider Hub - Canteen Express</title>
    <link rel="stylesheet" href="{% static 'css/rider.css' %}">
    <link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800;900&display=swap"
rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" />
    <!-- Leaflet powers the live route map on each job card (served locally, no CDN). -->
    <link rel="stylesheet" href="{% static 'vendor/leaflet/leaflet.css' %}" />
    <script src="{% static 'vendor/leaflet/leaflet.js' %}"></script>
</head>
```

```
<body class="bg-glow text-zinc-100 min-h-screen flex flex-col selection:bg-orange-500 selection:text-white"
x-data="riderHub()">

    <div class="fixed top-4 right-4 z-[70] space-y-2 w-80 max-w-[90vw]">

        <template x-for="toast in toasts" :key="toast.id">

            <div class="toast-enter glass-strong flex items-start gap-3 rounded-2xl p-4 shadow-glass"

                 :class="toast.type === 'success' ? '!border-green-500/40' : (toast.type === 'error' ?
'!border-red-500/40' : '!border-amber-500/40')">

                <div class="w-9 h-9 rounded-xl flex items-center justify-center shrink-0"

                     :class="toast.type === 'success' ? 'bg-green-500/20 text-green-400' : (toast.type === 'error'
? 'bg-red-500/20 text-red-400' : 'bg-amber-500/20 text-amber-400')">

                    <i :class="toast.type === 'success' ? 'fa-solid fa-circle-check' : (toast.type === 'error' ?
'fa-solid fa-circle-xmark' : 'fa-solid fa-circle-info')"></i>

                </div>

                <div class="flex-1 min-w-0">

                    <p class="text-sm font-black text-white" x-text="toast.title"></p>

                    <p class="text-xs text-zinc-400 mt-0.5" x-text="toast.desc"></p>

                </div>

                <button @click="removeToast(toast.id)" class="text-zinc-500 hover:text-white"><i class="fa-solid
fa-xmark"></i></button>

            </div>

        </template>

    </div>

    <!-- ============ HEADER (glass) ============ -->

    <header class="glass sticky top-0 z-40 shrink-0 border-b border-rider-border">

        <div class="max-w-7xl mx-auto px-4 py-3.5 space-y-3">

            <!-- Row 1: identity + status -->

            <div class="flex items-center justify-between gap-3">

                <div class="flex items-center gap-3 min-w-0">

                    <div class="w-11 h-11 rounded-2xl bg-gradient-to-br from-orange-400 via-orange-500
to-orange-600 text-white flex items-center justify-center font-black text-xl shadow-glow-orange shrink-0">

                        <i class="fa-solid fa-motorcycle"></i>

                    </div>

                    <div class="min-w-0">

                        <h1 class="text-sm sm:text-lg font-black text-white leading-tight truncate">Delivery Rider
Hub</h1>

                        <p class="text-[11px] font-bold uppercase tracking-wider mt-0.5"

                           :class="isAvailable ? 'text-green-400' : 'text-zinc-500'">

                            <span x-text="isAvailable ? 'Online & Ready' : 'Offline'"></span>

                            <span class="text-zinc-600">•</span>
```

```
                        <span class="text-zinc-400 truncate">{{
request.user.get_full_name|default:request.user.first_name|default:request.user.username }}</span>

                    </p>

                </div>

            </div>



            <div class="flex items-center gap-2 shrink-0">

                <button @click="showChangePasswordModal = true" class="bg-zinc-800 hover:bg-zinc-700
text-zinc-200 border border-zinc-700 w-10 h-10 rounded-xl text-xs font-bold transition flex items-center
justify-center" title="Change Password">

                    <i class="fa-solid fa-key"></i>

                </button>

                <a href="{% url 'accounts:delivery_logout' %}" class="bg-red-500/10 hover:bg-red-500/25
text-red-400 border border-red-500/30 backdrop-blur-md px-3 h-10 rounded-xl text-xs font-bold transition flex
items-center justify-center gap-1.5" title="Logout">

                    <i class="fa-solid fa-right-from-bracket"></i>

                    <span>Logout</span>

                </a>

            </div>

        </div>

        <div class="flex items-center justify-between gap-3 border-t border-white/5 pt-3">

            <button @click="toggleAvailability()" class="flex items-center gap-3 pl-3 pr-2.5 py-2 rounded-xl
border backdrop-blur-md transition active:scale-95"

                :class="isAvailable ? 'bg-green-500/10 border-green-500/30' : 'bg-zinc-800/60
border-rider-border'">

                <span class="flex items-center gap-2">

                    <span class="relative flex h-2.5 w-2.5">

                        <span x-show="isAvailable" class="animate-ping absolute inline-flex h-full w-full
rounded-full bg-green-400 opacity-75"></span>

                        <span class="relative inline-flex rounded-full h-2.5 w-2.5" :class="isAvailable ?
'bg-green-500' : 'bg-zinc-500'"></span>

                    </span>

                    <span class="text-[11px] font-black uppercase tracking-wider"

                          :class="isAvailable ? 'text-green-400' : 'text-zinc-400'" x-text="isAvailable ?
'Online' : 'Offline'"></span>

                </span>

                <span class="relative inline-flex items-center h-6 w-11 rounded-full transition"

                      :class="isAvailable ? 'bg-green-500 shadow-glow-green' : 'bg-zinc-700'">

                    <span class="inline-block w-4 h-4 transform rounded-full bg-white shadow transition"

                          :class="isAvailable ? 'translate-x-6' : 'translate-x-1'"></span>

                </span>
```

```
                </button>

                <a href="{% url 'deliveries:history' %}" class="bg-zinc-800/60 hover:bg-zinc-700/60 text-zinc-300
border border-white/10 backdrop-blur-md px-4 py-2.5 rounded-xl text-xs font-bold transition flex items-center gap-2
active:scale-95">

                    <i class="fa-solid fa-clock-rotate-left text-orange-400"></i> History

                </a>

            </div>

        </div>

    </header>

    <main class="flex-1 max-w-7xl mx-auto w-full px-4 py-6 space-y-6 sm:pb-6 pb-28">

        <div class="grid grid-cols-3 gap-2 sm:gap-3">

            <div class="card-lift bg-gradient-to-br from-orange-500 via-orange-600 to-orange-700 rounded-xl px-3
py-2.5 shadow-glow-orange">

                <div class="flex items-center gap-2">

                    <div class="w-7 h-7 rounded-lg bg-white/20 backdrop-blur-md text-white flex items-center
justify-center text-xs font-black shrink-0">₱</div>

                    <div class="min-w-0">

                        <span class="block text-[8px] sm:text-[9px] font-extrabold text-orange-100 uppercase
tracking-wider truncate">Earnings</span>

                        <span class="block text-sm sm:text-base font-black text-white truncate">₱<span
id="earnings-total">0</span></span>

                    </div>

                </div>

            </div>

            <a href="{% url 'deliveries:history' %}" class="card-lift glass rounded-xl px-3 py-2.5
hover:!border-green-500/40">

                <div class="flex items-center gap-2">

                    <div class="w-7 h-7 rounded-lg bg-green-500/15 text-green-400 flex items-center justify-center
text-xs shrink-0"><i class="fa-solid fa-circle-check"></i></div>

                    <div class="min-w-0">

                        <span class="block text-[8px] sm:text-[9px] font-extrabold text-zinc-400 uppercase
tracking-wider truncate">Delivered</span>

                        <span class="block text-sm sm:text-base font-black text-green-400"><span
id="delivered-count">{{ completed_count }}</span></span>

                    </div>

                </div>

            </a>

            <div class="card-lift glass rounded-xl px-3 py-2.5">

                <div class="flex items-center gap-2">

                    <div class="w-7 h-7 rounded-lg bg-orange-500/20 text-orange-400 flex items-center
justify-center text-xs shrink-0"><i class="fa-solid fa-person-biking"></i></div>
```

```
                    <div class="min-w-0">

                        <span class="block text-[8px] sm:text-[9px] font-extrabold text-zinc-400 uppercase
tracking-wider truncate">Active</span>

                        <span class="block text-sm sm:text-base font-black text-orange-400">{{
active_deliveries_count }}/3</span>

                    </div>

                </div>

            </div>

        </div>

        {% if not is_available %}

        <div class="glass-strong rounded-2xl p-4 text-xs font-bold flex items-center gap-3 !border-amber-500/30">

            <i class="fa-solid fa-wifi text-lg text-amber-400"></i>

            <span class="text-amber-300">You're <span class="text-white">offline</span>. Go <span
class="text-green-400">Online</span> to receive new requests.</span>

        </div>

        {% endif %}

        <div class="glass rounded-2xl p-1.5 w-full sm:w-fit flex items-center gap-1.5 shadow-glass">

            <button @click="activeTab = 'incoming'"

                    class="flex-1 sm:flex-none flex items-center justify-center gap-2 px-4 py-2.5 rounded-xl
text-xs font-black transition active:scale-95"

                    :class="activeTab === 'incoming' ? 'bg-gradient-to-r from-orange-500 to-orange-600 text-white
shadow-glow-orange' : 'text-zinc-400 hover:text-white hover:bg-white/5'">

                <i class="fa-solid fa-bell"></i>

                Incoming

                <span class="px-1.5 py-0.5 rounded-md text-[9px] font-black"

                      :class="activeTab === 'incoming' ? 'bg-white/25 text-white' : 'bg-orange-500/20
text-orange-400'">{{ reachable_loc_count }}</span>

            </button>

            <button @click="activeTab = 'active'"

                    class="flex-1 sm:flex-none flex items-center justify-center gap-2 px-4 py-2.5 rounded-xl
text-xs font-black transition active:scale-95"

                    :class="activeTab === 'active' ? 'bg-gradient-to-r from-green-500 to-green-600 text-white
shadow-glow-green' : 'text-zinc-400 hover:text-white hover:bg-white/5'">

                <i class="fa-solid fa-person-biking"></i>

                My Deliveries

                <span class="px-1.5 py-0.5 rounded-md text-[9px] font-black"

                      :class="activeTab === 'active' ? 'bg-white/25 text-white' : 'bg-green-500/20
text-green-400'">{{ in_transit_deliveries|length }}</span>

            </button>

        </div>
```

```
        <div x-show="activeTab === 'incoming'" x-cloak x-transition.opacity class="tab-fade">

            <div class="space-y-4">

                <div class="flex items-center justify-between">

                    <h3 class="text-xs font-black text-zinc-400 uppercase tracking-wider flex items-center gap-2">

                        <i class="fa-solid fa-inbox text-orange-400"></i> Incoming Requests

                    </h3>

                    <span class="text-[10px] text-zinc-500 font-bold uppercase tracking-wider"><i class="fa-solid
fa-clock mr-1 text-orange-400"></i>Offers rotate if not accepted</span>                </div>

                {% if active_deliveries_count >= 3 %}

                <div class="p-3 bg-amber-500/10 border border-amber-500/30 rounded-2xl text-amber-400 text-xs
font-bold text-center">

                    <i class="fa-solid fa-triangle-exclamation mr-1"></i> Max 3 active deliveries reached. Complete
one first.

                </div>

                {% endif %}

                {% include "partials/pending_delivery_cards.html" %}

            </div>

        </div>

        <div x-show="activeTab === 'active'" x-cloak x-transition.opacity class="tab-fade">

            <div class="space-y-6">

                <div class="flex items-center justify-between">

                    <h3 class="text-xs font-black text-zinc-400 uppercase tracking-wider flex items-center gap-2">

                        <i class="fa-solid fa-person-biking text-green-400"></i> Active Deliveries

                    </h3>

                    <span class="text-[10px] text-green-400 font-bold uppercase tracking-wider">{{
in_transit_deliveries|length }} in transit • {{ completed_deliveries|length }} delivered</span>

                </div>

                <div class="space-y-3">

                    <h4 class="flex items-center gap-2 text-[10px] font-black text-zinc-500 uppercase
tracking-wider">

                        <i class="fa-solid fa-route text-green-400"></i> Being Delivered

                        <span class="px-1.5 py-0.5 rounded-md bg-green-500/20 text-green-400 text-[9px]
font-black">{{ in_transit_deliveries|length }}</span>

                    </h4>

                    <div class="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-4">

                        {% for d in in_transit_deliveries %}

                        <div class="card-lift glass rounded-3xl p-6 space-y-4 shadow-glass
hover:!border-green-500/40 flex flex-col !border-green-500/30" data-location-id="{{ d.id }}">

                            <div class="flex justify-between items-start border-b border-white/5 pb-3">
```

```
                                <button @click="openDetail({{ d.id }})" class="text-left group" title="View order
details">

                                    <span class="text-xs font-black text-green-400 uppercase tracking-wider
group-hover:underline">Order {{ d.order.order_number }} <i class="fa-solid fa-up-right-from-square text-[9px]
opacity-60"></i></span>

                                    <h4 class="text-base font-black text-white mt-0.5"><i class="fa-solid
fa-building text-green-400 mr-1"></i> {{ d.delivery_location }}</h4>

                                    <p class="text-[11px] text-zinc-400 flex items-center gap-1.5 mt-1">

                                        <i class="fa-solid fa-user-tie text-[9px] text-green-400/70"></i>

                                        Customer: <span class="font-bold text-white">{% if d.order.customer %}{{
d.order.customer.get_full_name|default:d.order.customer.username }}{% elif d.order.guest_name %}{{
d.order.guest_name }}{% else %}Guest{% endif %}</span>

                                    </p>

                                </button>

                                <span class="bg-green-500/15 text-green-400 border border-green-500/30 text-[10px]
font-extrabold px-2.5 py-1 rounded-full uppercase shrink-0"

                                  data-delivery-status="{{ d.raw_status }}"

                                  x-text="liveStatus({{ d.id }})"></span>

                            </div>

                            {% if d.dest_lat and d.dest_lng %}

                            <div class="space-y-1.5">

                                <p class="text-zinc-500 font-bold uppercase tracking-wider text-[10px] flex
items-center gap-1.5">

                                    <i class="fa-solid fa-route text-orange-400"></i> Live Route to Customer

                                    <span id="route-meta-{{ d.id }}" class="ml-auto text-[10px] font-black
text-orange-400 normal-case tracking-normal tabular-nums">Locating…</span>

                                </p>

                                <div class="live-route-map w-full h-44 rounded-2xl overflow-hidden border
border-white/10 bg-zinc-900"

                                     data-route-id="{{ d.id }}"

                                     data-route-url="{% url 'deliveries:api_live_route' d.id %}"

                                     data-dest-lat="{{ d.dest_lat }}"

                                     data-dest-lng="{{ d.dest_lng }}"

                                     data-dest-label="{{ d.delivery_location|escapejs }}"></div>

                            </div>

                            {% endif %}

                            <div class="text-xs space-y-1.5 text-zinc-300">

                                <p class="text-zinc-500 font-bold uppercase tracking-wider text-[10px]">Items</p>

                                {% for item in d.order.items.all %}

                                    <div class="flex items-center justify-between gap-2">

                                        <span>• {{ item.quantity }}x {{ item.item_name }}</span>
```

```
                                    <span class="text-zinc-400 font-bold shrink-0">₱{{ item.total_price
}}</span>

                                </div>

                            {% endfor %}

                        </div>

                        <div class="pt-2 text-xs space-y-1 text-zinc-400">

                            <div class="flex justify-between items-center">

                                <span>Subtotal</span>

                               <span class="font-bold text-zinc-300">₱{{ d.order.total_amount }}</span>

                            </div>

                            <div class="flex justify-between items-center">

                                <span>Delivery Fee (Rider)</span>

                                <span class="font-black text-orange-400">₱{{ d.order.delivery_fee }}</span>

                            </div>

                            <div class="flex justify-between items-center pt-1 border-t border-white/5">

                                <span class="text-zinc-500 font-bold uppercase tracking-wider">Total
Payment</span>

                                <span class="font-black text-orange-400 bg-orange-500/15 border
border-orange-500/30 px-2.5 py-0.5 rounded-lg">₱{{ d.order.total_payment }}</span>

                            </div>

                        </div>

                        <div class="pt-4 border-t border-white/5 mt-auto flex flex-col gap-2">

                            <div class="grid grid-cols-3 gap-2">

                                <button @click="openChat({{ d.id }}, '{{ d.order.order_number }}', '{% if
d.order.customer %}{{ d.order.customer.get_full_name|default:d.order.customer.username|escapejs }}{% elif
d.order.guest_name %}{{ d.order.guest_name|escapejs }}{% else %}Guest{% endif %}')" class="relative bg-zinc-800/60
hover:bg-zinc-700/80 text-white font-bold px-2 py-2.5 rounded-xl text-xs transition flex items-center
justify-center gap-1.5 border border-white/10 active:scale-95">

                                    <i class="fa-solid fa-comments text-green-400"></i> Chat

                                    <span x-show="(unread[{{ d.id }}] || 0) > 0"

                                          x-text="unread[{{ d.id }}] || 0"

                                          class="absolute -top-2 -right-2 bg-red-500 text-white text-[9px]
font-black min-w-[18px] h-[18px] px-1 rounded-full flex items-center justify-center border border-white/30"></span>

                                </button>

                                <a href="{% url 'deliveries:track_order' d.id %}" class="bg-zinc-800/60
hover:bg-zinc-700/80 text-white font-bold px-2 py-2.5 rounded-xl text-xs transition flex items-center
justify-center gap-1.5 border border-white/10 active:scale-95">

                                    <i class="fa-solid fa-location-crosshairs text-orange-400"></i> Track

                                </a>

                                <button @click="confirmAction('{% url 'deliveries:cancel_delivery' d.id %}',
'Release this delivery?', 'It will be returned to the pool for another rider.', 'release')"
```

```
                                        class="bg-red-500/10 hover:bg-red-500/25 text-red-400 border
border-red-500/30 px-2 py-2.5 rounded-xl text-xs font-bold transition active:scale-95 flex items-center
justify-center gap-1.5" title="Release back to pool">

                                    <i class="fa-solid fa-rotate-left"></i> Release

                                </button>

                            </div>

                            <button type="button"

                                    onclick="openCameraCapture('{% url 'deliveries:complete_delivery' d.id
%}')"

                                    class="btn-shine w-full bg-gradient-to-r from-green-500 to-green-600
hover:from-green-600 hover:to-green-700 text-white font-black px-3.5 py-3 rounded-xl text-xs shadow-glow-green
transition active:scale-95 flex items-center justify-center gap-1.5" title="Capture a live photo as proof of
delivery (gallery uploads are not accepted)">

                                <i class="fa-solid fa-camera"></i> Delivered + Photo

                            </button>

                        </div>

                    </div>

                    {% empty %}

                    <div class="col-span-full glass rounded-3xl p-16 text-center text-zinc-500 space-y-2">

                        <i class="fa-solid fa-box-open text-4xl mb-2 text-zinc-600"></i>

                        <p class="text-xs font-bold">No active deliveries yet.</p>

                        <p class="text-[10px] text-zinc-600">Accept a request from the Incoming tab to start
earning.</p>

                    </div>

                    {% endfor %}

                </div>

            </div>

            {% if completed_deliveries %}

            <div class="space-y-3">

                <h4 class="flex items-center gap-2 text-[10px] font-black text-zinc-500 uppercase
tracking-wider">

                    <i class="fa-solid fa-circle-check text-emerald-400"></i> Delivered

                    <span class="px-1.5 py-0.5 rounded-md bg-emerald-500/20 text-emerald-400 text-[9px]
font-black">{{ completed_deliveries|length }}</span>

                </h4>

                <div class="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-4">

                    {% for d in completed_deliveries %}

                    <div class="card-lift glass rounded-3xl p-6 space-y-4 shadow-glass
hover:!border-emerald-500/30 flex flex-col opacity-70 !border-emerald-500/20">

                        <div class="flex justify-between items-start border-b border-white/5 pb-3">
```

```
                                <button @click="openDetail({{ d.id }})" class="text-left group" title="View order
details">

                                    <span class="text-xs font-black text-emerald-400 uppercase tracking-wider
group-hover:underline">Order {{ d.order.order_number }} <i class="fa-solid fa-up-right-from-square text-[9px]
opacity-60"></i></span>

                                    <h4 class="text-base font-black text-white mt-0.5"><i class="fa-solid
fa-building text-emerald-400 mr-1"></i> {{ d.delivery_location }}</h4>

                                </button>

                                <span class="bg-emerald-500/15 text-emerald-400 border border-emerald-500/30
text-[10px] font-extrabold px-2.5 py-1 rounded-full uppercase shrink-0">Delivered <i class="fa-solid fa-check
ml-0.5"></i></span>

                            </div>

                            <div class="flex items-center justify-between text-[11px] text-zinc-500">

                                <span><i class="fa-regular fa-clock mr-1.5"></i>{{ d.delivered_at|date:"d M h:i
A"|default:"Completed" }}</span>

                                <span class="font-black text-emerald-400"><i class="fa-solid fa-wallet
mr-1"></i>+₱{{ d.order.delivery_fee }}</span>

                            </div>

                            <div class="pt-2 text-xs space-y-1 text-zinc-400 border-t border-white/5">

                                <div class="flex justify-between items-center">

                                    <span>Subtotal</span>

                                    <span class="font-bold text-zinc-300">₱{{ d.order.total_amount }}</span>

                                </div>

                                <div class="flex justify-between items-center">

                                    <span>Delivery Fee (Rider)</span>

                                    <span class="font-black text-orange-400">₱{{ d.order.delivery_fee }}</span>

                                </div>

                                <div class="flex justify-between items-center pt-1 border-t border-white/5">

                                    <span class="text-zinc-500 font-bold uppercase tracking-wider">Total
Payment</span>

                                    <span class="font-black text-white">₱{{ d.order.total_payment }}</span>

                                </div>

                            </div>

                            <div class="pt-3 border-t border-white/5 flex items-center justify-between gap-2
mt-auto">

                                <span class="text-[11px] font-bold text-emerald-400"><i class="fa-solid
fa-circle-check mr-1.5"></i>Completed</span>

                                <button @click="openDetail({{ d.id }})" class="bg-zinc-800/60 hover:bg-zinc-700/80
text-zinc-300 border border-rider-border px-3 py-2 rounded-xl text-xs font-bold transition active:scale-95"
title="View details">

                                    <i class="fa-solid fa-receipt"></i> Details

                                </button>
```

```
                            </div>

                        </div>

                        {% endfor %}

                    </div>

                </div>

                {% endif %}

            </div>

        </div>

    </main>

    <footer class="sm:hidden glass-strong fixed bottom-0 left-0 right-0 z-50 shadow-glass border-t
border-white/10">

        <div class="max-w-7xl mx-auto px-4 pb-[env(safe-area-inset-bottom)] flex items-stretch justify-around">

            <button @click="activeTab = 'incoming'"

                    class="flex flex-col items-center justify-center gap-1 py-3.5 px-4 min-w-[72px] transition"

                    :class="activeTab === 'incoming' ? 'text-white' : 'text-zinc-500'">

                <span class="w-11 h-8 rounded-xl flex items-center justify-center transition"

                      :class="activeTab === 'incoming' ? 'bg-gradient-to-r from-orange-500 to-orange-600
shadow-glow-orange' : ''">

                    <i class="fa-solid fa-inbox text-xl" :class="activeTab === 'incoming' ? 'scale-110' : ''"></i>

                </span>

                <span class="text-[11px] font-black uppercase tracking-wide">Incoming</span>

            </button>

            <button @click="activeTab = 'active'"

                    class="flex flex-col items-center justify-center gap-1 py-3.5 px-4 min-w-[72px] transition"

                    :class="activeTab === 'active' ? 'text-white' : 'text-zinc-500'">

                <span class="w-11 h-8 rounded-xl flex items-center justify-center transition"

                      :class="activeTab === 'active' ? 'bg-gradient-to-r from-green-500 to-green-600
shadow-glow-green' : ''">

                    <i class="fa-solid fa-person-biking text-xl" :class="activeTab === 'active' ? 'scale-110' :
''"></i>

                </span>

                <span class="text-[11px] font-black uppercase tracking-wide">Delivering</span>

            </button>

        </div>

    </footer>

    <div x-show="detailOpen" x-cloak class="fixed inset-0 z-50 glass-backdrop flex items-center justify-center p-4"
@click.self="closeDetail()">

        <div class="glass-strong rounded-3xl max-w-md w-full overflow-hidden shadow-glass flex flex-col
max-h-[85vh]">
```

```
            <div class="border-b border-white/10 p-4 flex items-center justify-between">

                <div>

                    <h3 class="font-black text-white text-sm" x-text="'Order ' + (detailData.order_number ||
'')"></h3>

                    <p class="text-[10px] text-zinc-400 font-bold" x-text="detailData.delivery_location ||
'Delivery order'"></p>

                </div>

                <button @click="closeDetail()" class="text-zinc-400 hover:text-white p-1.5"><i class="fa-solid
fa-xmark text-lg"></i></button>

            </div>

            <div class="flex-1 p-4 overflow-y-auto space-y-3 bg-black/20">

                <template x-if="detailData.customer_name">

                    <div class="glass rounded-2xl p-3 flex items-center gap-2.5 border-l-2
!border-l-orange-500/60">

                        <div class="w-9 h-9 rounded-xl bg-orange-500/15 text-orange-400 flex items-center
justify-center shrink-0">

                            <i class="fa-solid fa-user"></i>

                        </div>

                        <div class="min-w-0">

                            <p class="text-[9px] text-zinc-500 uppercase tracking-widest font-bold">Customer</p>

                            <p class="text-sm font-black text-white truncate"
x-text="detailData.customer_name"></p>

                        </div>

                        <a x-show="detailData.customer_phone" :href="'tel:' + detailData.customer_phone"
class="ml-auto text-[10px] font-black bg-orange-500/15 text-orange-400 border border-orange-500/30 px-2.5 py-1.5
rounded-lg flex items-center gap-1 shrink-0">

                            <i class="fa-solid fa-phone"></i> Call

                        </a>

                    </div>

                </template>

                <div class="space-y-1.5">

                    <template x-for="item in detailData.items" :key="item.name + item.qty">

                        <div class="flex items-center justify-between gap-2 text-xs">

                            <span class="text-zinc-300"><b class="text-white" x-text="item.qty + 'x'"></b> <span
x-text="item.name"></span></span>

                            <span class="text-zinc-400 font-bold shrink-0" x-text="'₱' + item.total"></span>

                        </div>

                    </template>

                </div>

                <div class="rounded-xl bg-black/30 border border-white/5 p-3 space-y-1.5">
```

```
                    <div class="flex items-center justify-between text-xs">
                        <span class="text-zinc-400 font-bold">Subtotal</span>
                        <span class="text-zinc-300 font-bold" x-text="'₱' + detailData.subtotal"></span>
                    </div>
                    <div class="flex items-center justify-between text-xs">
                        <span class="text-zinc-400 font-bold flex items-center gap-1.5">
                            <i class="fa-solid fa-motorcycle text-orange-400"></i> Delivery Fee
                            <span class="text-[8px] text-orange-400/70 uppercase tracking-widest font-black">(Rider earning)</span>
                        </span>
                        <span class="text-orange-400 font-black" x-text="'₱' + detailData.delivery_fee"></span>
                    </div>
                    <div class="pt-2 border-t border-white/10 flex items-center justify-between">
                        <span class="text-[10px] text-zinc-500 font-bold uppercase tracking-wider">Total Payment</span>
                        <span class="text-lg font-black text-orange-400" x-text="'₱' + detailData.total_payment"></span>
                    </div>
                </div>
                <p class="text-[10px] text-zinc-500 font-bold flex items-center gap-1"><i class="fa-solid fa-calendar-day"></i> <span x-text="detailData.created_at"></span></p>
            </div>
        </div>
    </div>
    <div x-show="confirmOpen" x-cloak class="fixed inset-0 z-[60] glass-backdrop flex items-center justify-center p-4">
        <div class="glass-strong rounded-3xl max-w-sm w-full p-6 shadow-glass text-center space-y-4">
            <div class="w-14 h-14 rounded-2xl mx-auto flex items-center justify-center text-xl"
                 :class="confirmData.type === 'release' ? 'bg-red-500/15 text-red-400' : (confirmData.type === 'reject' ? 'bg-zinc-800 text-zinc-400' : 'bg-orange-500/15 text-orange-400')">
                <i :class="confirmData.type === 'release' ? 'fa-solid fa-rotate-left' : (confirmData.type === 'reject' ? 'fa-solid fa-xmark' : 'fa-solid fa-triangle-exclamation')"></i>
            </div>
            <div>
                <h3 class="font-black text-white text-base" x-text="confirmData.title"></h3>
                <p class="text-xs text-zinc-400 mt-1" x-text="confirmData.desc"></p>
            </div>
            <div class="flex items-center gap-2">
                <button @click="closeConfirm()" class="flex-1 bg-zinc-800/60 hover:bg-zinc-700/80 text-zinc-300 border border-white/10 font-bold py-2.5 rounded-xl text-xs transition active:scale-95">Cancel</button>
```

```
                <button @click="doConfirm()" class="flex-1 btn-shine font-black py-2.5 rounded-xl text-xs
transition active:scale-95 text-white"

                        :class="confirmData.type === 'release' ? 'bg-gradient-to-r from-red-500 to-red-600' :
(confirmData.type === 'reject' ? 'bg-gradient-to-r from-zinc-600 to-zinc-700' : 'bg-gradient-to-r from-orange-500
to-orange-600')">

                    <span x-text="confirmData.type === 'release' ? 'Release' : (confirmData.type === 'reject' ?
'Reject' : 'Confirm')"></span>

                </button>

            </div>

        </div>

    </div>

    <div id="photo-capture-modal" class="fixed inset-0 z-[70] glass-backdrop hidden flex items-center
justify-center p-4">

        <div class="glass-strong rounded-3xl max-w-md w-full overflow-hidden shadow-glass flex flex-col
max-h-[88vh]">

            <div class="border-b border-white/10 p-4 flex items-center justify-between">

                <div>

                    <h3 class="font-black text-white text-sm">Capture Proof of Delivery</h3>

                    <p class="text-[10px] text-green-400 font-bold">Live camera photo only - gallery uploads not
accepted</p>

                </div>

                <button onclick="closePhotoCapture()" class="text-zinc-400 hover:text-white p-1.5"><i
class="fa-solid fa-xmark text-lg"></i></button>

            </div>

            <div class="flex-1 p-4 overflow-y-auto space-y-3 bg-black/20">

                <div id="capture-live-wrap" class="relative rounded-2xl overflow-hidden bg-black/60 aspect-[4/3]">

                    <video id="camera-live" autoplay playsinline muted class="w-full h-full object-cover"></video>

                    <div class="absolute inset-0 flex items-center justify-center pointer-events-none">

                        <div class="w-10 h-10 text-zinc-400/80">

                            <i class="fa-solid fa-camera-rotate text-4xl"></i>

                        </div>

                    </div>

                </div>

                <canvas id="camera-snapshot" class="hidden w-full rounded-2xl border border-white/10
bg-black/40"></canvas>

                <div id="capture-error-box" class="hidden rounded-xl bg-red-500/10 border border-red-500/30 p-3
text-xs text-red-400 font-bold"></div>

                <p class="text-[10px] text-zinc-500 font-bold text-center">The photo is taken from your device
camera in real time.</p>

            </div>

            <div class="p-4 border-t border-white/10 flex items-center gap-2">
```

```
                <button id="capture-retake-btn" onclick="retakePhoto()" class="hidden bg-zinc-800/60
hover:bg-zinc-700/80 text-zinc-300 border border-white/10 font-bold px-3.5 py-2.5 rounded-xl text-xs transition
active:scale-95">

                    <i class="fa-solid fa-rotate-left mr-1"></i> Retake

                </button>

                <button id="capture-shoot-btn" onclick="shootPhoto()" class="flex-1 btn-shine bg-gradient-to-r
from-green-500 to-green-600 hover:from-green-600 hover:to-green-700 text-white font-black px-3.5 py-2.5 rounded-xl
text-xs shadow-glow-green transition active:scale-95 flex items-center justify-center gap-1.5">

                    <i class="fa-solid fa-camera"></i> Capture Photo

                </button>

                <button id="capture-submit-btn" onclick="submitCapturedPhoto()" class="hidden flex-1
bg-gradient-to-r from-orange-500 to-orange-600 hover:from-orange-600 hover:to-orange-700 text-white font-black
px-3.5 py-2.5 rounded-xl text-xs shadow-glow-orange transition active:scale-95 flex items-center justify-center
gap-1.5">

                    <i class="fa-solid fa-check"></i> Complete Delivery

                </button>

                <button onclick="closePhotoCapture()" class="bg-red-500/15 hover:bg-red-500/30 text-red-400 border
border-red-500/30 px-3 py-2.5 rounded-xl text-xs font-bold transition active:scale-95">

                    Cancel

                </button>

            </div>

        </div>

    </div>

    <div x-show="chatOpen" x-cloak class="fixed inset-0 z-50 glass-backdrop flex items-center justify-center p-4">

        <div class="glass-chat rounded-3xl max-w-md w-full overflow-hidden flex flex-col h-[540px]
border-green-500/20">

            <div class="border-b border-white/10 p-4 flex items-center justify-between">

                <div>

                    <h3 class="font-black text-white text-sm" x-text="'Chat - Order ' + currentOrderNo"></h3>

                    <p class="text-[10px] text-green-400 font-bold" x-text="'Messaging with ' +
(currentCustomerName || 'Customer')"></p>

                </div>

                <button @click="closeChat()" class="text-zinc-400 hover:text-white p-1.5"><i class="fa-solid
fa-xmark text-lg"></i></button>

            </div>

            <div id="chat-messages-box" class="flex-1 p-4 overflow-y-auto space-y-3 bg-white/[0.03] border-y
border-white/5">

                <template x-for="msg in messages" :key="msg.id">

                    <div :class="msg.is_me ? 'flex justify-end' : 'flex justify-start'">

                        <div :class="'max-w-[82%] px-3.5 py-2 rounded-2xl text-xs text-white ' + (msg.is_me

                            ? 'bg-gradient-to-br from-green-500 to-green-600 rounded-tr-none shadow-glow-green'
```

```
                            : 'bg-zinc-800 border border-white/10 rounded-tl-none')">

                            <p class="font-bold text-[9px] mb-0.5" :class="msg.is_me ? 'text-green-200' :
'text-orange-400'">

                                <span x-text="msg.is_me ? 'You' : ((msg.sender_name ? msg.sender_name : (msg.sender
|| 'Customer')) + ' (' + senderRoleLabel(msg.sender_role) + ')')"></span>

                                <span class="opacity-60" x-text="msg.time ? ' • ' + msg.time : ' • now'"></span>

                            </p>

                            <p class="whitespace-pre-wrap break-words" x-text="msg.message"></p>

                        </div>

                    </div>

                </template>

                <div x-show="messages.length === 0" class="text-center py-20 text-zinc-500 text-xs">

                    <i class="fa-solid fa-comments text-3xl mb-2 text-zinc-600 block"></i>

                    No messages yet. Send a message to coordinate delivery!

                </div>

            </div>

            <div class="px-3 pt-2 bg-black/30 flex gap-1.5 overflow-x-auto no-scrollbar whitespace-nowrap border-t
border-white/5">

                <button @click="newMessage = 'On my way to your location 🏍️'; sendMessage()" class="px-2.5 py-1
bg-white/5 hover:bg-green-500/20 border border-white/10 hover:border-green-500/40 text-white/80 hover:text-white
rounded-full text-[10px] font-bold transition">🏍️ On my way</button>

                <button @click="newMessage = 'Arrived at your building 🏢'; sendMessage()" class="px-2.5 py-1
bg-white/5 hover:bg-green-500/20 border border-white/10 hover:border-green-500/40 text-white/80 hover:text-white
rounded-full text-[10px] font-bold transition">🏢 Arrived</button>

                <button @click="newMessage = 'Please meet me at the lobby 📍'; sendMessage()" class="px-2.5 py-1
bg-white/5 hover:bg-green-500/20 border border-white/10 hover:border-green-500/40 text-white/80 hover:text-white
rounded-full text-[10px] font-bold transition">📍 At lobby</button>

                <button @click="newMessage = 'Thank you! Enjoy your meal 🍽️'; sendMessage()" class="px-2.5 py-1
bg-white/5 hover:bg-green-500/20 border border-white/10 hover:border-green-500/40 text-white/80 hover:text-white
rounded-full text-[10px] font-bold transition">🍽️ Enjoy meal</button>

            </div>

            <div class="p-3 border-t border-white/10 flex items-center gap-2">

                <input type="text" x-model="newMessage" @keyup.enter="sendMessage()" placeholder="Type your
message..."

                    class="flex-1 px-4 py-3 glass rounded-xl text-xs text-white placeholder-zinc-500
focus:outline-none focus:border-orange-500/50 font-medium">

                <button @click="sendMessage()" class="btn-shine bg-gradient-to-r from-green-500 to-green-600
hover:from-green-600 hover:to-green-700 text-white px-5 py-3 rounded-xl text-xs font-black shadow-glow-green
transition active:scale-95">

                    <i class="fa-solid fa-paper-plane"></i>

                </button>

            </div>

        </div>
```

```
    </div>

    <script src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js" defer></script>

    <div x-show="showChangePasswordModal" class="fixed inset-0 z-50 flex items-center justify-center bg-black/70
backdrop-blur-md p-4" x-cloak>

        <div class="glass-strong rounded-3xl max-w-md w-full p-6 shadow-glass space-y-4 text-zinc-100 border
border-white/10" @click.outside="showChangePasswordModal = false">

            <div class="flex items-center justify-between pb-3 border-b border-white/10">

                <h3 class="text-base font-black flex items-center gap-2"><i class="fa-solid fa-key
text-orange-400"></i> Change Password</h3>

                <button @click="showChangePasswordModal = false" class="text-zinc-400 hover:text-white font-bold
text-sm">✕</button>

            </div>

            <form @submit.prevent="

                fetch('{% url "accounts:change_password" %}', {

                    method: 'POST',

                    headers: {

                        'Content-Type': 'application/json',

                        'X-CSRFToken': '{{ csrf_token }}'

                    },

                    body: JSON.stringify({

                        current_password: currentPassword,

                        new_password: newPassword,

                        confirm_password: confirmPassword

                    })

                })

                .then(async res => {

                    const data = await res.json().catch(() => ({ success: false, error: 'Invalid server response'
}));

                    if (res.ok && data.success) {

                        addToast('success', 'Success', 'Password successfully updated!');

                        showChangePasswordModal = false;

                        currentPassword = '';

                        newPassword = '';

                        confirmPassword = '';

                    } else {

                        addToast('error', 'Error', data.error || 'Failed to update password');

                    }

                })
```

```
                .catch(err => addToast('error', 'Error', 'Error connecting to server: ' + err.message))

            " class="space-y-4">

                <div>

                    <label class="block text-xs font-bold text-zinc-400 mb-1">Current Password</label>

                    <div class="relative">

                        <input type="password" id="rider-cp-current" x-model="currentPassword" required
placeholder="••••••••" style="color: #111827 !important; background-color: #ffffff !important;" class="w-full
px-3.5 py-2.5 pr-11 bg-white border border-zinc-300 rounded-xl text-xs text-zinc-900 font-bold focus:ring-2
focus:ring-orange-500 focus:outline-none">

                        <button type="button" onclick="togglePassword('rider-cp-current', this)" title="Show/Hide
password" class="absolute right-2.5 top-1/2 -translate-y-1/2 text-zinc-500 hover:text-zinc-800 cursor-pointer
text-base p-1.5 bg-zinc-100 hover:bg-zinc-200 rounded-lg active:scale-90 transition">

                            <i class="fa-solid fa-eye"></i>

                        </button>

                    </div>

                </div>

                <div>

                    <label class="block text-xs font-bold text-zinc-400 mb-1">New Password</label>

                    <div class="relative">

                        <input type="password" id="rider-cp-new" x-model="newPassword" required
placeholder="••••••••" style="color: #111827 !important; background-color: #ffffff !important;" class="w-full
px-3.5 py-2.5 pr-11 bg-white border border-zinc-300 rounded-xl text-xs text-zinc-900 font-bold focus:ring-2
focus:ring-orange-500 focus:outline-none">

                        <button type="button" onclick="togglePassword('rider-cp-new', this)" title="Show/Hide
password" class="absolute right-2.5 top-1/2 -translate-y-1/2 text-zinc-500 hover:text-zinc-800 cursor-pointer
text-base p-1.5 bg-zinc-100 hover:bg-zinc-200 rounded-lg active:scale-90 transition">

                            <i class="fa-solid fa-eye"></i>

                        </button>

                    </div>

                </div>

                <div>

                    <label class="block text-xs font-bold text-zinc-400 mb-1">Confirm New Password</label>

                    <div class="relative">

                        <input type="password" id="rider-cp-confirm" x-model="confirmPassword" required
placeholder="••••••••" style="color: #111827 !important; background-color: #ffffff !important;" class="w-full
px-3.5 py-2.5 pr-11 bg-white border border-zinc-300 rounded-xl text-xs text-zinc-900 font-bold focus:ring-2
focus:ring-orange-500 focus:outline-none">

                        <button type="button" onclick="togglePassword('rider-cp-confirm', this)" title="Show/Hide
password" class="absolute right-2.5 top-1/2 -translate-y-1/2 text-zinc-500 hover:text-zinc-800 cursor-pointer
text-base p-1.5 bg-zinc-100 hover:bg-zinc-200 rounded-lg active:scale-90 transition">

                            <i class="fa-solid fa-eye"></i>

                        </button>

                    </div>
```

```
                </div>

                <div class="flex justify-end gap-2 pt-2 border-t border-white/10">

                    <button type="button" @click="showChangePasswordModal = false" class="px-4 py-2 rounded-xl
bg-zinc-800 text-zinc-300 text-xs font-bold hover:bg-zinc-700 transition">Cancel</button>

                    <button type="submit" class="px-5 py-2 rounded-xl bg-gradient-to-r from-orange-500
to-orange-600 text-white text-xs font-bold hover:from-orange-600 hover:to-orange-700 transition
shadow-glow-orange">Update Password</button>

                </div>

            </form>

        </div>

    </div>

    <script src="{% static 'js/ui-loading.js' %}"></script>

</body>

</html>
```

APPENDIX L. CURRICULUM VITAE

VINCE ALSHIE RIVERA

LEAD DEVELOPER / WEB DEVELOPER

CONTACT

+63 9946902599

vincealshierivera@gmail.com

https://github.com/snyb1010

https://www.facebook.com
/vincealshie.valdez.1

SKILLS

- Core Engineering
- Responsive Engineering
- Mobile-Friendly Architecture
- Workflow Management
- UI/UX Design
- Team Direction
- Communication
- Problem-Solving
- Crisis Management & Adaptability
- Arduino

HOBBY

- Coding

LANGUAGES

- English
- Filipino

PROFILE

A 22-year-old aspiring Full Stack Developer from Palawan, Philippines with a strong foundation in both front-end and back-end development, experienced in building responsive web applications, developing RESTful APIs, managing databases, and implementing modern software development practices. Skilled in collaborating within development teams to create scalable, user-centered solutions while continuously learning emerging technologies and contributing to the successful delivery of digital projects.

WORK EXPERIENCE

Academic -Thesis (Canteen Express) 2026 – PRESENT
Lead Developer & Web Developer

- Primarily responsible for leading project development and writing core code for responsive web applications, while managing digital workflows via Git and GitHub to guide team collaboration, ensure UI/UX integrity and deliver seamless technical results.

Render Haus 2025
Full Stack Developer

- RenderHaus Studio is a creative digital service specializing in professional video editing solutions for creators, brands, and agencies. Through a dedicated-editor model, the company delivers reliable, scalable, and brand-consistent video content, helping clients improve content production efficiency and audience engagement.

EDUCATION

Bachelor of Science in Information Technology 2022 – Present
Palawan State University - Puerto Princesa City

Senior High School Diploma 2016 – 2022
Palawan National School - Puerto Princesa City

JONATHAN FAJICULAY

SYSTEM ADMINISTRATOR / PROOFREADER

CONTACT

+63 9164552317

jonathanfajiculay0809@gmail.com

https://github.com/jon-faji

https://www.facebook.com/jonz.faji

SKILLS

- Digital Systems & File Management
- Network Technology Foundations
- Proofreading & Copyediting
- Microsoft Word & Excel
- Great Communicator
- Adaptability to Change

HOBBY

- Technology & Networking
- Creative Literature & Media

LANGUAGES

- English
- Filipino

PROFILE

A 23-year-old multimedia production and proofreader from Palawan, Philippines with a strong background in live broadcasting and digital content localization, experienced in collaborating with cross-functional teams and global organizations to manage complex digital workflows and deliver high-quality, error-free media production.

WORK EXPERIENCE

Flame Scan - Freelance 2022 – 2023
Proofreaders

- Primarily responsible for conducting meticulous proofreading of translated scripts via Google Drive, collaborating with translators and typesetters to ensure absolute grammatical compliance, natural dialogue flow and zero-error delivery under fast-paced release schedules.

PSU Arsenals - Organization 2024 – 2025
Technical Head

- Primarily responsible for managing the technical integrity and digital workflow logistics of live, high-pressure broadcasts, ensuring all on-air content strictly conforms to industry standards and formatting regulations through rapid error detection and crisis management.

Academic - Thesis (Canteen Express) 2026 – Present
Lead Documentation Editor

- Primarily responsible for configuring server infrastructure and deployment environments while directing the final compilation, structural formatting and precise editorial review of the thesis documentation to ensure absolute compliance with university thesis manual .

EDUCATION

Bachelor of Science in Information Technology 2022 – Present
Palawan State University - Puerto Princesa City

Senior High School Diploma 2016 – 2022
Palawan National School - Puerto Princesa City

NORELYN MADIA

UI|UX DESIGNER / FRONT-END DEVELOPER

CONTACT

+63 9165705108

nyx828174@gmail.com

https://github.com/NexieMadia23

https://www.facebook.com/nexiegh

SKILLS

- Technical Core: HTML, CSS, JavaScript
- Responsive Design: Mobile-Layouts
- Git & GitHub
- UI/UX Design
- Great Communicator
- Adaptability
- Problem-Solving

PROJECTS

Visit this link to see projects

https://github.com/NexieMadia23

HOBBY

- Film Editing
- Reading
- Sports

LANGUAGES

- English
- Filipino

PROFILE

A 22-year-old information technology enthusiast and web programmer from Palawan, Philippines with a strong background in digital product improvement and system functionality, experienced in working within collaborative environments to manage modern development workflows and drive professional project goals.

WORK EXPERIENCE

Academic -Thesis (Canteen Express) 2026 - PRESENT

UI/UX Designer & Front-End Developer

- Primarily responsible for designing intuitive UI/UX wireframes, developing responsive front-end layouts, and maintaining critical project documentation, while utilizing Git and GitHub to collaborate with team members and ensure a seamless digital workflow.

EDUCATION

Bachelor of Science in Information Technology 2022 – Present

Palawan State University | Puerto Princesa City

2016 – 2022

Junior - Senior High School Diploma

Langogan National High School | Puerto Princesa City

JULIET LOU TUMALAC

WEB DESIGNER / DOCUMENT WRITER

CONTACT

+63 9701203661

julietloutumalac@gmail.com

https://github.com/jiyu0903

https://www.facebook.com/jloutmlc

SKILLS

- Mobile-Layouts
- Git & GitHub
- UI/UX Design
- Great Communicator
- Adaptability
- Problem-Solving

HOBBY

- Web Designing
- Reading

LANGUAGES

- English
- Filipino

PROFILE

A 21-year-old information technology enthusiast and web designer from Palawan, Philippines with a strong background in digital product improvement and system functionality, experienced in working within collaborative environments to manage modern development workflows and drive professional project goals.

WORK EXPERIENCE

Academic -Thesis (Canteen Express) 2026 – Present

Web Designer & Document Writer

- Primarily responsible for co-managing project workflows and authoring core technical documentation, while assisting in the design and development of web applications to ensure absolute compliance with project requirements and team goals.

EDUCATION

Bachelor of Science in Information Technology 2023 – Present

Palawan State University - Puerto Princesa City

Senior High School Diploma 2017 – 2023

Inagawan National High School - Puerto Princesa City